

**УЧЕБНИКИ
НГТУ**

Серия основана в 2001 году

**РЕДАКЦИОННАЯ КОЛЛЕГИЯ
СЕРИИ «УЧЕБНИКИ НГТУ»**

д-р техн. наук, проф. (председатель) *А.А. Батаев*
канд. техн. наук, доц. (зам. председателя) *В.В. Янпольский*

д-р техн. наук, проф. *С.В. Брованов*
д-р техн. наук, проф. *А.Г. Вострецов*
д-р техн. наук, проф. *А.А. Воевода*
д-р физ.-мат. наук, проф. *В.Г. Дубровский*
д-р филос. наук, проф. *В.И. Игнатьев*
д-р техн. наук, проф. *Н.В. Пустовой*
д-р филос. наук, проф. *М.В. Ромм*
д-р техн. наук, проф. *Ю.Г. Соловейчик*
д-р физ.-мат. наук, проф. *В.А. Селезнев*
д-р техн. наук, проф. *А.А. Спектор*
д-р техн. наук, доц. *В.С. Тимофеев*
д-р техн. наук, проф. *А.Г. Фишов*
д-р экон. наук, проф. *М.В. Хайруллина*
канд. экон. наук, доц. *С.С. Чернов*
д-р техн. наук, проф. *А.Ф. Шевченко*
д-р техн. наук, проф. *Н.И. Щуров*

Ю. В. ШОРНИКОВ

ТЕОРИЯ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

**ПРОЕКТИРОВАНИЕ
И РЕАЛИЗАЦИЯ**



**НОВОСИБИРСК
2022**

УДК 004.43(075.8)

Ш 795

Рецензенты:

В. Е. Зюбин, д-р техн. наук, зав. лабораторией киберфизических систем
Института автоматизации и электрометрии СО РАН

В. И. Гужов, д-р техн. наук, профессор кафедры ССОД НГТУ

Шорников Ю. В.

Ш 795 Теория языков программирования: проектирование и реализация : учебное пособие / Ю. В. Шорников. – Новосибирск : Изд-во НГТУ, 2022. – 290 с. – (Учебники НГТУ).

ISBN 978-5-7782-4817-5

Учебное пособие подготовлено в соответствии с Государственным образовательным стандартом по направлениям 09.03.01 «Информатика и вычислительная техника», 09.03.03 «Прикладная информатика» для цикла дисциплин информационных специальностей. Основой учебного пособия стал материал, прочитанный автором студентам соответствующих специальностей в Новосибирском государственном техническом университете и Казахстанско-Британском техническом университете в курсах «Теория формальных языков и компиляторов», «Системное программное обеспечение», «Лингвистическое обеспечение».

В учебном пособии рассмотрена теория порождающих грамматик, конечных автоматов и регулярных выражений. Все теоретические механизмы анализа и синтеза языковых конструкций строго формализованы и составляют теоретические основы проектирования языков программирования. Реализация языков программирования представлена разработкой языковых процессоров. Переход от формальных языков к языковым процессорам выполнен через конструктивные методы анализа со строгими моделирующими алгоритмами, которые могут быть реализованы на языках высокого уровня или с помощью современных средств автоматизации программирования. В пособии рассмотрены средства ANTLR и FLEX & BIZON для автоматизации программирования парсера и лексера.

Несмотря на образовательную направленность, пособие может быть полезно всем, кто занимается проектированием и реализацией новых языков, языковых процессоров и конечно-автоматных распознавателей.

Работа подготовлена
на кафедре автоматизированных систем управления

УДК 004.43(075.8)

DOI 10.17212/978-5-7782-4817-5

ISBN 978-5-7782-4817-5

© Шорников Ю. В., 2022

© Новосибирский государственный
технический университет, 2022

ОГЛАВЛЕНИЕ

Введение.....	7
1. СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ	11
1.1. Определения и логические связи языковых процессоров	11
1.2. Системные исследования в России	16
1.3. Место языковых процессоров	28
1.4. Потребность в разработке языковых процессоров	32
1.5. Современные архитектурные решения.....	35
2. ПОРОЖДАЮЩИЕ ГРАММАТИКИ И ЯЗЫКИ	43
2.1. Обозначения и определения	43
2.2. Порождающие грамматики.....	46
2.3. Языки порождающих грамматик	52
2.4. Прямая и обратная задачи формальных языков и грамматик	53
2.5. Геометрическая интерпретация синтаксического разбора	58
2.6. Эквивалентность и однозначность.....	63
2.7. Классификация Хомского	70
2.8. Графы автоматной грамматики	73
3. КОНЕЧНО-АВТОМАТНЫЕ РАСПОЗНАВАТЕЛИ И ЯЗЫКИ.....	77
3.1. Автоматные распознаватели и грамматики Хомского	77
3.2. Конечные автоматы	79
3.3. Синтаксические диаграммы и конечные автоматы	81
3.4. Недетерминированные конечные автоматы	86
3.5. Детерминированный конечный автомат	87
3.6. Моделирование НКА.....	96
3.7. Минимизация ДКА	96
3.8. МП-автоматы	99
3.9. Детерминированные МП-автоматы	102
4. ЯЗЫКИ И РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ	107
4.1. Регулярные множества и языки.....	109
4.2. Регулярные выражения	111



4.3. Автоматные грамматики и регулярные выражения	113
4.4. Регулярные выражения и конечные автоматы.....	115
4.5. Анализ алгоритмов.....	122
4.6. Целесообразность перехода от НКА к ДКА	127
5. СИНТАКСИЧЕСКИЙ И СЕМАНТИЧЕСКИЙ АНАЛИЗ	131
5.1. Процессор числовых констант	131
5.2. Сканер.....	135
5.3. Организация таблиц символов	145
5.4. Рекурсивный спуск.....	152
5.5. Диагностика и нейтрализация синтаксических ошибок	157
5.6. Вычисление арифметических выражений.....	159
5.7. Восходящие методы анализа	166
6. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ	175
6.1. Препроцессор к станкам с ЧПУ	175
6.2. Интерпретатор задачи Коши	182
6.3. ИСМА 2015	201
7. ГЕНЕРАТОРЫ ЯЗЫКОВЫХ ПРОЦЕССОРОВ	217
7.1. ANTLR.....	218
7.2. Листинг грамматики <i>LISMA_PDE</i> в формате ANTLR	222
7.3. FLEX & BISON	226
Библиографический список	229
Приложения.....	237
Приложение А.....	237
Приложение Б.....	287

ВВЕДЕНИЕ

В настоящее время все более актуальными становятся вопросы проектирования систем программирования, без которых невозможно представить себе современный информационно-вычислительный комплекс. Традиционно и достаточно грубо вычислительную машину можно охарактеризовать двумя основными компонентами: она включает в себя программное обеспечение (ПО, software) и техническое, или аппаратное, обеспечение (ТО, hardware). В прошлом (1950–1970-е гг.) преобладающей доминантой в этой характеристике считалось ТО. В настоящее время акценты доминирующего по важности направления резко сместились в сторону ПО, причем с появлением интернет-технологий системы программирования стали развиваться не просто быстро, а ускоренными темпами. В настоящем учебном пособии рассматриваются вопросы разработки той части ПО, которая связана с проектированием и реализацией языков программирования системными средствами – языковыми процессорами.

Одним из первых и надежных средств общения с вычислительной техникой является символьный язык программирования. Проектирование языка программирования в учебном пособии рассматривается в трех концептуальных определениях. Первый и исторически самый ранний способ определения языка – порождающие грамматики. Второй рассмотренный способ – конечные автоматы и автоматы с магазинной памятью как механизмы порождения и анализа так называемых сентенциальных форм, или допускаемых языковых цепочек (предложений). Наконец, последний аспект порождения ограниченного класса языковых конструкций – регулярные выражения.

В учебном пособии представлен сравнительный анализ механизмов порождения языковых конструкций и показана их взаимосвязь через строгие формальные определения. Практическая часть содержит реализацию синтаксиса и семантики в соответствии



с теоретическими схемами анализа. Рассматриваются традиционные методы анализа сверху вниз или слева направо и методы восходящего распознавания – снизу вверх. Приводятся рекомендации по выбору соответствующего метода для эффективного анализа.

Вопросы практической реализации языковых процессоров неразрывно связаны с теоретической частью. Так, обязательным моментом перед реализацией является вопрос определения однозначности и безвозвратности разбора – эти понятия подробно проанализированы в теоретической части с иллюстрациями. Кроме того, в практикуме языковых процессоров представлена техника практической разработки конкретных предметных языков и языковых процессоров. Рассмотрены три примера: конструирование языка подготовки управляющих программ для обработки деталей на станках с ЧПУ и семантический анализатор исполнительного механизма как приложение в промышленной сфере; язык *LISMA* описания задачи Коши и язык *LISMA_PDE* описания гибридных систем с возможностью программирования определенного класса систем уравнений в частных производных. Зарегистрированные и защищенные в кандидатских диссертациях проекты символьных языков *LISMA* и *LISMA_PDE* реализованы в инструментальной среде ИСМА и используются в научных исследованиях и образовании.

Рассмотренные в учебном пособии вопросы нашли свое отражение в трудовой функции А/02.6 «Разработка компиляторов, загрузчиков и сборщиков» профессионального стандарта 06.028 «Системный программист», входящего в перечень профессиональных стандартов Федерального государственного образовательного стандарта высшего образования (ФГОС ВО) по направлению 09.03.01 «Информатика и вычислительная техника» в рамках дисциплины «Теория формальных языков и компиляторов». Несомненно, рассматриваемые вопросы востребованы и в других направлениях, в которых используются информационные технологии. Такая новая государственная образовательная ориентация вселяет надежду на исправление негативной тенденции резкого относительного снижения интересов отечественной науки в развитии языковых средств и окружения, что в свое время было отмечено главным российским координатором в указанном направлении – профессором И. В. Поттосиным (Институт систем информатики СО РАН, Новосибирск). Кроме того, в современных сложных условиях импорто-



замещения все более актуальными становятся отечественные ИТ-технологии, способные обеспечить России достойный информационный суверенитет, что полностью соответствует политике государства. В связи с этим современные приоритеты, установленные указами правительства РФ, направлены на развитие отечественного ПО. Важная роль в этом отводится качественной подготовке высококвалифицированных специалистов в области ИТ-технологий.

Учебный материал, ставший основой для пособия, уже на протяжении нескольких лет читается студентам бакалавриата всех форм обучения специальности 09.03.01 «Информатика и вычислительная техника» и магистратуры направления 09.04.01 «Информатика и вычислительная техника» с программой подготовки «Теория формальных языков и компиляторов» и «Технология разработки программного обеспечения» в Новосибирском государственном техническом университете. По существу, представленный материал является переработанным и дополненным вариантом курса лекций и ранее изданных учебных пособий и методических материалов, указанных в списке литературы.

Изучение материалов учебного пособия должно неразрывно сопровождаться решением практических задач, приведенных в упражнениях после каждого раздела. Их решение позволит читателю самостоятельно оценить степень усвоения теоретического материала. Упражнения раздела 6 подобраны таким образом, что для их выполнения необходимы знания всего рассмотренного теоретического материала, и поэтому они могут служить критерием комплексной проверки освоения всей теории. Решение задач раздела 6 включает в себя обязательную программную реализацию компонентов языковых процессоров с проектированием и реализацией пользовательского интерфейса, примеры которого приведены в настоящем учебном пособии.

Программная реализация оконного интерфейса с комментариями показана в приложении А, листинги программ представляют собой фрагмент практического материала курсовых и выпускных квалификационных работ студентов специальности 09.03.01. Приведенный в приложении А материал выполнен как часть курсового проекта по курсу «Теория формальных языков и компиляторов». Эта часть касается проектирования программы пользовательского интерфейса с редактором входного текста программы на разрабо-



танном варианте языка программирования и полем диагностики синтаксических ошибок, а также результатами работы основной программы. В приложении А рассмотрены два варианта реальных курсовых проектов: реализация интерпретатора арифметических выражений в ПОЛИЗ на C++ и реализация парсера оператора if на C#.

В приложении Б приведена грамматика спроектированного нового оригинального языка моделирования *LISMA_PDE*, который позволяет в том числе описывать определенный класс уравнений в частных производных в отличие от базового языка *LISMA*, настроенного на описание обыкновенных дифференциальных уравнений. Рассмотренные новые языки зарегистрированы в Федеральной службе по интеллектуальной собственности (Роспатент) и защищены кандидатскими диссертациями под руководством автора по специальности 05.13.11 «Математическое и программное обеспечение вычислительных машин, комплексов и компьютерных систем».

1. СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

В этом разделе рассмотрены некоторые типы языковых процессоров как примеры механизма преобразования программ на языках программирования. Дается краткая характеристика языковых процессоров в зависимости от предметной ориентации задач пользователя. В подразделе 1.2 приводится аналитический обзор современных научных направлений и соответствующих научно-исследовательских отечественных центров. Кратко обозначены актуальные вопросы разработки системных средств.

1.1. ОПРЕДЕЛЕНИЯ И ЛОГИЧЕСКИЕ СВЯЗИ ЯЗЫКОВЫХ ПРОЦЕССОРОВ

Определение 1.1. *Языковым процессором называется системная программа обработки текстов на языках программирования.*

Структура и логические связи языковых процессоров непосредственно связаны с языками программирования. Языки программирования, в свою очередь, зависят от предметной области (см. таблицу) [86].

Системная программа выполняет перекодировку входного формата в некоторый выходной набор данных. Различают *компиляторы*, *интерпретаторы*, *ассемблеры* и *препроцессоры*. Такие программы разрабатываются на основе синтаксически-ориентированных методов.

Компилятор – это такой языковой процессор, в котором входной формат данных – язык программирования высокого уровня, а выходной формат – объектный код.



Т а б л и ц а

**Использование языков программирования
в различных областях**

Область	Основные языки
Обработка деловой информации	<i>COBOL, C/C++, Java, C#, Kotlin, Python, 4GL, SpreadSheet, PL/1, VBA</i>
Научные вычисления	<i>ALGOL, Fortran, C/C++, BASIC, Java, MATLAB, Python, R, Julia</i>
Системное программирование	<i>C/C++, Java, C#, Rust, Ada, Modula, Assembler, Forth, РЕФАЛ</i>
Искусственный интеллект и машинное обучение	<i>LISP, Prolog, Python, R, C/C++</i>
Языки командной строки	<i>BASH, Shell, TCL, AWK, Powershell</i>
Программирование веб-сайтов	<i>HTML, CSS, JavaScript, TypeScript, CoffeeScript</i>
Программирование на графических процессорах	<i>GLSL, HLSL, C/C++</i>
Разработки с графическим интерфейсом	<i>C/C++, Kotlin, Java, C#, HTML, CSS, JavaScript, QML, XAML</i>

Компиляторы являются системными исполнительными программами с оптимально спроектированным кодом. Поэтому они остаются наиболее универсальным решателем в отличие от интерпретаторов, которые чаще используются для отладки алгоритмов. Логические связи компилятора приведены на рис. 1.1.

Охарактеризуем каждый блок в отдельности и при этом определим сопутствующие понятия.

1. Исходный текст, или программа, поступает в компилятор на некотором входном языке – первоначально в сканер или лексический анализатор (блок 1), который выполняет декомпозицию текста на *лексемы (токены)* или просто символы.



Рис. 1.1. Логические связи компилятора

Лексема (токен, token) – это некоторая программная единица, которая характеризует определенные категории языка. Традиционно лексемами являются идентификаторы, числовые константы, ключевые слова, знаки операций и др. Понятие «токен» широко используется в иностранной литературе, например в [11]. В отечественных источниках вместо него употребляют эквивалентное понятие «лексема» [12]. Токен, или лексема, в общем случае является подмножеством множества терминальных символов, строгое определение которых будет приведено ниже. В свою очередь, лексемы состоят из литер.

Литера (литерал) – это неделимая информационная единица. Из литер набирается текст программы; в простом информационном понятии это терминальное поле на клавиатуре компьютера или компьютерный алфавит ASCII и EBCDIC. Иначе говоря, литерами



являются знаковые клавиши на клавиатуре терминала, за исключением служебных и функциональных. Токены (лексемы) могут состоять из одной или более литер, более строгое математическое понятие этой категории будет приведено ниже.

Функция сканера – передать определенный внутренний код определенным объектам программного сегмента. Так, ключевые слова в языке *C* имеют один внутренний код, разделители – другой, числовые константы – третий и т. д. Кроме того, сканер *фильтрует* (очищает) программу от незначащих символов – пробелов, знаков табуляции и переводов на новую строку.

2. *Синтаксис*. Синтаксический анализатор проводит анализ на соответствие программы на входном языке грамматическим правилам этого языка, а также совместно с процессором синтаксических ошибок занимается обработкой несоответствия синтаксиса грамматике, нейтрализацией таких мест в программе и сообщает пользователю об этих ошибках.

3. *Семантика*. Семантический анализатор предполагает смысловую обработку. Из синтаксически правильных конструкций генерируется промежуточный набор данных, который представлен в виде триад, тетрад, ПОЛИЗ и в другом внутреннем представлении (чаще древесном), удобном для последующей обработки. Промежуточный набор данных служит для упрощения генерации кода.

4. *Подготовка к генерации кода*. Все внешние модули, макросы, встроенные и внешние функции объединяются, и таким образом программа из нескольких функций (процедур) собирается в единый программный сегмент.

5. *Генератор кода* преобразует промежуточный код в набор объектных кодов, т. е. переводит программный сегмент в объектный код.

В компиляторе, как показано на рис 1.1, тексты программ проходят все стадии обработки до объектного кода. Кодовый образ программы после компиляции оказывается эффективным. В связи с этим компилятор предназначен для решения вычислительных задач, когда загрузочный модуль эффективно распределяет вычислительные ресурсы.

Интерпретатор – это такой языковой процессор, в котором входной формат данных – язык программирования высокого уровня, а выходной формат – промежуточный или исполнительный код.



Процесс интерпретации состоит в обработке каждого оператора и его немедленном исполнении. Интерпретатор используется в отладочных целях. Поскольку в интерпретаторе используются обычно только блоки 1, 2, 3 (рис. 1.1), то он является более быстрым процессором на стадии трансляции, нежели компилятор, и как решатель задач используется чаще в пооператорных режимах.

Ассемблер – это такой языковой процессор, в котором входной формат данных – команды ассемблера, близкие к кодам машины. Выходной формат – исполнительный код или машинные команды, доступные для загрузки в оперативную память и исполнения. Ассемблеры используются в задачах управления ресурсами вычислительной техники. Поскольку входные программы на ассемблере близки к кодам машины, они обладают наивысшей степенью доступа к командам управления всей периферией вычислительной техники.

Препроцессор – это такой языковой процессор, в котором входной формат данных – специализированные (проблемные) языки, выходной – объектный код (как у компилятора) либо данные, аналогичные выходным данным интерпретатора. Структура препроцессора представлена на рис. 1.2.



Рис. 1.2. Структура препроцессора

Препроцессоры используются для решения предметных задач с языков пользователя, когда пользователь не является профессионалом в области вычислительной техники и программирования и имеет доступ лишь к предметным категориям своего языкового процессора.

Одним из основных инструментов при проектировании языковых процессоров являются синтаксически-ориентированные методы,



которые, в свою очередь, базируются на теории формальных языков и грамматик и являются научно обоснованной методологией проектирования языковых процессоров.

1.2. СИСТЕМНЫЕ ИССЛЕДОВАНИЯ В РОССИИ

Представленные материалы помогут предметной ориентации по разработке системных средств в части проектирования языков, языковых процессоров и их окружения. Далее приведена история отечественных исследований, которая в основном базируется на анализе [27], выполненном главным российским координатором в этом направлении – профессором И. В. Поттосиным, а также перечислены современные исследования в России из материалов открытых источников.

Институт систем информатики (ИСИ) СО РАН (Новосибирск)

ИСИ СО РАН – ведущий коллектив в России по проектированию языков, языковых процессоров и их окружений. ИСИ СО РАН стал преемником работ, проводившихся под руководством академика А. П. Ершова в Вычислительном центре СО АН (Новосибирск). Работы по теории трансляции [5, 15] и оптимизации программ, а также по многоязыковым транслирующим системам принесли группе академика А. П. Ершова мировую признательность.

В лаборатории *теоретического программирования* института ведутся исследования по *верификации* программ и языкам спецификации. Разработан язык спецификации систем реального времени *REAL 92* [5]. Верификация трансляторов в этой системе заключается в доказательстве правильности программных модулей трансляторов относительно их спецификаций по методу Хоара [55]; предложенные средства использованы для спецификации трансляторов с подмножеств языка *Pascal* и языка *BASIC*, проведено аннотирование этих трансляторов и доказана их корректность с помощью системы верификации.

В лаборатории также ведутся работы над исследованием формальных моделей и методов описания семантики, спецификации и верификации программ и систем. Развивается онтологический



подход к формальной операционной семантике языков программирования. Разработаны и реализованы следующие продукты:

- система верификации программ на языке *C-light*;
- методы и средства анализа и верификации мультиагентных систем;
- система программного комплекса *SRDSV2* (*SDL/REAL* Distributed Systems Verifier), предназначенного для моделирования, анализа и верификации SDL-спецификаций, который использует разработанный ранее язык *Dynamic-REAL* (*dREAL*) в качестве промежуточного языка;
- система программного комплекса *ASV* (Automata Systems Verifier), предназначенного для анализа и верификации автоматных спецификаций телекоммуникационных систем.

В группе *смешанных вычислений* проведены исследования по развитию методов преобразования императивных и функциональных программ путем смешанных вычислений. Найден новый метод организации смешанных вычислений, основанный на поливариантной схеме, что открыло путь для построения автоматических самоприменимых процессоров смешанных вычислений. По отношению к трансляции это позволяет выводить и обосновывать специфические понятия, такие как таблица символов, шаблоны кодогенерации и другие, исходя из универсальных механизмов смешанных вычислений. Инструментом для экспериментов служит автопроектор *Similix*, разработанный в Копенгагенском университете для языка *Scheme* и модифицированный в данной группе.

В лаборатории *оптимизации и преобразований программ* (руководитель В. Н. Касьянов) ведутся теоретические исследования и разработка экспериментальных систем по созданию процессоров преобразования программ [5]. Были исследованы проблемы корректности конкретизирующих преобразований, их классификации, типов аннотирующих утверждений. В связи с проблемами оптимизации и преобразования программ [31] проведен сравнительный анализ форм промежуточных представлений программ, ориентированных на оптимизацию и параллельную обработку, а также подготовлен каталог реструктурирующих и оптимизирующих преобразований программ для различных архитектур, включая параллельные.

В лаборатории ведутся работы по созданию анализаторов семантических свойств. Такие анализаторы выявляют подозрительные



семантические конструкции в тексте программы и сообщают пользователю о свойствах состояний вычислений в интересующих его точках. Создан анализатор свойств *Modula*-программ [38].

В настоящее время в лаборатории *конструирования и оптимизации программ* проводятся исследования проблем систем программирования в следующих направлениях:

- развитие теории трансформационного программирования и разработка методов и средств конструирования эффективных и надежных программ;
- создание инструментально-информационной системы по оптимизирующим и реструктурирующим преобразованиям программ для ЭВМ параллельных архитектур;
- разработка экспериментальной версии среды параллельного и функционального программирования для поддержки облачных супервычислений. Разработан язык *Cloud Sisal*, который является входным языком системы параллельного программирования *CPPS*;
- создание системы *ATG* (Automatic Test Generator), которая позволяет в автоматическом режиме тестировать компилятор для нового языка *Cloud Sisal* при помощи порождаемых генератором тестов.

В лаборатории *системного программирования* (заведующий лабораторией В. И. Шелехов) работы по трансляции выполнены в рамках более общего проекта, имеющего целью создание методики и инструментальной поддержки разработки надежных и качественных программ для бортовых ЭВМ. Работа над проектом включает в себя как исследования и экспериментальные разработки, так и создание развитого окружения программирования для бортовых ЭВМ методами кросс-трансляции. Система *СОКРАТ* разработана для языка *Modula-2* [38], однако большинство инструментов системы языково-независимо и ориентировано на класс языков со статическим контролем типов *Modula-2*, *Oberon* [33, 58], *Ada* [41], *C++* [68]. Разработан и реализован глобальный оптимизатор как отдельный языковой процессор, слабо зависящий от входного языка. Оптимизирующий генератор кода осуществляет большое число архитектурно-зависимых оптимизаций. Наличие различных архитектур бортовых ЭВМ требует сравнительно легкой настройки генератора кода на архитектуру объектной ЭВМ.

Высокие требования к надежности разработки и эффективности бортовых программ обусловили включение в систему ряда процес-



соров автоматической обработки программ – специализаторов, анализаторов свойств, о которых говорилось выше. Группой под руководством А. Е. Недори [33] разработана двухязыковая система *Mithrill*. Система предназначена для разработки переносимого программного обеспечения на языках *Oberon-2* и *Modula-2* и позволяет смешивать в одном проекте модули на разных языках. Система включает в себя несколько генераторов кода и генератор текста на языке *ANSI C*.

В лаборатории системного программирования в настоящее время ведутся работы по созданию методов и экспериментальных инструментов конструирования и спецификаций программ в окружениях надежного программирования. Разработаны предикатный язык программирования *P* и синтаксически-ориентированный редактор предикатных программ, а также методы и алгоритмы оптимизации предикатных программ с трансляцией на язык *C++*.

ИСИ СО РАН – ведущий российский центр по работам, связанным с языком *Modula-2* [37, 38] и его преемниками. В этом коллективе был разработан первый советский транслятор для *Modula-2*, созданы семейства процессоров и рабочая станция КРОНОС с архитектурой, ориентированной на поддержку *Modula*-программ, система программирования с языка *Modula-2* для ЭВМ. Для семейства КРОНОС *Modula-2* была базовым языком программирования, все системное программное обеспечение писалось на этом языке. Коллектив ИСИ СО РАН поддерживает рабочие контакты с группой автора *Modula-2* профессора Н. Вирта из Цюрихского технического университета. Институт является организатором российской группы по стандартизации *Modula-2*.

В ИСИ СО РАН им. академика А. П. Ершова в настоящий момент ведется разработка языка программирования *Cloud Sisal* [95]. Цель проекта – дать возможность широкому кругу лиц, находящихся в удаленных населенных пунктах или в местах с недостаточными вычислительными средствами, но имеющих выход в Интернет, дистанционно, без установки дополнительного программного обеспечения на своих недорогих вычислительных устройствах в визуальном стиле создавать и отлаживать переносимые параллельные программы на языке *Cloud Sisal*. Затем пользователи могут дистанционно (в облаке) осуществлять эффективное решение своих задач, исполняя созданные и отлаженные переносимые программы на языке



Cloud Sisal на некоторых доступных удаленных супервычислителях, предварительно адаптировав их под используемые супервычислители с помощью облачного оптимизирующего кросс-компилятора, предоставляемого системой параллельного программирования.

В институте систем информатики им. академика А. П. Ершова также разрабатывается язык параллельного программирования *СИИХРО* [96] для обучения концепциям параллельного программирования, ведутся работы по верификации программ на различных языках программирования. В частности, проведены исследования концептуального базиса улучшенного трехуровневого метода верификации *C#* программ [97]. Институт также занимается исследованиями в области применения языка *Dynamic-REAL* [98].

Институт прикладной математики (ИПМ) РАН (Москва)

В институте разработаны и реализованы транслятор для языка *Simula 67*, транслятор для расширения языка *Fortran* применительно к параллельным и векторным вычислениям с векторизатором, бортовая операционная система для космического корабля «Буран». Разработанная система *RAMPA* [28] интегрирует три различных подхода к языкам программирования для распределенных и параллельных систем, а именно:

- языки параллельного программирования для распределенных систем, основанные на коммуникации с посылкой сообщений;
- расширения последовательных языков, связанные с разделением вычислений между процессорами в виде специальных комментариев;
- декларативные языки спецификаций.

Для первых двух задач разработано два расширения *Fortran 77* – *Fortran GNS* и *Fortran DMV*. Разработаны трансляторы для *Modula-2* и *C* на транспьютероподобные RISC-процессоры. Выполняются разработки трансляторов декларативных языков и реализация их на языке *РЕФАЛ* [59, 60] с переводом *РЕФАЛ*-программы в код абстрактной *РЕФАЛ*-машины. Язык *РЕФАЛ* [61] был предложен В. Ф. Турчиным (университет Нью-Джерси) [60, 62].

В настоящее время в институте проводятся исследования *DVM*-системы, предназначенной для проектирования параллельных программ научно-технических расчетов на языках *C-DVMH*



и *Fortran-DVMH*. Разрабатывается компилятор для языка *НОРМА*, который является средством автоматизации решения задач математической физики на вычислительных системах с параллельной архитектурой.

**Вычислительный центр РАН
Отдел систем программного обеспечения (Москва)**

В учреждении проводятся исследования по атрибутивным грамматикам для высокопараллельных многопроцессорных архитектур. Разработаны системы проектирования трансляторов (СПТ) *Super* для *Modula*-подобных языков [30], *Modula*-транслятор и транслятор для языка программирования баз данных *Modula-90* (расширение *Modula-2*). Для параллельных вычислений в научных приложениях разработан язык *SYNAPSE* [29] – расширение языка *C*. Создано окружение программирования для языка *РЕФАЛ*, аналогичное турбосистемам. Все продукты реализованы для MS Windows.

**Московский инженерно-физический институт.
Факультет кибернетики**

Под руководством О. Н. Перминова в институте выполнена реализация языка *Ada* [41, 42] и его окружения, разработан транслятор с языков *Pascal* и *Ada*. Затем были созданы транслятор и окружение для *Ada* на IBM PC – совместимых компьютерах, разработана кросс-система программирования для языка *Ada* на Cray-подобные и транспьютерные архитектуры. Инструментальной машиной выступает IBM PC. Группа также ведет исследования по автоматической параллелизации *Ada*-программ.

**Институт вычислительной математики
и математической геофизики СО РАН (Новосибирск).**

**Отдел программного обеспечения
высокопроизводительных ЭВМ (supercomputers)**

В институте под руководством В. Э. Малышкина [62] ведутся работы по созданию средств программирования для крупноблочных многопроцессорных систем. В подобных архитектурах вычислительная система строится как иерархия, элементами которой



являются ЭВМ. В соответствии с этим подходом была разработана мультипроцессорная вычислительная система «Сибирь», а для программирования на этой системе – язык *ИНЯ* [62–64]. Язык, являясь расширением *Fortran*, дополнительно содержит средства для инициации процессов, параллельно исполняемых на подпроцессорах иерархии, их синхронизации, декомпозиции данных и т. п. Система программирования для этого языка включает в себя препроцессор, переводящий *ИНЯ*-программы в *Fortran*-программы на базовой машине, и библиотеку программ, поддерживающих распределение процессоров и данных в вычислительной системе.

Для описания и моделирования алгоритмов для ассоциативных *SIMD*-процессоров разработан *STAR* – паскалеподобный язык, имеющий такие типы данных, как битовые матрицы, вырезки (*slice*), слова и операции, осуществляющие ассоциативную параллельную обработку этих данных. Планируется создание соответствующей системы для моделирования *STAR*-программ.

Разработана мобильная библиотека *СПАРФ* (Сверхточной Параллельной АРиФметики), характерной чертой которой является реализация принципа переносимости программного обеспечения, что позволяет использовать *СПАРФ*-библиотеку на различных вычислительных архитектурах как последовательного, так и параллельного действия. Библиотека передана в НИИ «Квант» (Москва), Пермский государственный университет, университет Aix-Marseille (Марсель, Франция) и используется в перечисленных организациях.

В настоящее время отдел математического обеспечения высокопроизводительных вычислительных систем (МО ВВС) занимается проведением исследований и разработкой моделей, технологий, языков и систем параллельного программирования для поддержки реализации крупномасштабных численных моделей для их исполнения на суперкомпьютерах. Первый крупный проект отдела – разработка архитектуры и создание высокопроизводительной вычислительной системы (8-процессорный кластер в современной терминологии) «Сибирь» с пиковой производительностью примерно 100 мегафлопс и ее программного обеспечения. Разрабатываются и изучаются локальные алгоритмы организации параллельных вычислений, реализующих большие численные модели на пета- и эксафлопсных мультикомпьютерах. Ведется разработка проекта HPC Community Cloud (HPC2C) – проекта по разработке программ-



ного инструментария для объединения ресурсов различных высокопроизводительных вычислительных систем (ВВС) в единый сервис и предоставления сторонним программным системам программного интерфейса (API).

**Иркутский государственный университет.
Математический факультет**

Группа под руководством А. В. Манциводы занимается проблемами функционального и логического программирования. Разработан язык *Flang* [56], основывающийся на недетерминированных функциях, в которых обычные для функциональных программ определения функций расширены возможностью употреблять отношения языка *Prolog*, а вычисления функций могут быть связаны с бэктрекингом и поиском альтернативных решений. В язык также введены средства (специальные примитивы) для задания ограничений (constraints). Все эти средства языка *Flang* позволяют исследовать совокупность современных парадигм программирования. Разработан *Flang*-транслятор. Для языка *Flang* предложена машина (*Flang*-машина), альтернативная машине Уоррена (WAM). В экспериментах *Flang*-транслятор показал возможность строить весьма эффективные программы (сравнимые при адекватности алгоритмов с программами, полученными на языке *Turbo Pascal*).

В настоящее время работы по разработке и исследованию системного программного обеспечения успешно продолжают. Разработан универсальный язык программирования *Libretto*.

Институт автоматики и процессов управления ДВО РАН

Работы под руководством А. С. Клещева, как правило, связаны с языками и системами искусственного интеллекта. Разработано несколько продукционных языков представления знаний, для которых созданы системы программирования. Последним является язык *РЕПРО* [54], основанный на декларативных системах продукций. В процессе работ исследовались проблемы анализа информационных связей в системе продукций; оптимизации логического вывода по числу операций вывода; преобразования продукционных программ, в том числе путем смешанных вычислений; генерации *Pascal*-программ по декларативному описанию базы знаний на языке *РЕПРО*.



В настоящее время лаборатория интеллектуальных систем имени А. С. Клещева работает над облачной платформой *IACPaaS* для разработки оболочек интеллектуальных сервисов. Выполняется реализация оболочки и портала знаний по верификации математических доказательств на платформе *IACPaaS*. Разрабатывается облачная платформа, поддерживающая единые технологические принципы проектирования, реализации и использования прикладных и инструментальных интеллектуальных сервисов, а также специализированная оболочка для создания интеллектуальных диагностических медицинских систем.

Институт точной механики и вычислительной техники РАН (Москва)

В институте создавались системы программирования для известного советского проекта суперкомпьютеров «Эльбрус». Для «Эльбрус-1» и «Эльбрус-2» был выполнен ряд интересных работ, созданы эффективный транслятор для специально ориентированного на архитектуру «Эльбрус» языка высокого уровня *Эль-76*, трансляторы с языков *Prolog* и *Ada*, оптимизирующий транслятор с *Fortran*, трансляторы с языков *COBOL*, *PL/I*, *АЛГОЛ* – *ЭЛЬБРУС* и др.

В Новосибирском филиале ИТМиВТ под руководством И. С. Голосова разработана система программирования «Эльбрус-3». Входной язык транслятора включал помимо традиционных диалектов *Fortran* также и конструкции *Fortran 90*. Особое внимание уделялось оптимизации программ, векторизации, компактификации кода.

Сейчас институт разрабатывает универсальную технологию оптимизирующей компиляции. Разработаны реализующие ее строительные блоки – анализатор, автоматический распараллеливатель и оптимизатор. К настоящему времени блоки уже оттестированы в технологической цепочке компиляторов *GCC*. Компиляция с языков *C*, *C++* и *Fortran* оттестирована в целевую архитектуру x86.

Институт математики и механики Санкт-Петербургского государственного университета

Группа под руководством В. О. Сафонова разработала широкий спектр систем программирования для «Эльбруса» – реализованы языки *Pascal*, *Modula-2*, *CLU*, *SNOBOL-4*, *РЕФАЛ*, *ABC* (язык символической обработки, предложенный С. С. Лавровым), а также ряд



интерактивных систем программирования для языков *ALGOL 60*, *BASIC*, *Forth*.

Была разработана так называемая TIP-технология (Technological Instrumental Package), предназначенная для создания больших модульных систем [57]. TIP-технология предполагает более содержательную спецификацию пакета как элемента сложной системы, чем в известных языках. При этом достаточно детально регламентируется интерфейс пакета, что облегчает его переиспользование. На основе TIP-технологии был разработан ряд трансляторов, причем с использованием одних и тех же пакетов в различных трансляторах. Группа сотрудничает с Sun Microsystems по разработке *Pascal*-транслятора для новой рабочей станции семейства SPARC.

**Санкт-Петербургский государственный университет.
Кафедра системного программирования**

Основное направление деятельности коллектива – создание технологии разработки программ для систем реального времени, прежде всего для систем связи (руководитель А. Н. Терехов) [34]. Основу создаваемой технологии составляют языки высокого уровня, обеспечивающие большую надежность разрабатываемого программного обеспечения, а именно языки с полным статическим контролем типов (*ALGOL 68*, *Modula-2*, *Ada*). Они, и в первую очередь *ALGOL 68*, положены в основу большинства создаваемых технологических инструментов. В коллективе созданы трансляторы для ряда подобных языков, наиболее интересными являются трансляторы для *ALGOL 68* и *Ada*.

Работа над технологией разработки программ, основанной на статических языках, привела к созданию архитектуры ЭВМ «Самсон» [58], ориентированной на программы, получаемые трансляцией с этих языков. В настоящее время «Самсон» выпускается как в виде отдельного устройства (процессор и локальная память), так и в виде троированного комплекса высокой надежности. «Самсон» – это микропрограммируемая ЭВМ. Средства микропрограммирования для нее разработаны на основе языка *ALGOL 68* и позволяют на порядок облегчить микропрограммирование по сравнению с традиционными микроассемблерами, что делает возможной микропрограммную оптимизацию конкретных программ. Сейчас на основе этого ведется проработка методики настройки ЭВМ «Самсон» на заданные предметные области.



Для языка *Ada* осуществлена полная реализация его и средств поддержки программирования в виде интегрированной инструментальной среды *Pallada*. Последняя содержит набор инструментов для разработки *Ada*-программ, в том числе отладчик в терминах входного языка, собственный текстовый редактор, систему конфигурационного контроля. Особое внимание в системе уделяется поддержке многобиблиотечного окружения. Специально для технологических целей в систему введены такие возможности, как гибкое управление проектом на основе абстрактных атрибутов, связанных с разделами библиотек, – поддержка запросов программ из библиотек, аналогичных принятым в базах данных. Система обладает высокой скоростью компиляции и перекомпиляции, в частности за счет автоматической минимизации необходимых перекомпиляций, вызванных изменениями в контексте, и реализации пошагового связывания (*incremental binding*). Среда *Pallada* реализована для IBM под VM/SP и под UNIX. Система в принципе доведена до состояния программного продукта.

В настоящее время заведующий кафедрой системного программирования механико-математического факультета СПбГУ профессор А. Н. Терехов руководит также компанией «Ланит-Терком» с годовым оборотом 7 млн долл. Основная деятельность компании – разработка программного обеспечения мультицелевых проектов с эффективными бизнес-планами.

Санкт-Петербургский политехнический университет (СПбГПУ)

Под руководством В. П. Котлярова в университете разработана технология построения программного обеспечения для встроенных ЭВМ (технология КОМФОРТ), основанная на модульном расширении языка *Forth*. Были созданы инструменты этой технологии, поддерживающие сборочное программирование. Группой разработчиков проведены исследования по применению методологии смешанных вычислений для построения эффективного программного кода.

Под руководством профессора Ю. Г. Карпова [103] разработан графический язык *Statechart* – карт состояний в оболочке инструментальных средств *AnyLogic*. Новое программное обеспечение было основано на передовых достижениях информационных технологий,



таких как объектно-ориентированный подход, элементы стандарта UML, базового языка программирования *Java* и современного GUI. *AnyLogic* поддерживает следующие формализмы: системную динамику, агентное моделирование и событийно-непрерывное моделирование, а также любую комбинацию этих подходов в пределах одной модели.

Большой шаг вперед был сделан в 2003 г., когда был выпущен *AnyLogic 5*, ориентированный на бизнес-моделирование. С помощью *AnyLogic* стало возможным разрабатывать модели в таких приложениях, как рынок и конкуренция; здравоохранение и фармацевтика; производство; логистика и цепочки поставок; бизнес-процессы и др. *AnyLogic 8* работает в интегрированной среде кросс-платформенных приложений *Eclips*. Инструмент *AnyLogic* нашел широкое применение на отечественном рынке и за рубежом. Однако это ПО ограничено в части моделирования непрерывных динамических процессов с жесткими режимами в условиях односторонних событий, где требуются нетрадиционные численные методы.

В СПбГПУ в группе под руководством Ю. Б. Сениченкова разработаны инструментальные средства *AnyDynamics* (<https://www.exponenta.ru>), которые лишены указанных недостатков *AnyLogic*. В основу формализма *AnyDynamics* положен подход гибридных автоматов. Графический язык спецификации гибридных моделей функционально аналогичен известной концепции современного объектно-ориентированного языка компонентного моделирования дискретно-непрерывных процессов *Modelica* (<https://www.openmodelica.org/doc/OpenModelicaUsersGuide/latest/ommatlab.html>).

Новосибирский государственный технический университет (НГТУ НЭТИ)

В НГТУ НЭТИ (<https://www.nstu.ru>) под руководством профессора Ю. В. Шорникова разрабатывается некоммерческое программное обеспечение инструментального моделирования сложных динамических систем.

Мультиязыковая среда инструментального исследования динамических процессов ИСМА [48, 51, 84] имеет предметно-ориентированную направленность к различным областям науки и техники. Исторически первым был разработан графический язык структурных схем *LISMA_STR* [51], принятый в инженерной практике



описания динамических процессов в системах автоматического управления, электромеханических и других технических системах. Предоставленный инженерам дружественный интерфейс GUI с традиционно понятными категориями описания модели и дальнейшего исследования динамических процессов освобождает предметных пользователей от недоступных процедур подготовки профессиональных вычислительных входных данных в виде программной графической модели, этапов трансляции и обработки результатов вычислительных экспериментов. Это позволяет предметным пользователям сосредоточиться на исследовании сути моделей и процессов в предметно-ориентированных задачах, что, несомненно, является главным достижением программного сервиса.

В мультипрограммной среде ИСМА вместе с *LISMA_STR* разработаны графический язык описания и моделирования электроэнергетических систем *LISMA_EPS* [104, 105], символьный язык описания и моделирования пространственно-временных процессов *LISMA_PDE* [106–108], а также символьный язык *LISMA+* [48, 51] для представления и моделирования процессов химической кинетики.

Архитектура ИСМА соответствует современным международным стандартам CSSL (Continuous System Simulation Language) и ориентирована в общем случае на вычисления событийно-непрерывных или гибридных систем. Программные модели с входного языка на первом этапе интерпретируются в базовый символьный язык *LISMA* [51], который на втором этапе трансляции *конвертируется* в *Java*-данные, являющиеся прототипом соответствующей системы обыкновенных дифференциальных уравнений с фактическими параметрами и начальными условиями. Эти данные передаются решателю с библиотекой традиционных и оригинальных численных методов. Решение и обработка вычислительного эксперимента происходит автоматически с предоставлением соответствующего сервиса решения без вмешательства пользователя.

1.3. МЕСТО ЯЗЫКОВЫХ ПРОЦЕССОРОВ

Из приведенного обзора актуальных направлений следует, что в целом системное программное обеспечение (СПО) включает в себя не только сами языки и языковые процессоры, но и их окружение. Окружением принято называть следующие системные компоненты:

- редакторы;
- компоновщики;



- библиотеки;
- загрузчики и отладчики.

Редакторы, или текстовые и графические процессоры, – это тоже своего рода трансляторы, потому что ввод символьной или графической информации всегда сопровождается определенным способом преобразования формата данных. Так, текст в редакторе MS Word может быть отформатирован в виде текстового файла с расширением .txt и документального – с расширением .doc, .rtf и др. Встроенный редактор текста в компиляторе с языка C++ форматирует файлы с расширением .cpp, с языка Pascal – .pas, Fortran – .for и т. д.

Компоновщик (linker) объединяет в единый файл несколько отдельных модулей, представленных после обработки языковым процессором в виде объектных файлов. Эти модули могут быть функциями (C++), процедурами (PL/I, Modula), подпрограммами (Fortran, Ada). Кроме того, все встроенные или библиотечные функции (процедуры) также komponуются в единый объектный файл. Такая процедура компоновки называется *редакцией связей*. Кроме того, компоновщик преобразует единый объектный модуль в исполняемый код, соответствующий определенной системе команд конкретной машины.

Загрузчик (loader) отвечает за размещение исполняемого кода в определенной области оперативной памяти и инициирование исполнения прикладной программы (ПП). Исполнения могут быть инициированы с помощью специальных средств – отладчиков.

В современном СПО все приведенное программное оборудование может работать как под управлением, так и в автономном режиме автоматического последовательного подключения соответствующих компонентов СПО.

Развитие средства обработки прикладных программ обеспечено поддержкой развитого графического интерфейса пользователя, взаимодействующего с функциями API (Application Program Interface) операционных систем. Пользователь в этой среде уже не управляет всем комплексом СПО, как это было на ранних стадиях эксплуатации СПО в так называемом пакетном режиме, когда процедуры редактирования, трансляции, компоновки и загрузки требовали дополнительной командной программы, заданной обработчиком прикладных задач. Теперь от пользователя требуется только знание языка программирования, остальное решается развитыми интерак-



тивными средствами через графический интерфейс. Это позволяет резко снизить трудозатраты, необходимые для разработки прикладного программного обеспечения. Показатель снижения трудозатрат в настоящее время считается более существенным, чем показатели, определяющие эффективность результирующей программы, разработанной средствами СПО [12].

На рис. 1.3 представлены совмещенная структура СПО и логические связи компонентов. Охарактеризуем структуру логических связей.

На первом уровне ПП проходят традиционные входящие в СПО этапы – редактирование и получение ПП на языке высокого уровня $L(G)$, трансляция программы в соответствии с порождающей грамматикой G с диагностикой, синтаксисом и получением в результате успешной трансляции объектных модулей. Далее выполняется компоновка объектных модулей, включая библиотечные функции, и получение последовательного кода (исполняемого файла ПП) единой программы решения задачи.

Второй уровень позволяет уменьшить трудозатраты предметного пользователя благодаря использованию ресурсов ПП.

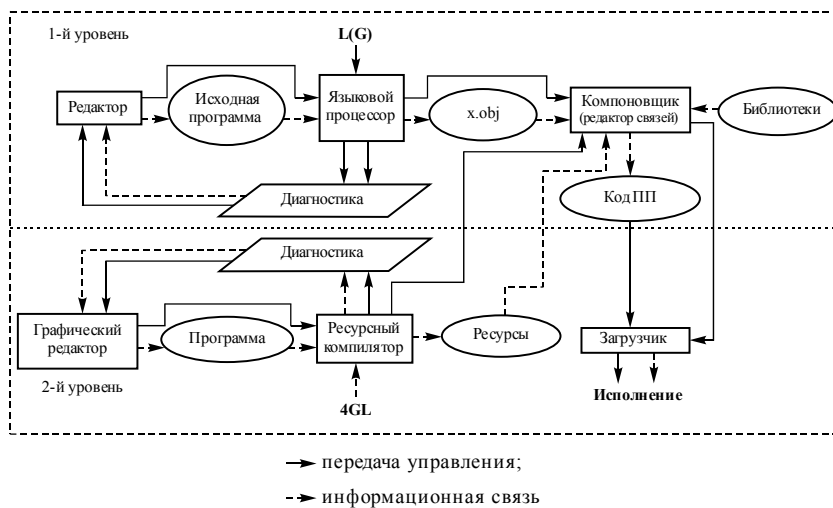


Рис. 1.3. Логические связи СПО

Определение 1.2. Ресурсами прикладной программы будем называть множество данных, обеспечивающих внешний вид интерфейса



пользователя этой программы и не связанных непосредственно с логикой выполнения программы [12, 67].

Характерными примерами таких ресурсов являются следующие:

- текстовое задание исходных данных;
- текстовые сообщения диагностики;
- конфигурационные файлы;
- ленты прокрутки и др.

Редакторы ресурсов осуществляют подготовку структуры данных, соответствующую языкам высокого уровня четвертого поколения – 4GL (four generation languages) [12]. Языки четвертого поколения строятся не на основе оперирования синтаксическими структурами языка и описаниями элементов интерфейса, а на основе представления их графического образа. На этом уровне проектировать и разрабатывать прикладное программное обеспечение может пользователь, не являющийся квалифицированным программистом. Такие языки являются оптимальными с точки зрения затрат предметного пользователя.

Примерами языков 4GL являются, например, языки в системах моделирования *SIMULINK* [72], *MVS* [73], *ISMA* [48,74], а также языки в визуальных средах *VC++*, *C++Builder*, *C#Builder*, *Delphi*, *Eclipse*, *JDeveloper* [66–69].

В целом языки 4GL решают уже более широкий класс задач, чем традиционное СПО. Они составляют части средств автоматизированного проектирования и разработки ПО, содержащие все этапы CASE-технологий проектирования [71]. Текст на языке 4GL поступает на вход компилятора ресурсов, который переводит исходный графический код в обычный код объектных модулей. При этом пользователь не пишет традиционно всю программу. Определенный код уже заготовлен заранее как отображение множества типовых графических элементов на множество текстовых операторов языков высокого уровня. Именно этот образ прикладной программы и переводится, как и в типовом традиционном подходе, в объектные модули. Здесь заканчивается разница между традиционным подходом (первый уровень, см. рис. 1.3) и новым (второй уровень). Компилятор ресурсов реализуется в визуальных средах, которые также содержат языки 4GL. Объектный код ресурсного компилятора ничем не отличается от традиционного, поэтому дальнейшие этапы компилирования и загрузки преемственны от первого уровня.



Таким образом, за исключением загрузчиков и библиотек, все системное программное обеспечение можно рассматривать с позиций трансляции или преобразования входного формата.

1.4. ПОТРЕБНОСТЬ В РАЗРАБОТКЕ ЯЗЫКОВЫХ ПРОЦЕССОРОВ

Разработка новых языков программирования и языковых процессоров во многом мотивирована появлением новых формализмов описания все более сложных технологических процессов. Так, новые фундаментальные достижения в теории автоматов позволили просто и адекватно представлять математические модели с логическими условиями. Для описания этих процессов лингвистическими средствами потребовалось ввести новые парадигмы в языках программирования, в том числе так называемое *автоматное программирование* [114]. Разработан формальный уровень автоматного программирования средствами *Event-B* [111] – формального метода программирования для моделирования и анализа на системном уровне сложных систем с логическими условиями. Ключевыми особенностями *Event-B* являются использование теории множеств и строгого математического доказательства проверки автоматной модели, которая лежит в основе программной реализации в технологии *Event-B*.

Следует также отметить появление новых методологий численного моделирования в реализации все более сложных математических моделей, так называемых *киберфизических* процессов. В математической постановке дискретно-непрерывных процессов появилось *понятие событийной функции* [109], которая строго математически предопределяет дискретное поведение системы. Для описания новых моделей потребовались новые формализмы, учитывающие не только непрерывное, но и дискретное поведение. Такой адекватный формализм был предложен Д. Харелом [110]. В *диаграммах Харела* (statecharts) непрерывное поведение учитывается в символьном описании узлов, а дуги суть условия событийных функций. Новый формализм стал базовой предпосылкой появления серии новых предметно-ориентированных языков программирования (моделирования) – *Modelica* (среда *Open Modelica* и *Dymola*) [113], *Statechart (AnyLogic)* [103], *Stateflow (MATLAB)* [51],



<https://www.mathworks.com/products/stateflow.html>], *LISMA* (*ISMA*) [48], *MTK* (*JuliaSim*) [112] и др.

Наконец, постоянно растущая потребность в новых языках и средствах их реализации связана с бурным развитием архитектур ЭВМ по различным направлениям. Традиционно увеличение вычислительной мощности мотивировалось численным моделированием сложных систем. Однако на сегодняшний день наиболее существенными становятся коммерческие приложения, которые включают в себя базы данных (особенно если они используются при принятии решений), видеоконференции, совместные рабочие среды, автоматизацию диагностирования в медицине, развитую графику и виртуальную реальность (особенно для промышленности). Хотя коммерческие приложения могут в достаточной мере определить архитектуру большинства будущих параллельных компьютеров, тем не менее традиционные научные приложения будут оставаться важными потребителями параллельной вычислительной технологии (например, США проводит ядерные испытания, используя лишь суперкомпьютеры и сети ЭВМ).

Потребность во все более мощных компьютерах и соответственно их программном обеспечении определяется запросами как коммерческих приложений, так и интенсивных по вычислениям научных и технических приложений. Требования этих областей человеческой деятельности постепенно сближаются: научные и технические приложения вовлекают все большие объемы данных, а коммерческие приложения начинают применять все более сложные вычисления.

До настоящего времени эффективность самых быстрых компьютеров возрастала почти по экспоненте. Однако архитектура вычислительных систем, определяющих этот рост, изменилась радикально – от последовательной до параллельной. Эра однопроцессорных компьютеров продолжалась до появления семейства Cray X-MP/Y-MP – слабо параллельных векторных компьютеров с 4–16 процессорами. Их в свою очередь сменили компьютеры с массовым параллелизмом, т. е. компьютеры с сотнями и тысячами процессоров. Эффективность компьютера зависит непосредственно от времени, требуемого для выполнения базовой операции, и числа базовых операций, которые могут быть выполнены одновременно. Время выполнения базовой операции ограничено временем выпол-



нения внутренней элементарной операции процессора (тактом процессора). Уменьшение такта ограничено физическими пределами, такими как скорость света. Чтобы обойти эти ограничения, производители процессоров реализовывают параллельную работу внутри чипа при выполнении элементарных и базовых операций. Однако теоретически было показано, что стратегия сверхвысокого уровня интеграции (VLSI – Very Large Scale Integration) является дорогостоящей, а время выполнения вычислений сильно зависит от размера микросхемы. Наряду с VLSI для повышения производительности компьютера используются и другие способы: конвейерная обработка (при которой различные стадии отдельных команд выполняются одновременно), многофункциональные модули (когда отдельные множители, сумматоры и тому подобное управляются одиночным потоком команды) и др.

Другая важная тенденция развития вычислений – это огромное увеличение производительности сетей ЭВМ.

Наряду с ростом быстродействия сетей повышается и надежность передачи данных. Это позволяет разрабатывать приложения, которые используют физически распределенные ресурсы, как будто они являются частями одного многопроцессорного компьютера. Например, коллективное использование удаленных баз данных или обработка графических данных на одном или нескольких графических процессорах, при этом вывод и управление в реальном масштабе времени осуществляется на рабочих станциях.

Рассмотренные тенденции развития архитектуры с использованием многопроцессорных компьютеров и компьютерных сетей привели к тому, что параллельность стала уделом не только суперкомпьютеров, но и рабочих станций и сетей ЭВМ. Программы используют как множество процессоров компьютера, так и процессоры, доступные по сети. Чтобы не пропадал накопленный опыт существующих алгоритмов и программ с использованием одного процессора, требуются новые алгоритмы – адаптеры и программы, способные *транслировать* разработанное ПО к современным мультипроцессорным архитектурам.

Количество процессоров в компьютерах увеличивается. В этой связи для защиты капиталовложений в программное обеспечение *масштабируемость* (scalability) программного обеспечения важна не менее, чем переносимость. Программа, способная использовать



только фиксированное число процессоров, является такой же плохой, как и программа, способная работать только на одном типе компьютеров.

Отличительным признаком многих параллельных архитектур служит то, что доступ к локальной памяти процессора дешевле, чем доступ к удаленной памяти. Следовательно, желательно, чтобы доступ к локальным данным был более частым, чем доступ к удаленным данным. Такое свойство программного обеспечения называют *локальностью* (locality). Наряду с параллелизмом и масштабируемостью это свойство является основным требованием к параллельному программному обеспечению.

Естественно, для новых вычислительных мощностей требуются новые языковые процессоры с новыми языками программирования. Здесь необходимо также отметить, что новые архитектуры требуют разработки совершенно новых подходов к созданию языков и языковых процессоров, поэтому наряду с собственно разработкой языковых процессоров ведется и большая научная работа по созданию новых языковых средств и методов трансляции в оригинальных современных архитектурах.

1.5. СОВРЕМЕННЫЕ АРХИТЕКТУРНЫЕ РЕШЕНИЯ

В настоящее время наиболее популярной процессорной архитектурой для персональных компьютеров является x86, которая берет свое начало с 32-битного CISC-процессора Intel 80386, выпущенного в далеком 1985 г. От него же унаследованы наборы инструкций, которые применяются даже на современных процессорах. Лицензией на их производство в настоящее время обладают только компании Intel и AMD. С этой архитектурой выпускаются процессоры как для рабочих станций, так и для высокопроизводительных серверов и суперкомпьютеров. В частности, x86-процессоры Intel Xeon и AMD Ерус задействованы в шести из десяти самых мощных суперкомпьютерах мира согласно рейтингу TOP500 [87].

Однако рынок вычислительных устройств не ограничивается настольными решениями, и очень сложно представить жизнь современных людей без смартфонов. В мобильных устройствах доминирующую позицию сейчас занимает архитектура ARM (32- и 64-битная) – это RISC-архитектура, представленная в 1985 г. и изначально



используемая в устройствах, которые разрабатывались с расчетом на компактные размеры и малое энергопотребление. Под эту категорию подходят мобильные телефоны, смартфоны, PDA и устройства, использующие микроконтроллеры. Однако сейчас сфера применения ARM-процессоров расширяется. За счет гибкой политики лицензирования они находят себе применение на рабочих станциях, в серверах и суперкомпьютерах. Согласно тому же рейтингу TOP500, самый производительный в мире суперкомпьютер Fugaku использует процессоры Fujitsu, выполненные на ARM-архитектуре.

Таким образом, господствующие позиции на рынке вычислительных систем занимают всего две архитектуры, которые имеют реализации практически во всех существующих типах вычислительных систем. Кажется, что унификация вычислительных систем должна была отобрать у разработчиков текстовых процессоров работу и что с 1980-х гг. мало что изменилось. На самом деле это верно лишь отчасти. Базовые наборы команд действительно остаются неизменными, и теоретически программа, собранная компилятором, созданным 30 лет назад, будет работать и на современных машинах, но вопрос в том, как она будет на них работать.

Процессоры все эти годы продолжали меняться, «обрастать» дополнительными возможностями и наборами инструкций. В частности, в них начали устанавливать несколько вычислительных ядер, что позволяет нативно выполнять многопоточные приложения. Еще один важный момент, на который обращают внимание гораздо реже, – современные процессоры стали поддерживать операции над векторными регистрами, позволяющие выполнять SIMD-инструкции на группах данных, что дает возможность в разы ускорить обработку массивов чисел даже на одном ядре. У процессоров также менялся конвейер обработки команд и размер кеша. Старые компиляторы «не знают» об этих возможностях и особенностях и, как следствие, просто не могут генерировать эффективный код для таких машин. Поэтому сейчас перед *разработчиками компиляторов* стоит задача не только создавать компиляторы для нескольких процессорных архитектур, но и делать генерируемый ими машинный код эффективным и обратно совместимым в пределах одной архитектуры.

Вопросы переносимости программ между архитектурами процессоров и операционными системами в настоящее время, как пра-



вило, решают либо с использованием языков промежуточного представления, либо применяя реализованные под каждую из платформ интерпретаторы.

Большой вклад в продвижение первого подхода сделали разработчики языка программирования *Java* и платформы *Java Virtual Machine (JVM)* [88], первые версии которых были выпущены в 1995 г. Особенностью этой платформы является применение виртуальной машины, которая запускается как отдельный процесс на целевой архитектуре и производит трансляцию кода промежуточного представления в машинные команды. В данном случае языком промежуточного представления является байт-код виртуальной машины, собираемый компилятором *Java* из исходного кода в .class-файлы. Эти файлы обычно собираются и распространяются в виде JAR-архивов. Таким образом, .class-файлы могут быть запущены на любой архитектуре процессора и операционной системе, где есть реализация *JVM*.

В начале 2000-х гг. корпорация Microsoft разработала похожую платформу для операционной системы Windows, которая получила название *.NET Framework*. Ее особенности: возможность более гибко работать с ресурсами ЭВМ, более глубокая интеграция с Windows, стандартный набор библиотек для создания пользовательских графических интерфейсов и сразу несколько языков программирования, в частности *C#* и *Visual Basic*. В остальном *.NET* практически повторяет концепции, заложенные в *Java*, – здесь также используется байт-код для промежуточного представления и он транслируется в машинные команды уже на пользовательской машине. Однако (в отличие от *Java*) *.NET Framework* доступен только в операционной системе Windows. Первые попытки реализовать его виртуальную машину на других платформах были предприняты в рамках проекта Mono компанией Novell при участии программиста Мигеля де Икасы [90]. Далее развитием Mono занималась компания Xamarin, которая сделала на ее основе кросс-платформенную версию для создания мобильных приложений на *C#*. Позднее Microsoft купила Xamarin и в 2016 г. представила кросс-платформенную версию *.NET* под названием *.NET Core*, которая была разработана с нуля и не является обратно совместимой с предыдущими версиями. В дальнейшем *.NET* и язык программирования *C#* стали развиваться относительно этой реализации.



К категории языков, использующих интерпретатор для выполнения программ, относится *Python*. Есть несколько его реализаций, в том числе использующие компиляцию в машинный код во время выполнения, но наиболее распространенным интерпретатором и де-факто эталонным является *CPython* [91], который выполняет программу непосредственно из исходного кода. Кросс-платформенность достигается за счет распространения программ в виде исходного кода и реализации *CPython* для всех необходимых платформ.

Гибридный подход используется в современных веб-браузерах для выполнения программ на языке *JavaScript*. В частности, речь идет о популярном *JS* [92], который не использует промежуточное представление и интерпретирует программу из исходного кода напрямую. Однако при необходимости он может скомпилировать некоторые ее участки в машинный код целевого процессора, что позволяет повысить скорость выполнения. Так же, как и в случае *Python*, программы распространяются в виде файлов с исходным кодом в текстовом виде. Программы на *JavaScript* выполняются на платформах, где есть реализация его интерпретатора.

Идеи использования промежуточного представления также повлияли на разработчиков компиляторов для языков низкого уровня. Они нашли применение в проекте *LLVM* с компилятором *Clang*, разработка которого началась в университете Иллинойса Крисом Латтнером и Викрамом Адве [93, 94]. Концептуально платформа *LLVM* похожа на *Java* и *.NET*, поскольку здесь также используется компиляция программы в промежуточное представление, которое затем преобразуется в машинные команды. Но особенность заключается в том, что *LLVM* позволяет использовать для этого языки низкого уровня, такие как *C* и *C++*. Промежуточное представление (*LLVM IR*) в своей текстовой форме похоже на язык ассемблера, инструкции которого напоминают наиболее распространенные инструкции реальных процессоров, но при этом лишены присущим им сложностей. Так, *IR* значительно упрощает работу с регистрами и предоставляет вспомогательные методы для наиболее распространенных операций, например арифметических. Такое промежуточное представление, в отличие от используемого в *Java* и *.NET*, остается максимально близким к архитектуре реальных ЭВМ и дает прямой доступ к большому количеству низкоуровневых операций. Однако вместе с тем оно остается более простым для разработчиков



компиляторов в сравнении с языком ассемблера. Самое важное, что промежуточное представление в виде байт-кода не имеет привязки к реальной архитектуре и может быть выполнено на любой платформе, для которой будет соответствующий компилятор или интерпретатор. В эталонной реализации *LLVM* используется компилятор, и *Clang* выступает первым этапом в сборке программы, генерируя промежуточное представление из программ на языке *C* или *C++*. После этого, как правило, запускается компилятор для полученного байт-кода под целевую платформу, который применяет для нее необходимые оптимизации. В результате программа в два независимых этапа преобразуется в машинный код. Такой подход дает серьезное преимущество создателям языков программирования, поскольку новые компиляторы, генерирующие байт-код *LLVM*, могут быть с минимальными затратами перенесены на другие платформы, для которых уже есть реализация байт-кода компилятора или интерпретатора. Аналогично при добавлении новой реализации платформы *LLVM* все существующие для нее компиляторы получают и ее поддержку. Однако важно понимать, что просто совместимости на уровне промежуточного представления недостаточно для того, чтобы компилятор работал на новой платформе. В отличие от *Java LLVM* не предоставляет доступа к системным API на уровне платформы, и библиотеки языков все-таки придется реализовывать независимо. К настоящему времени на базе *LLVM* созданы компиляторы *C/C++*, *Rust*, *Swift*, *Objective-C*, *Julia*, *Kotlin*, *Haskell* и многих других языков.

Следующий вопрос, который пытаются решить разработчики современных языков программирования и сред разработки для них, – повышение производительности труда программистов. Большую работу в этом направлении ведут компании *JetBrains* и *Microsoft*. Первая активно занимается продвижением своего нового языка программирования *Kotlin*, который изначально разрабатывался как альтернатива *Java* для платформы *JVM*. В нем существенно пересмотрены стандартные языковые конструкции и стандартная библиотека, а также сделан большой акцент на использовании функций как одного из основных типов данных в программировании. Как правило, программы на *Kotlin* получаются более компактными, чем на *Java*, но при этом как минимум не уступают им в читабельности. В особенности это касается работы с коллекциями и определениями типов данных.



Microsoft сейчас продолжает активно развивать свою среду разработки Visual Studio и язык программирования *C#*. Главная заслуга компании в последнем десятилетии – компилятор Roslyn с открытым исходным кодом, который написан на *C#*, имеет гибкую расширяемую архитектуру и возможность интеграции со средой разработки. Roslyn Compiler Platform позволяет при необходимости добавить пользовательские анализаторы исходного кода, которые будут использоваться при динамическом анализе в среде разработки или же на стадии компиляции. Базовый набор анализаторов встроен в компилятор начиная с *.NET*, он дает программисту контекстные подсказки по лучшим практикам и сценариям применения возможностей языка во время написания программы, что позволяет экономить время и получать на выходе более качественный код. Ведутся также и разработки, связанные с использованием анализаторов Roslyn для генерации исходных текстов программы на стадии компиляции, что позволит во многих случаях отказаться от использования дорогостоящих операций отражения, т. е. обращения к метаданным программы и генерации IL-кода в ходе выполнения программы. Кроме того, генерация на стадии компиляции позволит произвести проверку его корректности до непосредственного выполнения.

Первые многоядерные процессоры для массового рынка появились более десяти лет назад, и сейчас сложно представить себе современный персональный компьютер или смартфон, который поддерживает лишь один поток выполнения программы. С приходом многопоточности в массовое использование кое-что существенно изменилось. Если раньше на многопроцессорных машинах решались какие-то научные задачи, то теперь требуется разрабатывать уже прикладные программы, которые могли бы эффективно использовать такие ЭВМ. Сейчас мы говорим уже не столько о параллельности, сколько о многозадачности. Под многозадачностью в этом случае будем понимать одновременное выполнение какой-либо программой нескольких процессов, в том числе в фоновом режиме. Многозадачность была и во времена одноядерных процессоров, она достигалась за счет постоянных переключений контекста, когда процессор периодически передавался в пользование нескольким параллельно выполняемым процессам, создавая у пользователя иллюзию их одновременного выполнения. Однако использовалось



это обычно на уровне операционной системы и системных программ, а не конечных приложений, по причине низкой эффективности. Сейчас же такая возможность появилась, и вместе с ней появилась потребность в доработке механизмов управления потоками в языках программирования. Обычным сценарием подобной многозадачности является использование фоновых потоков выполнения. Например, в средах разработки было бы очень удобно выполнять статический анализ написанного кода в фоновом режиме, но при этом позволить пользователю продолжать взаимодействовать с графическим интерфейсом. Делать это с использованием классических механизмов управления потоками крайне неудобно, поскольку запускать задачи на выполнение теперь приходится гораздо чаще, а порождение и завершение потоков требует нетривиальных действий со стороны разработчиков. Получила свое развитие концепция асинхронного программирования и легковесных потоков, в частности через парадигму *promise-future*. При ее использовании программист не управляет потоками напрямую, а при вызове функции лишь обещает («*promise*») вернуть из нее результат, возвращая объект соответствующего типа. В будущем («*future*») он уведомляет вызывающий код (например, функцией обратного вызова) о завершении задачи и позволяет получить результат ее выполнения. Управление потоками осуществляется планировщиком задач через пул выделяемых по мере необходимости потоков. В этом плане приложение становится похоже на небольшую операционную систему реального времени, которая использует ресурсы настоящей операционной системы по мере необходимости. Такой подход сейчас реализован во многих языках, в том числе в *C++* и *Java* на уровне стандартных библиотек. Более детально к задаче внедрения этой возможности подошли разработчики *Kotlin* и *C#*, закрепившие *promise-future* (здесь он называется *async-await*) на уровне синтаксиса и сделавшие использование паттерна более дружелюбным для конечного пользователя.

Вопрос реализации параллельных алгоритмов в программах получил свое развитие с появлением особого вида процессоров, называемых графическими процессорами (GPU). Их особенность заключается в использовании очень большого количества вычислительных ядер (несколько тысяч) и низкой тактовой частоте. Изначально они использовались в задачах, связанных с компьютерной



графикой, однако в последние годы приобретают все большую популярность и в других сферах, преимущественно связанных с обработкой больших массивов данных. Программы для графических процессоров называются *шейдерами* и, как правило, пишутся на диалектах языка C, например *GLSL*, *HLSL*. Для технологий CUDA или OpenCL были также разработаны специальные расширения для языков C/C++.

УПРАЖНЕНИЯ

1. Что такое языковой процессор? Классификация языковых процессоров.
2. Какой языковой процессор самый быстрый?
3. Чем сканер отличается от синтаксического анализатора?
4. В каком языковом процессоре самый эффективный код?
5. Чем отличается компилятор от ассемблера?
6. Чем сканер отличается от синтаксического анализатора? В чем отличие интерпретатора от компилятора?
7. Какие тенденции присущи современным системам программирования?
8. Каковы отличительные особенности языковых процессоров по сравнению с физическими процессорами?
9. Какие особенности трансляции в препроцессорах?
10. Какие языковые процессоры требуют минимальных затрат при проектировании?
11. Что понимается под окружением языковых процессоров?
12. Какие направления в разработке языковых процессоров можно выделить как наиболее актуальные?
13. В чем состоит динамика проектирования языковых процессоров?
14. Как современные многопроцессорные системы влияют на актуальность задач проектирования языков и языковых процессоров?
15. В чем отличие систем программирования одно- и многопроцессорной архитектуры?
16. Какие архитектурные решения доминируют в современном ПО?

2. ПОРОЖДАЮЩИЕ ГРАММАТИКИ И ЯЗЫКИ

В этом разделе вводятся основные теоретические понятия, которые необходимы для синтаксически-ориентированных методов проектирования языковых или лингвистических процессоров. Материал является разделом математической лингвистики и может быть опущен без ущерба для дальнейшего изучения теми, кто знаком с этой дисциплиной.

2.1. ОБОЗНАЧЕНИЯ И ОПРЕДЕЛЕНИЯ

Нетерминальные символы (нетерминалы) несут смысловое содержание. Обозначаются нетерминалы печатными прописными буквами латинского алфавита $[A, \dots, Z]$ и (или) заключаются в угловые скобки $< >$. Например, обозначения $<\text{нетерминал}>$, A , B , $<\text{Coshy}>$, $<AB>$ являются нетерминальными символами. Обозначение AB следует читать как сочетание двух рядом стоящих нетерминалов A и B . От изменения обозначений нетерминальных символов грамматика не изменяется. Нетерминалы являются связующими элементами грамматики, и поэтому их обозначение выбирается произвольно автором грамматики так, чтобы смысловая нотация грамматики была предметно понятна и читабельна. Например, если разрабатывается грамматика числовой константы, то понятно, что одним из нетерминальных символов в этой грамматике будет нетерминал $<\text{числовая константа}>$, который можно обозначить и сокращенно $<\text{ЧК}>$, но сокращения менее читабельны и осложняют понимание грамматики.

Терминальные символы (терминалы) могут состоять из одной и более литер и являются субъектами языка. Тексты программ



на языках программирования можно рассматривать как строго упорядоченную последовательность операторов из терминальных символов. Обозначаются терминалы строчными буквами латинского алфавита $[a, b, \dots, z]$ или алфавитно-цифровыми и знаковыми клавиатурными аббревиатурами. Исключением для такого соглашения являются ключевые слова и идентификаторы, например, языков *Fortran*, *BASIC* и др. [39, 40]. В этом случае для выделения многолитерного терминального символа будем использовать курсив. Например, ключевые слова языка Fortran «DO», «IF», «THEN» и другие будем обозначать терминалами *DO*, *IF*, *THEN* с использованием другого шрифта.

СОГЛАШЕНИЯ

b – $A, B, \dots, Z, a, b, \dots, z$ – это буква, которая является терминальным символом из словаря латинского алфавита.

d – $0, 1, \dots, 9$ – это цифра, которая является терминальным символом цифрового регистра.

id – идентификатор.

Все обозначения множеств будем выделять полужирным шрифтом.

Словарь (алфавит) – это конечное непустое множество символов. Словарь обозначается заглавными прописными буквами латинского алфавита, чаще **V**, и Σ (возможно с индексами).

Термин *алфавит* как класс или группа символов введен для обозначения, например, букв русского $[A, B, \dots, Я]$ или латинского $[A, B, \dots, Z]$ алфавита, как прописных, так и строчных. В теории формальных языков эта категория приняла лишь более строгую и конкретную форму.

Пример 2.1

$A = \{a, b\}$ – словарь **A** состоит из двух терминальных символов a, b .

$B = \{0, 1\}$ – бинарный словарь.

Стандарты ASCII и EBCDIC являются примерами компьютерных алфавитов.

Отметим, что формальный словарь, о котором идет речь, не обязательно должен состоять только из терминальных символов.



В общем случае, как будет показано ниже, общий словарь V может включать в себя терминальный набор символов вместе с нетерминальным.

Цепочки (строки) – это конечная последовательность элементов словаря. Строки обозначаются греческими прописными буквами α , β , γ , τ ...

Пример 2.2

$\alpha = aa$, $\beta = abc$.

Длина цепочки α – это число символов в цепочке α , обозначается $|\alpha|$. Для приведенного примера длины цепочек соответственно равны $|\alpha| = 2$, $|\beta| = 3$. Длина *пустой цепочки* Λ равна 0, т. е. $|\Lambda| = 0$, эквивалентные обозначения *пустой цепочки* – e , ϵ .

Конкатенация (катенация) цепочек – присоединение цепочек слева направо. Это основная операция над цепочками.

Пример 2.3

Пусть $\alpha = ab$, $\beta = cab$, $\gamma = \Lambda$, тогда $\alpha\beta = abcab$, $\alpha\gamma = ab$.

Свойство *коммутативности* для операции конкатенации цепочек не выполняется в общем случае. Действительно, если $\alpha = ab$, $\beta = cab$, то $\alpha\beta = abcab$, но это не одно и то же, что $\beta\alpha = cabab$.

Выделим еще несколько операций над строками или цепочками символов.

Обращение – это запись символов цепочки α (обозначение α^R) в обратном порядке. Так, если $\alpha = ab$, то $\alpha^R = ba$.

Итерация – это операция n -кратной конкатенации символов цепочки α (обозначение α^n).

Например, если $\alpha = ab$, то $\alpha^3 = \alpha\alpha\alpha = ababab$.

Для пустой цепочки Λ справедливы следующие очевидные равенства:

- $\forall \alpha : \Lambda\alpha = \alpha\Lambda = \alpha$;
- $\Lambda^R = \Lambda$;
- $\forall n \geq 0 : \Lambda^n = \Lambda$;
- $\forall \alpha : \alpha^0 = \Lambda$.

Произведение словарей – это множество конкатенаций $\alpha\beta$ таких, что цепочка $\alpha \in \mathbf{A}$, $\beta \in \mathbf{B}$, т. е.

$$\mathbf{AB} = \{\alpha\beta \mid \alpha \in \mathbf{A}, \beta \in \mathbf{B}\}.$$



Если, например, $A = \{a, b\}$, $B = \{c, d\}$, тогда $AB = \{ac, ad, bc, bd\}$.

Степень словаря:

$$A^0 = \{\Lambda\}, A^1 = A, A^2 = AA, \dots, A^n = A^{n-1}A \neq AA^{n-1}.$$

Усеченная итерация словаря – это бесконечное объединение всех степеней словаря, кроме нулевой:

$$A^+ = A^1 \cup A^2 \cup \dots \cup A^n.$$

Итерация словаря – это бесконечное объединение всех степеней словаря, включая нулевую:

$$A^* = A^+ \cup A^0 = A^+ \cup \{\Lambda\}.$$

Пример 2.4

$$A = \{a, b\},$$

$$A^2 = \{aa, ab, ba, bb\},$$

$$A^3 = \{aaa, aab, aba, abb, \dots\},$$

.....

$$A^* = \{\Lambda, a, b, ab, aa, bb, aaa, \dots, aba, \dots, ab\dots\}.$$

Пусть $A = \{0, 1, 2, \dots, 9\}$, тогда $A^* = \{\Lambda, \langle \text{множество целых чисел} \rangle\}$.

В общем случае содержательный смысл итерации словаря включает множество всевозможных комбинаций всевозможной длины над элементами словаря.

2.2. ПОРОЖДАЮЩИЕ ГРАММАТИКИ

Рассмотрим предложение «Большой слон съел орех». Обычный разбор такого предложения состоит в декомпозиции предложения на составляющие части речи. Так, «слон» – существительное (подлежащее), «орех» – существительное (дополнение), «съел» – глагол (сказуемое) и, наконец, «большой» – определение (прилагательное).

Переставим одно слово в предложении: «Слон съел большой орех», и предложение поменяет свой смысл, или, как принято говорить, предложение будет иметь другую семантику.

С точки зрения синтаксиса оба предложения допустимы. В дальнейшем нас в первую очередь и будет интересовать именно синтаксис.

Набор правил, определяющих синтаксис, будем называть *грамматикой*. Грамматика имеет свой метаязык, т. е. язык для описания

Одним из наиболее распространенных способов описания синтаксиса языка является *форма Бэкуса – Наура* (J. W. Backus, P. Naur) [1, 19]. Этот способ был разработан для описания языка *ALGOL 60*, однако в дальнейшем он был использован для многих других языков. При записи грамматики в форме Бэкуса – Наура используются два типа объектов:

- основные символы (или терминальные символы, в частности ключевые слова);
- металингвистические переменные (или нетерминальные символы), значениями которых являются цепочки основных символов описываемого языка. Металингвистические переменные изображаются словами (русскими или английскими), заключенными в угловые скобки (< >) и разделенными металингвистическими связками («:=» и «|»).

Вернемся к нашему предложению и опишем его метаязыком в БНФ.

Пусть синтаксис, или грамматика, задается следующими правилами:

$\langle \text{предложение} \rangle ::= \langle \text{прилагательное} \rangle \langle \text{подлежащее} \rangle$
 $\langle \text{сказуемое} \rangle \langle \text{существительное} \rangle.$

Эта грамматика допускает лишь первый вариант рассмотренного выше предложения и не допускает второго варианта с другой семантикой.

Таким образом, на этом примере видно, что синтаксис может влиять на семантику, т. е. определять смысловое содержание предложения.

Существуют разные толкования грамматики.

В [13] грамматика рассматривается как математическая система, определяющая язык. В [16] грамматикой называется любой конечный механизм определения языка. Такое определение может противоречить словарному синониму слова «грамматика», так как



в классическом определении [17] грамматика (от греч. *grammatikē*) – это система языковых форм. Хотя, может быть, для алгоритмических языков можно уйти от классических категорий, как это и попытался сделать Ю. Г. Карпов [16].

Н. Вирт при определении синтаксиса языков *Pascal* и *Modula-2* [37, 38] использовал *расширенную форму Бэкуса – Наура* (EBNF) согласно следующим правилам:

- нетерминалы записываются как отдельные слова;
- терминалы записываются в кавычках, например, "BEGIN";
- вертикальная черта «|», как и прежде, используется для определения альтернатив;
- круглые скобки используются для группировки символов;
- квадратные скобки используются для определения возможного вхождения символа или группы символов;
- фигурные скобки используются для определения возможного повторения символа или группы символов;
- символ равенства используется вместо символа «::=»;
- символ «точка» используется для обозначения конца правила;
- комментарии заключаются между символами (* ... *);
- ε эквивалентно Λ.

Пример 2.5

```
Integer = Sign UnsignedInteger.  
UnsignedInteger = digit {digit}.  
Sign = ["+"|"–"].  
digit = "0"|"1"|"2"|"3"|"4"|"5"|"6"|"7"|"8"|"9".
```

В 1981 г. Британский институт стандартов (British Standards Institute, BSI) опубликовал стандарт EBNF. Стандарт BSI получился более наглядным, чем расширенная форма Бэкуса – Наура, предложенная Н. Виртом.

Согласно этому документу элементы правил разделяются запятыми. Правила заканчиваются точкой с запятой. Пробелы не являются значащими.

Пример 2.6

```
Constant Declaration = "CONST",  
    Constant Identifier, "=", Constant Expression, ";",  
    {Constant Identifier, "=", Constant Expression, ";"};  
Constant Identifier = identifier;  
Constant Expression = Expression;
```




Существуют и другие формы представления грамматик (например, грамматика Вайнгартена), которые использовались для описания синтаксиса *ALGOL 68* [6].

Теперь обратимся к строгому формальному определению грамматики.

Определение 2.1. *Грамматикой $G[Z]$ называется конечное непустое множество правил вывода, множества терминальных символов (терминальный словарь), множества нетерминальных символов и начального символа Z , который должен встречаться хотя бы один раз в левой части правила вывода, т. е.*

$$G[Z] = \{V_T, V_N, Z, P\}.$$

Здесь использованы следующие обозначения:

- V_T – конечное множество терминальных символов;
- V_N – непересекающееся с V_T конечное множество нетерминальных символов ($V_T \cap V_N = \emptyset$);
- P – конечный набор порождающих правил вида $(\alpha \rightarrow \beta)$, где $\alpha \in V^+$, $\beta \in V^*$;
- Z – начальный символ, $Z \in V_N$;
- $V = V_T \cup V_N$ – объединение словарей, или общий словарь терминалов и нетерминалов.

В дальнейшем будет использоваться *расширенная форма Бэкуса – Наура* (EBNF) со следующими дополнительными соглашениями:

- нетерминалы и терминалы соответствуют определениям п. 2.1;
- символ « $\rightarrow$ » используется вместо символа « $::=$ »;
- грамматика может быть задана с учетом соглашений, принятых в п. 2.1.

Пример 2.7

Определим грамматику целых чисел $G_1[Z]$ в нотации Хомского с множеством P :

- 1) $\langle \text{целое без знака} \rangle \rightarrow \langle \text{цифра} \rangle$;
- 2) $\langle \text{целое без знака} \rangle \rightarrow \langle \text{цифра} \rangle \langle \text{целое без знака} \rangle$.

Следуя введенному формальному определению грамматики, представим $G_1[Z]$ ее составляющими:

- $Z = \langle \text{целое без знака} \rangle$;
- $V_T = \{0, 1, 2, \dots, 9\}$;
- $V_N = \{\langle \text{целое без знака} \rangle, \langle \text{цифра} \rangle\}$.

**Пример 2.8**

Пусть грамматика Хомского $G_2[Z]$ задана следующим образом:

$$V_T = \{a, bc\};$$

$$V_N = \{A, B, C\};$$

$$P = \{1) Aab \rightarrow cb; 2) B \rightarrow ca; 3) BA \rightarrow c\}.$$

Правила вывода множества P часто называют продукциями. Впервые продукцией воспользовались в своих работах Дж. Бэкус и П. Наур.

В [19] отмечаются некоторые существенные свойства грамматик, которые учитываются в практическом построении грамматик реальных языков программирования. В первую очередь следует отметить свойство эквивалентности. Ниже это понятие будет строго определено.

ВЫВОДИМОСТЬ

Определение 2.2. Говорят, что из цепочки α непосредственно следует цепочка β и обозначается $\alpha \Rightarrow \beta$, если при $\alpha = \mu\tau\nu$ и $\beta = \mu\gamma\nu$ в правилах вывода P грамматики $G[Z]$ существует такая продукция, что из $\tau \rightarrow \gamma \in P$.

Пример 2.9

Пусть $\alpha = BAabcs$, $\beta = Bcbcs$. Докажем, что из α непосредственно следует β . Найдем $\alpha \Rightarrow \beta$.

$$G_2[Z]: \quad \alpha = B \quad Aab \quad c \Rightarrow \beta = B \quad cb \quad c$$

$$\mu \quad \tau \quad \nu \quad \mu \quad \gamma \quad \nu$$

$$Aab \rightarrow cb \in P: G_2[Z]. \text{ Тогда } \alpha \Rightarrow \beta.$$

Определение 2.3. Говорят, что из α выводится β ($\alpha \Rightarrow^* \beta$), если имеет место следующая непосредственная выводимость:

$$\alpha = \omega_0 \Rightarrow \omega_1 \Rightarrow \omega_2 \Rightarrow \dots \Rightarrow \omega_n = \beta,$$

т. е. n -кратная непосредственная выводимость порождает итерационную выводимость или просто выводимость.

В [19] дается более лаконичное определение понятий выводимости и итерационной выводимости.

Определение 2.4. Если $\alpha\beta\gamma \in L(G)$ – цепочка языка, а $\beta \rightarrow \delta \in P$ – правило грамматики G , то $\alpha\beta\gamma \Rightarrow_G \alpha\delta\gamma$ ($\alpha\delta\gamma$ непосредственно выводима из $\alpha\beta\gamma$ в G). Рефлексивное и транзитивное замыкание этого отношения обозначим как $\alpha \Rightarrow_G^* \beta$ (цепочка β выводима из α итерационно в грамматике G).

**Пример 2.10**

Рассмотрим грамматику $G_1[Z = \langle \text{целое без знака} \rangle]$:

1) $\langle \text{целое без знака} \rangle \rightarrow \mathbf{ц}$

2) $\langle \text{целое без знака} \rangle \rightarrow \mathbf{ц} \langle \text{целое без знака} \rangle$

Теперь поставим задачу анализа строк: принадлежит ли цепочка 123 множеству $\langle \text{целое без знака} \rangle$.

Анализ проведем, используя определение выводимости:

$$\begin{aligned} \langle \text{целое без знака} \rangle &\Rightarrow^{(2)} \mathbf{ц} \langle \text{целое без знака} \rangle \Rightarrow^{(2)} \\ &\Rightarrow^{(2)} \mathbf{цц} \langle \text{целое без знака} \rangle \Rightarrow^{(1)} \mathbf{ццц} \Rightarrow 123 \end{aligned}$$

Цифры в круглых скобках над стрелками выводимости соответствуют нумерации правил вывода. Итак, используя правила грамматики G_1 и понятие выводимости, нам удалось убедиться в принадлежности строки 123 множеству $\langle \text{целое без знака} \rangle$. Легко проследить с помощью той же процедуры выводимости принадлежность цепочки 0123 грамматике целых в позиционной системе исчисления. Следует иметь в виду, что константы, начинающиеся с цифры «0», в некоторых языках программирования относятся к десятичной системе исчисления. Это означает, что грамматика, которая определяет синтаксис, в определенных случаях включает в себя также и семантическую нагрузку. Для того чтобы и семантика была правильной в данном примере, преобразуем грамматику G_1 в $G_1'[\langle \text{ЦБЗ} \rangle = \langle \text{целое без знака} \rangle]$ так, что

$G_1'[\langle \text{ЦБЗ} \rangle]$:

1) $\langle \text{ЦБЗ} \rangle \rightarrow \langle \text{Ц0} \rangle \{ \langle \text{Ц0} \rangle \mid \mathbf{ц} \} \mid 0$

2) $\langle \text{Ц0} \rangle \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Преобразованная грамматика $G_1'[\langle \text{ЦБЗ} \rangle]$ не является эквивалентной G_1 , так как цепочки, выводимые из $\langle \text{ЦБЗ} \rangle$, не могут начинаться с цифры «0» в естественной форме. Таким образом, эта грамматика удовлетворяет правильности условий, синтаксиса и семантики целых чисел позиционной системы исчисления.

Определение 2.5. *Грамматика является леворекурсивной, если в ней есть такой нетерминал, что существует порождение $A \Rightarrow^* A\alpha$.*

Методы нисходящего разбора не могут работать с леворекурсивными грамматиками, так как практически реализация этих грамматик часто приводит к закликиванию алгоритма разбора. Поэтому от левой рекурсии избавляются преобразованием исходной



грамматики в эквивалентную, но без леворекурсивных правил. Идея такого преобразования состоит в следующем.

Пусть имеем леворекурсивную продукцию $A \rightarrow A\alpha \mid \beta$. Введем дополнительный нетерминал B , такой что

$$\begin{aligned} A &\rightarrow \beta B \\ B &\rightarrow \alpha B \mid \Lambda. \end{aligned}$$

Легко показать эквивалентность продукций, т. е. выводимость одних и тех же цепочек. В последних двух правилах вывода отсутствует левосторонняя рекурсия. Платой при этом явилось добавление нового правила вывода и нетерминала. Это оправданная цена, поскольку гарантируется отсутствие заикливания. Общий алгоритм преобразования леворекурсивных грамматик описан в [12, 13].

2.3. ЯЗЫКИ ПОРОЖДАЮЩИХ ГРАММАТИК

В рассмотренном примере принадлежности цепочки 123 множеству <целое без знака> мы уже по существу столкнулись с элементом языка – терминальной строкой 123. Следует отметить, что 123 в данном случае рассматривается не как семантическое понятие *целое*, а как синтаксическая конструкция, которая состоит из терминальных символов цифрового регистра.

Определение 2.6. *Языком $L(G[Z])$ называется множество терминальных цепочек x , порождаемых грамматикой $G[Z]$, таких что x итерационно выводится из начального символа Z . Цепочки x называются конечной сентенциальной формой.*

Строгое математическое определение языка имеет форму

$$L(G[Z]) = \{x, Z \Rightarrow^* x, x \in V_T^*\}.$$

В отличие от определения, данного для порождаемого языка в [1], здесь над терминальным словарем выполняется операция замыкания Клини. Это приводит к возможности порождения пустого языка.

ОПЕРАЦИИ НАД ЯЗЫКАМИ

Как видно из приведенного определения, язык представляет собой хоть и специфическое, но все-таки множество. Определим операции над этим множеством, причем для содержательной интерпретации в примерах для простоты положим, что язык $L1 = \{A, \dots, Z\}$ совпадает с алфавитом заглавных латинских букв и $L2 = \{0, \dots, 9\}$ – множество цифр позиционной системы.



Объединение: $L1 \cup L2 = \{s \mid s \in L1, s \in L2\}$.

Объединение языков представляет собой новый язык – множество заглавных букв латинского алфавита и цифр.

Конкатенация: $L1 L2 = \{s t \mid s \in L1, t \in L2\}$.

Конкатенация языков представляет собой множество строк из заглавных букв, за которыми следует цифра, причем длина этих цепочек равна двум.

Замыкание Клини (итерация): $L^* = L^0 \cup L^1 \cup \dots \cup L^n \cup \dots$

Итерация над языком L представляет собой все возможные строки из всевозможных комбинаций заглавных букв всевозможной длины (включая пустую цепочку).

Позитивное замыкание (усеченная итерация): $L^+ = L^1 \cup \dots \cup L^n \cup \dots$

Содержательная интерпретация нового языка L^+ та же, что и в предыдущем случае, только без пустой цепочки.

Пример 2.11 [13]

Обозначим множество букв латинского алфавита через $L_T = \{a, \dots, z, A, \dots, Z\}$, а множество цифр позиционной системы исчисления через $D_T = \{0, \dots, 9\}$ и будем рассматривать L_T и D_T как конечные языки из строк единичной длины букв латинского алфавита и цифр. Тогда следующие операции приводят к новым языкам:

- $L_T \cup D_T$ – язык, состоящий из букв и цифр;
- L_T^+ – язык из строк длиной $k = 1, 2, 3, \dots$, представляющих собой всевозможные комбинации из букв латинского алфавита;
- D_T^+ – язык цепочек без знака, включая цепочки, которые начинаются с нуля;
- L_T^4 – язык, который включает в себя всевозможные четырехбуквенные строки.

Из приведенных примеров следует, что новый язык может порождаться не только грамматикой, но и операциями над исходным языком.

2.4. ПРЯМАЯ И ОБРАТНАЯ ЗАДАЧИ ФОРМАЛЬНЫХ ЯЗЫКОВ И ГРАММАТИК

Задача построения грамматик по языку является прямой задачей. Соответственно выбор языка по грамматике является обратной задачей. Проиллюстрируем эту процедуру на формальных примерах и обозначим при этом некоторые особенности подбора. Начнем с обратной задачи.

**Пример 2.12**

Пусть грамматика $G_1[A]$ задана следующим набором:

$$V_T = \{0, 1\}, V_N = \{A\},$$

$$P: 1) A \rightarrow 01$$

$$2) A \rightarrow 0A1$$

или

$$A \rightarrow 01 \mid 0A1,$$

где V_T – терминальный словарь; V_N – нетерминальный словарь; P – множество правил вывода.

Требуется определить язык, порождаемый грамматикой $G_1[A]$, т. е. $L(G_1[A])$ – ?

В соответствии с правилами грамматики P определим множество цепочек, порождаемых грамматикой $G_1[A]$. Для этого продемонстрируем процедуру выводимости в соответствии с правилами вывода P :

$$\begin{array}{ccccccc} A & \Rightarrow & 0A1 & \Rightarrow & 00A11 & \Rightarrow & 000A111 \dots \\ \Downarrow & & \Downarrow & & \Downarrow & & \\ 01 & & 0011 & & 000111 & & \dots \end{array}$$

Очевидно, что это множество включает в себя вполне упорядоченный набор цепочек бинарного словаря $V_T = \{0, 1\}$:

$$\{01, 0011, 000111, \dots\}.$$

Отсюда легко видеть, что

$$L(G_1[A]) = \{0^n 1^n \mid n \geq 1\}.$$

Примечание: показатель степени n здесь и далее будет означать n -кратную итерацию этого терминального символа.

Приведем еще одну грамматику $G_{11}[A]$, которая порождает тот же язык $L(G_1[A]) = \{0^n 1^n \mid n \geq 1\}$.

$$P: 1) A \rightarrow 0B1$$

$$2) 0B \rightarrow 00B1$$

$$3) B \rightarrow \Lambda.$$

Тем же способом выводимости можно показать эквивалентность грамматик $G_{11}[A]$ и $G_1[A]$ с точки зрения порождения одного и того же языка. Строгое определение эквивалентности будет дано ниже, а здесь отметим приведенную особенность грамматик. Грамматика $G_1[A]$ обладает определенными преимуществами перед $G_{11}[A]$.



Во-первых в грамматике $G_1[A]$ меньшее количество нетерминальных символов (всего один A) и меньшее количество продукций, и это связано с меньшим количеством шагов при выводимости определенных цепочек языка. Во-вторых, отсутствие правил вида $B \rightarrow \Lambda$ делает грамматику более определенной. Наконец, при разработке грамматик следует стремиться к тому, чтобы в левой части продукций стоял нетерминальный символ, а не смешанная цепочка из терминалов и нетерминалов, как это приведено во втором правиле $G_{11}[A]$.

Пример 2.13

Пусть $G_2[Z]$ определена на множестве правил вывода P .

- P:** 1) $Z \rightarrow aA$
 2) $A \rightarrow bB$
 3) $B \rightarrow c$
 4) $A \rightarrow Bb$.

По аналогии с предыдущим примером

$$\begin{aligned} Z &\Rightarrow aA \Rightarrow abB \Rightarrow abc \\ &\Downarrow \\ &aBb \Rightarrow acb \end{aligned}$$

Поэтому легко видеть, что язык $L(G_2[Z])$ включает в себя только две терминальные цепочки:

$$L(G_2[Z]) = \{abc, acb\}.$$

Пример 2.14

Наконец, последний случай определим грамматикой $G_3[Z]$:

$$V_T = \{a, b\}, V_N = \{A, Z\},$$

- P:** 1) $Z \rightarrow aA$
 2) $A \rightarrow Ab$.

Вновь воспользуемся выводимостью:

$$Z \Rightarrow aA \Rightarrow aAb \Rightarrow aAbb \Rightarrow \dots$$

Таким образом никогда не удастся избавиться от нетерминала A , поэтому грамматика $L(G_3[Z])$ не порождает терминальных цепочек, а язык $L(G_3[Z])$ является пустым:

$$L(G_3[Z]) = \emptyset.$$



Из приведенных примеров следует, что грамматики могут порождать языки, включающие бесконечное множество определенных цепочек ($G_1 \mid n \rightarrow \infty$), ограниченное множество цепочек (G_2) и вообще не иметь терминальных цепочек ($G_3[Z]$).

ПРЯМАЯ ЗАДАЧА ПОСТРОЕНИЯ ГРАММАТИКИ

Перейдем к прямой задаче, когда по языку необходимо построить некоторую грамматику G , порождающую заданный язык L . Вновь обратимся к конкретным примерам.

Пример 2.15

Пусть $L(G_4[Z]) = \{a^n b^{2n-1} \mid n \geq 1\}$.

Необходимо определить $G_4[Z]$.

При подборе грамматики следует стремиться выполнить следующее:

- 1) ограничить множество P ;
- 2) ограничить множество V_N , по возможности избегая так называемых *цепных productions*.

Определение 2.7. *Цепными называются productions вида $A \rightarrow B$.*

Грамматика считается оптимально сконструированной, когда при минимальных P и V_N порождается один и тот же язык. Для языка $L(G_4[Z])$ характерными являются цепочки, которые представляют собой n -кратную итерацию терминального символа a и $(2n-1)$ -кратную итерацию символа b . По аналогии с примером 2.12 легко найти $G_4[Z]$:

$$V_T = \{a, b\},$$

$$V_N = \{Z\},$$

$$P: 1) Z \rightarrow ab$$

$$2) Z \rightarrow aZbb.$$

Или еще проще – пользуясь принятыми соглашениями:

$$P: 1) Z \rightarrow ab \mid aZbb.$$

Проверка правильности сконструированной грамматики осуществляется, например, уже рассмотренным методом выводимости. Действительно,

$$\begin{array}{ccccc} Z & \Rightarrow & aZbb & \Rightarrow & aaZbbbb & \Rightarrow & \dots \\ \Downarrow & & \Downarrow & & \Downarrow & & \\ ab & & aabbb & & a^3b^5 & & \end{array}$$



В следующем примере из класса прямых задач рассмотрим наиболее часто употребительные в инженерной и экономической практике решения задач традиционные формы алгебраических выражений, или, как принято именовать эти конструкции в языковых процессорах, – язык арифметических выражений ($\langle AB \rangle$).

Требуется решить прямую задачу разработки грамматики $G[\langle AB \rangle]$ для содержательных конструкций арифметических выражений вида

$$a + b - c;$$

$$a * (b / (c + d)) \text{ и т. д.}$$

Эта нетривиальная задача, в отличие от приведенных выше, решалась достаточно непросто. В ее решении принимали участие коллективы отечественных [15] и зарубежных ученых [11, 14].

Наиболее полно арифметические выражения представлены в языке *Fortran* (FORmula TRANslator). Приведем данную грамматику с усечениями, которые не нарушают общности. Усечение грамматики выполняется на уровне идентификаторов или операндов, в качестве которых будем использовать терминальные символы a, b, c .

Пусть $G[\langle AB \rangle]$ – усеченная грамматика языка *Fortran*.

$$V_T = \{+, -, *, /, (,), a, b, c\}$$

- P:**
- 1) $\langle AB \rangle \rightarrow T$ (T – терм)
 - 2) $\langle AB \rangle \rightarrow T + \langle AB \rangle$
 - 3) $\langle AB \rangle \rightarrow T - \langle AB \rangle$
 - 4) $T \rightarrow O$ (O – операнд)
 - 5) $T \rightarrow O * T$
 - 6) $T \rightarrow O / T$
 - 7) $O \rightarrow (\langle AB \rangle) \mid a \mid b \mid c$.

Грамматика языка *Fortran* в отличие от других языков высокого уровня (*C*, *C++*, *Pascal*, *BASIC*, *Java*) имеет операцию возведения в степень, которая в $G[\langle AB \rangle]$ не приведена. Воспользуемся принятыми соглашениями и приведем грамматику к более компактному виду:

- 1) $\langle AB \rangle \rightarrow T \{ \langle \text{знак} + \rangle T \}$
- 2) $\langle \text{знак} + \rangle \rightarrow + \mid -$
- 3) $T \rightarrow O \{ \langle \text{знак} * \rangle O \}$
- 4) $\langle \text{знак} * \rangle \rightarrow * \mid /$
- 5) $O \rightarrow (\langle AB \rangle) \mid a \mid b \mid c$.



Выбор той или иной формы грамматики $G[<AB>]$ связан с видом анализа. При синтаксическом анализе удобно пользоваться компактной формой. Например, при семантическом анализе, как будет показано ниже, можно пользоваться только безальтернативной расширенной формой.

В рассмотренных выше примерах решалась прямая или обратная задача поиска грамматики G по языку L или языка L по грамматике G . Теперь поставим задачу по-другому.

Пусть есть некоторая цепочка β и требуется установить ее принадлежность языку, порождаемому заданной грамматикой $L(G[Z])$, т. е. требуется установить, имеет ли место соотношение $\beta \in L(G[Z])$.

Обратимся в очередной раз к примеру.

Пример 2.16

Пусть задана грамматика $G_5 [I]$ с множеством продукций P :

- 1) $I \rightarrow abAI$
- 2) $I \rightarrow c$
- 3) $A \rightarrow bA$
- 4) $A \rightarrow c$

и пусть требуется установить, принадлежит ли цепочка $abbcc$ языку $L(G_5[I])$.

Воспользуемся все тем же приемом итерационной выводимости:

$$I \Rightarrow abAI \Rightarrow abAc \Rightarrow abbAc \Rightarrow abbcc \in L(G_5[I]).$$

Таким образом, легко установлено, что данная цепочка принадлежит языку $L(G_5[I])$. Во всех предыдущих примерах прямой и обратной задачи, а также задачи анализа они решались с использованием определения выводимости. Это не единственный способ анализа. Существуют и другие методы, например метод геометрической интерпретации. Остановимся на этом более подробно.

2.5. ГЕОМЕТРИЧЕСКАЯ ИНТЕРПРЕТАЦИЯ СИНТАКСИЧЕСКОГО РАЗБОРА

Существует несколько способов геометрической интерпретации анализа. В этом разделе рассмотрим синтаксические диаграммы и синтаксические деревья. Введем эти определения на арифметиче-

ских выражениях, которые заканчиваются терминалом #. Грамматику арифметических выражений $G[<AB>]$ представим в следующем виде:

$$\begin{aligned} 1) & <AB> \rightarrow E\# \\ 2) & E \rightarrow T \mid E + T \mid E - T \\ 3) & T \rightarrow O \mid T * O \mid T / O \\ 4) & O \rightarrow a \mid b \mid c \mid (E). \end{aligned}$$

В приведенной грамматике опущена фортрановская операция возведения в степень **. Однако это обстоятельство не нарушает общности.

Для построения синтаксической диаграммы приведем $G[<AB>]$ к так называемым LL(1)-грамматикам.

При этом попытаемся найти такую грамматику $G1[<AB>]$, которая, с одной стороны, будет удовлетворять требованиям LL(1)-грамматик, а с другой стороны, эта грамматика должна порождать тот же язык, что и $G[<AB>]$, или, как принято говорить [13–15], быть *эквивалентной*.

Требованием LL(1)-грамматик (Last Left (1) – самый левый первый символ), или, как их еще называют, *разделенных грамматик* [11, 13], является отсутствие альтернативных продукций с одинаковыми самыми левыми символами [3, 12]. Это достаточно жесткое условие, и легко видеть, что в исходной грамматике $G[<AB>]$ оно не выполняется.

В [3] приведена эквивалентная грамматика $G1[<AB>]$ со следующими продукциями:

$$\begin{aligned} 1) & <AB> \rightarrow E\# \\ 2) & E \rightarrow TA \\ 3) & A \rightarrow \Lambda \mid + TA \mid - TA \\ 4) & T \rightarrow OB \\ 5) & B \rightarrow \Lambda \mid * OB \mid / OB \\ 6) & O \rightarrow a \mid b \mid c \mid (E). \end{aligned}$$

Представленный пример приведения грамматики $G[<AB>]$ к эквивалентной $G1[<AB>]$ выполнен с использованием общего приема по устранению левой рекурсии [3, 16].

Построенная грамматика отвечает условиям LL(1)-грамматик. Легко убедиться в том, что вновь построенная грамматика $G1[<AB>]$ порождает то же множество цепочек, что и $G[<AB>]$. Новая, так



называемая эквивалентная (ниже будет дано точное определение понятия эквивалентности) грамматика приведена к виду LL(1)-грамматик, и в связи с этим синтаксический разбор можно представить с помощью *синтаксических диаграмм*.

СИНТАКСИЧЕСКИЕ ДИАГРАММЫ

В [16] приводится следующее определение.

Определение 2.8. Синтаксическая диаграмма – это направленный граф, дуги которого – элементы объединенного словаря $V = V_T \cup V_N$, а структура графа соответствует правилам вывода множества P .

Не нарушая общности, преобразуем грамматику $G1[AB]$ в $G[E]$, опуская эквивалентные операции ($/$ и $-$), и объединим a, b, c в токен **id**. Тогда

$$P = \{1. E \rightarrow TE', 2. E' \rightarrow +TE' \mid \varepsilon, 3. T \rightarrow OT', 4. T' \rightarrow *OT' \mid \varepsilon, 5. O \rightarrow (E) \mid \text{id}\}$$

Синтаксические диаграммы имеют вид, показанный на рис. 2.1.

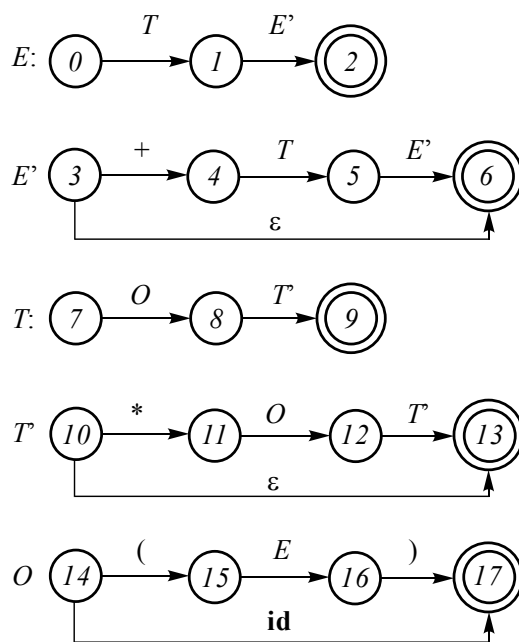


Рис. 2.1. Виды синтаксических диаграмм



Состояния 2, 6, 13, 17, выделенные двойной окружностью, являются заключительными. Возможен также преобразованный вариант (рис. 2.2).

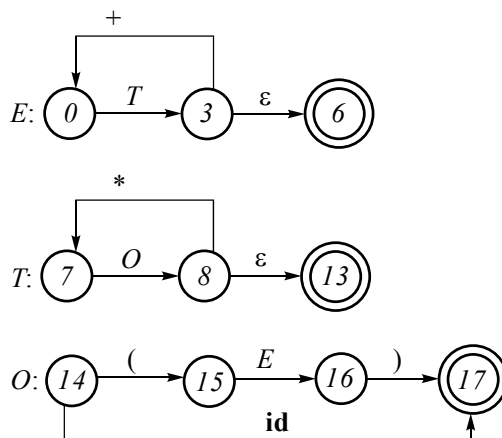


Рис. 2.2. Синтаксический граф преобразованной грамматики

Приведем другое определение [16] того же понятия синтаксической диаграммы.

Определение 2.9. Синтаксическая диаграмма – это направленный граф с одним входным ребром и одним выходным ребром и помеченными вершинами. Цепочки пометок при вершинах на пути от входного ребра к выходному считаются цепочками языка.

Тогда для $G[E]$ синтаксическая диаграмма будет иметь вид, представленный на рис. 2.3.

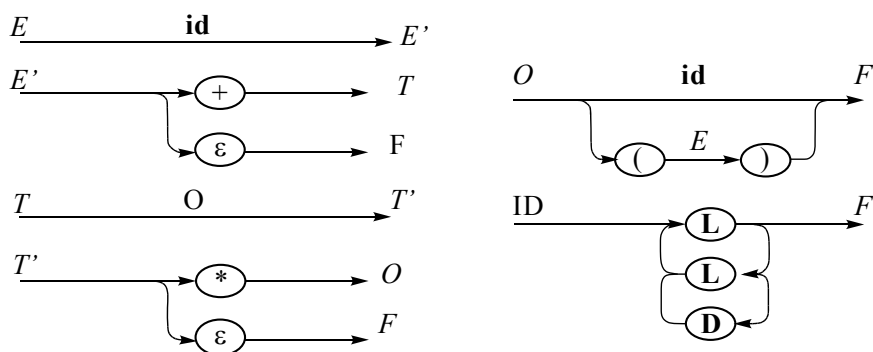


Рис. 2.3. Синтаксическая диаграмма для $G[E]$:

F – Finish, L – множество букв, D – цифры от 0 до 9



Приведем еще одну иллюстрацию синтаксических диаграмм $G[\langle \text{целое} \rangle]$:

- 1) $\langle \text{целое} \rangle \rightarrow [+ \mid -] \langle \text{целое без знака} \rangle$;
- 2) $\langle \text{целое без знака} \rangle \rightarrow \Pi \{ \Pi \}$.

Соответствующие синтаксические диаграммы имеют простой и понятный вид (рис. 2.4).

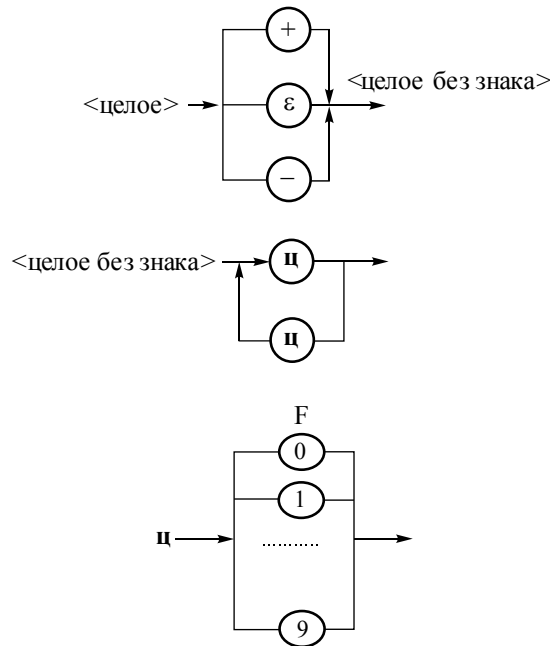


Рис. 2.4. Синтаксические диаграммы для грамматики $G[\langle \text{целое} \rangle]$

СИНТАКСИЧЕСКИЕ ДЕРЕВЬЯ

Геометрической интерпретацией синтаксического разбора также служат и синтаксические деревья. В отличие от синтаксических диаграмм для синтаксических деревьев нет необходимости приведения грамматики к определенному виду LL(1)-грамматик.

Определение 2.10. Синтаксическим деревом называется граф, узлами которого являются нетерминалы из V_N , а дуги (ветки) суть стрелки выводимости продукций множества P грамматики. Листья синтаксического дерева отражают цепочки языка $\beta_i \in V_T^*$, порождаемые заданной грамматикой с правилами вывода P .



На рис. 2.5 представлено синтаксическое дерево для грамматики классической арифметики $G[<AB>]$. Дерево как бы перевернуто «вверх ногами», и корень находится вверху, хотя листья – терминальные цепочки $\beta_i \in V_T^*$, все-таки нормально свисают с веток.

Определение 2.11. Терминальная цепочка $\beta_i \in V_T^*$ называется *сентенциальной формой*, если цепочка является языковой конструкцией или выводима из начального нетерминала, $\beta_i \in L(G[Z])$.

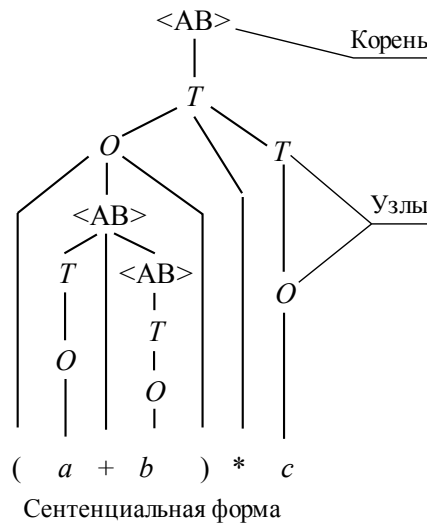


Рис. 2.5. Синтаксическое дерево с грамматикой $G[<AB>]$

Здесь $(a + b) * c$ – сентенциальная форма или основа; $<AB>$ – корень дерева; O, T – узлы дерева.

На рис. 2.5 представлено дерево синтаксического разбора цепочки $(a + b) * c$, для которой в соответствии с грамматикой $G[<AB>]$ и определением выводимости имеет место итерационное следование

$$<AB> \Rightarrow^* (a + b) * c.$$

2.6. ЭКВИВАЛЕНТНОСТЬ И ОДНОЗНАЧНОСТЬ

Определение 2.12. Грамматики G и $G1$ являются эквивалентными, если они порождают один и тот же язык, т. е. имеет место тождество множеств

$$L(G) = L(G1).$$



Определение 2.13. Грамматики G и $G1$ являются почти эквивалентными, если порождаемые ими языки отличаются не более чем на пустую цепочку Λ , т. е. имеет место тождество

$$L(G) = \{L(G1) \cup \{\Lambda\}\}.$$

Из приведенных иллюстраций двух грамматик G_1 и G_{11} в примере 2.12 видно, что каждая из них порождает один и тот же язык и поэтому эти грамматики эквивалентны. Однако не все эквивалентные грамматики можно использовать для реализации. Реализовывать можно только однозначные грамматики.

Если подобранная грамматика G удовлетворяет требованиям порождаемого ею языка $L(G)$, но при этом не является однозначной, то необходимо, если это возможно, подобрать эквивалентную однозначную грамматику $G1$.

К сожалению, доказано [12], что проблема эквивалентности грамматик в общем случае неразрешима. Это значит, что в принципе невозможно придумать в настоящем и будущем алгоритм, который бы проверил эквивалентность двух грамматик. Однако общий случай не распространяется на частные задачи.

Точно так же в общем случае неразрешима и проблема однозначности грамматик. Проблема алгоритмической неразрешимости эквивалентности и однозначности известна как «проблема соответствий Поста» [12] и формулируется следующим образом.

Пусть в некотором алфавите Σ имеется непустая последовательность пар цепочек

$$(\alpha_i, \beta_i) \mid \alpha_i, \beta_i \in \Sigma^+, i = \overline{1, n}.$$

Необходимо проверить, существует ли среди них такая подпоследовательность пар $(\alpha_1, \beta_1), (\alpha_2, \beta_2), \dots, (\alpha_m, \beta_m), m > 0$, что $\alpha_1 \alpha_2 \dots \alpha_m = \beta_1 \beta_2 \dots \beta_m$.

Американским математиком Э. Постом доказано, что не существует алгоритма, который бы за конечное число шагов мог в принципе дать ответ на этот вопрос. Однако если в общем проблема не решается, то это совсем не означает, что ее невозможно решить в частном случае.

Выполним попытку решения проблемы однозначности в частном случае.



Проблема однозначности. Проблема однозначности и безвозвратности разбора является важной задачей системного программирования в части проектирования языковых процессоров и состоит в следующем.

Безвозвратность предполагает спуск по синтаксическому дереву без подъемов или возвратов. Однозначность означает существование единственного синтаксического дерева для одной основы или одной сентенциальной формы.

Проиллюстрируем проблему однозначности на конкретном примере.

Пусть грамматика подкласса арифметических выражений $G[A]$ задана следующим альтернативным правилом:

$$A \rightarrow A + A \mid A * A \mid (A) \mid a.$$

Для простоты разбора не будем учитывать знаки «/» и «—», а также терминальные символы $b, c \dots$. Это усечение не означает неполноту грамматики и не нарушает ее общность.

Рассмотрим сентенциальную форму $a + a * a$ и выполним для нее геометрическую интерпретацию разбора в $G[A]$ (рис. 2.6).

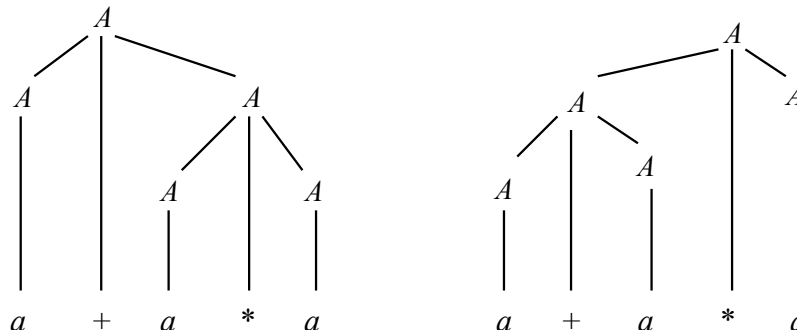


Рис. 2.6. Неоднозначность

Два синтаксических дерева для одной и той же основы и есть неоднозначность разбора. К чему может привести эта неоднозначность? Пусть результат – корень, т. е. в узле будем считать результат R . Сворачивая (вычисляя) нетерминалы снизу вверх, получим:

$$1) a * a = A$$

$$a = A$$

$$R = A + A$$

$$2) a + a = A$$

$$A = a$$

$$R = A * A$$



Таким образом, для разных синтаксических деревьев с одной и той же основой получены разные результаты R . Неоднозначная грамматика не предполагает семантического единства и верности синтаксического разбора.

По определению две рассмотренные грамматики $G[A]$ и $G[<AB>]$ являются эквивалентными, хотя строго математически эквивалентность в этом случае не показана. Просто очевидным является порождение этими грамматиками одного и того же множества цепочек арифметических выражений.

Рассмотрим еще один частный, но теперь уже формальный случай.

Пусть терминальный алфавит состоит из двух элементов $V_T = \{a, b\}$. Можно построить множество пар цепочек

$$\Sigma_1 = \{(abbb, b), (a, aab), (ba, b)\}$$

и найти одно из частных рещений

$$\Sigma_2 = \{(a, aab), (a, aab), (ba, b), (abbb, b)\}.$$

С учетом условия эквивалентности выполним конкатенацию соответствующих цепочек из словаря Σ_2 и получим

$$(a)(a)(ba)(abbb) = (aab)(aab)(b)(b)$$

или

$$a^2 b a^2 b^3 = a^2 b a^2 b^3.$$

Таким образом, теперь уже строго математически обоснована эквивалентность для частного формального примера. Однако для множества пар цепочек $\Sigma_3 = \{(ab, aba), (aba, baa), (baa, aa)\}$ очевидно [12], что решения не существует.

КРИТЕРИИ НЕОДНОЗНАЧНОСТИ

Выше отмечалось, что проблемы эквивалентности и однозначности алгоритмически в общем случае неразрешимы, лишь в частом случае можно добиться успеха. Удалось, однако, выделить множество продукций, по наличию которых в правилах вывода грамматик P можно судить об однозначности. Приведем этот уникальный набор продукций, который назовем P' :

$$1) H \rightarrow HH \mid \alpha$$



- 2) $H \rightarrow H \alpha H \mid \beta$
- 3) $H \rightarrow \alpha H \mid H \beta \mid \gamma$
- 4) $H \rightarrow \alpha H \beta H,$

где $\alpha, \beta, \gamma \in V^+ = (V_T \cup V_N)^+$.

Если в заданной грамматике встречается хотя бы одно правило из приведенных, то доказано [12], что такая грамматика неоднозначна.

В рассмотренном выше примере $G[A]$ альтернативные продукции

$$A^*A \mid A/A \mid A - A \mid A + A$$

полностью соответствуют второму правилу из P' . Именно поэтому удалось легко показать неоднозначность $G[A]$.

УСТРАНЕНИЕ НЕОДНОЗНАЧНОСТИ

Рассмотрим еще один пример из [13], где предложен алгоритм по устранению неоднозначности в частном случае.

Для иллюстрации рассмотрим грамматику условного оператора $G[\langle \text{stmt} \rangle]$:

- 1) $\langle \text{stmt} \rangle \rightarrow \text{IF } \langle \text{expr} \rangle \text{ THEN } \langle \text{stmt} \rangle$
- 2) $\langle \text{stmt} \rangle \rightarrow \text{IF } \langle \text{expr} \rangle \text{ THEN } \langle \text{stmt} \rangle \text{ ELSE } \langle \text{stmt} \rangle \mid \text{OTHER}$

Или в соответствующих формальных обозначениях $G[S]$:

- 1) $S \rightarrow a E b S$
- 2) $S \rightarrow a E b S c S \mid \text{OTHER}.$

Применим эту грамматику для цепочки $aEbScaEbScS$ и установим $aEbScaEbScS \in G[S]$?

Для этого попытаемся построить синтаксическое дерево с корнем S .

Применим эту грамматику для цепочки $\text{IF } \langle \text{expr} \rangle \text{ THEN } \langle \text{stmt} \rangle \text{ ELSE IF } \langle \text{expr} \rangle \text{ THEN IF } \langle \text{expr} \rangle \text{ THEN S2 ELSE S3 } \langle \text{stmt} \rangle$ (рис. 2.7).

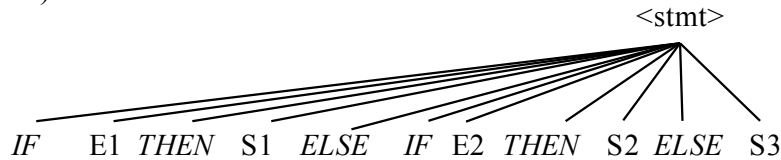


Рис. 2.7. Синтаксическое дерево (для иллюстрации)



В данном случае получено синтаксическое дерево для сен-
тенциальной формы приведенного примера.

Рассмотрим другую цепочку в качестве основы:

IF E1 THEN IF E2 THEN S1 ELSE S2

и построим для нее синтаксическое дерево (рис. 2.8).

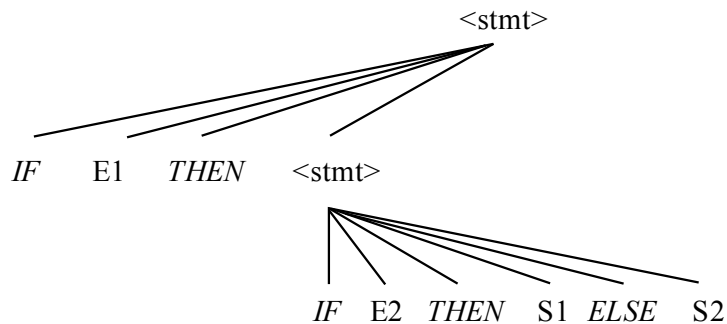


Рис. 2.8. Первое синтаксическое дерево

Попытаемся построить второе синтаксическое дерево для той
же основы (рис. 2.9).

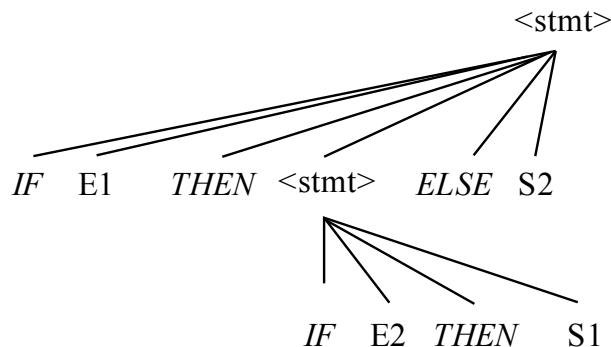


Рис. 2.9. Второе синтаксическое дерево
для одной основы в $G[<stmt>]$

Легко видеть неоднозначность для сен-
тенциальной формы

IF E1 THEN IF E2 THEN S1 ELSE S2.

Устранить неоднозначность можно лишь путем изменения пра-
вил исходной грамматики $G[<stmt>]$, причем воспроизводимые но-
вой грамматикой цепочки должны воспроизводить тот же язык, что
и $G[<stmt>]$.



Введем следующую грамматику $G_1[<stmt>]$:

- 1) $<stmt> \rightarrow <Сбалансированный\ Оператор> \mid <Несбалансированный\ Оператор>$
- 2) $<Сбалансированный\ оператор> \rightarrow IF\ <expr>\ THEN\ <Сбалансированный\ Оператор>\ ELSE\ <Сбалансированный\ Оператор>\ \mid\ other$
- 3) $<Несбалансированный\ Оператор> \rightarrow IF\ <expr>\ THEN\ <stmt>\ \mid\ IF\ <expr>\ THEN\ <Сбалансированный\ Оператор>\ ELSE\ <Несбалансированный\ Оператор>$

Грамматика $G_1[<stmt>]$ является эквивалентной для грамматики $G[<stmt>]$.

Хотя в приведенной грамматике $G_1[<stmt>]$ введены дополнительные нетерминальные символы $<Сбалансированный\ Оператор>$ и $<Несбалансированный\ Оператор>$, это позволило добиться однозначной грамматики, не нарушая условий эквивалентности.

Приведенный прием является эвристическим. Однако плата за избыточность нетерминалов в новой грамматике $G_1[<stmt>]$ оправдана, так как благодаря этому удалось избавиться от неоднозначной продукции $S \rightarrow a E b ScS$ или $S \rightarrow \alpha ScS$ ($\alpha = a E b$) в исходной формальной грамматике $G[S]$.

Действительно, в соответствии с продукциями 3, 4 правил вывода P' , характеризующих неоднозначность, следует $H \rightarrow \alpha H$. Применяя правило 3 грамматики P' , имеем

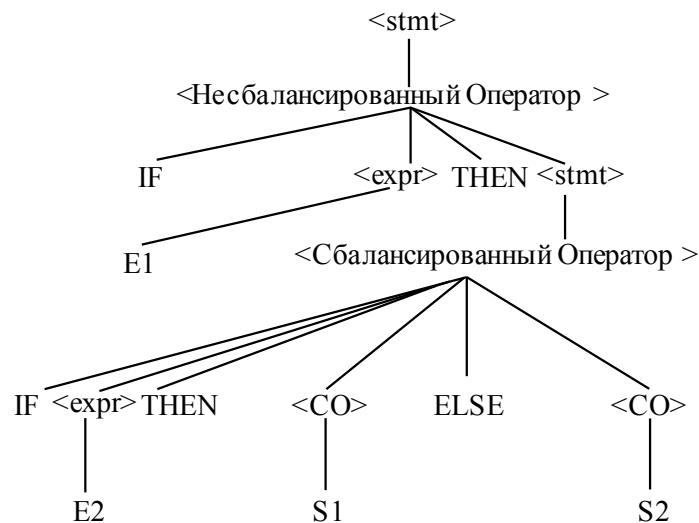
$$H \rightarrow \alpha H \alpha H.$$

Построим синтаксическое дерево (рис. 2.10) по грамматике $G_1[<stmt>]$ для основы

IF E1 THEN IF E2 THEN S1 ELSE S2.

Нетрудно убедиться в том, что построить второе синтаксическое дерево для той же основы невозможно [13].

Однако при проектировании языковых процессоров каждый раз трудно эмпирически подобрать грамматику, которая будет отвечать требованиям однозначности. Частичное решение этой проблемы было предложено Хомским через классификацию грамматик.

Рис. 2.10. Синтаксическое дерево для $G_1[<stmt>]$

Классификация предполагает определение однозначности и безвозвратности разбора без проверки построения синтаксических деревьев для одной и той же основы. С учетом классификации Хомского проект языкового процессора застрахован от неоднозначности. Вот почему после построения грамматики ее необходимо классифицировать. В случае неудачной классификации необходимо изменить либо язык, либо грамматику так, чтобы классификация удовлетворяла требованиям однозначности и безвозвратности в соответствии с иерархией Хомского.

2.7. КЛАССИФИКАЦИЯ ХОМСКОГО

1. Грамматики нулевого типа:

$$\alpha \rightarrow \beta, \quad \alpha \in V^*, \quad V = V_T \cup V_N, \quad \beta \in V^*.$$

Пример 2.17

$G[A]$:

$Aab \rightarrow cBd$.

Грамматики нулевого типа не имеют практического применения. Этот класс грамматик является неоднозначным.

2. Контекстно-зависимые грамматики (КЗ-грамматики):

$$\gamma_1 A \gamma_2 \rightarrow \gamma_1 \beta \gamma_2, \quad \beta \in V^+, \quad \gamma_1, \gamma_2 \in V^*.$$



Другая форма записи *КЗ-грамматик*, или *неукорачивающихся*, имеет вид [11]

$$\alpha \rightarrow \beta, \quad \alpha, \beta \in V^+, \quad |\alpha| \leq |\beta|.$$

Пример 2.18

$G[S]$:

$$G[S] = \{1. S \rightarrow aSBC \mid \in V^*. abC; 2. CB \rightarrow BC; 3. bB \rightarrow bb; 4. bC \rightarrow bc; 5. cC \rightarrow cc\}.$$

Грамматики этого класса также неоднозначны, но имеют ограниченное применение только в тех случаях, когда в частности можно показать однозначность.

3. Контекстно-свободные грамматики:

$$A \rightarrow \alpha, \quad A \in V_N, \quad \alpha \in V^*.$$

Пример 2.19

$G[R]$:

- 1) $R \rightarrow aa$
- 2) $R \rightarrow aAa$
- 3) $A \rightarrow b$
- 4) $A \rightarrow bA$.

Контекстно-свободные грамматики (КС-грамматики) имеют гораздо более широкое применение в отличие от предыдущих. Однако теоремой Поста [1] доказано, что в общем случае нельзя показать однозначность и безвозвратность КС-грамматик, поэтому они также имеют ограниченное применение. В индустрии проектирования процессоров широко используются подклассы КС-грамматик, для которых однозначность показана.

4. Автоматные, или регулярные, грамматики:

$$G[A]: A \rightarrow aB \mid a \mid \Lambda, \quad a \in V_T, \quad A, B \in V_N.$$

Для этого класса однозначность и безвозвратность доказана. Поэтому такой тип грамматик наиболее часто используется на практике.

Пример 2.20

$G[I]$:

- 1) $I \rightarrow aI$
- 2) $I \rightarrow bA$
- 3) $A \rightarrow bI$
- 4) $A \rightarrow a$.



При классификации грамматик все смысловые содержания должны быть переобозначены в формальные и классификация должна быть выполнена на приведенных формальных правилах. Эти правила должны иметь тот же вид, что и правила классификации грамматик по Хомскому.

Определим, принадлежит ли $L(G[L])$ цепочка $\{abbba\}$. Для этого построим дерево (рис. 2.11).

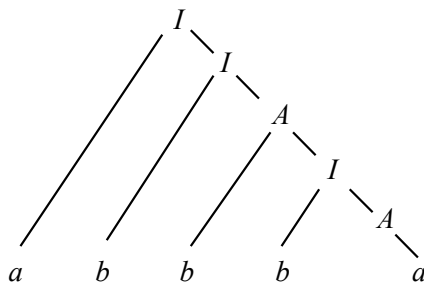
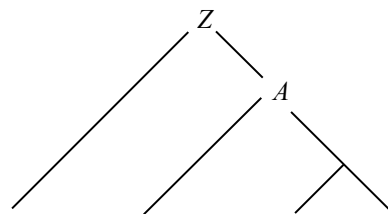


Рис. 2.11. Граф автоматной грамматики

Следовательно, цепочка $\{abbba\}$ принадлежит $L(G[L])$.

Из иллюстрации приведенного примера и особенностей правил вывода для автоматной грамматики следует, что морфология синтаксического дерева этого класса грамматик имеет вид, представленный на рис. 2.12.

Рис. 2.12. Структура графа автоматной грамматики



Сентенциальная форма

Рассмотренный тип автоматных грамматик по определению [12] является *праволинейным* типом автоматных грамматик.

Существуют также и *леволинейные* автоматные, или регулярные, грамматики. Продукции таких грамматик имеют вид

$$A \rightarrow Ba \mid a \mid \Lambda, \quad a \in V_T, \quad A, B \in V_N.$$

В [12] показан однозначный алгоритм перехода от леволинейной грамматики к праволинейной. Идея этого алгоритма рассмотрена выше для рекурсии (см. раздел 2.2). Поэтому здесь и в дальнейшем будем рассматривать только праволинейные грамматики.



По семантическому смыслу предметной задачи всегда можно избежать левостороннего проектирования грамматических конструкций.

В заключение отметим, что приведенная классификация – включающая [19], т. е. все контекстно-свободные грамматики являются и контекстно-зависимыми, все контекстно-зависимые грамматики являются грамматиками общего вида и т. д. Кроме того, можно показать, что существуют языки, принадлежащие к типу i ($0 \leq i \leq 3$), но не к типу $i+1$. Например, язык $G[S]$ является контекстно-зависимым, но не контекстно-свободным, т. е. не существует контекстно-свободной грамматики, порождающий этот язык. Наконец, отметим, что определение контекстно-зависимой грамматики запрещает использование правил вида $A \rightarrow \varepsilon$.

2.8. ГРАФЫ АВТОМАТНОЙ ГРАММАТИКИ

Графы, или диаграммы состояний, также представляют собой геометрическую интерпретацию синтаксического анализа слева направо. В отличие от синтаксических диаграмм и синтаксических деревьев графы используются для геометрической иллюстрации только автоматных грамматик. В этом смысле синтаксические деревья и синтаксические диаграммы являются более широким способом геометрической иллюстрации нисходящего разбора.

Определение 2.14. *Графом $\Gamma(G[Z])$, или диаграммой состояний, называется совокупность узлов и направленных дуг, соединяющих узлы. Узлы графа суть нетерминальные символы. Дуги графа направлены из одного узла в другой (или в тот же узел) таким образом, что из узла N дуга направляется в M и маркируется термином t , если*

$$N \rightarrow t M \in \mathbf{P}.$$

В этом случае имеет место отображение декартова произведения множества V_N и множества V_T на множество V_N , т. е.

$$\mathbf{P} : V_N \times V_T \rightarrow V_N.$$

Если $M \equiv \Lambda$, то на графе следует нетерминал K считать конечным узлом. В этом случае все правила вывода вида $N \rightarrow t$ следует заменить правилами $N \rightarrow tK$, т. е. ввести дополнительный нетерминал K , который определяет все заключительные узлы графа. При этом в $\mathbf{P}$ следует добавить $K \rightarrow \Lambda$.



Проиллюстрируем процедуру построения графа для автоматной грамматики $\mathbf{G}[I]$ (рис. 2.13.):

- 1) $I \rightarrow aI$
- 2) $I \rightarrow bA$
- 3) $A \rightarrow bI$
- 4) $A \rightarrow a$.

Введем в правило 4 нетерминал K , который определит заключительный узел на графе. Тогда в $\mathbf{G}[I]$ вместо правила 4 необходимо добавить

- 4) $A \rightarrow aK$
- 5) $K \rightarrow \Lambda$.

Соответственно терминальный и нетерминальный словари будут определены множествами

$$V_T = \{a, b, \Lambda\}, \quad V_N = \{I, A, K\}.$$

Граф $\Gamma(\mathbf{G}[I])$, или диаграмма состояний, в соответствии с определением будет иметь вид, представленный на рис. 2.13.

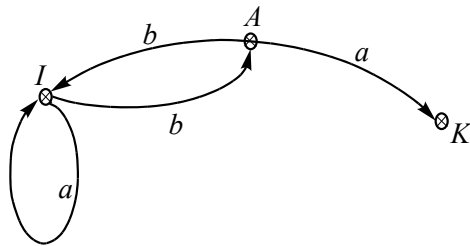


Рис. 2.13. Граф $\Gamma(\mathbf{G}[I])$

Диаграмма состояний (граф) автоматически показывает принадлежность цепочки языку, порождаемому заданной грамматикой. Для установления факта принадлежности следует выполнить «путешествие» из начального нетерминала I в заключительное состояние K . Если такое «путешествие» завершается в узле K , то это означает принадлежность цепочки языку, порождаемому грамматикой $\mathbf{G}[I]$, в противном случае цепочка не принадлежит языку.

УПРАЖНЕНИЯ

1. Можно ли в определении $L(\mathbf{G}[Z])$ вместо словаря V_T^* написать V_T^+ ?

2. Пусть $A = \{0, 1, 2\}$. Написать семь самых коротких цепочек из A^* и A^+ .



3. Пусть $A = \{0, 1, 2\}$, $B = \{1, 0\}$. Найти AB .

4. Пусть $A = \{a, e\}$. Найти A^+ .

5. Какие из приведенных ниже цепочек являются символами и (или) лексемами?

While, ?, :=, &&, a, ab, int

6. Пусть $A = \{a, e\}$. Найти A^2, A^3 .

В упражнениях № 7–12 подобрать $G([Z])$ по L .

7. $L = \{1^n 0^m, n, m > 0\}$.

8. $L = \{1^n 0^n 1^m 0^m, n, m > 0\}$.

9. $L = \{a^n b^m c^k, n, m, k > 0\}$.

10. $L = \{a^* + b^*\}$.

11. $L = \{a^n b^{3n-1} c^m, n, m > 0\}$.

12. $L = \{1^n 0^n\} + \{a^{2n} b^n\}, n > 0$.

В упражнениях № 13–17 подобрать L для порождающей грамматики $G[A]$.

13. P : 1) $Z \rightarrow 11XY0$ 2) $X \rightarrow 1X \mid 1 \mid \Lambda$ 3) $Y \rightarrow 1Y0 \mid \Lambda$.

14. P : 1) $Z \rightarrow AB - A$ 2) $X \rightarrow 1X \mid 1 \mid \Lambda$ 3) $Y \rightarrow 1Y0 \mid \Lambda$.

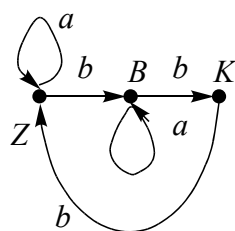
15. P : 1) $Z \rightarrow AB$ 2) $A \rightarrow 1A0 \mid 10$ 3) $B \rightarrow 1B0 \mid 1$.

16. P : 1) $Z \rightarrow A + B$ 2) $A \rightarrow aA \mid \Lambda$ 3) $B \rightarrow bB \mid \Lambda$.

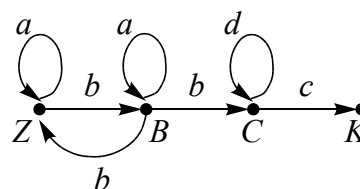
17. P : 1) $Z \rightarrow A - B$ 2) $A \rightarrow 1A00 \mid 10$ 3) $B \rightarrow aB \mid \Lambda$.

В упражнениях № 18–21 подобрать $L, G([Z])$ по графу $\Gamma(G[Z])$.

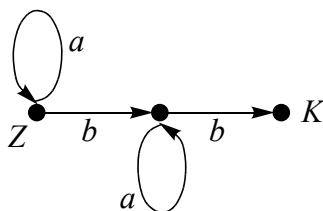
18.



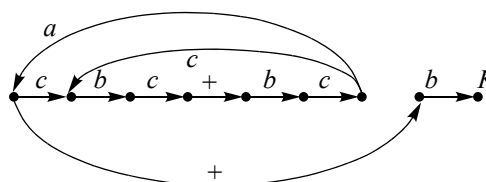
19.



20.



21.





$$\mathbf{G}[I]: \mathbf{P} = \{I \rightarrow aA, \quad I \rightarrow Ic, \quad I \rightarrow Ab, \quad A \rightarrow d\}.$$

3. КОНЕЧНО-АВТОМАТНЫЕ РАСПОЗНАВАТЕЛИ И ЯЗЫКИ

В этом разделе будут рассмотрены конечно-автоматные распознаватели предложений с языков, определенных не порождающими грамматиками, а конечными автоматами. Будут даны строгие определения детерминированных и недетерминированных автоматов. Языки, определяемые этими автоматами, уже описаны в классификации Хомского и называются автоматными. Кроме того, в этом разделе рассматриваются и КС-грамматики, которые распознаются или принимаются автоматами с магазинной памятью (МП-автоматами). Новый подход к языкам с точки зрения конечно-автоматных распознавателей не противоречит порождающим грамматикам, а существенно дополняет теорию формальных языков.

3.1. АВТОМАТНЫЕ РАСПОЗНАВАТЕЛИ И ГРАММАТИКИ ХОМСКОГО

Для языков $L(G[Z])$, порождаемых грамматиками, распознаватель в виде диаграммы состояний (или графа Γ) являлся, по существу, определяемым также порождаемой грамматикой $G[Z]$, т. е. правилами вывода автоматной грамматики вида $N \rightarrow tM$, причем $N, M \in V_N, t \in V_T$ (рис. 3.1).

В этом смысле справедливой является запись $\Gamma(G[Z])$.



Рис. 3.1. Производные от $G[Z]$



Если граф Γ задан другим способом, то он уже не является порождающим от продукции P порождающей грамматики. Такой граф называют автоматом-распознавателем A , а язык $L(A)$ определяется так называемой *конфигурацией* автомата. Если входная цепочка α , поданная на вход автоматного распознавателя, переводит A последовательно из некоторого начального состояния s_0 в одно из заключительных F , то $\alpha \in L(A)$. Считается, что если $\alpha \in L(A)$, то реакция автомата на эту цепочку «Да», в противном случае автомат отвечает «Нет».

Схема автомата-распознавателя A показана на рис. 3.2.

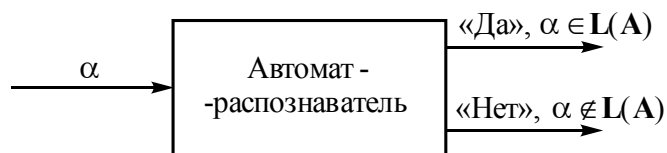


Рис. 3.2. Реакция автомата
на входную цепочку

Определение 3.1. *Распознаватель (recognizer) – это алгоритм, представленный в виде конечного автомата, который, обрабатывая входные цепочки, принимает их («Да») или отвергает («Нет»).*

При классификации Хомского порождаемые соответствующей грамматикой языки имеют тот же тип, что и грамматика $G[Z]$, которая их порождает. Связь порождающих грамматик из классификации Хомского и автоматов-распознавателей устанавливается через соответствующий язык.

С точки зрения конечных автоматов справедливы следующие утверждения:

- язык является автоматным, если он определяется (задается или распознается) конечным автоматом (возможно, и недетерминированным);
- язык является контекстно-свободным, если он определяется (задается или распознается) автоматом с магазинной памятью.

Конструктивное доказательство этих утверждений будет дано после определения конечных детерминированных и недетерминированных автоматов и автомата с магазинной памятью.



3.2. КОНЕЧНЫЕ АВТОМАТЫ

Идеология конечного автомата (КА) как некоторого алгоритмического устройства состоит в следующем. Имеется входная цепочка α , которая посимвольно читается слева направо считывающим механизмом A . При каждом входном символе автомат переходит в одно из своих состояний $s \in S$. Процесс чтения символов из цепочки α не бесконечен, так как длина всегда ограничена конечным символом $\#$ входной цепочки – длиной $|\alpha|$. Переход из одного текущего состояния в другое осуществляется под управлением функции переходов δ до тех пор, пока не будет обработан (считывание и переход) заключительный символ $\#$. Когда функция переходов δ допускает несколько переходов из текущего состояния при входном символе из α , тогда этот переход – любой из допустимых. В начале работы автомат всегда находится в начальном состоянии s_0 .

Конфигурация конечного автомата определяется в виде (s, ω, n) , где s – текущее состояние A , $s \in S$; ω – цепочка входных символов, $\omega \in \Sigma^+$; n – положение указателя в цепочке ω , $n \in \{0, 1, 2, \dots\}$, $n \leq |\omega|$.

Теперь дадим строгое математическое определение.

Определение 3.2. *Конечным автоматом-распознавателем A называется пятерка объектов:*

$$A = (S, \Sigma, s_0, \delta, F),$$

где S – конечное непустое множество; Σ – входной алфавит автомата (конечное непустое множество входных символов); $s_0 \in S$ – начальное состояние A ; $\delta : S \times \Sigma \rightarrow S$ – функция переходов; $F \subseteq S$ – множество заключительных (конечных) состояний.

Легко провести аналогию между рассмотренным выше графом переходов автоматной грамматики $\Gamma(G[Z])$ и конечным автоматом A .

Действительно, V_N и S – два семантически эквивалентных множества, в первом случае это множество нетерминальных символов, которые в графе $\Gamma(G[Z])$ становятся множеством состояний, причем определено и начальное состояние на графе Γ – это Z . Словари V_T и Σ – это два словаря терминальных символов. P и δ – объекты логического уровня, определенные соответственно графом и автоматом. Финальные состояния K и F для графа и автомата также имеют аналогию.



Принципиальной разницей между $\Gamma(G[Z])$ и конечным автоматом A является идентификация графа и автомата. Как уже отмечалось, граф $\Gamma(G[Z])$ определен, если определена порождающая грамматика, по которой строится граф $(N \rightarrow tM)$. Конечный автомат A имеет только терминальный словарь Σ (нетерминальный словарь отсутствует) и определен однозначно своей пятеркой $(S, \Sigma, s_0, \delta, F)$.

Определим итерацию функции переходов δ как отображение на множество состояний декартова произведения множеств состояний автомата S на итерацию словаря, т. е.

$$\delta^* : S \times \Sigma^* \rightarrow S.$$

Определение 3.3. Конечный автомат $A = (S, \Sigma, s_0, \delta, F)$ допускает входную цепочку $\alpha \in \Sigma^*$, если символы α переводят A в одно из заключительных состояний F так, что $\delta^*(s_0, \alpha) \in F$.

Теперь мы подошли к еще одному определению языка. Только в данном случае язык L будет являться производным от конечного автомата-распознавателя A .

Определение 3.4. Языком $L(A)$ над словарем Σ^* называется множество всех цепочек α , которые допускает (принимает) конечный автомат, т. е.

$$L(A) = \{\alpha \mid \alpha \in \Sigma^*, \delta^*(s_0, \alpha) \in F\}.$$

Это определение языка отличается от данного выше языка $L(G[Z])$, порождаемого грамматикой, хотя в обоих случаях язык определяется множеством цепочек, допускаемых в одном случае порождающей грамматикой, а в другом – конечным автоматом.

С точки зрения нового определения языка $L(A)$ как множества допускающих конечным автоматом цепочек соответственно изменяется определение автоматного языка в отсутствие порождающих его автоматных грамматик.

Определение 3.5. Автоматным языком называют язык $L(A)$. Иначе говоря, язык относится к классу автоматных, если существует конечный автомат A , принимающий этот язык.

Приведенное определение не противоречит уже данному определению $L(G[Z])$ для порождающих грамматик. Действительно, ранее было показано, что граф автоматной грамматики $\Gamma(G[Z])$ легко превращается в КА, если положить $s_0 = Z$; $\Sigma = V_T$, $F = \{K\}$, $S = V_N$. Тогда легко доказать, что множество продукций P эквивалентно $\{\delta\}$, так как правила вида $N \rightarrow tM$, где $N, M \in V_N$, $t \in V_T$, можно представить как отображение декартова произведения множества состояний и нетерминального словаря на множество состояний:



$$P : V_N \times N_T \rightarrow V_N.$$

Конечные автоматы представляются в виде графов, или диаграмм состояний, только вместо P используется $\{\delta\}$, вместо $V_T - \Sigma$ и т. д. Состояние автомата принято нумеровать или обозначать буквой, нетерминальные символы для этого неприменимы – в автомате нет такого понятия. Начальное состояние нумеруется как 0, а заключительные состояния нумеруют n (n – целое) и для отличия либо рисуют более жирно, либо обводят двойным кружком, как показано ниже.

① – Начальное состояние



– Заключительное состояние

3.3. СИНТАКСИЧЕСКИЕ ДИАГРАММЫ И КОНЕЧНЫЕ АВТОМАТЫ

При рассмотрении вопросов геометрической интерпретации выводимости цепочек языков с порождающей грамматикой (см. рис. 2.3, 2.4) синтаксические диаграммы рассматривались как направленные графы с одним входным ребром (дугой) и узлами, помеченными символами входной строки (цепочки).

Установим связь между конечным автоматом $A = \{S, \Sigma, \delta, s_0, F\}$ и синтаксической диаграммой. Для этого построим синтаксическую диаграмму для грамматики $G[I]$:

- P:** 1) $I \rightarrow aB$
 2) $B \rightarrow bC \mid aD$
 3) $C \rightarrow cD$
 4) $D \rightarrow b \mid cI$

Граф этой грамматики представлен на рис. 3.3.

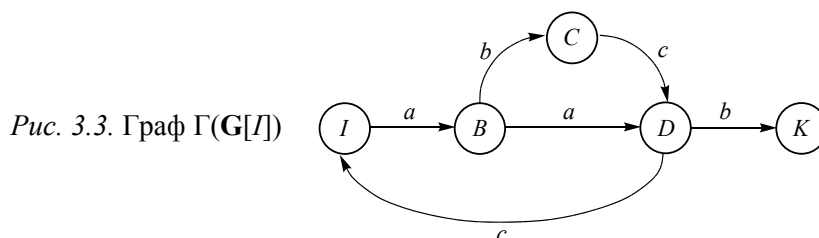


Рис. 3.3. Граф $\Gamma(G[I])$



Соответствующая синтаксическая диаграмма получается из $\Gamma(G[I])$, если узлами сделать помеченные дуги, а узлы графа убрать, соединив входную и выходную дуги в одну. На рис. 3.4 показана операция преобразования графа $\Gamma(G[I])$ в синтаксическую диаграмму. Дуги помечаются соответствующими обозначениями состояний.

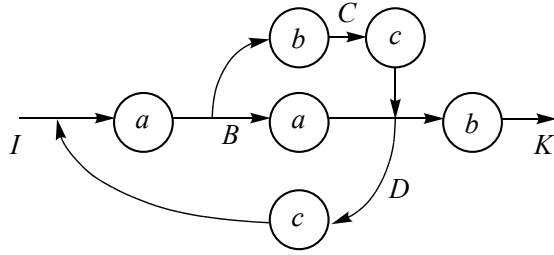


Рис. 3.4. Синтаксическая диаграмма

Язык $L(G[I])$ порождает множество цепочек $\{aab, aabaab, \dots, aab(aab)^*, abcb, abccab, \dots\}$.

Ранее было показано, что если в $\Gamma(G[I])$ нетерминалы принять множеством состояний $S = \{I, B, C, D, K\}$, то такой граф становится эквивалентным автоматным преобразователем (конечным автоматом), т. е. функция переходов $\delta(s, a)$, $\forall s \in S$, $\forall a \in \Sigma$ определена продукциями множества правил вывода P , причем $\Sigma = V_T$, $S = V_N$. В этом случае легко убедиться в эквивалентности графа автоматной грамматики и конечного автомата. Отметим, что эквивалентный конечный автомат в общем случае является недетерминированным (НКА) ввиду наличия в автоматной грамматике правила вывода $A \rightarrow \Lambda$. Таким образом, установлена связь между графами $\Gamma(G[I])$, синтаксическими диаграммами и конечными автоматами. При этом еще раз отметим, что $G[I]$ должна быть обязательно автоматной, тогда прямой и обратный переходы от $\Gamma(G[I])$ или $A = \{S, \Sigma, \delta, s_0, F\}$ к синтаксическим диаграммам определены однозначно, и все эти механизмы задания (определения или порождения) языка L эквивалентны.

Определение 3.6. КА называется полностью определенным, если в каждом его состоянии существует функция перехода для всего словаря Σ , т. е.

$$\forall a \in \Sigma, \forall s \in S \exists \delta(a, s) = R \mid R \subseteq S.$$

Рассмотрим КА в качестве иллюстрации приведенных определений.



Пусть

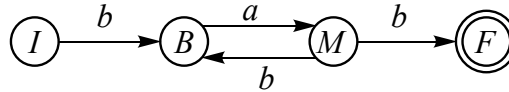
$$A_1 = (\{I, M, B, F\}, \{a, b\}, \delta, I, \{F\}).$$

Определим функцию переходов δ :

$$\delta(I, b) = B; \delta(B, a) = M; \delta(M, b) = \{B, F\}.$$

Граф состояний для КА представлен на рис. 3.5.

Рис. 3.5. Граф состояний для A_1



Как и раньше, в порождающих грамматиках автоматного типа данный автомат-распознаватель A_1 принимает язык $L(A_1)$, все цепочки которого образуются «путешествием» из начального состояния I в конечное F . Таким образом, $L(A_1) = \{bab, babab, bababab, \dots\}$. Здесь легко установить закономерность. Действительно, *префиксом* (или «головой») цепочек α является $b \in \Sigma$, *суффиксом* – ab , между префиксом и суффиксом цепочек бесконечное число раз может повторяться ab (или его может не быть вообще). Поэтому

$$L(A_1) = \{b(ab)^*ab\}.$$

Приведем еще одно определение [19], трактующее порождение языка L конечным автоматом A .

Определение 3.7. Слово $\omega = a_1 \dots a_k$ над алфавитом Σ допускается конечным автоматом $A = (S, \Sigma, \delta, s_0, F)$, если существует такая последовательность состояний $s_1, \dots, s_n$, что

$$\forall i, j : 1 \leq i < n, 1 \leq j < k, \exists \delta(s_i, a_j) = s_{i+1}, s_1 = s_0, s_n \in F.$$

Это определение следует понимать как существование функции переходов δ , которая под управлением входных символов $a_i (i = 1, \dots, n)$ последовательно переводит автомат из начального состояния s_0 в конечное $s_n \in F$.

Используя это определение и предполагая, что формальный язык представляется множеством слов (цепочек) ω , т. е. $L(A) = \{\omega\}$, $\omega \in \Sigma^*$, можно дать следующее определение языка.

Определение 3.8. Язык $L(A) = \{\omega\}$, $\omega \in \Sigma^*$, распознается конечным автоматом тогда и только тогда, когда каждое слово языка над алфавитом Σ допускается этим конечным автоматом.



Если A_1 принимает $\alpha \in L(A_1) = \{b(ab)^*ab\}$, то все другие цепочки $\beta \neq \alpha$, $\beta \in \Sigma^*$, $\Sigma = \{a, b\}$ должны переводить конечный автомат в состояние ошибки (ERROR или E). Отметим, что вновь введенное состояние E не является заключительным, так как в противном случае запрещенные цепочки β будут также принадлежать языку $L(A_1)$. Таким образом, на состояние E замыкаются все неопределенные переходы из множества состояний $S = \{I, M, B, F\}$. Для полностью определенного КА необходимо E замкнуть само на себя переходами $a \mid b$ и, кроме того, на E замкнуть переход из F по $a \mid b$. Тогда новая функция переходов будет иметь вид согласно табл. 3.1.

Таблица 3.1

Функция переходов

Состояния	Функция δ	
	a	b
I	E	B
B	M	E
M	E	F
F	E	E
E	E	E

Соответствующий граф конечного автомата показан на рис. 3.6.

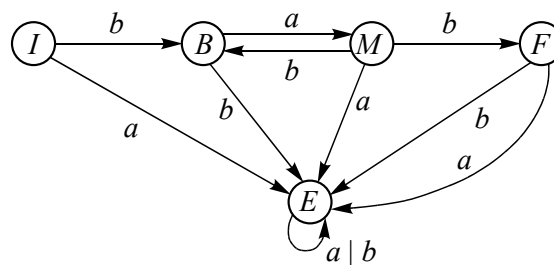


Рис. 3.6. Полностью определенный КА

Представление автомата таблицей переходов δ имеет свои преимущества и недостатки. К преимуществам относится быстрое определение конфигурации автомата по входному символу над алфавитом Σ и текущему состоянию $s \in S$. Недостаток – громоздкость



таблицы, когда алфавит велик и большинство переходов ведут в пустое множество состояний. Представление δ списком гораздо компактнее, однако при этом трудно определить конфигурацию конечного автомата. Приведем еще пример КА.

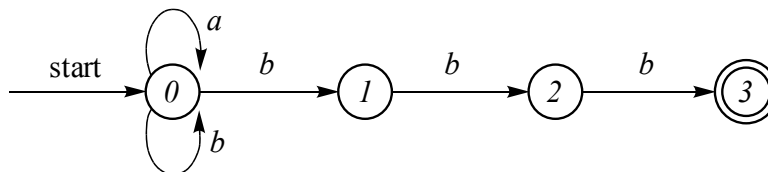
Пусть конечный автомат A_2 задан таблицей 3.2.

Таблица 3.2

Таблица переходов для A_2

Состояния	Функция δ	
	a	b
0	$\{0\}$	$\{0, 1\}$
1	—	$\{2\}$
2	—	$\{3\}$

В этом автомате, как видно из таблицы, отсутствуют переходы из состояний 1 и 2 по входу a . Соответствующий граф переходов НКА показан на рис. 3.7.

Рис. 3.7. Граф автомата A_2

Легко определяется язык $L(A_2)$, допускаемый автоматом A_2 :

$$L(A_2) = \{(a | b)^* bbb\}.$$

Все допустимые цепочки (слова) можно показать с помощью так называемых перемещений – *move*. Например, слово $bbb \in L(A_2)$ имеет перемещение

$$\begin{array}{ccc} b & b & b \\ 0 & \rightarrow 1 & \rightarrow 2 \rightarrow 3. \end{array}$$

Заметим, что *move* может и не привести в заключительное состояние. Так, цепочка abb имеет *move*:

$$\begin{array}{ccc} a & b & b \\ 0 & \rightarrow 0 & \rightarrow 1 \rightarrow 2. \end{array}$$



3.4. НЕДЕТЕРМИНИРОВАННЫЕ КОНЕЧНЫЕ АВТОМАТЫ

Представленный на рис. 3.8 [13] конечный автомат N_ε является недетерминированным. Недетерминированные конечные автоматы (НКА) характеризуются следующими особенностями: либо существует переход по $\varepsilon \in \Sigma$, либо из одного состояния выходит несколько переходов, помеченных одним и тем же символом из словаря Σ .

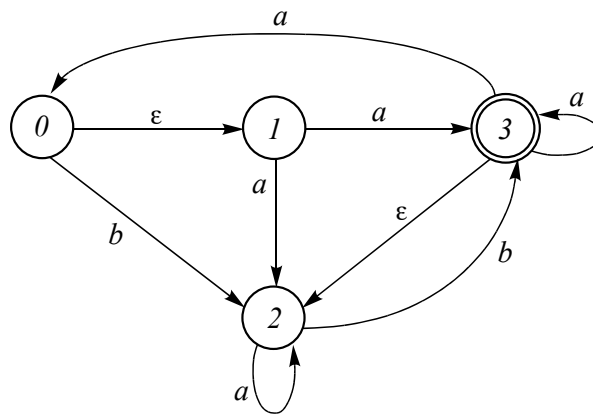


Рис. 3.8. Недетерминированный
конечный автомат

Переход из состояния 0 в состояние 1 может быть выполнен спонтанно в любой момент без подачи входного сигнала из Σ . Переход из состояния 1 по входному символу a разрешает автомату перейти либо в состояние 3, либо в 2 (еще одна неопределенность). Приведенный НКА не является полностью определенным.

Определение 3.9. Недетерминированным конечным автоматом N (*nondeterministic finite automaton*) называется пятерка

$$N = (S, \Sigma, I, \delta, F),$$

где S – конечное непустое множество (состояний); Σ – конечное непустое множество входных символов (входной словарь); $I \subseteq S$ – множество начальных состояний; $\delta : S \times (\Sigma \cup \varepsilon) \rightarrow 2^S$ – функция переходов. Здесь как 2^X обозначен булеан X , т. е. множество всех подмножеств множества X ; $F \subseteq S$ – множество заключительных состояний.



Слово «недетерминированный» нельзя читать как «неопределенный». Действительно, автомат, представленный на рис. 3.8, может спонтанно (без подачи входного символа) перейти в новое состояние 1 или остаться в прежнем состоянии 0. Автомат N_ε принимает, например, цепочку $\omega = abb$. При этом путь, помеченный ω , выглядит так:

$$\begin{array}{ccccccc} & \varepsilon & a & b & b & & \\ 0 & \rightarrow & 1 & \rightarrow & 2 & \rightarrow & 2 \rightarrow 3. \end{array}$$

Определение 3.10 *Недетерминированный конечный автомат N распознает входную цепочку ω , если существует путь, помеченный символами цепочки ω из начального в одно из заключительных состояний автомата, возможно, с учетом ε -переходов. Недетерминированный КА распознает язык $L(N)$, если он распознает все цепочки этого языка.*

Распознает ли автомат N_ε (рис. 3.8) цепочку bb ? Да, поскольку есть путь из начального состояния 0 в заключительное состояние 2.

3.5. ДЕТЕРМИНИРОВАННЫЙ КОНЕЧНЫЙ АВТОМАТ

На рис. 3.9 представлен детерминированный конечный автомат (ДКА) **D**.

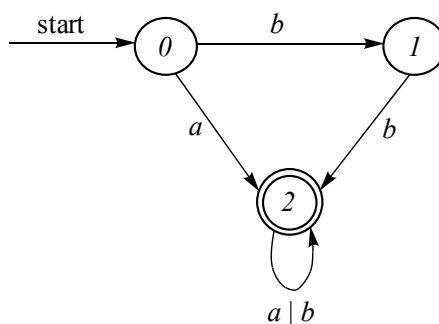


Рис. 3.9. Детерминированный конечный автомат

Автомат **D**, как легко убедиться, «путешествуя» по графу из начального состояния 0 в конечное состояние 2, допускает тот же



язык $L(D)$, что и недетерминированный автомат N_ε . Такие автоматы называют эквивалентными.

Определение 3.11. Два автомата $A_1 = (S_1, \Sigma_1, \delta_1, s_1, F_1)$ и $A_2 = (S_2, \Sigma_2, \delta_2, s_2, F_2)$ называются эквивалентными, если они распознают один и тот же язык.

Отметим особенности D в отличие от N_ε :

- отсутствие ε -переходов;
- для каждого состояния s и входного символа t существует не более одной дуги, исходящей из s , помеченной как t .

Детерминированный конечный автомат имеет для входного символа не более одного перехода из каждого состояния. Табличное представление D отличается от N тем, что каждая запись в таблице всегда имеет одно состояние. Поэтому проще, чем в N , проверить принадлежность цепочек языку $L(D)$, так как имеется не более одного пути из начального состояния в заключительное.

Определение 3.12. Конечный автомат $D = (S, \Sigma, \delta, s_0, F)$ называется детерминированным конечным автоматом (deterministic finite automaton), если в каждом его состоянии $s \in S$ функции переходов $\delta(s, a)$, для любого входного символа $\forall a \in \Sigma$ содержит не более одного состояния, т. е.

$$\forall a \in \Sigma, \forall s \in S : \delta(a, s) = \{r\}, r \in S$$

$$\text{или } \forall a \in \Sigma, \forall s \in S : \delta(a, s) = \emptyset.$$

ДКА значительно проще НКА и с точки зрения моделирования. Приведем алгоритм моделирования ДКА на псевдокоде (C-подобном).

Пример 3.1

Входные данные: входная строка string, завершаемая символом конца файла EOF (End Of File);

D: стартовое состояние $s_0 \in S$, множество заключительных состояний F .

Выходные данные: true – если D допускает (принимает) x , false – в противном случае.

Используемые функции

Функция `move(s, c)` – возвращает состояние, в которое переходит D при входном символе c .

Функция `getchar()` – возвращает очередной сканируемый символ входной строки string.



```

void()
{
    s = s0; /*стартовое состояние – начало «путешествия»
по графу */
    c = getchar(string); /*сканирование очередного символа*/
    while(c != EOF)
    {
        s = move(s, c); /*переход в новое состояние под управ-
лением «с»*/
        c = getchar(string);
    }
    if (s ∈ F)
        return (true); /*переход в заключительное состояние*/
    else
        return (false);
}

```

Приведенный алгоритм моделирования **D** достаточно прост и надежен. Он может быть применен, в общем, ко всем конечным автоматам, если недетерминированный конечный автомат можно привести к детерминированному.

Теорема 3.1. *Для любого недетерминированного конечного автомата существует эквивалентный ему детерминированный конечный автомат.*

Доказательство этой теоремы выполняется конструктивным методом. Это означает, что по заданному недетерминированному конечному автомату **N** строится эквивалентный детерминированный конечный автомат **D** по определенному конечному алгоритму.

Пусть задан недетерминированный конечный автомат $\mathbf{N} = (\mathbf{S}, \Sigma, s_0, \delta_\varepsilon, \mathbf{F})$.

Необходимо построить эквивалентный детерминированный конечный автомат $\mathbf{D} = (\mathbf{Q}, \Sigma, q_0, \delta, \mathbf{E})$. Итак, необходимо добиться, чтобы функция переходов δ в детерминированном конечном автомате **D** не содержала ε -переходов в отличие от δ_ε в недетерминированном **N**. Кроме того, каждое состояние $q \in \mathbf{Q}$ в детерминированном автомате **D** имеет единственный путь перехода в новое состояние при подаче любого входного символа c из входного алфавита $c \in \Sigma$.



Алгоритм перехода от **N** к эквивалентному **D** основан на последовательном конструировании таблицы **DT** (Deterministic Table).

Первый столбец таблицы – новые состояния $\mathbf{Q} = \{q\}$. Строим таблицу, начиная с элементов второго столбца состояния $\mathbf{U} = \{u\}$, в которые может перейти детерминированный конечный автомат **D** под воздействием входного символа $c \in \Sigma$. Причем в детерминированном автомате по определению каждый элемент таблицы **DT** $[q, c]$ определен одним состоянием $\mathbf{U} \in \mathbf{Q}$.

Структура новой таблицы, определяющей новую функцию переходов $\delta(q, c)$, показана на рис. 3.10.

DT $[q, c]$:	q_0	
	Q	$\delta(\mathbf{T}, c) = \mathbf{U}$
	E	

Рис. 3.10. Структура **DT** $[q, c]$

Введем следующие определения и обозначения.

Определение 3.13. ε -замыканием состояния s (ε -closure) $\Xi(s)$ называется множество состояний недетерминированного конечного автомата **N**, достижимых из состояния $s \in \mathbf{S}$ только по ε -переходам.

Определение 3.14. ε -замыканием множества состояний $\mathbf{T} \subseteq \mathbf{S}$ называется множество $\Xi(\mathbf{T})$ состояний недетерминированного конечного автомата **N**, достижимых из каждого s из $\mathbf{T}$ ($\forall s \in \mathbf{T}$).

При моделировании **D** использовалась функция $\text{move}(s, c)$. Эта функция возвращает состояние, в которое переходит автомат при каждом входном $c \in \Sigma$. Для недетерминированного автомата **N** эта функция имеет тот же смысл.

Пусть $\mathbf{M}(\mathbf{T}, c)$ – множество состояний, в которые переходит автомат **N** из состояния s из $\mathbf{T}$ ($s \in \mathbf{T}$), $\mathbf{T} \subseteq \mathbf{S}$ под воздействием c . Таким образом, $\mathbf{M}(\mathbf{T}, c) = \{\text{move}(s, c)\}$, $\forall s \in \mathbf{T}$, $\forall c \in \Sigma$.

С учетом принятых обозначений приведем алгоритм перехода от **N** к **D**.

```

 $q_0 = \Xi(s_0); \mathbf{U} = \{q_0\}; \mathbf{Q} = \emptyset;$ 
while ( $\mathbf{U} \neq \emptyset$ )
{

```



```

T = U; Q = Q ∪ U; U = ∅;
  for (t ∈ T) /* Цикл по всем новым состояниям T*/
  {
    for (c ∈ Σ) /* Цикл по всем элементам словаря
Σ*/
    {
      u = Ξ (M(t, c)); δ (t, c) = u;
      if (u ∉ Q) U = U ∪ {u};
    }
  }

```

Определим начальные условия. В ДКА стартовое состояние q_0 является первым новым состоянием, которое определяется как ε -замыкание от стартового состояния НКА $\Xi(s_0)$. Другими словами, спонтанный переход в каждое новое стартовое состояние из s_0 и определяет новые стартовые состояния, определенные на множестве $\{q_0\}$.

Первоначально множество новых текущих состояний **T**, полученных на последнем шаге, является еще не рассмотренным, т. е. пустым: **T** = {∅}. Аналогично и множество состояний **Q** ДКА еще не формировалось, поэтому **Q** = {∅}. Начальным значением множества новых состояний **U** является единственное принятое начальное q_0 . Поэтому **U** = { q_0 }.

Внешний цикл (цикл 1) **while** работает до тех пор, пока есть новые состояния, полученные на очередном шаге расчета. На первом шаге внешнего цикла **U** = { q_0 }. Внутренний цикл (цикл 2) перебирает все возможные новые состояния t из **T** = **U**. Во втором внутреннем цикле (цикл 3) просматриваются все символы c из словаря **Σ**. Если в результате определения **Ξ** обнаружены новые множества, они помечаются новым состоянием q_i ($i = 0, 1, 2, \dots$), и вновь, начиная с внешнего цикла, продолжается поиск новых состояний эквивалентного детерминированного автомата.

Пример 3.2

Рассмотрим недетерминированный автомат **N**, который принимает цепочки языка **L** = {(a | b)*abb}. На рис. 3.11 приведен соответствующий НКА.

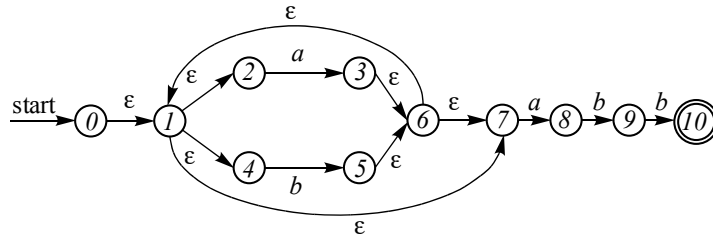


Рис. 3.11. НКА для $L = \{(a | b)^*abb\}$

Определим стартовое состояние q_0 эквивалентного детерминированного конечного автомата:

$$q_0 = \Xi(s_0) = \{0, 1, 2, 4, 7\}; \quad T = \{\emptyset\}; \quad Q = \{\emptyset\}; \quad U = \{q_0\}.$$

Начало внешнего цикла (цикла 1)

$$T = U = \{q_0\}; \quad Q = \{q_0\} \cup \{\emptyset\}; \quad U = \{\emptyset\};$$

Начало внутреннего цикла (цикла 2)

Начало внутреннего цикла (цикла 3)

$$\forall t \in T, T = \{q_0\}.$$

$$c = a \left\{ \begin{array}{l} M(t, a) = M(q_0, a) = M(\{0, 1, 2, 4, 7\}, a) = \{3, 8\} \\ \Xi(M(T, a)) = \Xi(\{3, 8\}, a) = \Xi(3, a) \cup \Xi(8, a) \\ \Xi(\{3\}) = \{1, 2, 3, 4, 6, 7\} \\ \Xi(\{8\}) = \{8\} \\ \Xi(\{3, 8\}) = \{\underline{1, 2, 3, 4, 6, 7, 8}\} - q_1 \text{ новое состояние,} \\ \text{отличное от } q_0 \end{array} \right.$$

$$\delta(t, a) = \delta(q_0, a) = q_1$$

$$U = U \cup \{q_1\} = \{q_1\}$$

$$c = b \left\{ \begin{array}{l} M(T, b) = M(q_0, b) = M(\{0, 1, 2, 4, 7\}, b) = \{5\} \\ \Xi(\{5\}) = \{\underline{1, 2, 4, 5, 6, 7}\} - \text{новое множество, которое} \\ \text{обозначим } q_2 \notin Q \end{array} \right.$$

$$\delta(t, b) = \delta(q_0, b) = q_2$$

$$U = U \cup \{q_2\} = \{q_1, q_2\}.$$



Конец цикла 3 ($U \neq \emptyset$) (выполнены две итерации по символам словаря a, b). Конец цикла 2 с одной итерацией по одному новому состоянию $t = q_0$.

Начало внешнего цикла (цикла 1), поскольку на предыдущем шаге получены новые состояния $U = \{q_1, q_2\}$.

Определяем очередную группу новых состояний $T = U = \{q_1, q_2\}$, общее множество состояний D становится $Q = \{q_0\} \cup U = \{q_0, q_1, q_2\}$. Сбрасываем стек новых состояний: $U = \emptyset$. Проверяем все ε -замыкания нового множества состояний $T \subseteq S$ для всех словарных символов a, b .

Начало внутреннего цикла (цикла 2)

Начало внутреннего цикла (цикла 3)

$$T = q_1 \left\{ \begin{array}{l} c = a \left\{ \begin{array}{l} M(T, a) = M(\{q_1, q_2\}, a) \\ M(\{1, 2, 3, 4, 6, 7, 8\}, a) = \{3, 8\} \\ u = \Xi(\{3, 8\}) = q_1; \delta(q_1, a) = q_1 \end{array} \right. \\ c = b \left\{ \begin{array}{l} M(q_1, a) = M(\{1, 2, 3, 4, 6, 7, 8\}, b) = \{5, 9\} \\ \Xi(\{5, 9\}) = \{1, 2, 4, 5, 6, 7, 9\} - q_3 / * \text{новое состояние} \end{array} \right. \end{array} \right.$$

$$\delta(q_1, b) = q_3; u = q_3; i! \in Q \Rightarrow U = \{\emptyset\} \cup \{q_3\}$$

Конец внутреннего цикла (цикла 3)

Начало внутреннего цикла (цикла 2)

$$T = q_2 \left\{ \begin{array}{l} c = a \left\{ \begin{array}{l} M(q_2, a) = M(\{1, 2, 4, 5, 6, 7\}, a) = \{3, 8\} \\ M(T, a) = \{3, 8\}; \Xi(3, 8) - q_1 / * \text{старое состояние} \\ u = q_1; \delta(q_2, a) = q_1 \end{array} \right. \\ c = b \left\{ \begin{array}{l} M(q_2, b) = M(\{1, 2, 4, 5, 6, 7\}, b) = \{5\} \\ \Xi(\{5\}) = \{1, 2, 4, 5, 6, 7\}; u = q_2; \\ \delta(q_2, b) = q_2 / * \text{старое состояние} \end{array} \right. \end{array} \right.$$

Конец внутреннего цикла (цикла 3)

Конец внутреннего цикла (цикла 2)



Определяем очередную группу новых состояний $T = U = \{q_3\}$, общее множество состояний D становится $Q = Q \cup U = \{q_0, q_1, q_2, q_3\}$. Сбрасывается стек новых состояний: $U = \emptyset$.

Проверяем все ε -замыкания нового множества состояний $T \subseteq S$ для всех словарных символов a, b .

Начало внутреннего цикла по одному новому состоянию

$T = \{q_3\}$, (цикла 2):

Начало внутреннего цикла для всех словарных символов

a, b (цикла 3)

$$c = a \begin{cases} M(t, a) = M(q_3, a) = M(\{1, 2, 4, 5, 6, 7, 9\}, a) = \{8\} \\ U = \Xi(\{3, 8\}) = q_1 - \text{старое состояние} \\ \delta(q_3, a) = q_1 \end{cases}$$

$$c = b \begin{cases} M(t, b) = M(q_3, b) = M(\{1, 2, 4, 5, 6, 7, 9\}, b) = \{5, 10\} \\ u = \Xi(\{5, 10\}) = \{1, 2, 4, 5, 6, 7, 9, 10\} q_4 - \text{старое состояние} \\ \delta(q_3, b) = q_4 \\ U = \{u\} = \{q_4\} \end{cases}$$

Конец внутреннего цикла (цикла 3)

Конец внутреннего цикла (цикла 2)

Определяем очередную группу новых состояний $T = U = \{q_4\}$, общее множество состояний D становится $Q = Q \cup U = \{q_0, q_1, q_2, q_3, q_4\}$. Сбрасывается стек новых состояний: $U = \emptyset$.

Проверяем все ε -замыкания нового множества состояний $T \subseteq S$ для всех словарных символов a, b .

Начало внутреннего цикла по одному новому состоянию $T = \{q_4\}$, (цикла 2)

Начало внутреннего цикла для всех словарных символов a, b (цикла 3)

$$c = a \begin{cases} M(q_4, a) = M(\{1, 2, 4, 5, 6, 7, 10\}, a) = \{3, 8\} \\ u = \Xi(\{3, 8\}) = q_1 / * \text{старое состояние} \\ \delta(q_4, a) = q_1 \end{cases}$$



$$c = b \begin{cases} \mathbf{M}(q_4, b) = \mathbf{M}(\{1, 2, 4, 5, 6, 7, 10\}, b) = \{5\} \\ u = \Xi(\{5\}) = q_2 \text{ /* старое состояние} \\ \delta(q_4, b) = q_2 \end{cases}$$

Конец внутреннего цикла (цикла 3)

Конец внутреннего цикла (цикла 2)

Конец внешнего цикла (цикла 1)

Так как стек новых состояний на очередном шаге поиска оказался пустым $U = \emptyset$, на этом одновременно заканчивается вся программа поиска новых состояний Q и функции переходов $\delta(q, c)$, $\forall q \in Q$, $\forall c \in \Sigma$ для эквивалентного детерминированного автомата **D**.

Итак, окончательная таблица детерминированного автомата **D** (табл. 3.3), состояние – входной символ, соответствующий новой функции переходов δ .

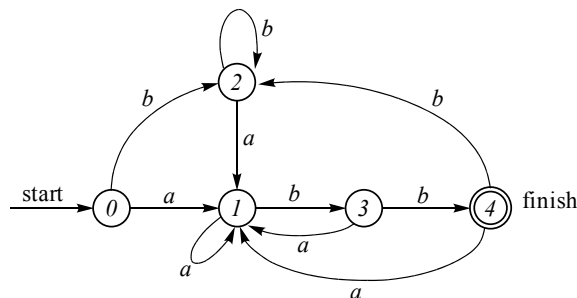
Таблица 3.3

Детерминированный автомат **D**

Состояние Q	Функция δ	
	a	b
q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	q_4
q_4	q_1	q_2

Новый детерминированный автомат представлен на рис. 3.12 (для упрощения состояния автомата пронумерованы индексами q_i , $i = 0, 4$).

Рис. 3.12. Эквивалентный ДКА для $L = \{(a | b)^*abb\}$





Приведем алгоритм моделирования НКА, пользуясь теми же обозначениями, что и раньше в алгоритме перехода от НКА к ДКА ($\mathbf{N} - \mathbf{D}$).

3.6. МОДЕЛИРОВАНИЕ НКА

Дано: недетерминированный конечный автомат $\mathbf{N} = (\mathbf{S}, \Sigma, s_0, \delta, \mathbf{F})$.

Входная цепочка x заканчивается символом конца файла **eof**.

Стартовое состояние автомата – s_0 , множество заключительных состояний – $\mathbf{F}$.

Задача: получить **true**, если $x \in \mathbf{L}(\mathbf{N})$, или **false**, если $x \notin \mathbf{L}(\mathbf{N})$. Другими словами, $\mathbf{N}$ либо допускает входную цепочку и при этом сообщает **true**, либо не допускает и при этом сообщает **false**.

```
N() /* процедура моделирования НКА*/
{
  S =  $\Xi(s_0)$ ; /* Определение  $\varepsilon$ -замыканий состояния  $s_0$  */
  c=getchar(); /* Сканирование первого символа из входного
потокa  $x^*$ */
  while (c  $\neq$  eof ) do
    { S =  $\Xi(M(S))$ ;
      c=getchar();
    }
  if S  $\cap$  F  $\neq \emptyset$  then
    return (true); /*  $\mathbf{N}$  допускает цепочку  $x \in \mathbf{L}(\mathbf{N})$  */
  else return (false); /*  $x \notin \mathbf{L}(\mathbf{N})$ 
}
```

В приведенном алгоритме моделирования НКА (в отличие от моделирования ДКА) разница состоит в дополнительном расчете ε -замыканий множества состояний $\mathbf{M}$. Стоимость этих расчетов порой значительно меньше, чем переход от НКА к ДКА. Затраты на реализацию приведенного алгоритма составляют $O(|x| * n)$, где $|x|$ – длина входной цепочки; n – количество состояний.

3.7. МИНИМИЗАЦИЯ ДКА

Ранее были введены понятия эквивалентности конечных автоматов, т. е. автоматов, которые распознают один и тот же язык. Очевидно, оправданной с точки зрения эквивалентности является



задача минимизации конечных автоматов. Суть минимизации состоит в том, чтобы добиться минимального количества состояний и дуг автомата так, чтобы исходный и минимизированный автоматы были эквивалентны.

Определение 3.15. Два состояния p и q из множества PQ множества состояний S , называются различимыми, если

$$\delta(p, c) \neq \delta(q, c), \quad c \in \Sigma, \quad p, q \in PQ, \quad PQ \subseteq S.$$

В противном случае состояния p и q являются неразличимыми.

Другими словами, если входную ленту с цепочкой ω и указателем n ($c = (\omega, n)$, $c \in \Sigma$) подать на вход D , то, проходя через подмножество различных состояний $PQ \subseteq S$, автомат всегда будет иметь разные конфигурации (s, c) , $s \in PQ$.

Определение 3.16. Состояние называется мертвым, если, не являясь заключительным, оно для всех входных символов может иметь только переходы в себя, т. е.

$$\delta(q, c) = P, \quad \forall c \in \Sigma, \quad P = \{p = q, \emptyset\}.$$

Мертвое состояние изображается как показано на рис. 3.13.

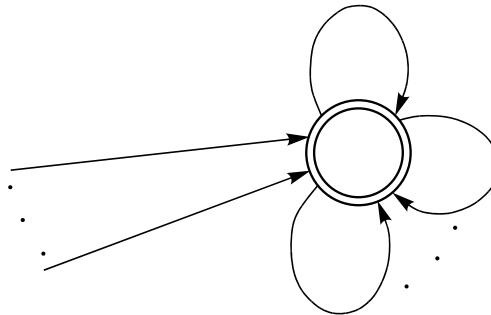


Рис. 3.13. Мертвое состояние

Определение 3.17. Недостижимыми называются все те состояния $\{q\}$, которые не достижимы из s_0 :

$$\delta(s_0, \omega) \neq \{q\}, \quad \omega \in \Sigma^*.$$

В этом определении цепочки ω являются цепочками словаря Σ . «Путешествие» по состояниям автомата D под воздействием словарных символов $c \in \omega$ (возможно, ω и не принимается детерминированным автоматом D) никогда не приведет во множество недостижимых состояний $\{q\}$.



АЛГОРИТМ МИНИМИЗАЦИИ

Исходные данные: детерминированный автомат $\mathbf{D} = \{S, \delta, s_0, F, \Sigma\}$.

Необходимо получить: $\mathbf{D}' = \{Q, \delta', q_0, E, \Sigma\}$.

1. Удалить все мертвые и недостижимые состояния и получить $\bar{S}$.
2. Выполнить разбиение $\bar{S}$ на два подмножества $\bar{S} = \bar{S} - F$ и F .
3. Провести разбиение множеств $\bar{S}$ и F на подмножества $S_1 \dots S_k, F_1 \dots F_m$ ($k = 1, 2, \dots; m = 1, 2, \dots$) причем все подмножества S_i ($i = \overline{1, k}$) таковы, что включают только различные состояния. В худшем случае может оказаться так, что $S_1 = \{s_1\}, \dots, S_k = \{s_k\}$, т. е. каждое подмножество разбиений множества S состоит из единственного состояния $s \in S$. В этом случае оптимальным является исходный автомат; перейти к п. 5.

4. Выбрать по одному состоянию из всех подмножеств S_i ($i = \overline{1, k}$) в качестве представителя нового состояния в $\mathbf{D}'$. Пусть $s \in S_i$ ($i = \overline{1, k}$) является представителем нового состояния в подмножестве S_i . Предположим, что для входного символа $c \in \Sigma$ существует переход $\delta(s, c) = t, \forall c$, причем $t \in S_j$ ($i \neq j, 1 \leq j \leq k$). Тогда t сделать представителем нового состояния для подмножества различных состояний S_j .

5. Стартовым q_0 сделать то подмножество состояний S_r , куда входит S_0 ($q_0 = S_0$). Заключительным q_F сделать любое состояние из F .

Иллюстрацией применения алгоритма минимизации является переход от автомата $\mathbf{D}$ (рис. 3.14) к эквивалентному оптимальному $\mathbf{D}'$ (рис. 3.15).

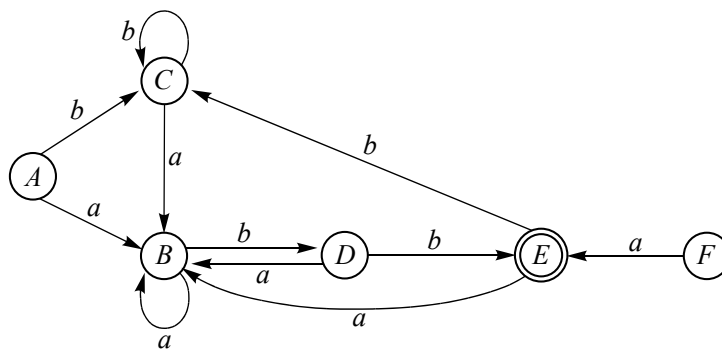


Рис. 3.14. Исходный неоптимальный детерминированный конечный автомат $\mathbf{D}$

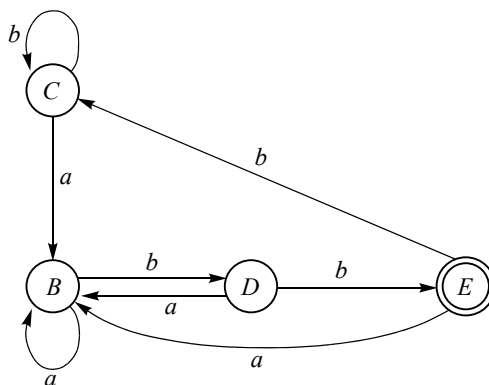


Рис. 3.15. Эквивалентный оптимальный (минимальный) D'

3.8. МП-АВТОМАТЫ

Рассмотрим еще один класс конечных автоматов. Такие автоматы называются магазинными автоматами, или автоматами с магазинной памятью (стеком). Сокращенно будем называть их МП-автоматы. Отличие МП-автоматов от обычных, уже рассмотренных, состоит в следующем.

Детерминированные и недетерминированные конечные автоматы принимают только автоматные языки и являются одновременно механизмом их порождения (генерации). Для КС-языков, порождаемых КС-грамматикой, механизма моделирования конечных ДКА и НКА недостаточно.

Приведем простой пример [2]. Пусть цепочка некоторого языка L имеет вид

$$L = \{w^n \alpha w_1^n \mid w \in W, \alpha \in V_T^*\},$$

где w – открывающая круглая скобка; w_1 – закрывающая круглая скобка; α – выражение без скобок.

Такие цепочки могут встречаться в арифметических выражениях языков *Fortran*, *C/C++*, *Pascal* и др. Условие баланса скобок сразу ограничивает применение схем ДКА и НКА к моделированию таких цепочек языка арифметики со сбалансированными скобками. Другими словами, в моделирующем механизме необходимо вести учет количества левых и правых круглых скобок. В МП-автоматах



для возможности реализации подобных проблем вводится дополнительная память (магазин, или стек) для хранения предыстории.

Стековый механизм нашел широкое применение в языковых процессорах, как будет показано ниже. Принцип организации магазинной памяти весьма прост. Это память (рис. 3.16) для хранения по принципу «первым вошел – последним вышел».

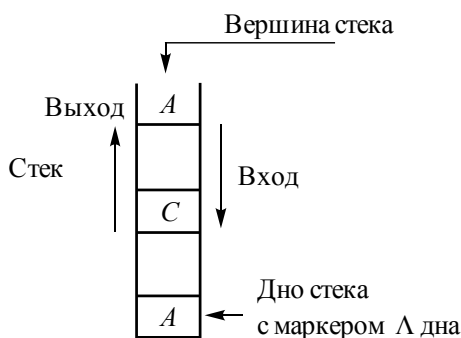


Рис. 3.16. Стековый механизм

Магазинные автоматы, известные также как автоматы с магазинной памятью (стеком), или как МП-автоматы, формально определяются следующим образом.

Определение 3.18. МП-автомат – это семерка $P = (S, \Sigma, M, \delta, s_0, Z_0, F)$, где S – конечное множество состояний; Σ – конечный входной алфавит; M – конечный алфавит магазинных символов; δ – функция перехода, отображение множества $S \times (\Sigma \cup \{\epsilon\}) \times M$ на множество конечных подмножеств множества $S \times M^*$; $s_0 \in S$ – начальное состояние управляющего устройства; $Z_0 \in M$ – символ, находящийся в магазине в начальный момент времени (начальный символ); $F \subseteq S$ – множество заключительных состояний.

Определение 3.19. Конфигурацией МП-автомата P называется тройка $(s, w, \alpha) \in S \times \Sigma^* \times M^*$, где s – текущее состояние управляющего устройства; w – неиспользованная часть входной цепочки (если $w = \epsilon$, то считается, что вся входная цепочка прочитана); α – содержимое магазина (самый левый символ цепочки α считается верхним символом магазина; если $\alpha = \epsilon$, то магазин считается пустым).

Именно по этой причине алгоритм моделирования МП-автомата в общем случае может быть не конечным, так как при окончании



входной цепочки автомат может осуществлять спонтанные (в том числе и бесконечные) переходы. При этом не исключена ситуация заикливания.

На рис. 3.17 каждый шаг (такт) процесса обработки входной цепочки над словарем Σ характеризуется функцией переходов δ , которая учитывает состояние s , вершину стека α , текущий символ входной цепочки c .

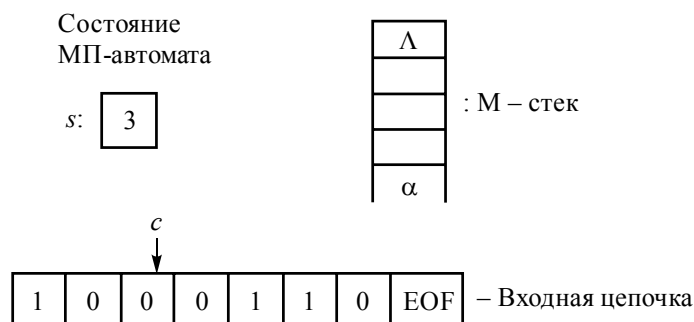


Рис. 3.17. Конфигурация автомата

На каждом шаге работы МП-автомат может либо занести что-то в магазин, либо снять какие-то значения с его вершины. Отметим, что МП-автомат может продолжать работать в случае окончания входной цепочки, но не может продолжать работу в случае опустошения магазина.

Доказано, что класс языков, распознаваемых МП-автоматами, в точности совпадает с классом языков, задаваемых КС-грамматиками. Доказательство этого можно найти, например, в [11].

Пример 3.2

Рассмотрим магазинный автомат, распознающий язык

$$L(G_1) = \{0^n 1^n \mid n \geq 0\}.$$

Пусть $P = (\{s_0, s_1, s_2\}, \{0, 1, e\}, \{1, \Lambda\}, \delta, s_0, \Lambda = \varepsilon, \{s_0\})$, где $\delta(s_0, 0, \varepsilon) = \{(s_1, 0\varepsilon)\}$; $\delta(s_1, 0, 0) = \{(s_1, 00\varepsilon)\}$; $\delta(s_1, 1, 0) = \{(s_2, 0\varepsilon)\}$; $\delta(s_2, 1, 0) = \{(s_2, \varepsilon)\}$; $\delta(s_3, e, \varepsilon) = \{(s_0, \varepsilon)\}$.

Работа автомата заключается в копировании в магазин начальных нулей из входной цепочки и последующем устранении по одному нулю из магазина на каждую прочитанную единицу.



Таким образом, дополнительная память (стек) должна обеспечить запоминание количества записанных нулей и затем отследить такое же количество единиц во входной цепочке.

3.9. ДЕТЕРМИНИРОВАННЫЕ МП-АВТОМАТЫ

МП-автоматы обладают одним существенным недостатком – они недетерминированы по своей природе. На практике хотелось бы иметь дело с детерминированными автоматами, в которых в каждой конфигурации возможно не более одного такта.

Управление памятью осуществляется функцией перехода δ , которая либо переводит МП-автомат в новое состояние $s \in S$, либо оставляет его в прежнем в зависимости от входного символа и состояния вершины стека.

Механизм управления МП-автоматом через $\delta(s, \alpha, c)$ удобно иллюстрировать таблицами, представленными на рис. 3.18. Эти таблицы называют управляющими [2].

Исходным состоянием является пустой стек $\alpha = \varepsilon$ (дно стека совмещено с его вершиной). Указатель чтения символов входной цепочки $n = 1$, состояние автомата – s_0 .

Пусть входная строка ограничена маркером e (eof). Состояние s_1 – это состояние записи или добавления (add) нулей в стек.

S	$s_0 :$	add s_1 $n++$	error	true	ε	M
	$s_1 :$	add s_1 $n++$	del s_2 $n++$	false	ε	
	$s_2 :$	add s_1 $n++$	del s_2 $n++$	false	ε	
		false	false	true s_0	ε	

Рис. 3.18. Механизм управления МП-автоматом

В состояние s_1 МП-автомат переходит под управлением первого нуля во входной строке из s_0 и последующем чтении нулей



из входной строки. Как только во входной строке появится 1, автомат переходит в состояние s_2 , при этом из памяти (стека) удаляется (del) 0, который к этому времени (такту) находится в вершине ($\alpha = 0$). Чтение 1 из входной строки продолжается до тех пор, пока не будет достигнут конец файла (e). При этом каждый раз с вершины стека снимается (удаляется) очередной 0-символ. Если строка $x \in L(G_1)$, то при достижении e в вершине стека окажется ε , т. е. автомат вновь перейдет в s_0 . Поэтому если в s_0, s_1 нет необходимости рассматривать $\alpha = \varepsilon$, то в s_2 следует рассмотреть случай, когда стек пустой ($\alpha = \varepsilon$) и непустой ($\alpha \neq \varepsilon$). Если автомат не принимает x , то в этом случае диагностируется **false**, иначе $x \in L(G_1)$ и выдается сообщение **true**. На рис. 3.19 показано состояние входной ленты (цепочки), стек и состояния **P** при обработке цепочки 0011e от начального до конечного символа.

0.	ε	$[s_0]$	0011e	
1.	$\varepsilon 0$	$[s_1]$	011e	
2.	$\varepsilon 00$	$[s_1]$	11e	
3.	$\varepsilon 0$	$[s_2]$	1e	
4.	ε	$[s_2]$	e	
5.	ε	$[s_0]$	e	true

Рис. 3.19. Состояния цепочки

Определим итерацию функции переходов δ^* для $P = (S, \Sigma, M, \delta, s_0, Z_0, F)$ как отображение декартова произведения множества состояний (**S**) на итерации множества словаря (Σ^*) и множества алфавита магазинных символов (M^*) в декартово произведение **S** и M^* , т. е.

$$\delta^* : S \times \Sigma^* \times M^* \rightarrow S \times M^*.$$

Тогда можно формально определить язык $L(P)$, который распознает МП-автомат **P**.

Определение 3.20. Конечный МП-автомат $P = (S, \Sigma, M, \delta, s_0, Z_0, F)$ допускает входную цепочку $\alpha \in \Sigma^*$, если α переводит **P** из начального (s_0) состояния с начальным магазинным символом в памяти (стеке) Z_0 в одно из заключительных состояний **F**, притом что магазинная память в результате обработки α вновь стала Z_0 :

$$L(P) = \{\alpha \mid \alpha \in \Sigma^*, \delta^*(s_0, \alpha, Z_0) \in (F \times Z_0)\}.$$



Определение 3.21. МП-автомат $P = (S, \Sigma, M, \delta, s_0, Z_0, F)$ называется детерминированным, если для каждого $s \in S$ и $Z \in M$ верно одно из следующих утверждений:

- $\delta(s, \alpha, Z)$ содержит не более одного элемента для каждого $\alpha \in \Sigma$ и $\delta(s, \varepsilon, Z) = \emptyset$;
- $\delta(s, \alpha, Z) = \emptyset$ для всех $\alpha \in \Sigma$ и $\delta(s, \varepsilon, Z)$ содержит не более одного элемента.

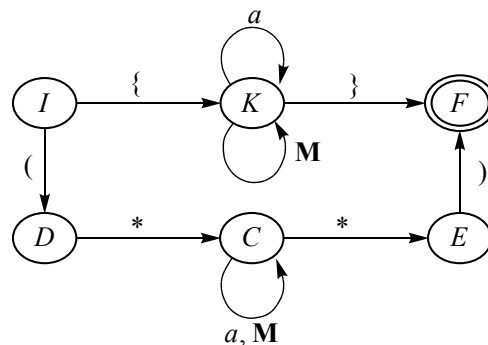
К сожалению, детерминированные МП-автоматы описывают только подмножество всего класса КС-языков – такое подмножество называется детерминированными КС-языками. Этот класс языков называют также $LR(k)$ -грамматиками, так как они могут быть однозначно разобраны путем просмотра цепочки слева направо с заглядыванием вперед не более чем на k символов.

Класс $LR(k)$ -грамматик чрезвычайно важен, так как на нем и некоторых его разновидностях основано большинство современных средств синтаксического разбора. В частности, эффективными средствами разбора для класса $LR(k)$ -грамматики являются методы снизу вверх или справа налево, которые будут рассмотрены далее.

УПРАЖНЕНИЯ

1. Пусть множество $M = \{ (,), *, \{, \} \}$, a – любой алфавитно-цифровой символ, кроме символов из M . Тогда представленный на рисунке недетерминированный конечный автомат принимает цепочки языка комментариев в *Borland Pascal* $N = \{S, I, \delta, F, \Sigma\}$, где I – начальное состояние; F – заключительное состояние; $\Sigma = \{a\}$ – словарь; δ – функции переходов, определенные графом.

2.





Задание:

- а) преобразовать N к D ;
- б) минимизировать D ;
- в) написать алгоритм моделирования N ;
- г) написать алгоритм моделирования D .

3. Задан конечный автомат

$A = \{\{S, R, Z\}, \{a, b\}, S, \{Z\}, \delta(S, a) = \{S, R\}, \delta(R, b) = \{R\}, \delta(R, a) = \{Z\}\}$,

- а) построить N ;
- б) преобразовать N к D ;
- в) минимизировать D .

4. Построить конечный автомат по продаже пива с возвратом сдачи [16]. Автомат принимает монеты достоинством 5 и 10 пенсов, кружка пива стоит 15 пенсов. Кроме отверстий для приема монет и выдачи у автомата есть кнопки «Налей» и «Сброс».

5. Построить конечный автомат, выбрасывающий пробелы в тексте.

6. Пусть грамматика $G_1[<число>]$ определена productions

$P_1: <число> \rightarrow <знак> 0 | <знак> 1 | <часть> . | <число> 0 | <число> 1$

$<часть> \rightarrow <знак> 0 | <знак> 1 | <часть> 0 | <часть> 1$

$<знак> \rightarrow \varepsilon | + | -$.

Построить конечный автомат и преобразовать его к детерминированному.

7. Пусть грамматика $G_2[<число>]$ определена productions

$P_2: <число> \rightarrow <часть> . | <осн.> 0 | <осн.> 1 | <часть> 0 | <часть> 1$

$<осн.> \rightarrow <часть> . | <осн.> 0 | <осн.> 1$

$<часть> \rightarrow 0 | 1 | <знак> 0 | <знак> 1 | <часть> 0 | <часть> 1$

$<знак> \rightarrow + | -$.

Построить конечный автомат и преобразовать его к детерминированному.

8. Построить недетерминированные конечные автоматы для приведенных ниже языков $L(A)$. Показать последовательность перемещений для каждого автомата при разборе входной строки *ababbab*:

- а) $L(A) = (a | b)^*$;
- б) $L(A) = (a^* | b^*)^*$;
- в) $L(A) = ((\varepsilon | a) b^*)^*$;
- г) $L(A) = (a | b)^* abb(a | b)^*$.



9. Преобразовать недетерминированные конечные автоматы, полученные из приведенных в упражнении 7(в) выражений, в детерминированные. Показать последовательность перемещений для каждого автомата при разборе входной строки *ababbab*.

10. Построить детерминированный конечный автомат по диаграмме переходов для приведенных токенов.

Регулярное выражение	Токен	Атрибут-значение
ws	—	—
if	if	—
then	then	—
else	else	—
id	id	Указатель на запись в таблице
num	num	Указатель на запись в таблице
<	relop	LT
<=	relop	LE
=	relop	EQ
>	relop	NE

11. Строка Фибоначчи определяется следующим образом:

$$s_1 = b$$

$$s_2 = a$$

$$s_k = s_{k-1}s_{k-2} \text{ для } k > 2.$$

Например, $s_3 = ab$, $s_4 = aba$, $s_5 = abaab$.

1. Чему равна длина s_n ?

2. Построить ДКА, допускающий $L(A) = s_6^*$.

4. ЯЗЫКИ И РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ

С точки зрения задания языка, как уже отмечалось выше, важную роль вместе с порождающими грамматиками и автоматами играют регулярные выражения (regular expression). Определение регулярного выражения созвучно с определениями арифметических и логических выражений. Действительно, все эти выражения строятся по строгим правилам своей аксиоматики. Так, над арифметическими выражениями существует принятая аксиоматика алгебры, в которой введены приоритеты выполнения арифметических операций и способ их переопределения с помощью круглых скобок. Операции в арифметических выражениях обладают свойствами коммутативности, ассоциативности и др. Точно так же для регулярных выражений ниже будут введены определенные аксиомы, следствием которых являются эквивалентные преобразования этих выражений.

Регулярные выражения формально задают определенные классы объектов по аналогии с объектно-ориентированной концепцией. Классами в регулярных выражениях являются множества цепочек или языки, объектами – сами цепочки из символов, определенных словарем.

Регулярные выражения являются более компактным способом определения подкласса (подмножества) языков, чем мощный аппарат порождающих грамматик. Словарь операций регулярных выражений ограничен операциями конкатенации, «или» ($\mid$) и замыкания Клини (*).

В качестве предметного обозначения некоторого объекта из языка приведем пример идентификатора, который представляет собой определенным образом построенную цепочку символов – букв (**б**) или цифр (**ц**), причем первым символом в цепочке является буква, а продолжает ее последовательность букв и цифр.



Порождающие грамматики в этом случае определяют эту конструкцию следующим образом:

$G[\langle \text{идентификатор} \rangle]$:

$V_T = \{\mathbf{b}, \mathbf{ц}\}$, $V_N = \{\langle \text{идентификатор} \rangle, \langle \text{хвост идентификатора} \rangle, \langle \text{Б Ц} \rangle\}$

P :

- 1) $\langle \text{идентификатор} \rangle \rightarrow \mathbf{b} \langle \text{хвост идентификатора} \rangle$
- 2) $\langle \text{хвост идентификатора} \rangle \rightarrow \langle \text{Б Ц} \rangle \langle \text{хвост идентификатора} \rangle \mid \Lambda$
- 3) $\langle \text{Б Ц} \rangle \rightarrow \mathbf{b} \mid \mathbf{ц}$.

Язык, порождаемый грамматикой $L(G[\langle \text{идентификатор} \rangle])$, и является идентификатором. Этот же язык идентификаторов можно задать проще в виде простой формулы

$$\mathbf{R}_1 = \mathbf{b} (\mathbf{b} \mid \mathbf{ц})^*,$$

соответственно язык, определенный формулой или регулярным выражением $\mathbf{R}_1$, определен как множество $L(\mathbf{R}_1) = \{\mathbf{b} (\mathbf{b} \mid \mathbf{ц})^*\}$, где, как и прежде, знак $*$ означает итерацию над буквой и (или) цифрой или замыкание Клини.

Такое простое задание языка не требует компонентов V_N , P , Z и в общем самой грамматики $G[Z]$. Необходимо только задать словарь $\Sigma = V_T$ и само выражение (формулу) $\mathbf{R}$, определяющее язык.

Так же просто можно определить целые без знака – как бесконечную итерацию цифр позиционной системы исчисления $\mathbf{R}_2 = a b^*$, где a – все цифры без нуля $[1-9]$, b – все цифры $[0-9]$. Соответственно язык целых представлен очень просто в виде $L(\mathbf{R}_2) = \{a b^*\}$.

Однако настолько просто можно задать далеко не все синтаксические конструкции языка в отличие от порождающих грамматик, для которых нет ограничений в иерархии (классификации) Хомского. Ограниченность языковых конструкций, порождаемых регулярными выражениями, и есть плата за простоту представления выражений. Несмотря на эти ограничения, регулярные выражения широко используются, особенно при лексическом анализе языков программирования.

Пример 4.1

Рассмотрим выражения $\mathbf{Q} = a^n b^n$, $n \geq 1$.

Близко к нему регулярное выражение $\mathbf{R} = a^* b^*$, однако $\mathbf{Q} \neq \mathbf{R}$. Это очевидно, потому что множество цепочек, порождаемых $\mathbf{Q}$, есть язык $L_{\mathbf{Q}} = \{\epsilon, a, b, aa, bb, ab, \dots\}$, т. е. совершенно другой язык.



Регулярные выражения широко используются во всех процессорах, в которых требуется шаблонно-ориентированный поиск, в первую очередь в лексических анализаторах. Для выделения шаблонов (идентификаторов, числовых констант, ключевых слов и др.) используют регулярные выражения. Например, в системах генераторов лексических анализаторов LEX [13], FLEX & BISON [101], ANTLR4 [99] все шаблоны определяются регулярными выражениями и генерируется эффективный конечный автомат, который распознает шаблоны.

Технология использования регулярных выражений при разработке лексических анализаторов используется и в других областях – таких, например, как языки запросов и информационно-поисковые системы [13].

Аппарат регулярных выражений и шаблонного поиска на их основе нашел применение в различных областях. Например, в [20] использован генератор лексических анализаторов на основе регулярных выражений для поиска дефектов печатных плат. Платы сканировались, и результат сканирования преобразовывался в строку отрезков – шаблонов. Лексический анализатор просматривал отрезки в поисках шаблона, соответствующего определенному дефекту.

Наконец, аппарат регулярных выражений используется в языках программирования лексеров в системах генерации FLEX & BISON и ANTLR4, представленных в разделе 7 настоящего учебного пособия.

Перейдем к строгим определениям.

4.1. РЕГУЛЯРНЫЕ МНОЖЕСТВА И ЯЗЫКИ

В этом разделе рассмотрим множества цепочек над конечным словарем, которые могут быть заданы формально, или, другими словами, посредством регулярных выражений. Такие множества будут называться регулярными.

Определение 4.1. Пусть P и Q – множества цепочек. Определим четыре операции над множествами P и Q :

- 1) объединение: $P \cup Q = \{\alpha \mid \alpha \in P \text{ или } \alpha \in Q\}$;
- 2) конкатенация: $PQ = \{pq \mid p \in P, q \in Q\}$;
- 3) итерация: $P^* = P^0 \cup P^1 \cup \dots \cup P^n, n \rightarrow \infty$ (замыкание Клини);
- 4) усеченная итерация: $P^+ = P^1 \cup P^2 \cup \dots \cup P^n, n \rightarrow \infty$ (позитивное замыкание).

**Пример 4.2** [13]

Пусть множество L – это множество букв (заглавных и прописных) латинского алфавита, $L = \{A, \dots, Z, a, \dots, z\}$, и D – множество цифр, $D = \{0, 1, \dots, 9\}$. Множества L и D могут рассматриваться по-разному. С одной стороны, это словари соответственно букв и цифр. С другой стороны, так как символы можно рассматривать как цепочку символов единичной длины, множества L и D – это множества цепочек, над которыми справедливы введенные определения 1–4.

Приведем содержательный смысл операций над множествами применительно к простым множествам L и D .

Объединение $L \cup D$ – новое множество букв и цифр $\{A, \dots, Z, a, \dots, z, 0, 1, \dots, 9\}$.

Конкатенация LD – множество строк длиной 2 с обязательным первым символом – буквой из L , вторым – цифрой из D .

L^3 – множество строк из всевозможных комбинаций букв из L , причем длина всех строк равна 3.

L^* – множество строк из всевозможных комбинаций букв из L всевозможной длины, включая пустую строку ϵ .

D^+ – множество всех строк одной или нескольких цифр.

Определение 4.2. Класс регулярных множеств над конечным словарем V определяется следующим образом:

- 1) $\emptyset$ – регулярное множество;
- 2) $\{\epsilon\}$ – регулярное множество;
- 3) $\{a\}$, $\forall a \in V$ – регулярное множество;
- 4) если P и Q – регулярные множества, то регулярными являются:

- объединение $P \cup Q$;
- конкатенация PQ ;
- итерация P^* и Q^* ;
- усеченная итерация P^+ и Q^+ ;

- 5) если множество не может быть построено путем конечного числа применения правил 1–4, оно не является регулярным.

Фактически регулярные множества, определенные как множества цепочек и построенные по вполне определенным правилам, являются языками. Введенные операции над регулярными множествами цепочек символов совпадают с операциями над языками



от порождающих грамматик. Поэтому по аналогии с определением «регулярные множества» можно употреблять термин *регулярные языки*.

4.2. РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ

Регулярные выражения формально задают регулярные множества, или регулярные языки, и неразрывно связаны с ними своими свойствами. Таким образом, если грамматики порождают языки, то регулярные языки как подмножество множества языков определяются (задаются) регулярными выражениями. Регулярное выражение строится индуктивно из более простых регулярных выражений. Каждое регулярное выражение R определяет (или задает) регулярный язык $L(R)$ (иногда пишут L_R).

Рассмотрим правила, которые определяют регулярные выражения над алфавитом Σ . Пусть R и S – регулярные выражения, определяющие соответственно языки (множества цепочек) $L(R)$ и $L(S)$. Тогда

$R = \varepsilon$ – регулярное выражение, определяющее язык $L(R) = \{\varepsilon\}$;

$R = a$ – регулярное выражение, определяющее язык $L(R) = \{a\}$;

$R | S$ – новое регулярное выражение, определяющее новый язык $L(R) \cup L(S)$;

R^* – новое регулярное выражение, определяющее новый язык $L^*(R)$.

Имеются свойства операций для преобразования регулярных выражений в эквивалентные. Эти свойства приведены в табл. 4.1 для регулярных выражений R, S, T .

Таблица 4.1

Свойства операций

№	Свойства	Комментарии
1	$R S = S R$	Оператор « » коммутативен
2	$R (S T) = (R S) T$	Оператор « » ассоциативен
3	$(RS)T = R(ST)$	Операция конкатенации ассоциативна
4	$R(S T) = RS RT$	Операция конкатенации дистрибутивна
5	$(S T)R = SR TR$	
6	$\varepsilon R = R \varepsilon = R$	Связь ε и конкатенации
7	$R = R \varepsilon$	Связь ε и объединения
8	$R^{**} = R^*$	Оператор $*$ идемпотентен



Как и в любом выражении (например, арифметическом) в регулярных выражениях определены приоритеты выполнения операций. Введенные операции имеют следующую приоритетную последовательность выполнения: первой выполняется операция итерации (наивысший приоритет), затем – конкатенации и последней – операция объединения (низший приоритет).

Пример 4.3

Пусть $\Sigma = \{a, b\}$.

1. $R = a \mid b$ – определяет язык $L(R) = \{a, b\}$.

2. $R = (a \mid b) a$ – определяет язык $L(R) = \{aa, ba\}$.

3. $R = (a \mid b)(a \mid b)$ – определяет язык $L(R) = \{aa, ab, ba, bb\}$; эквивалентное $R_1 = aa \mid ab \mid ba \mid bb$ также определяет язык $L(R_1) = \{aa, ab, ba, bb\}$.

В п. 2 круглые скобки введены для переопределения приоритетности операции, так как операция объединения $a \mid b$ будет выполняться первой и только после этого – конкатенация с символом a , хотя конкатенация имеет приоритет выше операции « $\mid$ ».

Учитывая, что $L(R) = L_1 \cup L_2$, где $L_1 = \{b^2\}$, $L_2 = \{ba^*b\} = \{b^2, bab, \dots\}$, приходим к эквивалентному регулярному выражению $R = ba^*b$.

Табл. 4.1 и введенные приоритеты операций задают алгебру операций над регулярными выражениями. Используя эту алгебру, можно, как и в алгебре арифметических выражений, выполнять эквивалентные преобразования регулярных выражений.

Пример 4.4

Выполним преобразование регулярного выражения $R = b(b \mid aa^*b)$ в соответствии с табл. 4.1. Получим

$$R = b(b \mid aa^*b) = b(b \mid a^*b) = b^2 \mid ba^*b.$$

Каждое регулярное выражение R обозначает один и только один регулярный язык $L(R)$. Однако один и тот же язык $L(R)$ может быть задан сколь угодно большим множеством регулярных выражений, определяющих этот язык. Действительно, например, если множество цепочек языка над алфавитом $\Sigma = \{a\}$ представляет собой цепочки, содержащие не менее двух словарных символов a , то все обозначенные ниже регулярные выражения, составленные в соответствии с правилами 1–4, будут обозначать один и тот же язык:

$$R_1 = aa^*a; \quad R_2 = a^*aa^*a^*; \quad R_1 = aaa^* \text{ и т. д.}$$



Все эти регулярные выражения эквивалентны с точки зрения определения языка $L(R) = \{aa, aaa, \dots, a \dots aa, \dots\}$.

Определение 4.3. Два регулярных выражения R_1 и R_2 называются эквивалентными тогда и только тогда, когда они определяют один и тот же язык, т. е. $L(R_1) = L(R_2)$.

Если использовать математическое обозначение эквивалентности « $\equiv$ », то это определение можно записать формально как $R_1 \equiv R_2 \Leftrightarrow L(R_1) = L(R_2)$.

4.3. АВТОМАТНЫЕ ГРАММАТИКИ И РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ

Рассмотрим язык $L = \{a^n b^n \mid n \geq 1\}$. Множество строк этого языка является подмножеством строк языка, определенного регулярным выражением $R = aa^*bb^*$. Действительно, в L все строки состоят только из тех последовательностей a и b , в которых символы a повторяются один и более раз, за ними следует последовательность символов b , которые точно повторяют предшествующее им количество символов a . Для $L(R)$ цепочки включают все последовательности символов a , за которыми следует b , причем в том числе и таких последовательностей, где количество может быть и не равным b . Поэтому в общем случае регулярное выражение R соответствует языку, порождаемому грамматикой $G[I]$, такому что $L(G[I]) = \{a^n b^m \mid n, m \geq 1\}$, с графом переходов, представленным на рис. 4.1.

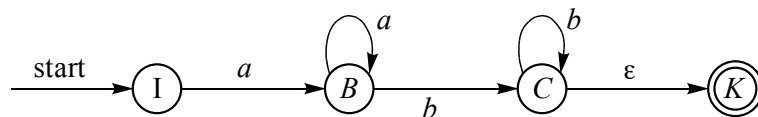


Рис. 4.1. Граф переходов

Соответствующая автоматная грамматика легко подбирается по графу $G[I]$:

- 1) $I \rightarrow aB$
- 2) $B \rightarrow aB \mid bC$
- 3) $C \rightarrow bC \mid \epsilon$.

Таким образом, автоматная грамматика $G[I]$ порождает тот же язык, что и язык $L(R)$, определенный регулярным выражением $L(R) = L(G[I])$. В этом случае можно говорить об эквивалентности



регулярного выражения $\mathbf{R}$ и автоматной грамматики $\mathbf{G}[L]$. Поэтому автоматную грамматику $\mathbf{G}[L]$ еще называют регулярной, что и было сделано в классификации Хомского [1].

ПРИМЕРЫ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ

Содержательными примерами использования регулярного выражения (РВ) являются построенные на принципах РВ современные лексические анализаторы. На понятиях РВ и регулярных определений построен язык спецификаций генератора лексических анализаторов LEX. Многие конструкции ОС UNIX используют РВ. Все шаблоны, где нет сбалансированных конструкций, могут легко представляться регулярными выражениями. Сбалансированными конструкциями в данном случае являются конструкции типа $\{a^n b^n \mid n \geq 1\}$. К таким конструкциям относится, например, инфиксная форма арифметических скобочных выражений. Другим примером сбалансированных конструкций является холеритовская константа языка *Fortran*.

Приведем конкретные примеры РВ. Рассмотрим $\mathbf{R}_1$:

$$\mathbf{R}_1 = \text{DT}^+ (. \text{DT}^+) ? (\text{E} (+ | -) ? \text{DT}^+) ? ,$$

где знак «?» соответствует умолчанию (в принятых ранее соглашениях для порождающих грамматик это соответствовало $[. \text{DT}^+]$).

Регулярное выражение $\mathbf{R}_1$ порождает по аналогии с порождающими грамматиками язык $\mathbf{L}(\mathbf{R}_1)$ числовых констант экспоненциального типа:

$$\mathbf{L}(\mathbf{R}_1) = \{ \text{DT}^+, \text{DT}^+ . \text{DT}^+, \text{DT}^+ . \text{DT}^+ \text{E} + \text{DT}^+, \dots \}.$$

Например, множество числовых констант, заданных регулярным выражением, может быть следующим:

$$\mathbf{L}(\mathbf{R}_1) = \{ 123, 123.456, 123.45\text{E} + 67, \dots \}.$$

Регулярное выражение $\mathbf{R}_2$ порождает язык идентификаторов $\mathbf{L}(\mathbf{R}_2)$:

$$\mathbf{R}_2 = \text{LT} (\text{LT} | \text{DT})^+$$

$$\mathbf{L}(\mathbf{R}_2) = \{ a, b, A, C, name, x1, \dots \}.$$

В сканере LEX DT и LT соответствуют классу цифр и букв латини и записываются как спецификации соответственно:



digit [0-9],
letter [A-Za-z].

Приведем еще один пример регулярного выражения R_3 для имен файлов, которые заканчиваются, например, сочетанием (суффиксом) «.о», причем имя может состоять из любых символов, кроме указанного суффикса. В этом случае

$$R_3 = (\wedge . | \wedge o | c)^*$$

где c – любой символ, а знак $\wedge$ означает отрицание соответствующих символов.

Примеров регулярных выражений можно привести достаточно много. Подробное описание теоретического и практического материала по этому вопросу можно найти, например, в [22, 99, 101].

4.4. РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ И КОНЕЧНЫЕ АВТОМАТЫ

Прямое соответствие между регулярными выражениями и конечными автоматами впервые было строго сформулировано в теореме Клини.

Теорема 4.1 (Теорема Клини). *Классы регулярных множеств и автоматных языков совпадают.*

Доказательство теоремы основано на установлении взаимно однозначного соответствия между регулярными выражениями R и соответствующими автоматами (в общем случае НКА, или N). Классы регулярных множеств, как уже отмечалось, вполне однозначно соответствуют языкам, которые, в свою очередь, являются производными от регулярных выражений. Учитывая, что автоматные языки однозначно связаны с конечными автоматами, для доказательства теоремы Клини достаточно показать совпадение регулярных выражений и конечных автоматов.

Переход от регулярных выражений к НКА является неоднозначной задачей. Это связано, во-первых, с тем, что одно и то же множество цепочек может быть представлено различными регулярными выражениями R_i , $i = 1, 2, \dots$. Как было показано в примере 4.4, множество цепочек $L_i(R_i)$, содержащее не менее двух символов a , может быть представлено следующими регулярными выражениями R_i :

$$R_1 = aa^*a, \quad R_2 = a^*aaa^*, \quad R_3 = aaa^*, \dots$$



Вторым фактором неоднозначности является множество алгоритмов перехода от регулярного выражения R к недетерминированному автомату НКА. Здесь будут рассмотрены два алгоритма – алгоритм Томпсона [13] и алгоритм Карпова [16]. Оба алгоритма имеют одну и ту же конструктивную идеологию, но при этом, как будет показано ниже, приводят в своих индуктивных следованиях к разным НКА. Следует отметить, что НКА являются разными только в их графическом изображении, но при этом принимают одни и те же цепочки языка, порождаемого одним и тем же регулярным выражением.

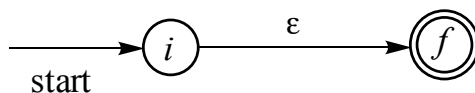
АЛГОРИТМ ТОМПСОНА (алгоритм 1)

Алгоритм позволяет построить НКА – N , допускающий язык $L(R)$, порождаемый регулярным выражением R над алфавитом Σ .

Идея алгоритма состоит в следующем.

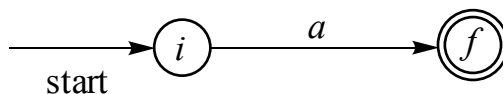
Вначале строится N для распознавания элементарных символов алфавита $R = \varepsilon$, $R = a$, $a \in \Sigma$. Затем определяется составной автомат для регулярных выражений R и S , содержащих операторы объединений $(R | S)$, конкатенации (RS) и замыкания Клини (R^*) .

T1. $R = \varepsilon$.



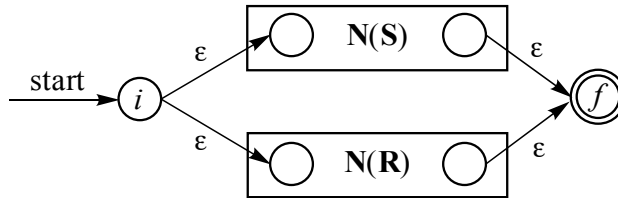
Здесь и в дальнейшем i (in – вход) – новое начальное состояние автомата A ; f (finish – выход) – новое заключительное состояние. Очевидно, что этот автомат A распознает множество цепочек $\{\varepsilon\}$.

T2. $R = a$, $a \in \Sigma$.



Автомат A в данном случае распознает любой символ a из алфавита Σ . Начальное i и конечное f состояния имеют ту же интерпретацию, что и в п. T1.

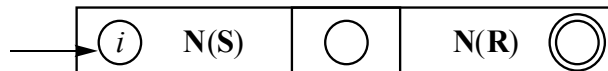
T3. Пусть $N(S)$ и $N(R)$ – автоматы для выражений соответственно S и R . Построим составной $N(S | R)$ для нового регулярного выражения, полученного объединением R и S .



Здесь i и f несут тот же смысл, что и в приведенных выше п. Т1 и Т2. Появляются новые переходы от i по ϵ к стартовым состояниям $N(S)$ и $N(R)$, которые не совпадают с начальным и заключительным состояниями составного $N(S | R)$. Представленная структура автомата $N(S | R)$ такова, что возможен переход от i к f либо через $N(S)$, либо через $N(R)$. Поэтому цепочки языка, которые принимает составной автомат $N(S | R)$, соответственно такие, что

$$\alpha \in L(S) \cup L(R).$$

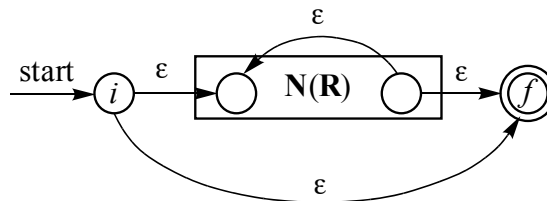
Т4. Пусть $N(S)$ и $N(R)$ – автоматы соответственно для выражений S и R . Построим составной автомат $N(S R)$ от конкатенации цепочек S и R .



Начальное состояние $N(S)$ совмещается с начальным состоянием составного автомата $N(S R)$. Конечное состояние $N(R)$ совмещается с конечным состоянием составного $N(S R)$. Заключительное состояние $N(S)$ совмещается с начальным состоянием $N(R)$. Распознавание цепочек α в $N(S R)$ происходит на пути следования от начального i к заключительному f . Таким образом, этот путь обязывает обработку цепочки вначале автоматом $N(S)$ и затем $N(R)$. Отсюда составной автомат $N(S R)$ принимает все α , такие что $\alpha \in L(S)L(R)$.

Т5. Последний случай – замыкание Клини R^* .

Построим $N(R^*)$:



Здесь i – новое начальное состояние; f – новое заключительное состояние.



Путь следования от начального состояния i к заключительному f может быть следующим:

а) $i - N(R) - f$, если $N(R)$ допускает $L(R)$, то в этом случае принимаются цепочки $\alpha \in \Sigma^*$, такие что $\alpha \in L(R)$;

б) $i - \varepsilon - f$. При этом легко видеть, что $\alpha \in L(R) = \{\varepsilon\}$;

в) $i - N(R) - \varepsilon - N(R) - \dots - \varepsilon - f$. Этот путь соответствует распознаванию всех цепочек α , которые могут пройти через $N(R)$ один раз или бесконечное число раз. Такой автомат настроен на язык $(L(R))^+$ – множество цепочек, соответствующих усеченному замыканию Клини (положительное замыкание) над языком $L(R)$.

Теперь совместим возможные пути следования от i к f (случаи «а», «б», «в»). При этом окажется, что автомат допускает следующие цепочки α :

$$\alpha \in \{\varepsilon\} \cup (L(R))^+ = (L(R))^*.$$

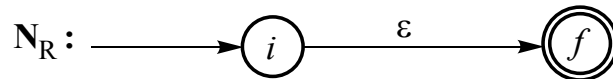
Однако язык $L^*(R)$ соответствует автомату $N(R^*)$. Следовательно, автомат принимает цепочки, порождаемые языком от регулярного выражения R^* . На этом завершается конструктивное доказательство теоремы Клини о соответствии регулярных выражений и конечных автоматов.

АЛГОРИТМ КАРПОВА (алгоритм 2)

В [16] Ю. Г. Карповым предложен другой алгоритм перехода от регулярного выражения к конечному автомату. Этот алгоритм устанавливает взаимно однозначное соответствие между регулярными выражениями и конечными автоматами и является еще одним способом конструктивного доказательства теоремы 4.1.

Установим это соответствие для пустой цепочки, когда $R = \varepsilon$, затем для любого словарного элемента $a \in \Sigma$, $R = a$, и далее для объединения $R = S \mid T$, конкатенации $R = ST$ и, наконец, итерации (замыкание Клини) R^* .

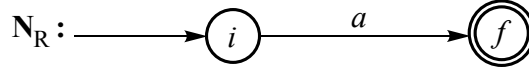
К1. Если $R = \varepsilon$, то соответствующий недетерминированный автомат N_R имеет одно начальное состояние i и одно конечное состояние f – с дугой, помеченной ε , как и в п. Т1 алгоритма Томпсона.



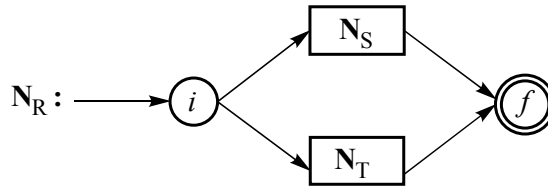
К2. $R = a, a \in \Sigma$.



В этом случае также нет никакого отличия от алгоритма Томпсона.

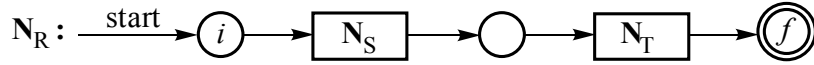


К3. $R = S \mid T$.



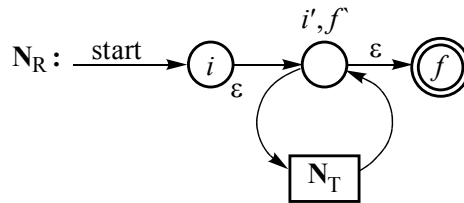
Здесь явное различие с п. Т3 для объединения регулярных выражений алгоритма Томпсона, так как начальные и конечные состояния тут совмещаются, а в п. Т3 алгоритма Томпсона появляются новое начальное состояние и новое конечное состояние.

К4. $R = ST$ – конкатенация регулярных выражений.



В данном случае происходит совпадение с п. Т.4 алгоритма Томпсона. Начальное состояние наследуется из N_S , конечное – из N_T . Заключительное f для N_S становится начальным для N_T .

К5. Замыкание Клини: $R = T^*$.



В этом случае имеется существенное различие составного недетерминированного автомата для регулярного выражения – замыкание Клини T^* . Действительное, бывшее начальное состояние i' для N_T совмещается с бывшим конечным f' для N_T . Таким образом, образуется петля, циклический проход по которой один или более раз принимает все цепочки $\alpha \in \Sigma$, такие что $\alpha \in (L(T))^+$.



Это новое промежуточное состояние $(i', f'$ – совмещенное старое начальное и заключительное) может быть пройдено из i в f переходом $\text{start} - i - \varepsilon - i', f' - \varepsilon - f$, и тем самым автомат N_R настроен на прием $L(T) = \{\varepsilon\}$. Объединим два возможных пути – по циклу и прямолинейный. В результате получим

$$(L(T))^+ \cup (L(T))^0 = (L(T))^*.$$

Таким образом, спроектированный на рисунке недетерминированный автомат построен на прием цепочек α из словаря Σ^* ($\alpha \in \Sigma^*$), таких что множество этих цепочек является замыканием Клини от языка L , порождаемого регулярным выражением $R = T$. Иначе говоря, автомат N_R принимает цепочки $\alpha \in L^*(T)$ из итерации словаря Σ^* . На этом заканчивается первая часть доказательства теоремы Клини.

Докажем конструктивно вторую часть.

Итак, алгоритмы Томпсона и Карпова устанавливают соответствие между регулярными выражениями и конечными автоматами. По существу доказано утверждение о том, что каждое регулярное множество является автоматным языком

$$M_R \subseteq M_A,$$

где M_R – регулярное множество; M_A – множество автоматных языков.

Докажем обратное утверждение.

Введем понятие графа регулярных выражений – граф РВ.

Определение 4.4. *Граф РВ ($\Gamma_{РВ}$) – это конечный граф, узлами которого являются состояния $s, p, q, \dots, f$. Дуги графа соответствуют направленному переходу из одного состояния в другое и помечены регулярными выражениями R_i ($i = 1, 2, \dots$) над словарем Σ .*

$\Gamma_{РВ}$ может в общем случае иметь одну начальную вершину и произвольное число заключительных вершин $f \in F$. Граф РВ допускает цепочку α , если α принадлежит множеству цепочек $\alpha \in M_R = \{R_1, R_2, \dots, R_n\}$, описываемых конкатенацией регулярных выражений, которые помечают путь из начальной вершины i в одну из заключительных вершин $f \in F$. Таким образом, новое определение языка, порождаемого регулярным выражением R , можно записать следующим образом:

$$L(R) = \{\alpha \mid \alpha \in \Sigma^*, \alpha \in M_R = \{R_1, R_2, \dots, R_n\}\}.$$



Приведем пример графа регулярного выражения $\Gamma_{\text{РВ}}$ (рис. 4.2) [16].

Пусть $\Sigma = \{a, b, c\}$:

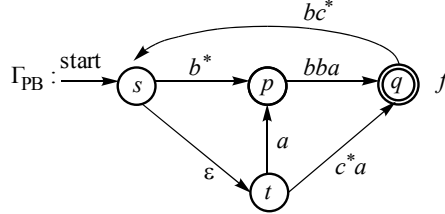


Рис. 4.2. Граф $\Gamma_{\text{РВ}}$

На рис. 4.2 изображен граф $\Gamma_{\text{РВ}}$, который показывает, например, что цепочка $\alpha = abba$ распознается по пути $s - t - p - q$, ведущему в заключительное состояние f . Этот путь помечен конкатенацией регулярных выражений $\mathbf{R}_1\mathbf{R}_2\mathbf{R}_3$, где $\mathbf{R}_1 = \varepsilon$, $\mathbf{R}_2 = a$, $\mathbf{R}_3 = bba$. Другой путь $s - t - q$ помечен конкатенацией $\mathbf{R}_1\mathbf{R}_4$, где $\mathbf{R}_4 = c^*a$. Таким образом, цепочка $abba \in L(A)$, $c^*a \in L(A)$.

Теорема 4.2. *Каждый автоматный язык является регулярным множеством, т. е.*

$$L(A) = M_R.$$

Доказательство теоремы выполним конструктивно [16]. Любой конечный автомат или граф регулярного выражения $\Gamma_{\text{РВ}}$ можно представить в так называемом [16] нормализованном виде с одной начальной вершиной i и одной заключительной вершиной f (рис. 4.3).

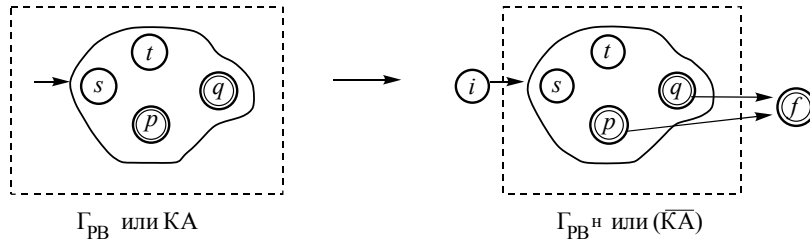
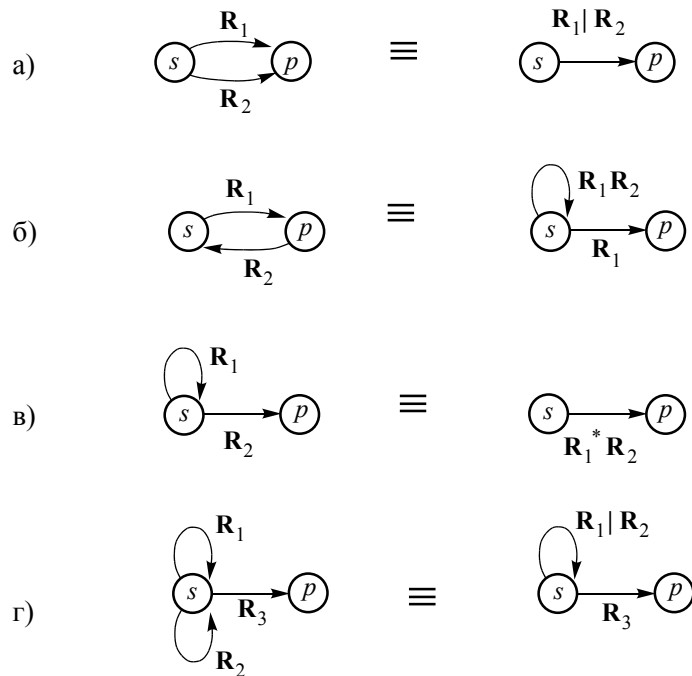


Рис. 4.3. Переход от $\Gamma_{\text{РВ}}$ к нормализованному $\Gamma_{\text{РВ}}^{\text{н}}$ ($\overline{КА}$)

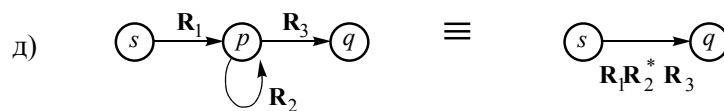
Над нормализованным графом $\Gamma_{\text{РВ}}^{\text{н}}$ ($\overline{КА}$) с одним начальным и одним заключительным состоянием могут быть выполнены следующие операции преобразования ребер и вершин.



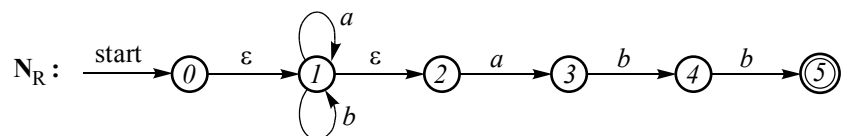
Преобразования ребер:



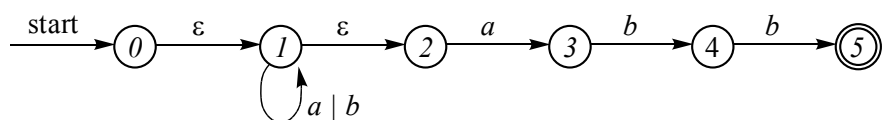
Преобразование вершин:



Пусть граф задан в виде недетерминированного N_R в виде

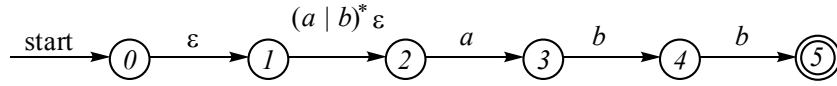


Используя свойство преобразования ребер г), получим

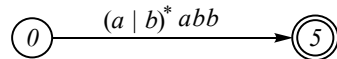




Воспользовавшись свойством преобразования ребер в), получим



Применим аксиомы 1–5 (см. табл. 4.1) для регулярных выражений и свойство конкатенации, получим окончательно



Язык, порождаемый регулярным выражением $\mathbf{R}$, является множеством цепочек α , таких что

$$\mathbf{L}(\mathbf{R}) = \{\alpha \mid \alpha \in \Sigma^* = \{a, b\}^*, \alpha \in \mathbf{M} = \{(a | b)^* abb\}\}.$$

Таким образом, теорема Клини доказана полностью.

4.5. АНАЛИЗ АЛГОРИТМОВ

Выполним алгоритмы Томпсона (алгоритм 1) и Карпова (алгоритм 2) для регулярного выражения $\mathbf{R} = b | (a | bb)(b | ab)^* a$ [13] и $\mathbf{R} = (a | b)^* abb$ [16] и дадим сравнительную оценку алгоритмов.

Пример 4.5

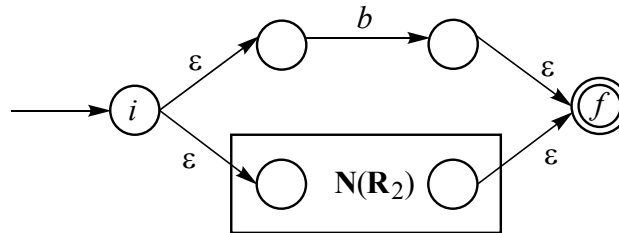
Пусть $\mathbf{R} = b | (a | bb)(b | ab)^* a$.

Спроектируем $\mathbf{N}$, используя алгоритм Томпсона.

Представим $\mathbf{R}$ как объединение двух регулярных выражений $\mathbf{R}_1$ и $\mathbf{R}_2$.

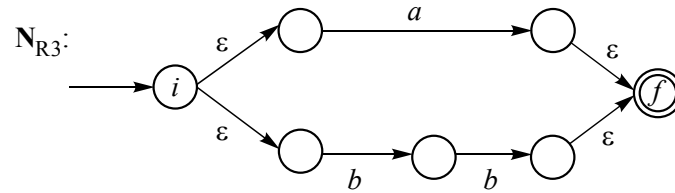
$\mathbf{R} = \mathbf{R}_1 | \mathbf{R}_2$, но $\mathbf{R}_1 = b$, тогда $\mathbf{R} = b | \mathbf{R}_2$, $\mathbf{R}_2 = (a | bb)(b | ab)^* a$.

В соответствии с правилом ТЗ алгоритма 1 (Томпсона) имеем

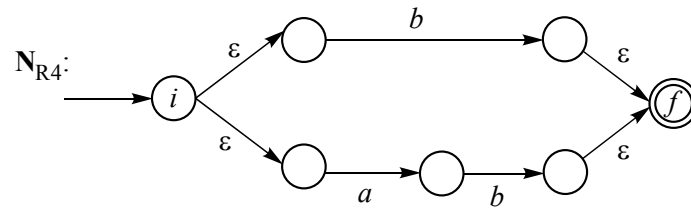


Пусть $\mathbf{R}_2 = \mathbf{R}_3 \mathbf{R}_4^* \mathbf{R}_5$, где $\mathbf{R}_3 = a | bb$, $\mathbf{R}_4 = (b | ab)$, $\mathbf{R}_5 = a$.

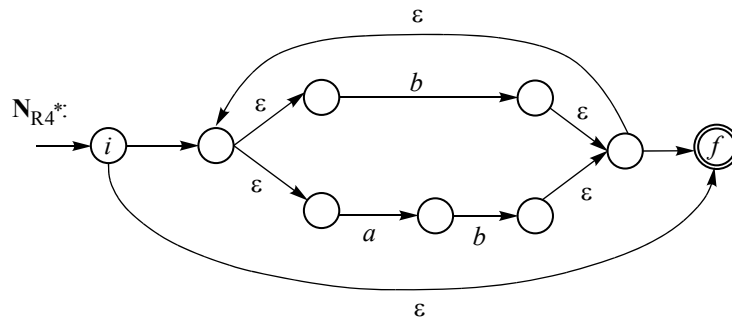
Рассмотрим $\mathbf{R}_3 = a | bb$, тогда, вторично применяя ТЗ, получим



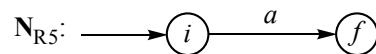
Для R_4 аналогично



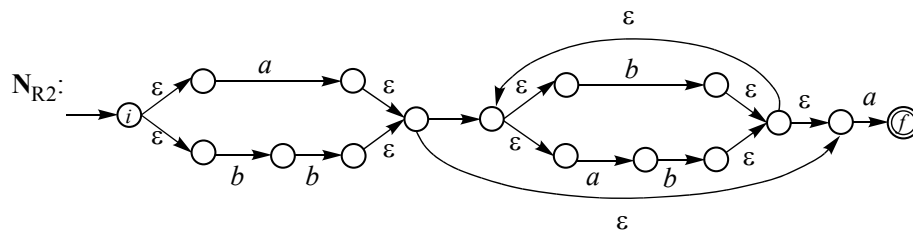
Для R_4^* , пользуясь T5, получим



Из правила T2 следует R_5

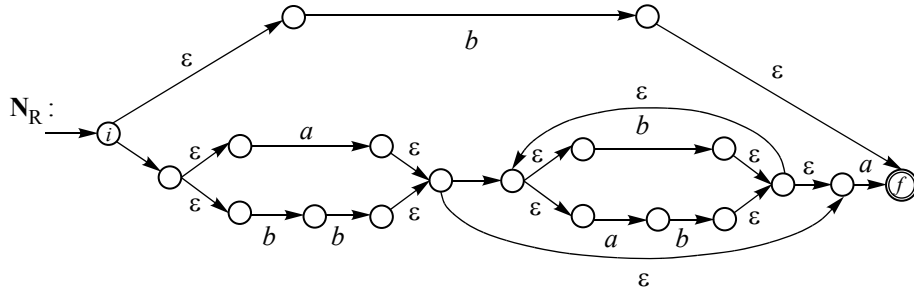


Для R_2 как конкатенации по правилу T4 имеем





Наконец, очередное применение правила T5 дает окончательное решение:



Этот недетерминированный автомат N_R допускает язык

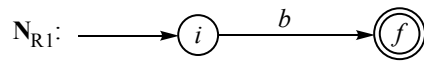
$$L(R) = b \mid (a \mid bb) (b \mid ab)^* a.$$

Действительно, легко порождаются регулярные циклы, производные от регулярного выражения R .

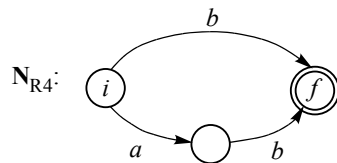
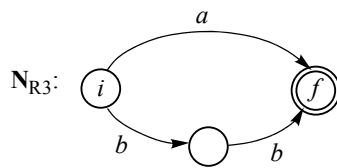
Теперь воспользуемся алгоритмом 2 (алгоритмом Карпова) для перехода от регулярного выражения $R = b \mid (a \mid bb) (b \mid ab)^* a$ к автомату N .

Пусть $R = R_1 \mid R_2$, где $R_1 = b$; $R_2 = (a \mid bb) (b \mid ab)^* a$.

Следуя правилу K2, получим для R_1

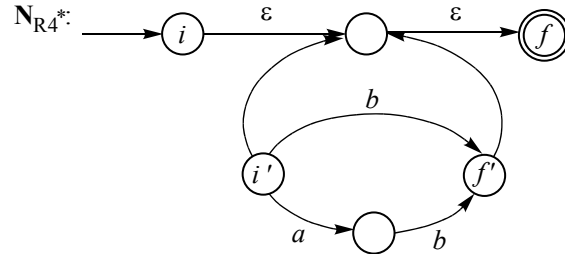


Положим $R_2 = R_3 R_4^* R_5$, где $R_3 = (a \mid bb) = a \mid bb$; $R_4 = (b \mid ab)$; $R_5 = a$. Тогда, применяя дважды правило K4, получим

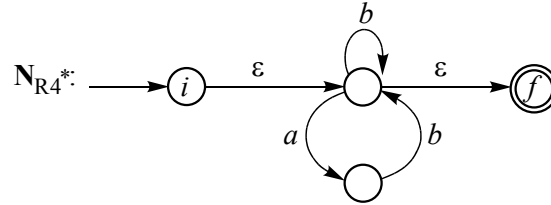




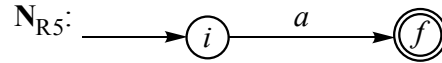
Применим правило K4 для итерации регулярного выражения R_4^* , получим



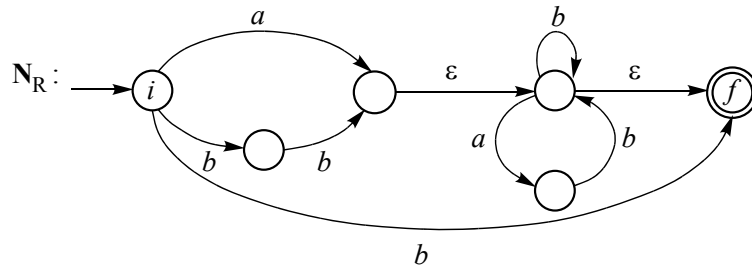
Или, что то же самое, в соответствии с редукцией предыдущего начального i' и предыдущего заключительного f' имеем



Используя правило K2, получим N_{R5} :



Конкатенация для регулярного выражения $R_2 = R_3 R_4^* R_5$ в соответствии с п. K1–K4 дает N_{R2} , а затем и N_R :



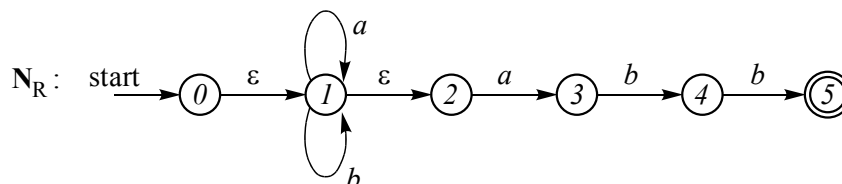
Легко видеть простоту алгоритма Карпова [16] по сравнению с алгоритмом Томпсона [13].

Аналогично для примера, приведенного в [13]:

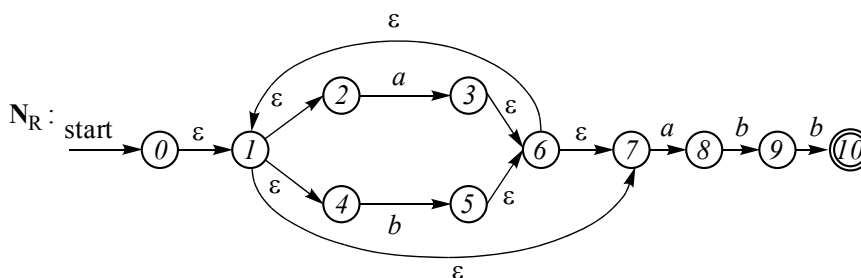
$$R = (a \mid b)^* abb.$$



По Карпову:



По Томпсону:



4.6. ЦЕЛЕСООБРАЗНОСТЬ ПЕРЕХОДА ОТ НКА К ДКА

К вопросу о необходимости перехода от недетерминированного автомата к детерминированному следует подходить из соображений разумной достаточности [12]. Моделирование **D** значительно проще, чем произвольного **N**, но при выполнении проведенных преобразований от **N** к **D** количество преобразований в худшем случае составит $2^n - 1$, где n – количество состояний исходного **N**. Может оказаться так, что затраты на моделирование **D** будут больше, чем затраты на моделирование **N**, если процесс преобразования **N** – **D** окажется трудоемким и суммарные затраты при этом для **D** будут превосходить затраты на моделирование **N**.

В табл. 4.2 [11] приведены оценки времени и памяти, затраченных для распознавателей **N** и **D**.

Таблица 4.2

Оценка затрат

Конечный автомат	Память	Время
N	$O(R)$	$O(R \cdot x)$
D	$O(2^{ R })$	$O(x)$



Здесь $\mathbf{R}$ – регулярное выражение; x – входная цепочка символов.

Остановимся на примере оценочных затрат $\mathbf{N}$ и $\mathbf{D}$ для регулярного выражения $\mathbf{R} = (a | b)^* a(a | b) (a | b) \dots (a | b)$. Это выражение завершается цепочкой $(a | b)^{n-1}$, $n > 1$, и описывает строку x , на n -м месте от правого конца которой находится символ a . Легко видеть, что $\mathbf{N}$ и $\mathbf{D}$ – автоматы-распознаватели – должны отслеживать как минимум цепочку x длиной $|x| = n$. В противном случае $x \notin \mathbf{L}(\mathbf{R})$. Отсюда очевидно, что для распознавания цепочки как минимум из n -символов в $\mathbf{D}$ потребуется по меньшей мере 2^n состояний для словаря из двух символов $\{a, b\}$. При этом

$$|\mathbf{R}| = 2(n - 1) + 1 = 2n - 1.$$

Воспользуемся оценками, приведенными в табл. 4.2 для памяти и времени недетерминированного (P_N , T_N) и детерминированного (P_D , T_D) автоматов:

$$P_N = O(n); P_D = O(2^n); T_N = O(n^2); T_D = O(n).$$

Из этих оценок следует, что недетерминированный автомат имеет n состояний (рис. 4.4).

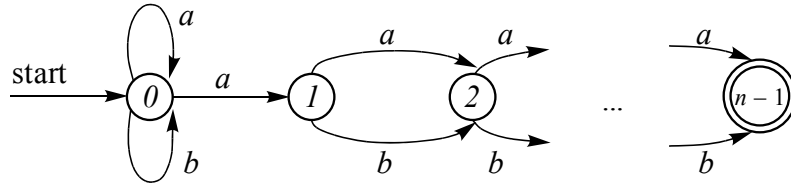


Рис. 4.4. Автомат $\mathbf{N}$ для $\mathbf{R} = (a | b)^* a(a | b) \dots (a | b)$

Таким образом, для данного регулярного выражения $\mathbf{R}$ возникает как раз тот неприятный случай, когда затраты перехода от $\mathbf{N}$ к $\mathbf{D}$ являются дорогостоящими, так как в конечном счете в $\mathbf{D}$ рост количества новых состояний определяется экспоненциальным законом (2^n). Если цепочка состоит из 10 символов, то время, затраченное на разбор x в $\mathbf{D}$, на порядок меньше, чем в $\mathbf{N}$; если $|x| = 100$, то время отличается на два порядка и т. д.

Если считать время распознавания множества цепочек для $\mathbf{R}$ из множества $\mathbf{L}(\mathbf{R})$, то время обработки с помощью $\mathbf{N}$ отличается на k ($k = 1, 2, \dots$) порядков, хотя память в $\mathbf{D}$ всегда независима от количества и общей длины обрабатываемых цепочек и будет фиксированно отличаться: $\mathbf{N} - O(n)$, $\mathbf{D} - O(2^n)$. Поэтому если, как предлагается в [11], использовать кеш-память, то временной критерий становится доминирующим. Время преобразования $\mathbf{N}$ в $\mathbf{D}$ тоже



фиксировано, и в соответствии с алгоритмом преобразования **N** – **D** (рис. 3.11) необходимо выполнить обработку 2^n новых состояний (внешний цикл) для словаря $\Sigma = \{a, b\}$ с двумя состояниями (внутренний цикл). Таким образом, на каждую итерацию внешнего цикла будет выполнено две итерации внутреннего. Общее число итераций алгоритма равно 2^{n+1} . Временная оценка перехода от **N** к **D** определяется при этом $O(2^n)$. Оценка общего времени перехода **N** – **D** обработки составляет

$$O(n) + O(2^n) = O(2^n).$$

Таким образом, сравнение по времени следует делать для функций $O(2^n)$ и $O(n^2)$. Отсюда легко видеть, что и временные затраты в **D** отражают (с ростом n на несколько порядков) временные затраты в **N**. Следовательно, для таких выражений **R** переход от **N** к **D** нецелесообразен и следует моделировать **N** для распознавания $x \in L(R)$.

УПРАЖНЕНИЯ

1. Определить эквивалентность регулярных выражений **R**₁ и **R**₂, если **R**₁ = $a(ba)^*b^*$, **R**₂ = $(ab)^*a(b^*)^*$.

2. Построить регулярные выражения, задающие множество всех таких слов над словарем $\Sigma = \{a, b, c\}$, в которых за символом b :

- 1) обязательно следует символ c ;
- 2) не может находиться символ c .

Привести **R** к эквивалентной автоматной грамматике.

3. Целые константы без знака в языке **C** определяются следующим образом [69]:

а) <десятичные> $\rightarrow$ <последовательность цифр, начинающихся не с 0>;

б) <восьмеричные> $\rightarrow$ 0 <последовательность цифр, кроме цифр 8, 9>;

в) <шестнадцатеричные> $\rightarrow$ 0 x <последовательность цифр и букв a – f или A – F > [**L**].

Если **L** умалчивается, то обыкновенная шестнадцатеричная константа, если не умалчивается – константа длинная.

Написать регулярное выражение для констант **C++** без знака, умолчание обозначается знаком «?».

Привести эквивалентную автоматную грамматику.

4. Построить **R**, если

а) $\Sigma = \{a, b, c\}$, **L**(**R**) = $\{abc, cc\}$;

б) $\Sigma = \{0, 1\}$, **L**(**R**) – <множество двоичных чисел>.



5. Представить регулярными выражениями комментарии языка C, C++.

6. Представить регулярным выражением **R** польскую инверсную запись.

7. Можно ли представить регулярным выражением язык

$$L = \{a^n b c^n \mid n \geq 1\}?$$

8. Описать языки, порождаемые следующими регулярными выражениями:

а) $0(1)^*0$

б) $((\varepsilon|00)1^*)^*$

в) $(0|1)^*0(0|1)(0|1)$

г) $0^*10^*10^*10^*$

9. Написать регулярные выражения **R** для следующих языков **L(R)**:

а) все строки из цифр, в которых есть не более одной повторяющейся цифры;

б) все строки из нулей и единиц, с четным числом нулей и нечетным числом единиц.

10. Доказать эквивалентность выражений **R**, **S**, **T**:

$$R = (a \mid b)^*, \quad S = (a^* \mid b^*)^*, \quad T = ((\varepsilon \mid a) b^*)^*.$$

11. На основании свойств регулярных выражений доказать тождества для произвольных **R**₁, **R**₂, **R**₃, **R**₄:

1) $(R_1 \mid R_2)(R_3 \mid R_4) = R_1 R_4 \mid R_1 R_3 \mid R_2 R_3 \mid R_2 R_4$

2) $R_4(R_1 \mid R_2) R_3 = R_4 R_1 R_3 \mid R_4 R_2 R_3$

3) $R_2 \mid R_2 R_1 \mid R_2 R_1^* = R_2 R_1^*$

4) $R_2 \mid R_2 R_1 R_1^* \mid R_2 R_1 R_1 R_1^* = R_2 R_1^*$

5) $R_4 R_1 R_3 \varepsilon^* \mid R_4 R_1^* R_3 \mid R_4 R_3 = R_4 R_1^* R_3$

6) $(\varepsilon^* \mid R_1 R_1^* \mid R_1 R_1 R_1^*)^* = R_1^*.$

12. Установить, являются ли истинными тождества для регулярных выражений **R**₁ и **R**₂:

1) $(R_1 \mid R_2)^* = (R_1^* \mid R_2^*)^*$

2) $(R_1 \mid R_2)^* = R_1^* \mid R_2^*$

3) $(R_1 \mid \varepsilon)^* = R_1^*$

4) $R_1^* R_2^* \mid R_2^* R_1^* = R_1^* R_2^* R_1^*$

5) $R_1^* R_1^* = R_1^*.$

5. СИНТАКСИЧЕСКИЙ И СЕМАНТИЧЕСКИЙ АНАЛИЗ

В этом разделе рассмотрены схемы проектирования и реализации языковых процессоров методами слева направо или сверху вниз и на их основе будет показан алгоритм диагностики и нейтрализации синтаксических ошибок. Методы анализа снизу вверх (так называемые восходящие анализаторы) рассмотрены в этом разделе в контексте грамматик предшествования. Атрибутная семантика, рассмотренная здесь же, будет непосредственно связана с синтаксисом.

5.1. ПРОЦЕССОР ЧИСЛОВЫХ КОНСТАНТ

Приведем грамматику числовых констант $\mathbf{G}[\langle \text{число} \rangle]$ в следующем виде:

- 1) $\langle \text{число} \rangle \rightarrow [+ | -] \langle \text{число без знака} \rangle$
- 2) $\langle \text{число без знака} \rangle \rightarrow \langle \text{десятичное число} \rangle [E \langle \text{целое} \rangle]$
 $\quad \quad \quad | E \langle \text{целое} \rangle$
- 3) $\langle \text{десятичное число} \rangle \rightarrow [\langle \text{целое без знака} \rangle]. \langle \text{целое без знака} \rangle$
 $\quad \quad \quad | \langle \text{целое без знака} \rangle$
- 4) $\langle \text{целое} \rangle \rightarrow [+ | -] \langle \text{целое без знаков} \rangle$
- 5) $\langle \text{целое без знака} \rangle \rightarrow \mathbb{C}\{\mathbb{C}\},$

где Ц – цифра.

Легко видеть, что грамматика $G[<\text{число}>]$ является автоматной, поэтому построим граф состояний (схема алгоритмизации, рис. 5.1, 5.2).

Правило 1 отражено на диаграмме рис. 5.1, где $\langle Ч \rangle$ – $\langle \text{число} \rangle$, $\langle ЧБЗ \rangle$ – $\langle \text{число без знака} \rangle$.

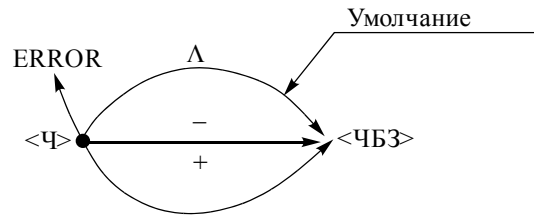


Рис. 5.1. Граф для правила 1 грамматики $G[<\text{Число}>]$

Правила 2–5 для $G[<\text{Число}>]$ реализованы на графе рис. 5.2.

Сплошные стрелки на графе характеризуют синтаксически верный разбор; пунктирные символизируют состояние ошибки (ERROR); непомеченные дуги предполагают любой терминальный символ, отличный от указанного из соответствующего узла. Состояние OUT символизирует успешное завершение разбора.

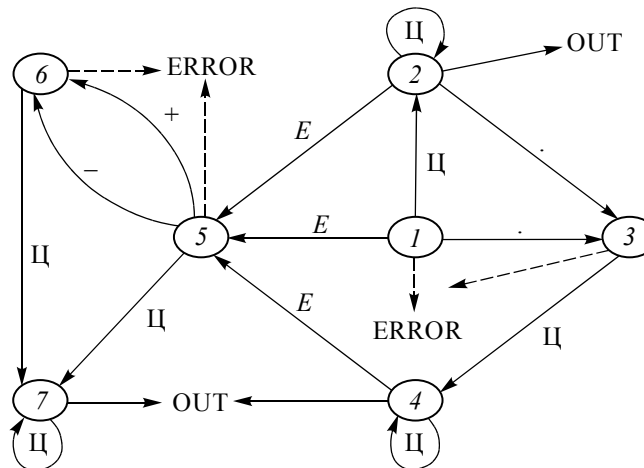


Рис. 5.2. Граф $G[<\text{Число}>]$, $\text{Ц} - [0 \dots 9]$

После синтаксической стадии обработки, как это указывалось выше (см. рис. 1.1), выполняется семантический анализ. Причем семантика в данном случае основывается на синтаксически верных конструкциях, которые обработаны синтаксическим анализатором на первом проходе.

Семантическая обработка текста предполагает смысловое наполнение вычислительного процесса в соответствии с содержанием



грамматики. Как правило, семантическая обработка основывается на синтаксических конструкциях. Для грамматик автоматного типа диаграммы состояний представляют собой «дерево», на которое словно «елочные украшения» добавляются семантические атрибуты. Такой «украшенный» граф называется семантическим графом.

Семантические процедуры проектируются в зависимости от смыслового содержания грамматики.

Семантические атрибуты числовой константы имеют следующий вид [78]:

$$n1: m = 10m + q;$$

$$n2: n = n + 1; m = 10m + q;$$

$$n3: p = 10p + q;$$

$$n4: m = 1;$$

$$n5: s = -1;$$

$$n6: R = m * 10^{(sp - n)}, \text{ } ^{\wedge} - \text{возведение в степень.}$$

Начальные условия:

$$m = p = n = 0; s = 1,$$

где m – текущее значение мантиссы; p – текущее значение порядка; n – счетчик числа десятичных цифр в мантиссе; s – знак порядка; q – значение сканируемого символа (лексемы).

Семантический граф отличается от синтаксического тем, что на нем нет состояния ERROR, и это не противоречит логике связей компилятора, т. е. семантическая обработка может начаться только в случае успешной синтаксической обработки. Поэтому компилятор в рассмотренной технологии является многопроходным: исходный текст на языке, порождаемом данной грамматикой, просматривается слева направо не менее двух раз. Первый проход – фаза синтаксического анализа, второй проход – фаза семантического анализа.

Рассмотрим пример разбора числовой константы в соответствии с рис. 5.2, 5.3.

Пример 5.1

Пусть числовая константа имеет вид 1.02E-2.

Тогда «путешествие» по графу (рис. 5.3) вызовет следующие семантические атрибуты:

$$n1: m = 1;$$

$$n2: n = 1; m = 10;$$

$$n2: n = 2; m = 102;$$

$$n6: R = 102 * 10^{(-2-2)} = 102 * 10^{-4} = 0,0102.$$

Представленный процессор числовых констант, выполняющий синтаксический и семантический анализ, является универсальным. Изменяя грамматику, можно преобразовать любые числовые константы, спроектированные для предметного пользования.

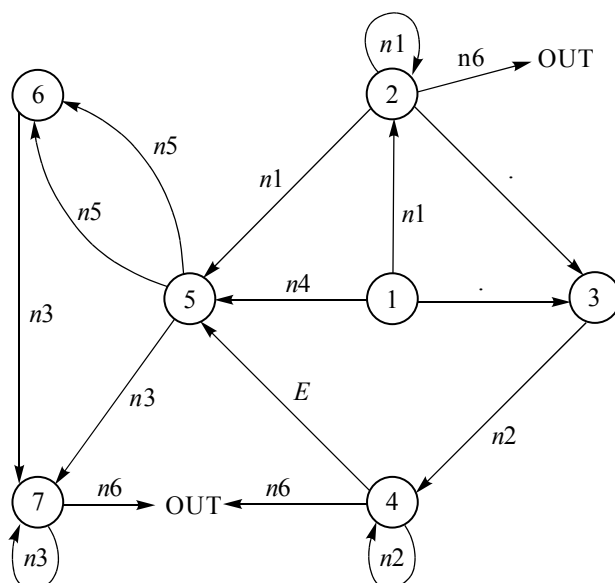


Рис. 5.3. Семантический граф для $G[\langle \text{число} \rangle]$

В современных языках программирования процедуры преобразования символа в числовой эквивалент встроили в компилятор как библиотеки стандартных функций. Однако эти процедуры жестко регламентированы форматом входных данных. Иначе говоря, пользователю нельзя изменить грамматику допустимых числовых констант. Например, в языке *C* существует функция **atoi(S)**, которая преобразует символьную цепочку **S** в ее числовой эквивалент. Ограничением является то, что **S** должна быть целым числом. Функция **atof(S)** также преобразует символьную цепочку в ее числовой эквивалент, но здесь числовые константы должны быть с плавающей точкой типа **float**. Приведем код библиотечной функции **atoi(S)** на языке *C* (рис. 5.4).



```

/* S – носитель входных данных */
int atoi(S)
char *S[ ];
{
    int i, n;
    n = 0;
    for (i = 0; S[i] >= '0' && S[i] <= '9'; ++i)
        n = 10*n + S[i] - '0';
    return (n);
}

```

Рис. 5.4. Листинг функции atoi(S)

5.2. СКАНЕР

Сканер как составная часть языкового процессора стоит на первой фазе обработки исходного текста. Функциональным назначением сканера является *лексическая свертка* исходного текста программы в символы, идентифицируемые в дальнейшем как ключевые слова, числовые константы, встроенные функции и др. Кроме того, на этапе лексического анализа происходит так называемая фильтрация *незначащей части текста*.

Незначащей частью текста будем называть вспомогательные символы, которые не являются носителями смыслового содержания текста. Такими символами являются символы табуляции «\t», символы перевода на новую строку «\n», пробелы « », комментарии и др. Эти символы используются только при редактировании исходного текста, когда необходимо соблюдать определенное форматирование текста и повысить читабельность.

На рис. 5.5 показана схема потоков при фильтрации исходного текста.

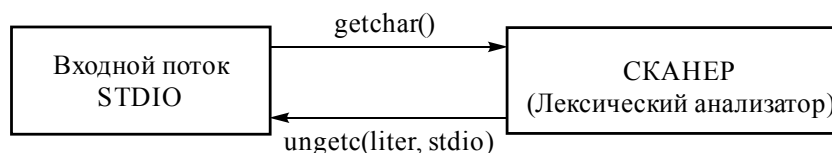


Рис. 5.5. Схема потоков сканера при фильтрации



Реализация процедуры фильтрации (очистки от *мусора* [13]) осуществляется с помощью стандартных функций `getchar()` и `ungetc()` из встроенной стандартной библиотеки `<stdio.h>`. На рис. 5.6 приведена функция `clear()`, которая очищает текст от символов табуляции, переводов строки и пробелов и возвращает отфильтрованный текст во входной поток.

```
#include <stdio.h>
int clear ( )
{
    int liter;
    while (1) /* цикл по всему входному потоку символов*/
    {
        liter = getchar( );
        if ( liter == ' ' || liter == '\n' || liter == '\t' )
            then ungetc (liter, stdio);
    }
}
```

Рис. 5.6. Процедура фильтрации

Часто в процессорах функцию сканера заменяет синтаксический анализатор, при этом нет необходимости в специальном просмотре исходного текста, который осуществляется на этапе синтаксического анализа. В отличие от синтаксического анализатора сканер определяет лишь принадлежность символов алфавиту языка и не устанавливает принадлежность языковых конструкций к грамматике.

Проиллюстрируем разработку сканера, используя грамматику арифметических выражений $G[<AB>]$ языка *Fortran* (** – возведение в степень):

- 1) $<AB> \rightarrow T \mid <AB> + T \mid <AB> - T$
- 2) $T \rightarrow S \mid T * S \mid T / S$
- 3) $S \rightarrow O \mid O ** S$
- 4) $O \rightarrow (<AB>) \mid <\text{идентификатор}> \mid <\text{целое без знака}>.$

Функцией сканера в этом примере является внутреннее представление символов арифметического выражения, причем под внутренним представлением будем понимать символический (условный) код, который ставится в соответствие идентификаторам, числовым константам и другим объектам языка [1]. Для рассматриваемого примера арифметических выражений примем следующие соглашения относительно лексем:



Символ	<ЦБЗ>	<идент-р>	+	-	/	*	**	(	)
Условный код	1	2	3	4	5	6	7	8	9

На рис. 5.7 приведена диаграмма состояний для грамматики $G[<AB>]$. На диаграмме представлена посимвольная декомпозиция арифметических выражений с генерацией соответствующего символического кода символов <идентификатор>, <целое без знака> и литеры «+», «-» и др. Непомеченные дуги на диаграмме соответствуют состоянию ERROR (отсутствие данного символа в словаре грамматики) либо выходу из обработки очередного символа и переходу на старт обработки следующего.

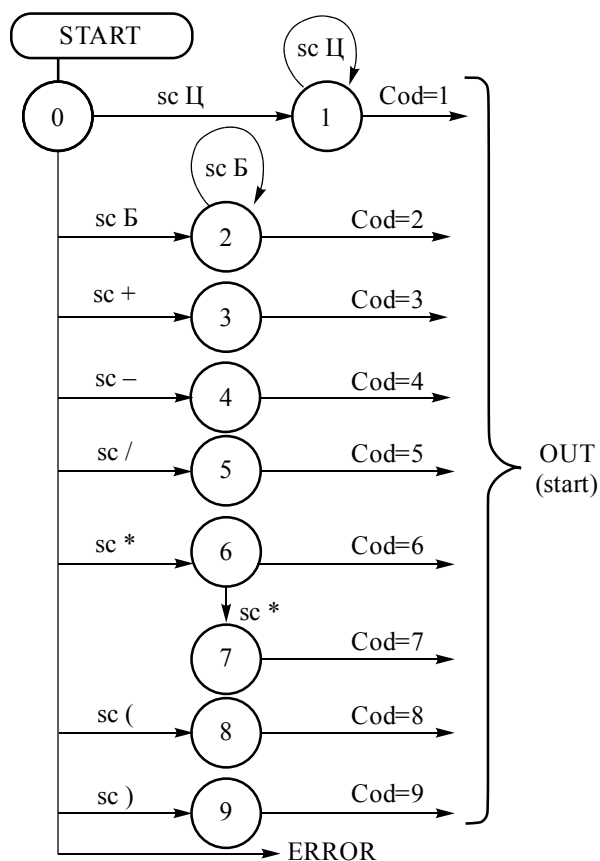


Рис. 5.7. Диаграмма состояний сканера

**Пример 5.2**

Пусть арифметическое выражение имеет вид $A1 * A2 \#$.

Проиллюстрируем движение по состояниям графа для данного примера:

0 – 2 – 2 – OUT – START – 6 – OUT – START – 2 – 2 – OUT.

Семантика сканера. На семантической диаграмме «навешаны» семантические атрибуты: SC – сканирование очередного символа, Cod – адресная генерация кода соответствующего символа.

Реализацию сканера по диаграмме состояний или по графу на рис. 5.6. приведем на C-подобном языке. Отметим, что данная реализация универсальна для графов, в которых переходы соответствуют терминальному символу из узла в узел либо по петле. Поэтому программа для графа <числовая константа> (см. рис. 5.3) мало чем будет отличаться от приведенной ниже.

```
# include <ctype.h>
# include <stdio.h>
# define ERROR 0
int i = 0; /*i =0 – признак ошибки*/
/*-----*/
main()
{
    while((i = scanner()) != ERROR) printf("%d", i);
}
/*-----*/
scanner( )
{
    int liter;
    liter=getchar( );
    if (isdigit(liter))

        { while(isdigit(liter = getchar()));
          ungetchar(liter);
          return(1);

        }
    else
        if (isalpha(liter))
```



```
    {
        while(isnum(liter = getchar()) || (isalpha(liter)));
        ungetchar(liter);
        return(2);
    }
else
    switch(liter)
    {
        case '+': return(3);
        case '-': return(4);
        case '/': return(5);
        case '*': if ((liter = getchar()) == '*')
                    return(7);
                    else
                    {
                        ungetchar(liter);
                        return(6);
                    }
        case '(': return(8);
        case ')': return(9);
        default : ungetchar(liter);
                    return(ERROR);
    }
}
```

В примере реализации сканирования символов грамматики выражений эта процедура сознательно упрощена из методических соображений и для того, чтобы не усложнять реализующую программу, которая в приведенном простом варианте доступна и читабельна.

Опишем дополнительные атрибуты сканера, используя для символов *идентификатор* и *целое* алгебру регулярных выражений.

ЛЕКСИЧЕСКАЯ СВЕРТКА ПРОГРАММЫ

Основным функциональным назначением сканера является лексическая свертка программы на ее терминальные составляющие: идентификаторы, ключевые слова, числовые константы, знаки операций и др. Эти языковые конструкции являются сентенциальными формами своих деревьев или терминальными символами. Причем



терминальные символы могут состоять из одной литеры (знаки операций «+», «-», «/», «*», ...) или нескольких литер (идентификаторы, числовые константы и др.). Для иллюстрации выполним лексическую свертку над символами арифметических выражений языка *Fortran*.

Грамматика арифметических выражений *Fortran* $G[<AB>]$ была рассмотрена ранее. Приведем ее с заменой нетерминала $<AB>$ на E . Тогда $G[E]$ имеет вид:

$$\begin{aligned} E &\rightarrow T \mid E + T \mid E - T \\ T &\rightarrow O \mid O * T \mid O / T \mid O ** T \\ O &\rightarrow (E) \mid \text{num} \mid \text{id}. \end{aligned}$$

В качестве операнда рассматриваются целые числовые константы (**num**) и идентификаторы (**id**). Это ограничение в операнде O не нарушает общности и выполнено с чисто методическими целями, чтобы был понятен основной механизм свертки. Увеличение размерности (наличие альтернатив) в операнде никак не влияет на алгоритмическую идеологию.

Основная свертка сентенциальной формы будет проходить на уровне операнда (O). Выделение знаковых конструкций производится на уровне выражения (E) и терма (T).

Приведенная грамматика $G[E]$ не может быть LL(1) и даже LL(k)-грамматикой [3], поскольку для любого $k = 1, 2, 3, \dots$ нетерминальному символу E может соответствовать выражение вида $((\dots(a)\dots)) + a$ или $((\dots(a)\dots)) - a$, содержащее $(k + 1)$ пар скобок, и, следовательно, просмотр этого выражения на k символов вперед не позволит определить альтернатив $E \rightarrow E + T$ или $E \rightarrow E - T$. Это неудобный вид КС-грамматики. Построим эквивалентную $G'[E]$:

$$\begin{aligned} E &\rightarrow TA \\ A &\rightarrow \varepsilon \mid + TA \mid - TA \\ T &\rightarrow OB \\ B &\rightarrow \varepsilon \mid * OB \mid / OB \mid ** OB \\ O &\rightarrow \text{num} \mid \text{id} \mid (E), \end{aligned}$$

где **num** = <целое без знака>, **id** = <идентификатор>.

Построенная грамматика является LL(1)-грамматикой, и для нее легко построить синтаксическую диаграмму (рис. 5.8). Эквивалентность G и G' очевидна, так как обе грамматики порождают один и тот же язык $L(G) = L(G')$. Забегая вперед, отметим, что синтаксический анализ LL(1)-грамматик проводится методом рекурсивного спуска, который, с одной стороны, является однозначным и безвозвратным, а с другой – весьма прост и эффективен при реализации.



Теперь по известным правилам (см. раздел 3.2) от синтаксической диаграммы (рис. 5.8) легко перейти к диаграмме состояний, представленной на рис. 5.9. В этой диаграмме целые и идентификаторы перенесены первыми, в остальном структура графа осталась без изменений.

Рис. 5.8. Синтаксическая диаграмма $G[E]$, $\text{num} = d^+$; $\text{id} = l(l|d)^*$:
 d – цифра, l – буква

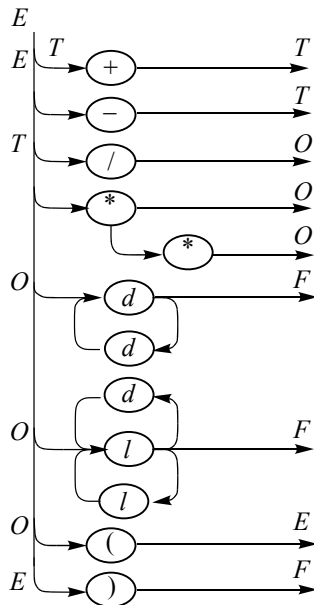
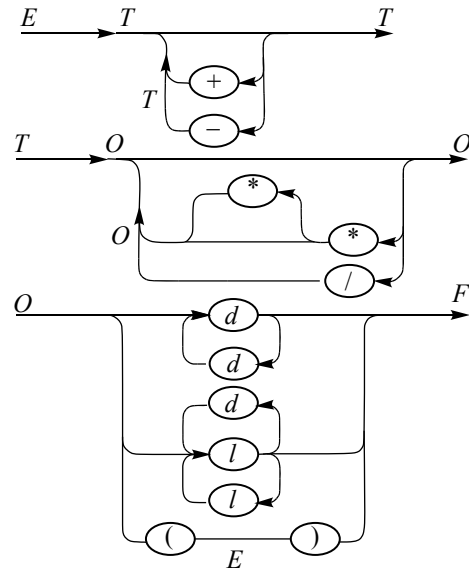


Рис. 5.9. Обобщенная синтаксическая диаграмма



Семантика простого кодирования может быть без нарушения алгоритма выполнена с использованием аппарата регулярных выражений. Покажем, как это выглядит для приведенных конструкций $\langle \text{ЦБЗ} \rangle = \text{num}$, $\langle \text{ид} \rangle = \text{id}$ и знаковых операций.

СЕМАНТИКА ЦЕЛОГО ЧИСЛА ПРИ СКАНИРОВАНИИ

Для *целого* в алгоритме предусмотрен генератор условного кода 1. Под кодом здесь понимается класс *целых*. Часто при обработке *целых* требуется не только их указатель (код), но и значение – `int.value`. Учитывая, что числовые значения (*value*) *целого* в выражении могут быть разными, при лексической обработке в этом случае необходимо каждый раз генерировать кортеж $\langle \text{код}, \text{значение} \rangle = \langle 1, \text{значение} \rangle$. Этот кортеж будем называть *таблицей констант*.

Соответствующие числовые значения, которые будут храниться в таблице констант, определяются по следующему алгоритму.

Семантическое правило для операнда (*O*) с определением атрибута значения для целого имеет вид

$$O \rightarrow dW (dW)^* y1,$$

где *W* – семантический атрибут записи (*Write*) очередного цифрового символа в символьную строку *S*; *y1* – семантический атрибут преобразования символьной строки в числовой эквивалент. Этим атрибутом может быть уже рассмотренная стандартная функция `atoi(S)`, которая принимает входной формальный параметр *S* и возвращает значение *целого*.

Такая удобная атрибутивная организация семантики позволяет практически без изменения общего алгоритма работать не только с целыми, но и десятичными числовыми константами. Только вместо библиотечной функции `atoi(S)` будет использоваться библиотечная функция `atof(S)`, которая возвращает числовые константы с плавающей точкой.

Каждая ветка на графе рис. 5.10 помечена не только терминальным символом, но и процедурой записи (*W*) очередного терминала в S^R (стек с обратной записью). Определенная таким образом строка *S* принимается в качестве входного формального параметра встроенной функции `atof(S)`, которая возвращает десятичное число *n* – значение очередной обрабатываемой числовой константы.

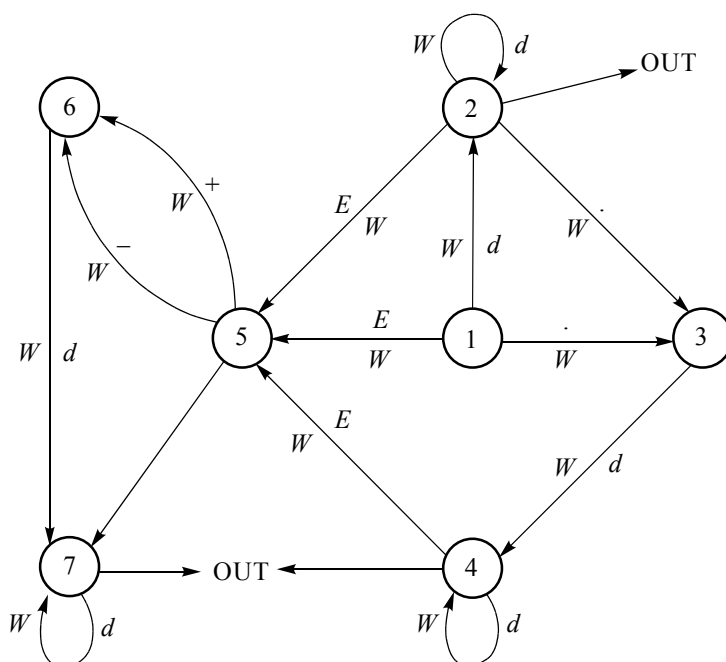


Рис. 5.10. Семантический граф

Наконец, если не пользоваться стандартными библиотечными функциями, то необходимо воспользоваться методикой преобразования цепочки символов в числовой эквивалент, описанный в разделе 5.1.

ЛЕКСИЧЕСКИЙ АНАЛИЗ ИДЕНТИФИКАТОРА

Выделение идентификаторов кодом 2 не является полной информацией о каждом идентификаторе в отдельности. На этапе лексического анализа информационной характеристикой идентификатора является кортеж <имя, адрес>.

Общих атрибутов идентификатора больше, чем представленный кортеж на этапе лексического анализа. Эти атрибуты предоставляют сведения об отведенной конкретному идентификатору памяти, его типе, области видимости (глобальный или локальный) и др.

Кроме того, имена процедур (функций) также обозначаются идентификаторами. К обычным атрибутам имени процедуры (функции) здесь добавляются аргументы или формальные параметры,



которые также должны быть охарактеризованы количеством, типом, способом передачи, памятью и др. Все эти данные сводятся для идентификаторов в так называемую *таблицу символов*.

Таблица символов представляет собой структуру данных, которая содержит записи имен идентификаторов с атрибутными полями.

Таким образом, на этапе лексического анализа в таблицу символов заносится уникальное имя идентификатора, а в качестве атрибута – его адрес или ссылка. Если организовать линейную структуру записи идентификаторов в таблицу символов, то каждый раз при сканировании очередного идентификатора с уникальным именем таблица будет заполняться последовательно, начиная с первого, определенного условным кодом (адресом), сгенерированным на рис. 5.7 ($Cod = 2$).

Синтаксическая конструкция идентификаторов в рассмотренных арифметических выражениях языка *Fortran* с учетом семантических атрибутов на этапе лексического анализа имеет вид:

$$IW(IW|dW)y_2,$$

где W – процедура записи соответствующей буквы (I) или цифры (d) в стек S^R ; y_2 – семантическая процедура обработки идентификатора.

Процедура y_2 представляет собой следующий алгоритм:

1) проверка наличия идентификатора с именем S по таблице символов (проверка на уникальность имени). Если идентификатор с именем S уже есть в таблице символов, перейти к п. 5;

2) запись имени S в позицию «имя идентификатора» таблицы символов;

3) определение адреса (ссылки) для нового идентификатора с именем S ;

4) запись адреса в позицию «АДРЕС» таблицы символов;

5) конец обработки очередного идентификатора.

ЛЕКСИЧЕСКИЙ АНАЛИЗ ОПЕРАЦИЙ

Лексический разбор бинарных операций, как правило, ограничивается уже описанным алгоритмом генерации адресного (условного) кода. Следует добавить лишь, что в сгенерированные адреса (ссылки) необходимо записать команду, соответствующую операции. Условно эти команды обозначены так, как показано в таблице.



+	–	/	*	**
com	sub	div	mult	exp

Таким образом, в адрес с условным кодом 3 будет записана операция сложения **com**, в адрес с условным кодом 4 – операция вычитания **sub** и т. д.

Аналогично на этапе лексического анализа генерируются адреса и происходит наполнение этих адресов соответствующими кодами других символов, такими как скобки, знак окончания операторов «;», лексические знаки $<$, $>$, $\leq$, $\geq$, $=$, $:=$ и др.

Ключевые слова вместе с идентификаторами резервируют определенные адреса в таблице символов перед наполнением этой таблицы идентификаторами и их атрибутами. Поэтому идентификаторы не могут называться именами, которыми идентифицированы ключевые слова: **while**, **if**, **do** и др.

5.3. ОРГАНИЗАЦИЯ ТАБЛИЦ СИМВОЛОВ

Проверка правильности семантики и генерация кода требуют знания характеристик идентификаторов, констант, имен функций (библиотечных и внешних) и др.

Определение 5.1. *Таблица символов – это структура данных, которая применяется для хранения информации о характеристиках символов [1].*

В этом разделе в качестве символов рассмотрим идентификаторы, организация которых в таблице символов представляет наибольший интерес.

Над таблицей символов должны выполняться как минимум две операции: вставка и поиск. Вставка добавляет новую запись в таблицу, поиск возвращает индекс записи для искомой строки или признак отсутствия строки, если она не найдена.

Операция поиска требует большого внимания, так как применяется многократно и может вызывать существенные затраты времени, что снижает скорость компиляции.

Атрибутами или характеристиками для идентификаторов – имен переменных могут быть различные семантические категории, например:

- тип (**int**, **real**, **char** , ...);
- вид (простая переменная, массив, структура и др.);
- адрес или ссылка на адрес переменной и др.



СТРУКТУРЫ ЛИНЕЙНОГО ФОРМАТА

Для структур линейного формата существует два способа хранения имен. Первый – хранить символы имени в записях таблицы символов (рис. 5.11), второй – в записи для имени размещать только указатель на отдельный массив символов, дающий позицию первого символа лексемы (рис. 5.12) [13].

Первый способ используется, если в языке имеется ограничение на предельный размер имени. В случае если ограничение на длину имени отсутствует (или предельный размер очень редко достигается), второй способ оказывается более эффективным с точки зрения использования памяти.

Имя									Атрибуты
<i>s</i>	<i>o</i>	<i>R</i>	<i>T</i>						
<i>a</i>									
<i>r</i>	<i>e</i>	<i>A</i>	<i>D</i>	<i>a</i>	<i>r</i>	<i>r</i>	<i>a</i>	<i>y</i>	
<i>i</i>									

Рис. 5.11. Сохранение имени

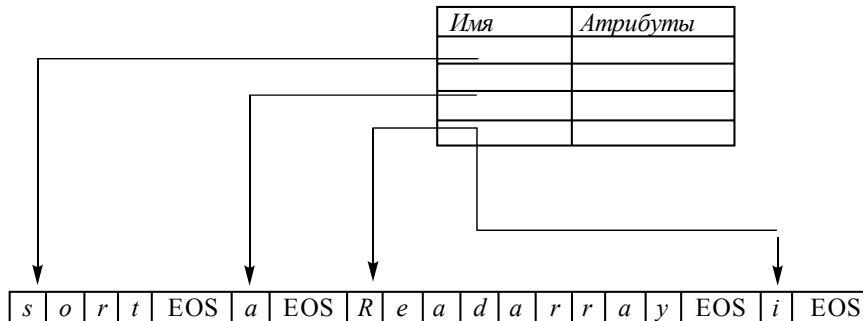


Рис. 5.12. Сохранение адреса

БЛОЧНЫЕ СТРУКТУРЫ

В некоторых языках один и тот же идентификатор может быть описан и использован много раз в различных блоках и процедурах. Каждое такое описание должно иметь единственный связанный с ним элемент в таблице символов, в этом случае используется блочная структура [13]. Описание, соответствующее идентифика-



тору, находится следующим образом: сначала просматривается текущий блок, в котором идентификатор используется, затем – объемлющий блок, и так до тех пор, пока не будет найдено описание данного идентификатора.

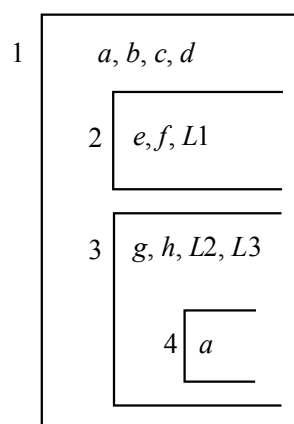
SURRNO – номер блока, объемлющего данный блок.

NOENT – число элементов в таблице символов для данного блока.

POINT – указатель на элементы.

На рис. 5.13 изображена блочная структура, а на рис. 5.14 приведена таблица символов для нее.

Рис. 5.13. Блочная структура



1	0	4			$e, f, L1$
2	1	3			a
3	1	4			$g, h, L2, L3$
4	3	1			a, b, c, d

Рис. 5.14. Таблица символов блочной структуры

ДРЕВОВИДНЫЕ СТРУКТУРЫ

Существует способ представления таблиц с использованием двоичных деревьев. Каждый узел дерева представляет собой заполненный элемент таблицы, причем корневой узел является первым элементом. Добавление и поиск вершин выполняются в соответствии с некоторым отношением $<$, заданным на множестве идентификаторов.



Пример таблицы, в которую были добавлены последовательно идентификаторы G, D, M, E, A, B и F , приведен на рис. 5.15.

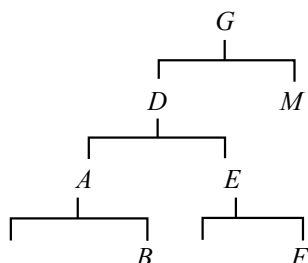


Рис. 5.15. Таблица с двоичным деревом

Число требуемых сравнений в процессе поиска во многом зависит от порядка, в котором поступают идентификаторы. Чтобы уменьшить максимальный поиск, можно перестраивать дерево [11].

ПОИСК ОБЪЕКТОВ В ТАБЛИЦЕ СИМВОЛОВ

Таблица символов просматривается всякий раз, когда в исходном тексте встречается некоторое имя. При обнаружении нового имени или новой информации об имеющемся имени в таблицу символов вносятся соответствующие изменения.

Механизм преобразования таблицы символов должен обеспечивать эффективный поиск и добавление в таблицу символов.

НЕУПОРЯДОЧЕННЫЙ ПОИСК

Неупорядоченный поиск – это простейший способ организации таблицы символов. Он состоит в том, чтобы добавлять элементы для аргументов в порядке их поступления, без каких-либо попыток упорядочения. Поиск в этом случае требует сравнения с каждым элементом таблицы, пока не будет найден подходящий. Для таблицы, содержащей n элементов, в среднем будет выполнено $n/2$ сравнений. Если n велико (20 или более), этот способ неэффективен [1].

Общее время, необходимое для внесения n имен и выполнения e запросов к таблице символов, не больше, чем $cn(n + e)$, где константа c представляет собой время, необходимое для выполнения нескольких машинных операций. В случае больших n и e время работы может оказаться недопустимо долгим.



БИНАРНЫЙ ПОИСК

Поиск может быть выполнен более эффективно, если элементы таблицы упорядочены (отсортированы) согласно некоторому естественному порядку аргументов.

Эффективным методом поиска в упорядоченном списке из n элементов является так называемый бинарный, или логарифмический, поиск. Символ S , который следует найти, сравнивается с аргументом элемента $(n + 1) / 2$ в середине таблицы. Если этот элемент не является требуемым, мы должны просмотреть только блок элементов, пронумерованных от 1 до $(n + 1) / 2 - 1$, или блок элементов от $(n + 1) / 2 + 1$ до n , – в зависимости от того, меньше искомый элемент S или больше элемента, с которым его сравнивали. Затем процесс повторяется над блоком меньшего размера. Поскольку на каждом шаге число элементов, которые будут содержать S , сокращается наполовину, максимальное число сравнений равно $1 + \log 2n$ [1].

Алгоритм бинарного поиска на языке C представлен на рис. 5.16.

```
int Find(int len, char **table, char *id)
{
    int end=len, begin=0, middle;
    for(int i=0;i<len;i++)
    {
        middle=(end-begin)/2;
        if(strcmp(table[i],id)==0) return i;
        if(strcmp(table[i],id)>0) end = middle;
        if(strcmp(table[i],id)<0) begin = middle;
    }
    return -1;
}
```

Рис. 5.16. Алгоритм бинарного поиска

ХЕШ-АДРЕСАЦИЯ

Метод хеш-адресации в целом более эффективен, чем линейные списки, и используется для таблиц символов в большинстве случаев [13]. Схема открытого хеширования (хеш-таблица размера 211) приведена на рис. 5.17.



Массив заголовков
списков, индексруемый
хеш-значением



Рис. 5.17. Хеш-функция

Структура данных состоит из двух частей: хеш-таблицы и блоков.

1. Хеш-таблица представляет собой фиксированный массив из m указателей на записи таблицы.

2. Записи таблицы организованы в виде m отдельных связанных списков, именуемых блоками (некоторые блоки могут быть пустыми).

Каждая запись в таблице символов встречается только в одном из этих списков.

Для определения, существует ли в таблице символов запись для строки s , мы вычисляем хеш-функцию h от строки s , возвращающую целое число от 0 до $m - 1$. Если строка s имеется в таблице символов, то она находится в списке с номером $h(s)$. Если s в таблице символов еще отсутствует, то она вносится в таблицу путем создания записи в начале списка с номером $h(s)$ [13].

Используя такой алгоритм, можно сделать время обращения к элементу таблицы константой. Пусть средняя длина списка при n записях в таблице символов и хеш-таблице размера m равна m/n . Можно выбрать m настолько большим, чтобы n/m было ограничено небольшой величиной, например 2.

Пространство, занимаемое таблицей символов, определяется как $m + cn$, где c – количество слов в одной записи таблицы символов.

Следует уделить особое внимание созданию хеш-функции, так как наибольшая эффективность работы алгоритма хеш-адресации достигается при равномерном распределении идентификаторов между списками и приемлемом времени вычисления значений



хеш-функции. С точки зрения реализации хеш-функция – это некоторое преобразование строки символов в значение номера элемента таблицы. Нельзя не отметить, что хеш-функция, учитывающая все символы строки, выглядит более разумной, чем функция, учитывающая только несколько символов.

Текст хеш-функции из компилятора С. Вайнбергера на языке С приведен на рис. 5.18 [13]:

```
int RHeshTable::hashpjw(char *s)
{ char *p;
  unsigned int h = 0, q;
  for (p=s;*p!=0;p++)
  { q = (h=(h<<4)+*p);
    if(h & 0xF0000000)
      h^=q >> 24^q; }
  return h%211; }
```

Рис. 5.18. Хеш-адресация

СРАВНЕНИЕ СПОСОБОВ ОРГАНИЗАЦИИ ТАБЛИЦ СИМВОЛОВ

Прямой поиск прост в реализации, но он самый неэффективный, так как время поиска прямо пропорционально размерности таблицы, а количество сравнений в среднем равно половине элементов таблицы. Бинарный поиск более эффективный. Однако существует серьезный недостаток, который обуславливает малую применимость метода двоичного поиска в компиляторах, – он состоит в том, что добавление новых записей обычно требует переупорядочивания записей таблицы. Поэтому этот метод обычно используется для почти зафиксированных таблиц, в которые записи добавляются редко.

Самым эффективным из рассмотренных методов является хеш-адресация, она используется для таблиц символов в большинстве случаев. Как можно ожидать, при использовании этого метода память, требуемая для структуры данных, растет с увеличением количества имен. Однако, подбирая размерность таблицы, можно свести время обращения к элементу таблицы к константе.



5.4. РЕКУРСИВНЫЙ СПУСК

Рассмотрим подкласс КС-грамматик, которые называются *S*-грамматиками.

Определение 5.2. *S*-грамматики – это подкласс контекстно-свободных грамматик, таких что:

1) правая часть каждого правила начинается с терминала

$$A \rightarrow a\beta;$$

2) если в грамматике есть альтернативные правила, то они обязательно начинаются с разных терминальных символов

$$A \rightarrow a\beta \mid b\alpha, \text{ причем } a \neq b.$$

S-грамматики часто называют разделенными или простыми.

Пример 5.3

$G[S]$:
1) $S \rightarrow aT$
2) $S \rightarrow TbS$
3) $T \rightarrow bT$
4) $T \rightarrow ba$.

$G[S]$ не является *S*-грамматикой, так как в правиле 2 первый символ нетерминальный, а это противоречит условию 1 для *S*-грамматик. Правила 3 и 4 тоже не удовлетворяют условию 2, так как два альтернативных правила начинаются с одного терминального символа.

Пример 5.4

$G'[S]$:
1) $S \rightarrow abR$
2) $S \rightarrow bRbS$
3) $R \rightarrow a$
4) $R \rightarrow bR$.

$G'[S]$ – удовлетворяет всем условиям и является *S*-грамматикой.

Анализ *S*-грамматик весьма эффективен методом рекурсивного спуска.

Метод рекурсивного спуска. Основная идея этого метода состоит в том, что каждому нетерминалу грамматики ставится в соответствие определенная программная единица, процедура или функция (например в языке *C* – функция), которая распознает цепочку, порождаемую этим нетерминалом. Такие процедуры (функции) вызываются



в соответствии с правилами грамматики, причем иногда вызывают сами себя рекурсивно. Следовательно, языком реализации этого метода может быть лишь такой язык, который допускает рекурсию.

Пример 5.5

$G[S]$: 1) $S \rightarrow aAS$
 2) $S \rightarrow b$
 3) $A \rightarrow cASb$
 4) $A \rightarrow \Lambda$.

Легко видеть, что $G[S]$ – это S -грамматика. Приведем анализ $G[S]$ методом рекурсивного спуска (рис. 5.19).

На рис. 5.19 проиллюстрирован алгоритм работы функции (нетерминалов) A , S и корневого сегмента на некотором псевдоязыке. В соответствии с правилами 1, 2 грамматики $G[S]$ процедура S должна начинаться с терминалов a или b и не может начинаться с терминала c или любого другого. Поэтому переход по таким символам влечет ошибку, т. е. переход на метку ERROR.

Корневой сегмент	Процедура S		Процедура A	
«Начало» Установим вход = первый входной символ; S; Если вход $\neq \downarrow$ то ERROR; Иначе STOP; /* $\downarrow$ – признак окончания программы */	Переход по символу:		Переход по символу:	
	Символ	Метка	Символ	Метка
	a	M1	a	M4
	b	M2	b	M4
	c	ER	c	M3
	$\downarrow$	ER	$\downarrow$	ER
	Другой	ER	Другой	ER
	M1: Scan; A; S; Возврат; M2: Scan; Возврат; ER: ERROR; Возврат;		M3: Scan; A; S; Если вход $\neq 'b'$ то ERROR; Иначе Scan; Возврат; M2: Возврат;	

Рис. 5.19. Алгоритм работы рекурсивной функции S()



Проиллюстрируем реализацию метода рекурсивного спуска для грамматики $G[\langle \text{инструкция} \rangle]$:

- 1) $\langle \text{инстр} \rangle \rightarrow \langle \text{пер} \rangle = \langle \text{выр} \rangle \mid$
 $\text{IF } \langle \text{выр} \rangle \text{ THEN } \langle \text{инстр} \rangle [\text{ELSE } \langle \text{инстр} \rangle]$
- 2) $\langle \text{пер} \rangle \rightarrow i[(\langle \text{выр} \rangle)]$
- 3) $\langle \text{выр} \rangle \rightarrow T \{+T\}$
- 4) $T \rightarrow O \{ * O \}$
- 5) $O \rightarrow \langle \text{пер} \rangle \mid (\langle \text{выр} \rangle).$

По классификации Хомского легко показать, что $G[\langle \text{инстр} \rangle]$ является контекстно-свободной и разделенной, т. е. грамматикой типа LL(1) (Last Left – самый крайний левый символ). В общем случае рассматривают грамматики $LL(k)$, в которых по k левым символам определяется принадлежность цепочки к соответствующему правилу грамматики. Грамматики этого класса удовлетворяют главному критерию однозначности и безвозвратности разбора. Эти грамматики, так же как и S -грамматики, удобно анализировать методом рекурсивного спуска. Учитывая приведенный выше алгоритм рекурсивного спуска, покажем реализацию синтаксического процессора методом рекурсивного спуска на алголоподобном языке со следующими принятыми допущениями:

- 1) переменная nx ($next$) всегда содержит очередной символ, по которому определяются альтернативы $LL(1)$ -грамматики;
- 2) процедура **scan** готовит очередной символ исходной программы и помещает его в nx ;
- 3) процедура **error**, обрабатывает ошибки синтаксиса (управление передается в вызывающую программу).

Следуя методу рекурсивного спуска, все нетерминалы грамматики $G[\langle \text{инстр} \rangle]$ обозначим своими процедурами, несущими ту же аббревиатуру, что и нетерминалы.

```
instr      /* <инстр> */
{
    if (nx="IF") then
        begin
            scan;
            expr;
            if (nx!="THEN") then
```



```

        error;
    else
        begin
            scan;
            instr;
            if (nx="ELSE") then
                begin
                    scan;
                    instr;
                end;
            end;
        end;
    else
        begin
            var;
            if (nx!="=") then
                error;
            else
                begin
                    scan;
                    expr;
                end;
            end;
        end;
    }
    /*-----*/
var      /* <пер> */
{
    if (nx!="i") then error;
    else
        begin
            if (nx!="(") then
                begin
                    scan;
                    expr;
                    if (nx!=")") then error;
                    else scan;
                end;
        end;
end;

```



```
}
/*-----*/

expr      /* <выр> */
{
  t;
  while (nx="+") do
    begin
      scan;
      t;
    end;
}
/*-----*/
t          /*  T  */
{
  o;
  while (nx="*") do
    begin
      scan;
      o;
    end;
}
/*-----*/
o;         /*  O  */
{
  if (nx("(") then
    begin
      scan;
      expr;
      if (nx!=")") then error;
      else scan;
    end;
  else var;
}
```



5.5. ДИАГНОСТИКА И НЕЙТРАЛИЗАЦИЯ СИНТАКСИЧЕСКИХ ОШИБОК

Диагностика – это установка места возникновения и типа синтаксической ошибки. Кроме того, обработанная ошибка должна быть визуализирована пользователем в виде, удобном для ее обнаружения.

Нейтрализация предполагает исключение синтаксически неверной конструкции в тексте безболезненно для дальнейшего разбора всего текста.

Систематических методов нейтрализации не существует вообще, поэтому в каждом языковом процессоре разработчику предоставляется самостоятельная задача по нейтрализации ошибки.

Н. Айронс в 1968 г. предложил метод [1] локализации и отсеечения «больных» кустов дерева при нисходящем разборе программы. Метод не претендует на универсальность, однако в нем выработаны здравые концепции нейтрализации ошибок при нисходящем разборе.

МЕТОД АЙРОНСА

Основная идея – по контексту без возврата отбрасывать литеры, которые привели к тупиковой ситуации (когда продолжение анализа по грамматике невозможно), и продолжать разбор. Для иллюстрации метода рассмотрим следующую грамматику:

$G[P]$:

- 1) $P \rightarrow A$
- 2) $A \rightarrow i = E$
- 3) $E \rightarrow T\{+T\}$
- 4) $T \rightarrow O\{*O\}$
- 5) $O \rightarrow i \mid (E)$.

Представленная грамматика $G[P]$ является усечением арифметического оператора присваивания, причем усечение выполнено на уровне операнда и операций. Это не нарушает общности грамматики и сделано для того, чтобы не увеличивать размерность синтаксического дерева.

Пример 5.6

$i = i +)$

Этот оператор присваивания явно не соответствует грамматике $G[P]$. Построим синтаксическое дерево для этой основы (рис. 5.20).

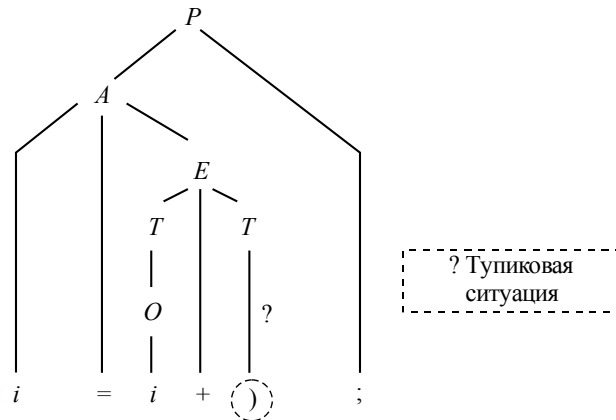


Рис. 5.20. Дерево с диагностикой ошибки

В общем случае обнаружение ошибки соответствует следующей схеме разбора:

$$Z \Rightarrow^* x_1 x_2 \dots x_{i-1} x_i \dots x_n,$$

где $(x_1 x_2 \dots x_{i-1})$ – построенная часть куста; $(x_i \dots x_n)$ – недостроенная часть куста), которую нельзя построить с помощью $G[Z]$.

Часто при диагностике в тупиковой ситуации на экран выводится недостроенная часть, или «хвост» сентенциальной формы. Задачей проектировщика процессора ошибок при нейтрализации является следующее:

- 1) недопущение ошибки с целью дальнейшего разбора;
- 2) исправление допущенной ошибки и дальнейший нормальный анализ.

АЛГОРИТМ АЙРОНСА ПО ИСПРАВЛЕНИЮ ОШИБОК

Пусть xju – куст исходной программы, где x – построенная часть, ju – недостроенная часть, $j \in V_T$.

1. Строим список L из литер недостающих частей неполных кустов.

2. Головной терминальный символ j в цепочке ju проверяется и отбрасывается (при этом каждый раз получается новая цепочка ju) до тех пор, пока не найдется такой символ j , что будет иметь место $V \Rightarrow^* j\dots$ (выводимость).



3. Определяется неполный куст, ставший причиной появления ошибки.

4. Определяется терминальная цепочка q . Если ее поставить перед j , то продолжение разбора приведет к правильной привязке к неполному кусту, найденному на шаге 3, и всем кустам поддерева. Для каждого такого куста генерируется цепочка терминалов до полного поддерева, а конкатенация этих цепочек дает q .

5. Цепочка q вставляется непосредственно перед j и разбор продолжается начиная с головного символа цепочки q , который становится входным.

Для нашего примера цепочка $X \equiv \{i = i +\}$

$j \equiv) \quad j y \equiv)$;

$xjy, j \in V_T$

1) $L = \{;, T, +\}$

2) $j \equiv)$; $P \rightarrow A$.

Необходимо вставить цепочку q , чтобы дополнить $E \rightarrow T\{+T\}$, при этом проще всего в качестве цепочки q рассмотреть символ i ($q = i$).

Тогда дерево будет иметь следующий вид (рис. 5.21).

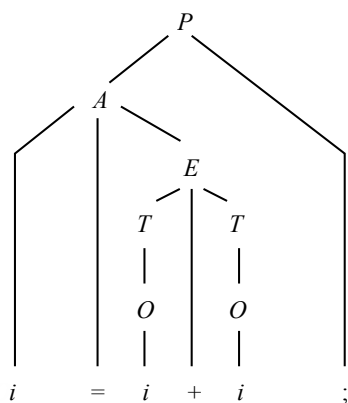


Рис. 5.21. Дерево с нейтрализацией ошибки

5.6. ВЫЧИСЛЕНИЕ АРИФМЕТИЧЕСКИХ ВЫРАЖЕНИЙ

Вычисление арифметических выражений $\langle AB \rangle$ происходит в два этапа. На первом этапе обычная традиционная, или *инфиксная*, форма записи $\langle AB \rangle$ преобразуется в *постфиксную* запись,



или *польскую инверсную запись* (ПОЛИЗ), названную так в честь польского математика Яна Лукасевича. На втором этапе ПОЛИЗ преобразуется в результат $\langle AB \rangle$.

Примеры инфиксной и постфиксной записи арифметических выражений приведены в таблице.

Инфиксная	Постфиксная
$a * b + c$	$ab * c +$
$a + b * c$	$abc * +$
$a + b * c - d / (a + b)$	$abc * + dab + / -$

Отличие ПОЛИЗ от инфиксной формы состоит в отсутствии круглых скобок () и порядке следования операндов и операций. Вычислять выражения на основе ПОЛИЗ значительно проще, так как отсутствие скобок, переопределяющих приоритет выполнения операций, исключает дополнительный алгоритм порядка выполнения операций. В ПОЛИЗ алгоритм выполнения жестко определен по правилу: операция справа – результат выполнения двух операндов слева, при этом цепочка ПОЛИЗ просматривается слева направо.

Пример 5.7

Рассмотрим польскую запись $ab * c +$

Просматривая цепочку слева направо, дойдем до знака *. Эта операция выполняется для операндов a и b , в результате образуется цепочка $Zc +$, где $Z = a + b$. Просматривая теперь эту цепочку до знака +, выполняем операцию $R = Z + c$.

Перевод инфиксной формы в ПОЛИЗ. Существует много способов перевода инфиксной записи в постфиксную. Наиболее удачными являются алгоритмы Дейкстры и Гриса. Алгоритм Дейкстры основан на использовании стекового механизма с учетом приоритетов (рис. 5.22).

Алгоритм Дейкстры:

Знак	(	)	$\supset$	$\vee$	$\wedge$	$\neg$	$\geq = <$ $\geq \neq \leq$	$+$ $-$	$*$ $/$	**
Приоритет	0	1	2	3	4	5	6	7	8	9

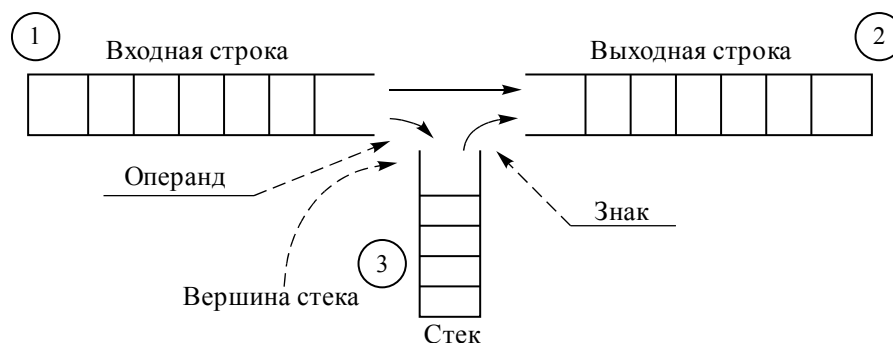


Рис. 5.22. Стекковый алгоритм

1. Входная строка – арифметическое или логическое выражение – помещается в стек и анализируется слева направо, т. е. в вершине стека находится самый левый символ.

2. Операнды переписываются в стек 2 без изменений. Знаки операций помещаются вначале в стек 3, а затем пересылаются в стек 2 по следующему правилу.

Если приоритет входного знака равен 0 или больше приоритета знака, находящегося в вершине, то новый знак добавляется в стек 3, в противном случае из стека 3 выталкивается в стек 2 знак, находящийся в вершине, а также следующие за ним знаки с приоритетом, большим приоритета входного знака или равным ему. После этого входной знак добавляется в стек операций.

3. Открывающаяся скобка «(» заносится в стек операций сразу, так как имеет минимальный приоритет, равный нулю. Ее не может вытолкнуть ни один знак, кроме закрывающейся скобки «)», которая имеет приоритет, равный 1, и не превосходит другие приоритеты операций. Поэтому появление на входе закрывающейся скобки «)» вызывает выталкивание всех знаков из стека 3 до тех пор, пока не появится открывающаяся скобка «(», единственная, которая имеет меньший приоритет. В стек закрывающаяся скобка «)» не заносится. Появление в вершине стека 1 закрывающейся скобки «)» и в вершине стека 3 открывающейся скобки «(» вызывает их взаимное уничтожение, и разбор продолжается с пункта 1.

Таким образом, в стеке 2 образуется ПОЛИЗ без скобок.



Пусть дано выражение $a + b * c - d / (a + b)$. Требуется перевести его в ПОЛИЗ. Рассмотрим перевод из инфиксной записи в постфиксную по шагам.

Шаг 1		
②		a+b*c-d/(a+b) ①
③		
Шаг 2		
②	a a	+b*c-d/(a+b) ①
③		
Шаг 3		
②	a a	b*c-d/(a+b) ①
③	+ +	
Шаг 4		
②	ab ab	*c-d/(a+b) ①
③	+ +	
Шаг 5		
②	ab ab	c-d/(a+b) ①
③	+* +*	
Шаг 6		
②	abc abc	-d/(a+b) ①
③	+* +*	



Шаг 7

$$\textcircled{2} \quad \boxed{abc^*} \quad \boxed{-d/(a+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{+}$$

Шаг 8

$$\textcircled{2} \quad \boxed{abc^*+} \quad \boxed{-d/(a+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{}$$

Шаг 9

$$\textcircled{2} \quad \boxed{abc^*+} \quad \boxed{d/(a+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{-}$$

Шаг 10

$$\textcircled{2} \quad \boxed{abc^*+d} \quad \boxed{/(a+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{-}$$

Шаг 11

$$\textcircled{2} \quad \boxed{abc^*+d} \quad \boxed{(a+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{-/}$$

Шаг 12

$$\textcircled{2} \quad \boxed{abc^*+da} \quad \boxed{a+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{-/(}$$

Шаг 13

$$\textcircled{2} \quad \boxed{abc^*+da} \quad \boxed{+b)} \quad \textcircled{1}$$

$$\textcircled{3} \quad \boxed{-/(}$$



Шаг 14 _____

② ①

③

Шаг 15 _____

② ①

③

Шаг 16 _____

② ①

③

Шаг 17 _____

② ①

③

Шаг 18 _____

② ①

③

Шаг 19 _____

② ①

③

Итак, инфиксное выражение $a + b * c - d / (a + b)$ в ПОЛИЗ имеет вид

$$abc^* + dab + / -.$$



После перевода инфиксной записи в ПОЛИЗ алгоритм вычисления арифметических выражений упрощается и состоит в следующем.

ПОЛИЗ просматривается слева направо. Если анализируемый символ – операнд, то просматривается следующий элемент. Если рассматриваемый символ – операция, то ее необходимо выполнить над двумя слева стоящими от нее операндами. Результат рассматривается как операнд в анализируемой строке. Это повторяется до последнего анализируемого символа в строке.

Пример 5.8

Выполним расчет ПОЛИЗ $abc* + dab + / -$

$$1) \quad \underbrace{abc*}_{R=b*c} + dab + / -$$

$$2) \quad a \underbrace{R+dab}_{Z=a+R} + / -$$

$$3) \quad Z \underbrace{dab}_{R=a+b} + / -$$

$$4) \quad Z \underbrace{dR}_{X=d/R} -$$

$$5) \quad \underbrace{ZX}_{R=Z-X} -$$

Приведем еще один алгоритм преобразования инфиксной формы в постфиксную – алгоритм Гриса [1], или транслирующих грамматик, который использует формальные методы анализа и в связи с этим имеет строгую простую реализацию.

Алгоритм Гриса (транслирующих грамматик)

Пусть грамматика арифметики $G[<AB>]$ определена следующим образом:

$$1) \quad <AB> \rightarrow <AB> + T$$

$$2) \quad <AB> \rightarrow T$$

$$3) \quad T \rightarrow T * O$$

$$4) \quad T \rightarrow O$$

$$5) \quad O \rightarrow (<AB>)$$

$$6) \quad O \rightarrow a \mid b \mid c.$$

Введем семантические атрибуты в правилах 1, 3, 6:

$$1) \quad <AB> \rightarrow <AB> + Ty_2$$

$$3) \quad T \rightarrow T * Oy_1$$

$$6) \quad O \rightarrow ay_3 \mid by_3 \mid cy_3,$$

где y_i ($i = 1, 2, 3$) – запись соответствующего терминального символа в стек ПОЛИЗ.



Пример 5.9. Проанализируем выражение $(a + b)*c$ с помощью процедуры выводимости, учитывая семантическую атрибутику:

$$\begin{aligned} \langle AB \rangle &\Rightarrow T \Rightarrow T*O \Rightarrow (\langle AB \rangle)*O_{y_1} \Rightarrow (\langle AB \rangle + T_{y_2})*O_{y_1} \Rightarrow \\ &\Rightarrow (ay_3 + T_{y_2})*O_{y_1} \Rightarrow (ay_3 + by_3y_2)*O_{y_1} \Rightarrow (ay_3 + by_3y_2)*cy_3y_1. \end{aligned}$$

Выстроим семантические атрибуты в той последовательности, в которой они появляются в процедуре разбора. В результате получим ПОЛИЗ:

$$\begin{array}{ccccc} y_3 & y_3 & y_2 & y_3 & y_1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ a & b & + & c & * \end{array}$$

Алгоритм транслирующих грамматик, как это видно из примера, отличается простотой, оригинальностью и, самое главное, имеет строгую формальную схему. В отличие от эвристики Дейкстры этот алгоритм основан на теории синтаксически ориентированных методов.

Отметим, что ПОЛИЗ используется как промежуточная форма кода не только для арифметических выражений, но и для других языковых конструкций – деклараций, операторов цикла, условных операторов и др.

5.7. ВОСХОДЯЩИЕ МЕТОДЫ АНАЛИЗА

Анализ методами слева направо является более приоритетным и используется в 60–80 % случаев для грамматик определенного класса. Менее популярными являются методы анализа снизу вверх или восходящие методы. Однако определенный класс грамматик (например, грамматики предшествования) эффективнее анализируется методами снизу вверх.

ГРАММАТИКИ ПРОСТОГО ПРЕДШЕСТВОВАНИЯ. ОТНОШЕНИЯ ПРЕДШЕСТВОВАНИЯ

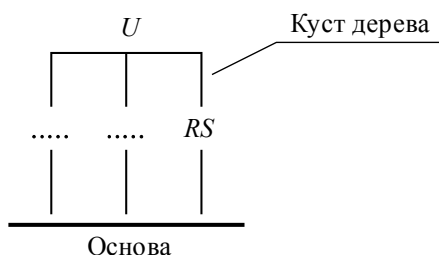
Отношения предшествования – такие же отношения, как и логические конъюнкция, дизъюнкция, и несут смысл определенных операций.

Здесь и в дальнейшем будем рассматривать пару рядом стоящих символов $R, S \in V = V_T \cup V_N$.

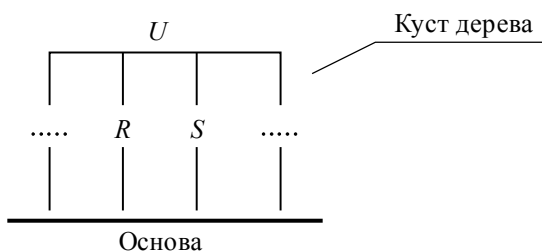


Определение 5.3. Говорят, что R предшествует S ($R \bullet > S$) (т. е. R сворачивается раньше, чем S), если R является частью основы синтаксического дерева, а S – нет.

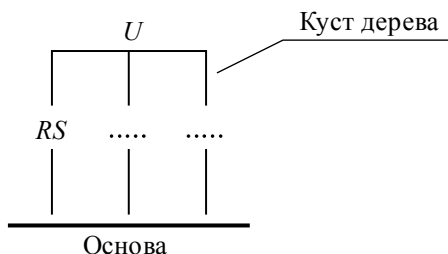
На рис. 5.23 показано, что R должен быть последним символом правой части некоторого правила $U \rightarrow \dots R$. Таким образом, символ R редуцируется раньше, чем символ S (т. е. предшествует ему).

Рис. 5.23. $R \bullet > S$ 

Определение 5.4. Говорят, что R сворачивается одновременно с S ($R \dot{=} S$), если оба символа входят в основу одновременно (рис. 5.24).

Рис. 5.24. $R \dot{=} S$

Определение 5.5. Говорят, что S предшествует R ($R < \bullet S$), или R сворачивается позже, чем S , если S в основе, а R еще нет (рис. 5.25).

Рис. 5.25. $R < \bullet S$ 



Символ S является первым символом некоторого правила $U \rightarrow S \dots$, причем S редуцируется раньше, чем R .

Свойства отношения предшествования. Отношения предшествования в отличие от всех основных отношений не обладают коммутативными свойствами, т. е. нельзя применять свойства симметрии к отношениям предшествования. Действительно, если мы говорим, что R предшествует S ($R \bullet > S$), то это совсем не означает, что $S < \bullet R$. Хотя для определенных символов словаря это имеет место, но только в частном случае.

Сформулируем аналитические определения отношений.

Определение 5.6. *Отношение $U \text{ FIRST } S$ имеет место тогда и только тогда, когда имеет место выводимость $U \rightarrow S\alpha$, $S \in V_N$, $\alpha \in V^*$ (иначе говоря, S – первый нетерминал).*

Определение 5.7. *Отношение $U \text{ FIRST}^+ S$ имеет место тогда и только тогда, когда существует итерационная выводимость $U \Rightarrow^+ S\alpha$, $S \in V_N$, $\alpha \in V^*$.*

Определение 5.8. *Отношение $U \text{ LAST } S$ имеет место тогда и только тогда, когда имеет место выводимость $U \rightarrow \alpha S$, $S \in V_N$, $\alpha \in V^*$.*

Определение 5.9. *Отношение $U \text{ LAST}^+ S$ имеет место тогда и только тогда, когда существует итерационная выводимость $U \Rightarrow^+ \alpha S$, $S \in V_N$, $\alpha \in V^*$.*

Определение отношений предшествования между символами по заданной грамматике производится в соответствии со свойствами отношений.

Свойство 1. $R \doteq S$, когда в грамматике существуют следующие отношения:

$$U \rightarrow \alpha RS\beta \in P; \text{ причем } \alpha, \beta \in V^*; R, S \in V.$$

Свойство 2. $R < \bullet S$, если в грамматике существуют следующие отношения:

$$U \rightarrow \alpha RW\beta; \text{ причем } \alpha, \beta \in V^*; W \text{ FIRST}^+ S; R, S, W \in V.$$

Свойство 3. $R \bullet > S$, если в грамматике существуют следующие отношения:

$$U \rightarrow \alpha QW\beta; \text{ причем } \alpha, \beta \in V^*; Q \text{ LAST}^+ R; \\ W \text{ FIRST}^+ S; R, S, W, Q \in V.$$



Рассмотрим операторные грамматики.

Определение 5.10. Грамматика называется операторной, если это контекстно-свободная грамматика, причем в правых частях редукций не встречаются рядом стоящие нетерминалы, т. е. отсутствует правило вида $C \rightarrow \alpha AB\beta$.

Определение 5.11. Операторная грамматика является грамматикой простого предшествования в следующих случаях:

- а) если для каждой пары терминальных символов a, b устанавливается не более одного отношения предшествования;
- б) среди правил грамматики G нет правил с одинаковыми правыми частями.

Теорема 5.1. Грамматики простого предшествования всегда однозначны.

(Доказательство см. [1].)

В качестве примера рассмотрим грамматику $G[Z]$:

- 1) $Z \rightarrow bMb$
- 2) $M \rightarrow (L \mid a$
- 3) $L \rightarrow Ma$.

Легко видеть, что $G[Z]$ является грамматикой простого предшествования (это легко доказывается.)

Язык, порождаемый данной грамматикой:

$$L(G[Z]) = \{bab, b(aa)b, b((aa)a)b, \dots\}.$$

Построим сентенциальные деревья из цепочек, принадлежащих $L(G[Z])$ (рис. 5.26–5.28).

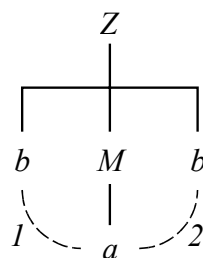


Рис. 5.26. Первое сентенциальное дерево

Основа	a
Отношения предшествования	$1: a \bullet > b$ $2: a < \bullet b$

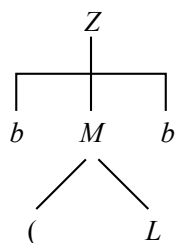


Рис. 5.27. Второе
сентенциальное
дерево

Основа	$(\quad L$
Отношения предшествования	$b \prec \bullet ($ $(\stackrel{\bullet}{=} L$ $L \succ \bullet b$

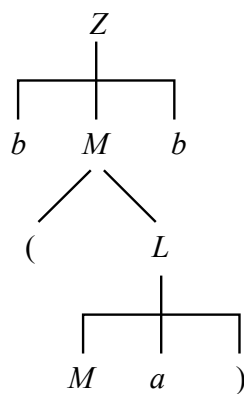


Рис. 5.28. Третье
сентенциальное
дерево

Основа	$M \quad a \quad)$
Отношения предшествования	$b \prec \bullet ($ $M \succ \bullet)$ $M \stackrel{\bullet}{=} a$ $a \stackrel{\bullet}{=})$ $) \succ \bullet b$



В соответствии с этим построим таблицу отношений R/S – матрицу предшествования.

$R \backslash S$	b	M	L	a	(	)
b		$\doteq$		$<\bullet$	$<\bullet$	
M	$\doteq$			$\doteq$		
L	$\bullet>$			$\bullet>$		
a	$\bullet>$			$\bullet>$		$\doteq$
(		$<\bullet$	$\doteq$	$<\bullet$	$<\bullet$	
)	$\bullet>$			$\bullet>$		

**АЛГОРИТМ РАЗБОРА
ДЛЯ ВОСХОДЯЩЕГО РАСПОЗНАВАТЕЛЯ
С ИСПОЛЬЗОВАНИЕМ ГРАММАТИК ПРЕДШЕСТВОВАНИЯ.
АЛГОРИТМ ФЛОЙДА (1963)**

Матрица предшествования представляется в виде таблицы Q с элементами q_{ij} , такими что:

$q_{ij} = 0$, если S_i и S_j несравнимы;

$q_{ij} = 1$, если $S_i <\bullet S_j$;

$q_{ij} = 2$, если $S_i \doteq S_j$;

$q_{ij} = 3$, если $S_i \bullet> S_j$.

Сами правила должны находиться в таблице, структура которой позволяет по заданной грамматике (правой части) найти содержимое и правило, а затем указать соответствующую левую часть.

Алгоритм Флойда. Символы входной цепочки просматриваются слева направо и помещаются в стек S до тех пор, пока символ вершины стека S_i не окажется в отношении предшествования « $\bullet>$ » с очередным анализируемым символом R . Это означает, что верхний символ стека является «хвостом» основы и, следовательно, основа уже в стеке.



Основу находят в списке правил приготовленной таблицы редукций и заменяют нетерминалом U . Процесс повторяется до тех пор, пока в качестве U не окажется Z (рис. 5.29).

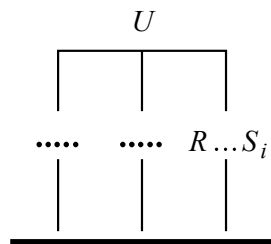


Рис. 5.29. К алгоритму Флойда

Блок-схема распознавателя в соответствии с алгоритмом Флойда изображена на рис. 5.30.

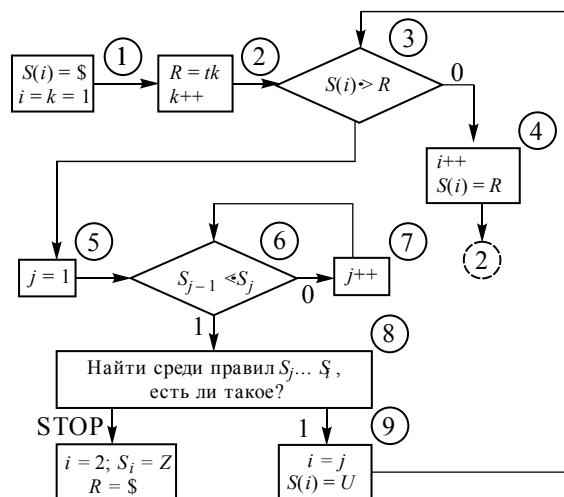


Рис. 5.30. Блок-схема распознавателя

На рис. 5.30 используются следующие обозначения: S – стек; i – счетчик стека; j – индекс адресации внутренних элементов стека; $\$t_1, \dots, t_n\$$ – формат анализируемого предложения; U – нетерминал, к которому приводится основа, найденная на шаге разбора; Z – начальный символ грамматики.

Значение блоков:

1, 2 – задание начальных условий;



3, 4 – выполнение условия: если S_i и R не находятся в отношении $S_i \bullet > R$, то не вся основа в стеке, поэтому символ R заносится в стек и поиск продолжается;

5, 6, 7 – S_i предшествует R ($S_i \bullet > R$), следовательно, основа уже в стеке. Необходимо найти «голову» основы (т. е. такое j , что $S_{j-1} < \bullet S_j$);

8, 9 – проверка цепочки $S_j \dots S_i$; если она не является правой частью редукции, то работа закончена с «хвостом» (ошибкой), в противном случае нужно исключить из стека $S_j \dots S_i$ и занести в стек символ U , который является левой частью ($U \rightarrow S_j \dots S_i$).

УПРАЖНЕНИЯ

Перевести в ПОЛИЗ инфиксные выражения 1–3.

1. $(x^2 + \sqrt{xy}) / (x + y)$.
2. $((x + y) * (x - y)) / (x^2 + y^2)$.
3. $(a + b)^2 - (a - b)^2$.

Вычислить выражения 4–7, записанные в форме ПОЛИЗ, причем в качестве операнда здесь используются цифры позиционной системы счисления.

4. 1 2 * 3 5 4 * + +
5. 1 2 3 5 4 * * + +
6. 1 2 + 3 * 5 + 4
7. 1 2 + 3 * 5 4 + -

8. Для каждого выражения ПОЛИЗ из упражнений 4–7 написать инфиксную форму.

9. Пусть $G[S]$ определена следующими продукциями:

- а) $S \rightarrow 0S1 \mid 01$
- б) $S \rightarrow +SS \mid -SS \mid a$
- в) $S \rightarrow S(S)S \mid \varepsilon$.

Построить для $G[S]$ синтаксически управляемый анализатор, работающий по методу рекурсивного спуска.

10. Построить алгоритм синтаксически управляемой трансляции перевода целых чисел в римские.

11. Написать алгоритм преобразования инфиксной цепочки словаря $V_T = \{a, b, +, *, (,)\}$ в постфиксную.

12. Написать алгоритм преобразования постфиксной цепочки словаря $V_T = \{a, b, +, *, (,)\}$ в инфиксную.



13. Написать алгоритм реализации синтаксического анализа арифметических выражений со словарем $V_T = \{a, b, +, *\}$ и соответствующими семантическими атрибутами преобразования инфиксной формы в ПОЛИЗ, используя схему Дейкстры.

14. Для оператора `if stmt then stmt [else stmt];` (`stmt` – оператор) представить схему алгоритма синтаксического анализа с диагностикой ошибок.

15. В языке C++ возможны два типа комментариев: обычный комментарий, ограниченный символами `«/*»` и `«*/»`, и строчный – начинающийся символами `«//»` и заканчивающийся концом строки. Построить сканер, который исключал бы из текста на языке C++ все комментарии.

16. Построить сканер, который исключал бы из текста на языке *Borland Pascal* все строковые константы. Строковые константы на этом языке состоят из строк, единичных символов и кодов, которые могут быть объединены между собой символом `«+»`. Строки ограничены символами `«“ ”»` (двойными кавычками), символы – `«' »` (одинарными кавычками), а коды начинаются с символа `«#»` (диз) и содержат только цифры.

17. Написать на языке C++ алгоритм бинарного поиска в таблице символов.

18. Что такое тупиковая ситуация и как ее избежать?

19. Какие существуют способы организации таблиц символов? Дать их сравнительную характеристику.

20. Какие проблемы необходимо разрешить при построении сканера на основе регулярных выражений?

21. В чем состоит нейтрализация ошибок и чем нейтрализация отличается от диагностики?

22. Позволяет ли КА организовать процедуру нейтрализации и диагностики ошибок?

23. Построить алгоритм в виде графа синтаксического анализа длинных целых констант языка C++.

24. В чем разница нисходящих и восходящих методов анализа?

25. Для каких грамматик целесообразно использовать восходящие методы анализа?

26. Дать определение грамматик простого предшествования.

27. Можно ли МП-автоматы использовать для анализа снизу вверх?

28. Каким образом изменится процедура восстановления от ошибок при восходящих схемах анализа?

6. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

В этом разделе приведены практические аспекты приложения теории языковых процессоров в производственной, научно-исследовательской и учебной практике. Приложение теоретических положений языковых процессоров в промышленной деятельности представлено препроцессором к станкам с ЧПУ. В научной и научно-производственной области широко используются инструментальные средства проектирования и анализа сложных динамических процессов. В разделе рассматривается инструментальная среда ИСМА [48, 51, 74] с оригинальными языками моделирования *LISMA* и *LISMA_PDE* [106]. Препроцессор описания на предметно-ориентированном языке *LISMA* [48] и анализа задачи Коши позволяет предметному пользователю проводить машинные эксперименты из области систем обыкновенных дифференциальных уравнений (ОДУ) и алгебро-дифференциальных уравнений (АДУ). Редактирование и анализ программных моделей на символично-графическом языке *LISMA* в оригинальном пользовательском интерфейсе (GUI) освобождает пользователя от необходимости дополнительных знаний численного анализа и программирования. В последнем подразделе рассмотрена новая версия языка моделирования гибридных систем *LISMA_PDE*, в котором к ОДУ и АДУ добавлена возможность описания определенного класса дифференциальных уравнений в частных производных (ДУЧП) [107]. Показано конструирование порождающей грамматики сложных динамических процессов.

6.1. ПРЕПРОЦЕССОР К СТАНКАМ С ЧПУ

Традиционно на станках с ЧПУ обработка деталей включала следующий предварительный технологический цикл:

1) составление инженером-технологом технологической карты с эскизом или чертежом соответствующей детали;

2) программирование инженером-программистом эскиза или чертежа детали в кодах процессора ЧПУ;

3) отладка на ЧПУ программы и устранение ошибок, допущенных в пунктах 1 и 2;

4) серийное изготовление детали по программе на станках с ЧПУ.

Недостатком такой технологической подготовки является низкая производительность подготовительного этапа в связи с тем, что два специалиста не могут исправить чужие ошибки: программист – технолога, а технолог – программиста.

Новая технология подготовительного этапа предполагает разработку препроцессора с языка, доступного непрофессиональному в программировании пользователю (инженеру-технологу). Тем самым необходимость в штатной единице программиста отпала. Инженер-технолог теперь может по своему эскизу или чертежу сам написать программу на предметном языке и, кроме того, не тратить времени на решение рутинных задач по расчету сложных точек пересечения геометрических фигур. Таким образом, производительность новой технологии в значительной степени выше, хотя бы потому, что штат уменьшился вдвое, а процессы написания программы и отладки резко сократились.

На рис. 6.1 приведен эскиз детали с геометрическими размерами для обработки на станке с ЧПУ. Движение фрезы начинается из точки T_0 в точку T_1 , T_2 и так далее в T_8 , T_1 . На чертеже представлены только те геометрические размеры, которые соответствуют необходимым и достаточным условиям определения траектории движения резца исполнительного механизма станка с ЧПУ.

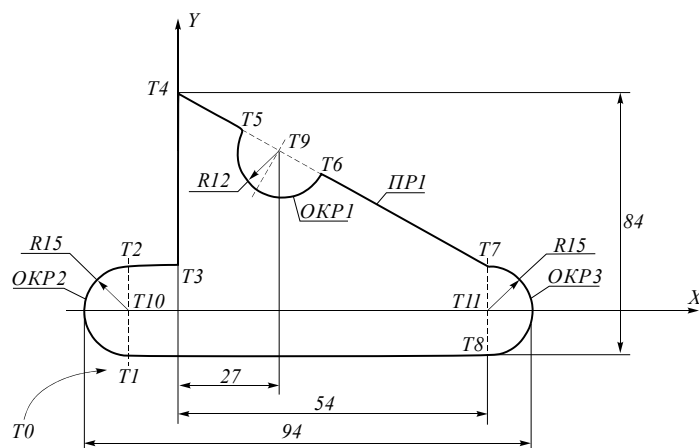


Рис. 6.1. Эскиз детали для обработки на станке с ЧПУ



Напишем программу, доступную не профессиональному в программировании пользователю-технологу, кодирующую геометрию детали.

```

ГЕОМ:  T0=-100, -200;
        T1=-10, -15;
        T2=-10, 15;
        T3=0, 15;
        T4=0, 69;
        T5=ПР1*ОКР1;
        T6=ОКР1*ПР1;
        T7=54, 15;
        T8=54, -15;
        T9=27, 27;
        T10=-10, 0;
        T11=54, 0;
        ОКР1=T9, R12;
        ОКР2=T10, R15;
        ОКР3=T11, R15;
        ПР1=T4, T7;

```

```

ТЕХН: T0, T1, ОКР2, T2, T3, T4, T5, ОКР1, T6, T7, ОКР3, T8, T1
КОНЕЦ

```

Знак отрицания «—» обозначает движение резца против часовой стрелки.

Подберем грамматику $G[<ПР>]$ по предложенному языку:

- 1) $<ПР> \rightarrow \text{ГЕОМ: } <\text{ТЕЛО ГЕОМ}> \text{ ТЕХН: } <\text{ТЕЛО ТЕХН}> \text{КОНЕЦ}$
- 2) $<\text{ТЕЛО ГЕОМ}> \rightarrow <\text{ЭЛ-Т}>; \{<\text{ЭЛ-Т}>;\}$
- 3) $<\text{ЭЛ-Т}> \rightarrow <\text{ТЧК}> \mid <\text{Объект}>$
- 4) $<\text{Объект}> \rightarrow <ПР> \mid <ОКР>$
- 5) $<\text{ТЧК}> \rightarrow \text{Т}<\text{ЦБЗ}> = <\text{Число}>, <\text{Число}> \mid$
 $\text{Т}<\text{ЦБЗ}> = <\text{Геом. Ф}>*<\text{Геом. Ф}>$
- 6) $<\text{Геом. Ф}> \rightarrow \text{ПР}<\text{ЦБЗ}> \mid \text{ОКР}<\text{ЦБЗ}>$
- 7) $<ПР> \rightarrow \text{ПР}<\text{ЦБЗ}> = \text{Т}<\text{ЦБЗ}>, \text{Т}<\text{ЦБЗ}>$
- 8) $<ОКР> \rightarrow \text{ОКР}<\text{ЦБЗ}> = \text{Т}<\text{ЦБЗ}>, \text{Р}<\text{ЦБЗ}>$
- 9) $<\text{ТЕЛО ТЕХН}> \rightarrow \text{Т}<\text{ЦБЗ}>\{, <\text{ТОКР}>\}$
- 10) $<\text{ТОКР}> \rightarrow \text{Т}<\text{ЦБЗ}> \mid \text{ОКР}<\text{ЦБЗ}>[-]$.

Принятые сокращения: ТЧК – точка, ОКР – окружность, ПР – прямая, ЦБЗ – целое без знака.

По классификации Хомского такая грамматика является контекстно-свободной.

СЕМАНТИЧЕСКИЙ АНАЛИЗАТОР

Функциями семантического анализатора в препроцессоре к оборудованию с ЧПУ являются организация таблиц ТЧК (точек) и ОКР (окружностей) для геометрии деталей, а также генерация параметров траектории движения резца для исполнительного механизма с ЧПУ с учетом сегмента ТЕХН.

Формирование таблицы ТЧК. Таблица имеет следующую структуру:

N	X	Y

Здесь N – номер точки; X , Y – ее координаты.

Воспользуемся первой альтернативой правила 5 от $G[<ПР>]$. Тогда

$$<ТЧК> \rightarrow T<ЦБЗ>_{y_1} = <Число>_{y_2}, <Число>_{y_3},$$

где y_1 – семантическая процедура преобразования ЦБЗ из символьной формы в числовую и запись этого целого в соответствующую позицию N таблицы ТЧК; y_2 – процедура преобразования числа из символьной формы в числовую и запись в позицию X таблицы ТЧК; y_3 – то же самое, но запись в позицию Y таблицы ТЧК.

Вторая альтернатива правила 5 семантически обрабатывается на втором проходе препроцессора.

Формирование таблицы ОКР. Таблица имеет следующую структуру:

N	ТЦ		R	S
	X	Y		

Здесь N – номер окружности; R – радиус; S – признак движения резца по часовой стрелке ($S = 0$), или против нее ($S = 1$). По умолчанию $S = 0$. ТЦ – точка центра окружности с координатами (X , Y).

Добавим к синтаксису окружности семантические атрибуты, тогда

$$<ОКР> \rightarrow ОКР<ЦБЗ>_{y_4} = T<ЦБЗ>_{y_5}, R<ЦБЗ>_{y_6},$$



где y_4 – процедура преобразования ЦБЗ и запись целого числа в позицию N таблицы ОКР; y_5 – преобразование ЦБЗ в целое i , а также поиск i по таблице ТЧК значений координат точек (X_i, Y_i) и их запись в позицию ТЦ таблицы ОКР; y_6 – преобразование числа и его запись в позицию R таблицы ОКР.

Формирование таблицы ПР. Таблица имеет следующую структуру:

N	$X1$	$Y1$	$X2$	$Y2$

Из синтаксиса с учетом семантики имеем

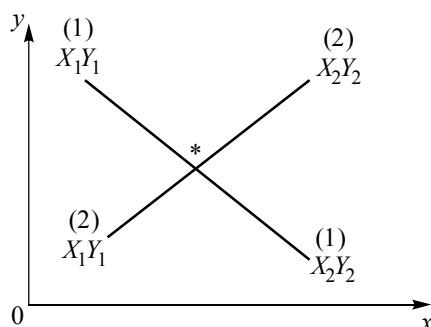
$$\langle \text{ПР} \rangle \rightarrow \text{ПР} \langle \text{ЦБЗ} \rangle y_7 = \text{T} \langle \text{ЦБЗ} \rangle y_8, \text{T} \langle \text{ЦБЗ} \rangle y_9,$$

где y_7 – процедура преобразования $\langle \text{ЦБЗ} \rangle$ и запись целого числа в позицию N таблицы ПР; y_8 – преобразование $\langle \text{ЦБЗ} \rangle$, поиск соответствующего номера N по таблице ТЧК и запись найденных координат (X_i, Y_i) в позиции соответственно X_1 и Y_1 таблицы ПР; y_9 – то же самое, что и y_8 , только запись в X_2 и Y_2 .

Уточнение координат точки. Пересечение геометрических фигур. Второй проход:

$$\begin{aligned} &\langle \text{ТЧК} \rangle \rightarrow \langle \text{Объект} \rangle * \langle \text{Объект} \rangle | \\ &\langle \text{ПР} \rangle * \langle \text{ПР} \rangle y_{10} | \\ &\langle \text{ОКР} \rangle * \langle \text{ПР} \rangle y_{11} | \\ &\langle \text{ПР} \rangle * \langle \text{ОКР} \rangle y_{12} | \\ &\langle \text{ОКР} \rangle * \langle \text{ОКР} \rangle y_{13}. \end{aligned}$$

Процедура y_{10} : пересечение прямых.



Для уравнений прямых, заданных двумя точками, имеем

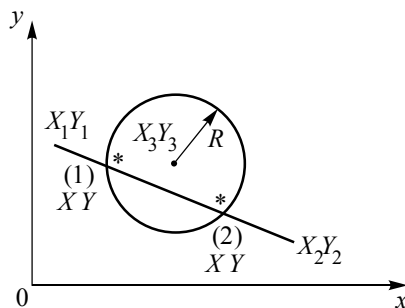
$$\frac{X - X_1^{(1)}}{X_1^{(1)} - X_2^{(1)}} = \frac{Y - Y_1^{(1)}}{Y_1^{(1)} - Y_2^{(1)}}; \quad (6.1)$$

$$\frac{X - X_1^{(2)}}{X_1^{(2)} - X_2^{(2)}} = \frac{Y - Y_1^{(2)}}{Y_1^{(2)} - Y_2^{(2)}}.$$

Совместное решение уравнений (6.1) дает точку пересечения (X^*, Y^*) .

Таким образом, процедура y_{10} организует поиск по таблице ПР соответствующих пар точек $X_1^{(1)}, X_2^{(1)}, Y_1^{(1)}, Y_2^{(1)}$ для первой прямой и точек второй прямой. Далее процедура вызывает стандартный пакет решения линейных алгебраических уравнений и, учитывая найденные точки как исходные данные, определяет точку пересечения (X^*, Y^*) и заносит эту точку в таблицу ТЧК.

Процедура y_{11} : окружность пересекается с прямой.



В этом случае имеем

$$\frac{X - X_1}{X_1 - X_2} = \frac{Y - Y_1}{Y_1 - Y_2}; \quad (6.2)$$

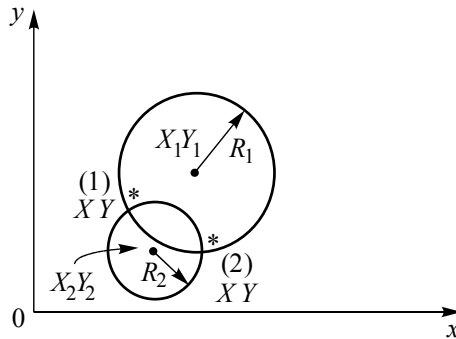
$$(X - X_3)^2 + (Y - Y_3)^2 = R^2.$$



Совместное решение уравнений (6.2) дает две пары точек $(X^{(1)}, Y^{(1)})$, $(X^{(2)}, Y^{(2)})$. Таким образом, процедура y_{11} осуществляет поиск по таблицам ПР и ОКР координат (X_1, Y_1) , (X_2, Y_2) , (X_3, Y_3) и R . Затем выполняется вызов стандартного пакета для совместного решения уравнений (6.2), определение пар $(X^{(1)}, Y^{(1)})$, $(X^{(2)}, Y^{(2)})$ и запись пары $X_1^{(1)}, Y_1^{(1)}$ в таблицу ТЧК.

Процедура y_{12} работает аналогично y_{11} , только выполняется запись значений $X_2^{(2)}, Y_2^{(2)}$.

Рассмотрим пересечение окружностей.



Для уравнения окружностей имеем

$$\begin{aligned} (X - X_1)^2 + (Y - Y_1)^2 &= R_1^2; \\ (X - X_2)^2 + (Y - Y_2)^2 &= R_2^2. \end{aligned} \quad (6.3)$$

Совместное решение уравнений (6.3) дает следующее решение: две пары точек $(X^{(1)}, Y^{(1)})$, $(X^{(2)}, Y^{(2)})$.

Таким образом, y_{12} осуществляет поиск данных по таблице ОКР для первой и второй окружности (R и точка центра). По стандартному пакету решения нелинейных систем уравнений определяется пара $(X^{(1)}, Y^{(1)})$, $(X^{(2)}, Y^{(2)})$ и в зависимости от S выполняется выбор соответствующей пары и ее запись в таблицу ТЧК.

Семантика технологии. Добавим семантические атрибуты в соответствующие продукции (9, 10) грамматики $G[<ЧПУ>]$:

$$\begin{aligned} <ТЕЛО ТЕХН> \rightarrow T<ЦБЗ>y_{15} \{,<ТОКР>\} \\ <ТОКР> \rightarrow T<ЦБЗ>y_{16} \mid ОКР<ЦБЗ>[\neg y_{14}]y_{17}, \end{aligned}$$



где задача y_{14} – установить $S = 1$ и записать в соответствующее место таблицы ОКР; y_{15} – преобразовать ЦБЗ, по таблице ТЧК найти точку, передать координаты точки в исполнительный механизм привода станка с ЧПУ, привести в движение исполнительный механизм, установить резец в данную точку; y_{16} – аналогично y_{15} ; y_{17} – по таблице ОКР передать данные окружности в исполнительный механизм привода станка с ЧПУ и произвести движение резцом по окружности в соответствующем направлении в зависимости от данных.

6.2. ИНТЕРПРЕТАТОР ЗАДАЧИ КОШИ

Постановка интерпретации решения задачи Коши состоит в следующем. Рассмотрим задачу Коши

$$y' = f(t, y), \quad y(t_0) = y_0, \quad t_0 \leq t \leq t_k, \quad (6.4)$$

где y и f – гладкие вещественные N -мерные вектор-функции; t – независимая переменная, изменяющаяся на заданном интервале $[t_0, t_k]$.

В этом классе систем обыкновенных дифференциальных уравнений могут быть представлены модели из различных областей. Например, модели популяций [21] и иммунных ответов [44] в биосистемах, динамические процессы в технических системах (автоматика [19, 46], электромеханика [25, 51], машиностроение и др.). Далеко не все предметные специалисты (биологи, медики, электромеханики) владеют достаточными знаниями для реализации предметных моделей на ЭВМ. К тому же если системы из класса задач (6.4) обладают жесткостью [24, 50], то при этом необходимо использовать специальные нетрадиционные схемы интегрирования [20–25, 48].

Ставится задача обеспечения предметного пользователя инструментом, который позволит ему ограничиться только задачами адекватной идентификации своих моделей с активным привлечением машинного эксперимента для синтеза и анализа систем.

Сформулируем требования к проектированию интерпретатора.

1. Язык описания задачи (6.4) должен быть простым и естественным с точки зрения близости к математическому описанию.



2. Численное интегрирование системы дифференциальных уравнений должно опираться на библиотеку разнообразных численных схем для решения гладких и жестких задач.

3. Интерпретатор должен быть реализован как открытая система для других приложений.

4. Синтаксический анализатор интерпретатора должен обеспечить содержательную и наглядную диагностику и нейтрализацию ошибок.

5. Результаты решения должны быть наглядными, с широкими возможностями геометрической интерпретации и манипуляции графическими данными.

Далее приведем методику реализации интерпретатора с удовлетворением перечисленных требований.

ЯЗЫК LISMA

Программа на этом языке состоит из трех частей:

- описание макросов правых частей нефазовых переменных;
- описание начальных условий фазовых переменных и констант;
- описание системы дифференциальных уравнений.

Макросом будем называть описание конкретных арифметических выражений на входном языке. Ниже приведено точное формальное определение всех категорий, в том числе и макросов. После каждого определения макроса, начальных условий, констант и описания дифференциального уравнения должна стоять точка с запятой. Текст программы может быть отредактирован в любом текстовом редакторе, не должен содержать никаких специальных символов и сохраняется как текстовый файл. В тексте программы можно использовать комментарии. При обработке введенного текста комментарии игнорируются.

С учетом определенных ограничений приведем грамматику $G[<LISMA>]$ языка описания задачи Коши и макросредств.

Грамматика языка $G[<LISMA>]$

1. $<LISMA> \rightarrow <Предложение> \{<Предложение>\}$
2. $<Предложение> \rightarrow <Макрос> ; | <Начальные условия> ; | <Уравнение> | <Комментарий>$
3. $<Макрос> \rightarrow \text{масго } <ИД> (<ИД> \{, <ИД>\}) = <AB>$
4. $<Начальные условия> \rightarrow <ИД> = <Число>$



5. $\langle \text{Уравнение} \rangle \rightarrow \langle \text{ИД} \rangle ' = \langle \text{AB} \rangle$
6. $\langle \text{Комментарий} \rangle \rightarrow // \{ \langle \text{Любой терминал} \rangle \} \langle \text{Конец строки} \rangle$
7. $\langle \text{AB} \rangle \rightarrow \langle \text{T} \rangle \{ \langle \text{знак} + - \rangle \langle \text{T} \rangle \}$
8. $\langle \text{T} \rangle \rightarrow \langle \text{C} \rangle \{ \langle \text{знак} * / \rangle \langle \text{C} \rangle \}$
9. $\langle \text{знак} + - \rangle \rightarrow + | -$
10. $\langle \text{знак} * / \rangle \rightarrow * | /$
11. $\langle \text{C} \rangle \rightarrow \langle \text{O} \rangle ^ \langle \text{O} \rangle$
12. $\langle \text{O} \rangle \rightarrow (\langle \text{AB} \rangle) | \langle \text{ИД} \rangle | \langle \text{Вызов функции или макроса} \rangle | \langle \text{ЧислоБЗ} \rangle$
13. $\langle \text{Вызов функции или макроса} \rangle \rightarrow \langle \text{ИД} \rangle (\langle \text{AB} \rangle \{ , \langle \text{AB} \rangle \})$
14. $\langle \text{Число} \rangle \rightarrow + | - \langle \text{ЧислоБЗ} \rangle$
15. $\langle \text{ЧислоБЗ} \rangle \rightarrow \langle \text{ЦБЗ} \rangle [. \langle \text{ЦБЗ} \rangle] [\langle \text{Степень} \rangle]$
16. $\langle \text{ЧислоБЗ} \rangle \rightarrow . \langle \text{ЦБЗ} \rangle [\langle \text{Степень} \rangle] | \langle \text{Степень} \rangle$
17. $\langle \text{Степень} \rangle \rightarrow \text{E} [+ | -] \langle \text{ЦБЗ} \rangle$
18. $\langle \text{ИД} \rangle \rightarrow \langle \text{Б} \rangle \{ \langle \text{Б} \rangle | \langle \text{Ц} \rangle \}$
19. $\langle \text{ЦБЗ} \rangle \rightarrow \langle \text{Ц} \rangle \{ \langle \text{Ц} \rangle \}$
20. $\langle \text{Б} \rangle \rightarrow \text{A} | \text{B} | \text{C} | \dots | \text{X} | \text{Y} | \text{Z} | \text{a} | \text{b} | \text{c} | \dots | \text{x} | \text{y} | \text{z}$
21. $\langle \text{Ц} \rangle \rightarrow 0 | 1 | 2 | \dots | 7 | 8 | 9$

Идентификаторы

Имена переменных и констант, названия макросов и встроенных функций представляют собой строку текста на английском языке с некоторыми ограничениями и дополнениями. Идентификаторы обязательно должны начинаться с буквы латинского алфавита, а далее могут включать буквы, цифры и символ подчеркивания «_». Длина значения не имеет. В дальнейшем идентификаторы будут встречаться в составе многих программных единиц языка и обозначаться $\langle \text{ИД} \rangle$.

Представление чисел

Каких-либо типов данных при записи чисел в языке *LISMA* не существует. Числа представлены в их естественном виде и могут быть описаны по одному из следующих форматов.

- $$\begin{aligned} \langle \text{ЧислоБЗ} \rangle &\rightarrow \langle \text{ЦБЗ} \rangle [. \langle \text{ЦБЗ} \rangle] [\langle \text{Степень} \rangle] \\ \langle \text{ЧислоБЗ} \rangle &\rightarrow . \langle \text{ЦБЗ} \rangle [\langle \text{Степень} \rangle] | \langle \text{Степень} \rangle \end{aligned}$$



<Степень> $\rightarrow$ E[+ | -] <ЦБЗ>

Примеры чисел, записанных по данному формату: 5567, 45.789, 124.567E-765, 345E+10, 345E 10, E-10, E-32.

Арифметическое выражение

Арифметические выражения используются при описании дифференциальных уравнений (правая часть) и при определении макросов. Запись на языке *LISMA* похожа на математическую форму. В арифметических выражениях можно использовать все математические операции: «+» – сложение; «-» – вычитание; «*» – умножение; «/» – деление; «^» – возведение в степень.

В качестве операндов арифметических выражений могут быть встроенные функции: $\exp(x)$ – вычисление экспоненты; $\sin(x)$ – вычисление синуса (угол в радианах); $\cos(x)$ – вычисление косинуса (угол в радианах); $\operatorname{tg}(x)$ – вычисление тангенса (угол в радианах); $\arcsin(x)$, $\arccos(x)$, $\operatorname{arctg}(x)$ – вычисление соответствующих обратных тригонометрических функций; $\operatorname{hsin}(x)$ – вычисление гиперболического синуса; $\operatorname{hcos}(x)$ – вычисление гиперболического косинуса; $\operatorname{th}(x)$ – вычисление гиперболического тангенса; $\lg(x)$ – вычисление десятичного логарифма; $\ln(x)$ – вычисление натурального логарифма.

В качестве операндов в арифметических выражениях также используются числа без знака, переменные, константы и арифметические выражения в круглых скобках.

Далее арифметические выражения будут встречаться в описаниях форматов многих программных конструкций и обозначаться <AB>.

Запись дифференциальных уравнений

Каждое дифференциальное уравнение имеет следующий формат:

<ИД>' = <AB>.

Имя фазовой переменной – это «ИДентификатор». Далее ставится знак «'» и знак равенства, после которого следует запись выражения для производной от соответствующей переменной. Правая часть уравнения является арифметическим выражением. Пример записи системы дифференциальных уравнений приведен на рис. 6.2.

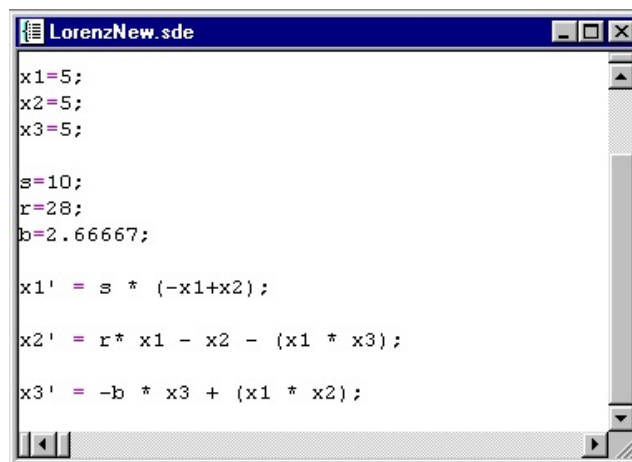


Рис. 6.2. Запись системы дифференциальных уравнений

Описание начальных условий переменных

Описание начальных условий имеет следующий синтаксис:

$\langle \text{ИД} \rangle \rightarrow \langle \text{Число} \rangle$.

Слева пишется имя переменной, справа – ее значение в начальный момент времени. Задание начальных условий возможно в любой части программы. Повторная запись начальных условий однозначно контролируется анализатором. По умолчанию если начальные условия для какой-то фазовой переменной не заданы, то ее значение считается равным нулю. На рис. 6.2 видно задание начальных условий для переменных x_1 , x_2 , x_3 в первых трех операторах присваивания.

Описание констант

Часто необходимо использовать символьное обозначение параметров модели, а значение параметра задавать и изменять в одном месте. Для этого используются константы. Описание константы аналогично записи начальных условий для переменной:

$\langle \text{ИД} \rangle \rightarrow \langle \text{Число} \rangle$.

На рис. 6.2 приведен пример использования констант s , r , b .



Использование макросов

Наиболее быстро и удобно строить программы позволяет использование макросов. Часто повторяющиеся фрагменты систем дифференциальных уравнений можно оформить в виде отдельного описания – макроса, а затем вставить вызов этого макроса в нужное место. Рекомендуется также использовать макросы для описания частей уравнений, имеющих какое-то законченное логическое значение. Например, входной сигнал, сложный расчет скорости или давления и т. п. Кроме того, при вызове макроса в него можно передать какие-либо данные – значения переменных, констант или рассчитываемые арифметические выражения. Макросы могут быть описаны в любом месте программы (даже после их вызова в дифференциальных уравнениях). Их синтаксис:

macro <Имя макроса>(<Список параметров>) = <АВ>.

Ключевое слово **macro** определяет задание макроса. За ним следует имя макроса, являющееся идентификатором. Далее в круглых скобках записывается список параметров, представляющий собой один идентификатор или несколько, разделенных запятыми. После знака «=» находится арифметическое выражение, в нем можно использовать переменные из списка параметров. Вычислив арифметическое выражение, программа возвращает значение макроса. Определение и пример вызова макроса приведен на рис. 6.3.

```

//Параметры транзистора (КП913А)
k=0.7; //коэффициент аппроксимации
S=2.72;
us=12.6; is=11; //координаты точек перегиба
                // передаточной характеристики
//C11-входная емкость транзистора
C11=3.5E-10;
C22min=1.3E-10; C=6.59E-10; D=0.33;
C12min=8E-12; A=3.5E-11; B=0.088;

//C22-выходная емкость транзистора
macro C22 (U) =C22min+C*exp (-D*U) ;
//C12-проходная емкость транзистора
macro C12 (U) =C12min+A*exp (-B*U) ;

//Вольт-Амперная характеристика транзистора
macro Is (uzi, usi) =M(uzi) * (1-exp (-k*usi*S/M(uzi))) ;
macro M (uzi) =is* (1+th (S* (uzi-us) / is)) ;
  
```

Рис. 6.3. Использование макросов



На рис. 6.3 представлена часть описания ключа на МДП-транзисторе, видно описание четырех макросов. В макросе **ls** используется макрос **M**, причем вызов макроса **M** происходит до его описания.

Использование комментариев

Для облегчения понимания текста программы и снабжения ее пояснительными надписями в языке предусмотрены такие элементы, как комментарии. Комментарием считается строка или часть строки, начинающаяся с символов «**//**». Комментарии можно использовать в любом месте программы – после точки с запятой в описании макросов, начальных условий переменных, констант и дифференциальных уравнений и в качестве самостоятельных элементов. Пример использования комментариев представлен на рис. 6.3.

Предопределенные константы и переменные

Предопределенные переменные и константы предоставляют удобный доступ к сервису системы. Они дополняют диалоговые средства задания параметров моделирования, а также позволяют записывать переменные, производные которых зависят от времени. В пользовательской программе можно использовать следующие имена:

- 1) **time** – переменная, значение которой равно текущему времени моделирования, ее можно использовать в правой части дифференциальных уравнений;
- 2) **step** – константа, значение шага моделирования;
- 3) **fulltime** – константа, значение длины интервала моделирования.

Константы **step** и **fulltime** использовать не обязательно. Если их нет в программе, то значение предлагается системой. В любом случае значение этих параметров моделирования можно изменить в диалоговом режиме.

ОПИСАНИЕ ИНФОРМАЦИОННОГО ОБЕСПЕЧЕНИЯ

Рассмотрим информационную модель и логические связи интерпретатора с входного языка *LISMA*. На схеме (рис. 6.4) показан процесс прохождения информации и передачи управления между модулями в системе. От пользователя на вход текстового редактора,



реализованного в пользовательском интерфейсе GUI, поступают команды написания и редактирования дифференциальных уравнений. Отредактированная на входном языке программа поступает на вход интерпретатора. Интерпретатор анализирует полученный текст на наличие синтаксических ошибок.

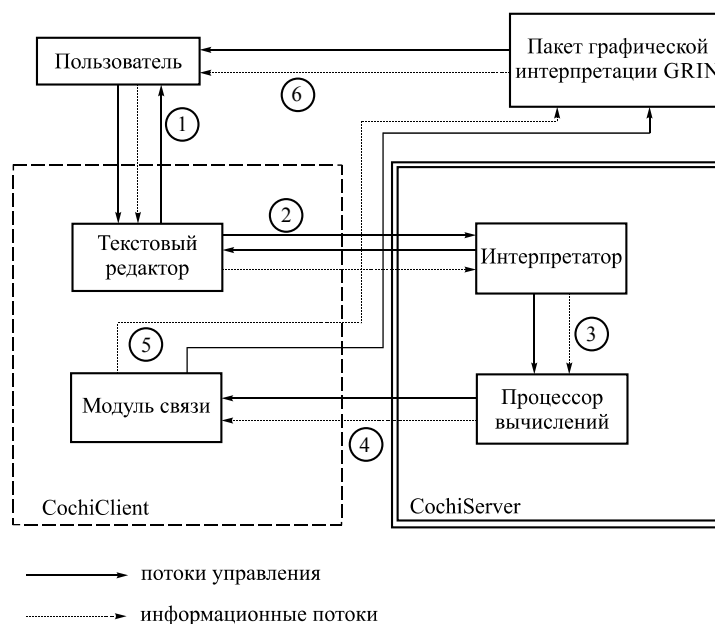


Рис. 6.4. Информационные связи

Если ошибки есть, то управление передается обратно текстовому редактору с сообщением о положении и типах ошибок. При отсутствии синтаксических ошибок интерпретатор генерирует внутреннее представление системы дифференциальных уравнений и разрешает клиенту использование процессора вычислений. Далее управление передается процессору вычислений для получения решения в числовом массиве. Массив точек решения передается на вход модуля связи с интерпретатором решений, который далее передает его в модуль графической интерпретации решений GRIN [80, 81]. Информационные потоки, отмеченные цифрами в окружности на рис. 6.4, имеют следующее значение:

- 1) информация о вводе и изменении текста;
- 2) текст программы на языке описания задачи Коши;

3) внутреннее представление задачи Коши в виде таблиц переменных, констант, макросов, функций, массива дифференциальных уравнений;

4) массив точек графика;

5) графическая интерпретация результатов моделирования.

Блок «Интерпретатор» изображен на рис. 6.5.



Рис. 6.5. Интерпретатор

На вход блока подается исходный текст программы на языке *LISMA*. Сканер производит декомпозицию исходного текста на определенные символьные единицы (лексемы) и передает на выход информацию в виде кодов лексем. Если сканер не может сопоставить текстовый элемент ни с одной из известных лексем, то выдается сообщение об ошибке. Синтаксический анализатор проверяет соответствие программных единиц грамматике языка, при возникновении ошибки о ней выдается информация. Управление семантическому анализатору передается, только если не было ошибок на предыдущих этапах. В этом блоке происходит смысловая обработка входного текста, при возникновении ошибок о них выдается информация.



Наконец, блок генерации занимается формированием внутреннего представления системы дифференциальных уравнений – с помощью синтаксически ориентированного алгоритма Гриса переводит арифметические выражения в ПОЛИЗ. Затем полученные данные передаются в процессор численного анализа.

СКАНЕР

Функции сканера (лексера) в данном случае связаны в основном с очисткой от «мусора». Производится очистка введенного текста от комментариев, а также сортировка введенных параметров в тексте по типам и по порядку следования. Первая функция состоит в удалении ненужных символов, вторая функция упрощает грамматику. Поставленные перед сканером цели обосновывают назначение сканера как однопроходного лексического анализатора [1], результатом работы которого является очищенная от вспомогательных символов программа, соответствующая той же самой грамматике с той лишь разницей, что при дальнейшем анализе нет смысла проверять правильность порядка следования операторов описания числовых констант, начальных условий, макросов и дифференциальных уравнений.

СИНТАКСИЧЕСКИЙ АНАЛИЗАТОР

Целью синтаксического анализатора являются проверка соответствия программы грамматике $G[<LISMA>]$ и диагностика синтаксических ошибок.

По классификации Хомского [1] грамматика $G[<LISMA>]$ относится к классу контекстно-свободных, поэтому основными требованиями при анализе являются однозначность и безвозвратность разбора. Известно, что для грамматик этого типа эффективным методом анализа является рекурсивный спуск [75, 79], который обладает следующими преимуществами над остальными:

- метод является однозначным и безвозвратным;
- реализация рекурсивного спуска на языке высокого уровня облегчает программирование и отладку. В частности, нет необходимости создавать стековый механизм и обеспечивать управление им;
- работа автомата с магазинной памятью заменяется здесь вызовом соответствующих процедур. Для разработанной грамматики



это тем более важно, что количество процедур не так велико и основная из них – разбор арифметического выражения;

- метод рекурсивного спуска обладает семантической совместимостью. Таким образом, процедуры синтаксиса не изменяются, а только дополняются семантическими процедурами, что значительно упрощает этап синтеза при трансляции.

Эти преимущества позволяют легко реорганизовать некоторые части анализатора при модификации грамматических правил вывода. Все сказанное обосновывает выбор метода рекурсивного спуска для анализа программ.

СЕМАНТИЧЕСКИЙ АНАЛИЗАТОР

При семантической обработке происходит синтез структур данных для процессора вычислений. Как уже отмечалось, выбранный метод рекурсивного спуска для разбора входного текста обладает гибкостью и по отношению к семантическому анализу. С этой целью в любое место синтаксических процедур можно включить семантические атрибуты.

Соответствующие рекурсивные процедуры, которые используются в синтаксическом анализаторе, дополняются вызовом семантических. Семантика при обработке <AB> подробно рассмотрена в разделе 5.6. Важной особенностью разработанного интерпретатора является оптимизация ПОЛИЗ для <AB>.

ОПТИМИЗАЦИЯ РАСЧЕТА ПОЛИЗ

Как уже отмечалось, структурирование данных, в дальнейшем неоднократно участвующих в расчетах, – это процесс, который требует особого внимания со стороны разработчика. По сути, на скорость расчетов влияет много аспектов. Одним из самых важных параметров при постоянном обращении к данным является способ представления этих данных. В нашем случае данные содержат в себе не простое числовое или текстовое значение, а особую структуру, которая описывает арифметическое выражение. В этом выражении кроме числовых значений имеются еще и переменные, значения которых нужно подставить из списка переменных. На расчет значения такой структуры в этом случае тратится большая часть времени. Это дополнительно усугубляется тем, что



с повышением сложности выражения время расчета правой части существенно возрастает.

Кроме того, специфика записи арифметического выражения в виде ПОЛИЗ требует ее «свертки» при каждом новом расчете выражения. Это приводит к постоянному изменению объема используемой приложением оперативной памяти и, поскольку такой объем может быть достаточно большим, также существенно влияет на быстродействие всей системы.

Таким образом, одного представления данных в виде ПОЛИЗ недостаточно для эффективного проведения расчета. Действительно, сложные выражения, содержащие много чисел и констант, могут быть значительно упрощены, если воспользоваться приемом так называемых *константных вычислений*. Остановимся на этом более подробно.

Пусть c – символьная константа; num – числовая константа по определению. Тогда при просмотре ПОЛИЗ для $\langle AB \rangle$ необходимо предварительно (перед численным расчетом) свернуть, или произвести вычисления синтаксических конструкций вида

$$(c \mid num)^2 \langle \text{знак операции} \rangle.$$

После таких расчетов правая часть примет вид

$$(\langle \text{операнд} \rangle \mid \langle \text{знак операции} \rangle)^n, n > 2$$

или вообще станет числовой константой – num .

Этот алгоритм преобразования правой части, несомненно, облегчит дальнейший пошаговый численный расчет. Причем чем выше порядок системы дифференциальных уравнений и больше дискретных шагов расчета, тем сравнительно меньшее время будет затрачено на решение задачи Коши.

Эффективность оптимизации правой части дифференциальных уравнений становится еще более очевидной при использовании многошаговых методов, когда вычисление правой части на одном шаге интегрирования представляет собой многоитерационную процедуру.

Этап семантического анализа является последним этапом обработки входного текста и синтеза образа модели в виде упорядоченных структур данных. На каждом этапе трансляции программной модели могут возникнуть ошибки, на которые система моделирования должна соответствующим образом отреагировать. Остановимся более подробно на диагностике различного рода ошибок.



ДИАГНОСТИКА И НЕЙТРАЛИЗАЦИЯ ОШИБОК

Одной из целей синтаксического анализатора является диагностика синтаксических ошибок [1, 6], причем он должен сообщить все синтаксические ошибки программы, а для этого нужно произвести нейтрализацию встретившейся ошибки и продолжить разбор. Известно [1, 3, 6, 75], что нет общих методов нейтрализации синтаксических ошибок. В каждом трансляторе эта проблема решается по-своему в зависимости от грамматики и выбранного метода трансляции. Для данной грамматики представляется удобным использовать идею, впервые предложенную Айронсом [1] для нисходящих распознавателей. Особенность грамматики заключается в построчном описании операций, разделенных символом «;». При этом нейтрализация ошибок в каждой строке состоит в исключении терминалов до «;» – тем самым производится исключение только одного неверного оператора. Синтаксически верные тексты подвергаются следующему этапу трансляции – семантическому анализу [75].

На стадии семантического анализа могут возникнуть семантические, или смысловые, ошибки. Нейтрализация этих ошибок производится аналогично синтаксическим, так как семантический распознаватель работает в соответствии с правилами формальной грамматики в синтаксически-управляемом режиме.

ПРОЦЕССОР ВЫЧИСЛЕНИЙ

После синтаксической и семантической обработки текста, когда модель в виде внутренней структуры данных уже построена, необходимо получить данные о параметрах моделирования системы (интервал моделирования, шаг, метод, точность и др.). После того как данные интерактивно получены от пользователя, процессор готов к проведению вычислений. Вычисления происходят следующим образом. По запросу клиентского приложения процессор вычисляет значения всех фазовых переменных на текущем шаге, а также осуществляет увеличение переменной времени моделирования на величину шага, после чего отправляет полученные данные (информацию об ошибке или об окончании моделирования) программе-клиенту. Повторный запрос клиента даст значения переменных на следующем шаге и т. д. Вычисления дискретных значений фазовых переменных могут производиться по различным



алгоритмам, выбор которых определяется выбором метода интегрирования. Библиотека численных схем в системе ограничена следующими методами: метод Эйлера; модифицированный метод Эйлера; метод Рунге – Кутты; метод Рунге – Кутты – Мерсона; метод Stek (модификация метода Мерсона); метод Фельберга; метод Милна – Симпсона; метод Адамса – Башфорта.

ИНТЕРПРЕТАЦИЯ РЕШЕНИЯ ЗАДАЧИ КОШИ

Модуль графической интерпретации GRIN (GRaphical Interpretation) [81] предназначен для графической обработки результатов моделирования. Основной целью разработки этой части является удобное для конечного пользователя представление данных. Графическое представление информации позволяет ему быстро и легко обрабатывать решения задач, наглядно представлять и понимать суть происходящих процессов.

Разработанный сервис позволяет пользователю:

- отображать графику линиями различной ширины и цвета; сопровождать графику текстовой информацией;
- выполнять сплошное и сеточное фонирование графического поля с произвольным шагом сетки;
- выполнять трассировку (получать точные координаты анализируемых процессов из их графического представления);
- устанавливать контрольные точки (маркеры);
- копировать графику из одного окна в другое;
- управлять масштабом рабочей области и отображением осей координат;
- сохранять графическую интерпретацию;
- получать жесткую копию;
- управлять окнами и их количеством;
- фрагментировать графическое поле.

Число графиков в одном окне не ограничивается и определяется самим пользователем. При большом числе графиков в одном окне информация становится сложной для восприятия, поэтому пользователь может переносить графики в другие окна. Число окон также не ограничивается. Таким образом, пользователь может настроить отображение графического представления результатов моделирования наиболее удобным для себя образом.



Многозадачный графический интерфейс пользователя (Graphical User Interface, GUI) позволяет реализовать более богатую графику и наполнить программу функционально. Передача данных между частями пакета, осуществляющими расчет модели, и модулем GRIN осуществляется через файлы на жестком диске. Формат файла фиксирован, поэтому исходные числовые данные для графической интерпретации могут передаваться модулю GRIN из любой другой прикладной программы. Гибкость системы заключается в том, что она не привязана к конкретному типу отображаемых данных и источнику, от которого эти данные получены. С помощью GRIN можно обрабатывать любую численную информацию, для этого необходимо только записать ее в файл во входном формате программного модуля. Таким образом, GRIN может выступать в качестве самостоятельного приложения и служить для графической обработки различной цифровой информации.

МАШИННЫЕ ЭКСПЕРИМЕНТЫ

Рассмотрим конкретные задачи Коши для иллюстрации подготовки данных для интерпретатора Coshy и графической интерпретации решения. Подготовка данных в приведенных задачах осуществляется встроенным в систему редактором CoshyEdit. Программная модель в окне редактора соответствует языку порожаемой грамматики $G[<LISMA>]$. Окно встроенного редактора содержит необходимые кнопки и соответствующую инструментальную панель. Кроме того, в окне подготовлены два поля: верхнее и нижнее. Верхнее поле предназначено для редактирования входного текста. Нижнее поле заполняется диагностическими сообщениями при работе синтаксического анализатора.

Машинные эксперименты охватывают различные области науки и техники, включая машинные эксперименты с моделями повышенной жесткости [46, 49, 51].

Пример 6.1

Теория ядерных реакций.

Исходная система уравнений:

$$y_1' = 0,01 - [1 + (y_1 + 1000) * (y_1 + 1)] * (0,01 + y_1 + y_2);$$

$$y_2' = 0,01 - (1 + y_2^2) * (0,01 + y_1 + y_2).$$



Начальные условия:

$$y_1(0) = 0; \quad y_2(0) = 0.$$

Запись в виде программной модели на языке LISMA:

```

Редактор уравнений в формате Коши - [02.diff]
Файл  Правка  Вид  Модель  Окно  Помощь

y1' = 0.01 - (1 + (y1 + 1000) * (y1 + 1)) * (0.01 + y1 + y2);
y2' = 0.01 - (1 + y2^2) * (0.01 + y1 + y2);

Строка : 4, Столбец : 1
  
```

Результаты моделирования представлены на рис. 6.6.

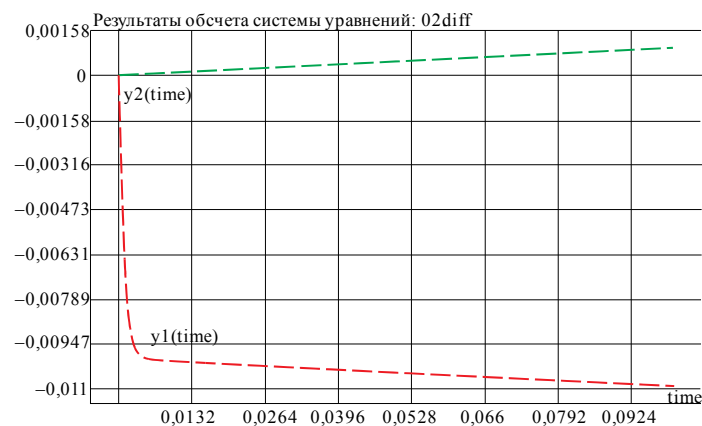


Рис. 6.6. Результаты моделирования фазовых переменных $y_1(t)$, $y_2(t)$

Пример 6.2

Микробиология (рост противоборствующих популяций).

Исходная система уравнений:

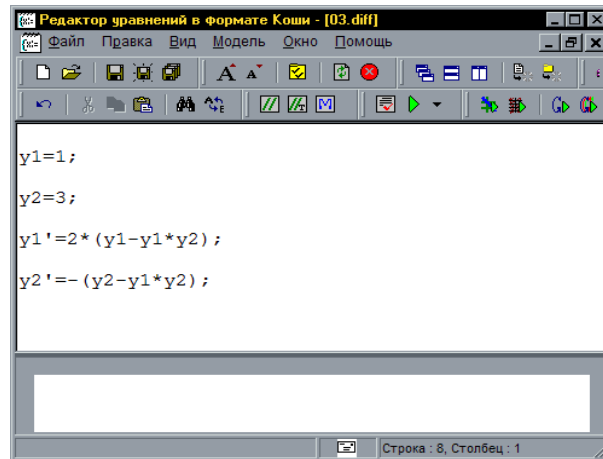
$$y_1' = 2 (y_1 - y_1 y_2);$$

$$y_2' = -(y_2 - y_1 y_2).$$

Начальные условия:

$$y_1(0) = 1; \quad y_2(0) = 3.$$

Запись в виде программной модели на языке LISMA:



```

y1=1;
y2=3;

y1'=2*(y1-y1*y2);
y2'=- (y2-y1*y2);
    
```

Результаты моделирования представлены на рис. 6.7.

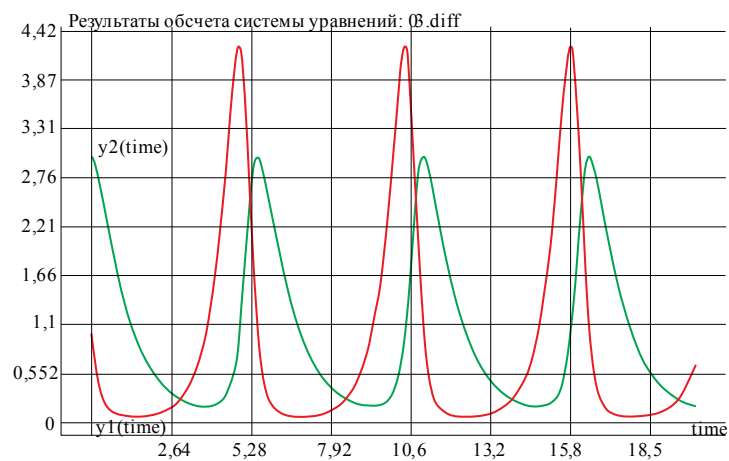


Рис. 6.7. Результаты моделирования популяций

Пример 6.3

Асинхронный двигатель (пуск под нагрузкой).

Исходная система уравнений:



$$F'_{sx} = 1 - R_{s1}F_{sx} + R_{s2}F_{rx} + F_{sy};$$

$$F'_{sy} = -R_{s1}F_{sy} + R_{s2}F_{ry} - F_{sx};$$

$$F'_{rx} = (1 - W_p)F_{ry} - Rr_3F_{rx} + Rr_2F_{sx};$$

$$F'_{ry} = -(1 - W_p)F_{rx} - Rr_3F_{ry} + Rr_2F_{sy};$$

$$W'_p = 1/T_j * (a_2(F_{sy}F_{rx} - F_{sx}F_{ry}) - M_c),$$

где $R_{s1} = 0,00484$; $R_{s2} = 0,00473$; $Rr_3 = 0,0569$; $Rr_2 = 0,0559$;
 $T_j = 6,689$; $a_2 = 4,3$; $M_c = 0$.

Начальные условия:

$$F_{sx}(0) = F_{sy}(0) = F_{rx}(0) = F_{ry}(0) = 0.$$

Запись в виде программной модели на языке LISMA:

```

Редактор уравнений в формате Коши - [09.dif]
Файл  Правка  Вид  Модель  Окно  Помощь
[Icons]
Fsx' = 1-Rs1*Fsx+Rs2*Frx+Fsy;
Fsy' = -Rs1*Fsy+Rs2*Fry-Fsx;
Frx' = (1-wp)*Fry-Rr3*Frx+Rr2*Fsx;
Fry' = -(1-wp)*Frx-Rr3*Fry+Rr2*Fsy;
wp' = 1/Tj*(a2*(Fsy*Frx-Fsx*Fry)-Mc);
Rs1=0.00484;
Rs2 = 0.00473;
Rr2 = 0.0559;
Rr3 = 0.0569;
Tj = 6.689;
a2 = 4.3;
Mc=0;
  
```

Результаты моделирования в виде графика «угловая скорость / время» представлены на рис. 6.8.

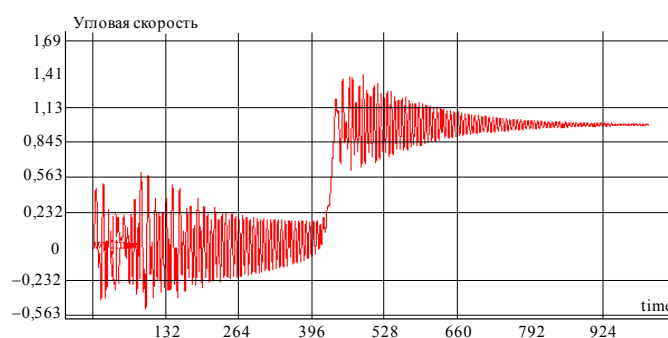
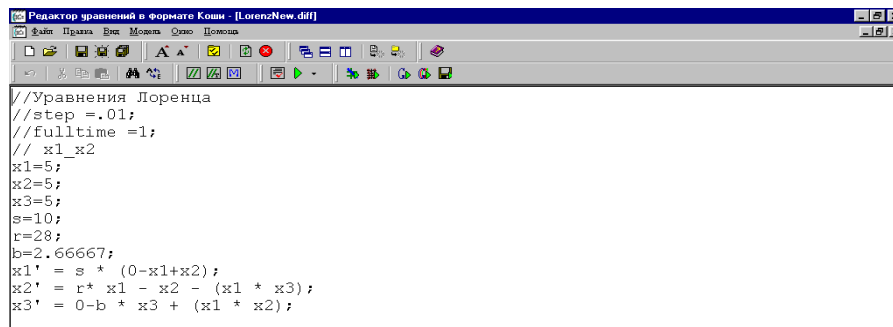


Рис. 6.8. Переходный процесс угловой скорости $W(t)$

Пример 6.4

Уравнения Лоренца с решением в виде фазового портрета.

Ниже приведена программная модель уравнений Лоренца (рис. 6.9). Первые шесть строк программы сразу после комментариев определяют начальные условия и соответствующие параметры. Далее следует система дифференциальных уравнений, которая численно интегрируется на интервале $[0, 1]$ с шагом 0,01.



```
//Уравнения Лоренца
//step =.01;
//fulltime =1;
// x1_x2
x1=5;
x2=5;
x3=5;
s=10;
r=28;
b=2.66667;
x1' = s * (0-x1+x2);
x2' = r * x1 - x2 - (x1 * x3);
x3' = 0-b * x3 + (x1 * x2);
```

Результаты моделирования в интерпретаторе GRIN в виде фазового портрета представлены на рис. 6.9.

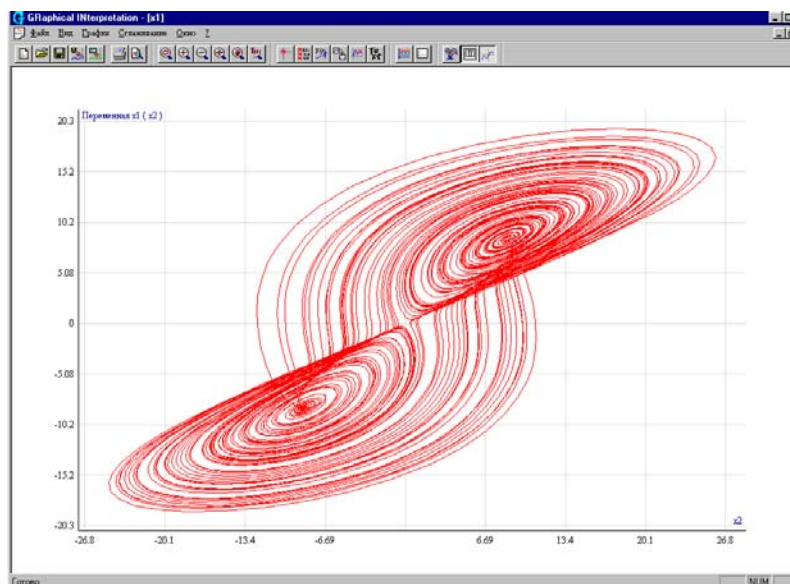


Рис. 6.9. Программная модель и фазовый портрет уравнений Лоренца



6.3. ИСМА 2015

В среде моделирования ИСМА разработан и реализован ряд языков спецификации динамических гибридных систем (ГС). К ним относятся структурные схемы [48], диаграммы Харела [51], текстовая спецификация на языке *LISMA* [48], графический язык спецификации принципиальных схем электроэнергетических систем *LISMA_EPS* [104, 105] и язык описания уравнений химических реакций [48, 51]. Среди всех способов спецификации базовым является текстовый, или символьный, язык. В настоящем пособии рассмотрим символьный язык *LISMA_PDE* для описания гибридных динамических систем из класса задачи Коши (6.4) с ограничениями

$$\begin{aligned} y' &= f(t, y), \quad y(t_0) = y_0, \quad t_0 \leq t \leq t_k; \\ pr &: g(y, t) < 0, \end{aligned} \quad (6.5)$$

где $g(y, t) : R \times R^n \rightarrow R^s$, $s = 1, 2, \dots$ – событийная функция, характеризующаяся предикатом $pr \in B = \{\text{false}, \text{true}\}$; $f : R \times R^n \rightarrow R^n$, $y \in R^n$.

Основным свойством большинства языков программирования является принадлежность классу контекстно-свободных (КС) языков. Решается задача конструирования порождающей КС-грамматики языка типа LL(2). Такая грамматика является однозначной и позволяет строить эффективные нисходящие анализаторы. Ввиду активного расширения языка *LISMA_PDE* и наполнения его новыми языковыми конструкциями процесс построения анализаторов должен быть оптимизированным. Для этого следует применить технологию генерации анализаторов или технологию универсальной интерпретации грамматик. В обоих подходах для расширения языка от пользователя требуется дополнение описание грамматики и методов семантического анализа. Этапы лексического и синтаксического анализа выполняются автоматически.

ЯЗЫК СИМВОЛЬНОЙ СПЕЦИФИКАЦИИ

Для описания задач из класса (6.4, 6.5) и задач, представленных в виде параболических дифференциальных уравнений в частных производных (ДУЧП), была разработана и зарегистрирована новая версия языка – *LISMA_PDE* (Language of ISMA with PDE)

[106–108]. Обновленный язык унаследовал набор конструкций, описывающих непрерывное поведение системой ОДУ и ДАУ, блоки состояний ГС, событийное управление, макроподстановки и многое другое. В *LISMA_PDE* впервые введены элементы, описывающие пространственные переменные, краевые условия и частные производные. Подробно изучены все этапы анализа текстовой модели – от лексического анализа до интерпретации модели (рис. 6.10).

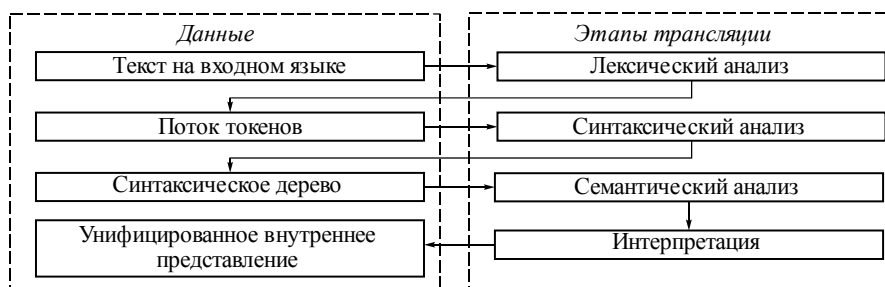


Рис. 6.10. Этапы трансляции текстовой модели ГС

Для этапов лексического и синтаксического анализа применены современные технологии автоматизации построения анализаторов на основе программных средств ANTLR, рассмотренных в разделе 7. В результате работы интерпретатора *LISMA_PDE* пользователь получает корректно сформированную структуру данных в виде *универсального внутреннего представления* (УВП). Данные УВП подаются на вход контроллера процессора численного анализа, и проводится численный эксперимент с использованием разработанных оригинальных методов численного анализа выражений (6.4, 6.5).

ГРАММАТИКА ЯЗЫКА

Программная модель на языке *LISMA_PDE* представляет собой последовательность операторов (**statement**) описания ГС. Грамматика языка для построения программных моделей выглядит следующим образом:

```

lisma → (statement)+
statement → constant | init_cond | equation | state
           | pseudo_state | approximated_var | edge | macros
           | linear_eq | linear_vars
  
```



Модель содержит описание непрерывного и дискретного поведения. Для описания дискретного поведения системы предусмотрены выражения, описывающие состояния ГС (**state**) и условные блоки (**pseudo_state**), реализующие событийное управление моделью. К конструкциям языка, описывающим непрерывное поведение, относятся константы (**constant**), системы ОДУ с алгебраическими функциями (**equation**) и системы линейных алгебраических уравнений (**linear_eq** | **linear_vars**). К ним также относятся введенные в *LISMA_PDE* конструкции, описывающие начально-краевые задачи с ДУЧП: объявление пространственных переменных (**approximated_var**), объявление краевых условий (**edge**), объявление начальных условий (**init_cond**) и описание систем ДУЧП (**equation**).

Рассмотрим подробно основные элементы языка *LISMA_PDE*. Последовательно будут представлены конструкции языка, как унаследованные от исходного *LISMA*, так и новые. Каждый элемент языка будет сопровождаться кратким описанием, соответствующими правилами грамматики и примерами. Унаследованные конструкции языка *LISMA* подробно описаны в [84].

Выражения. Базовой единицей в описании систем различных типов является выражение. Оно используется при описании правых частей уравнений, констант, начальных и краевых условий. Частым случаем является логическое выражение, используемое для описания предикатов событийных функций ГС.

Константы представляют собой неизменяемые значения, каждому из которых присвоено определенное имя. Правая часть констант может быть задана выражением, которое может быть вычислено на этапе синтаксического анализа. Практика разработки текстовых моделей ГС показала, что порой требуется вводить несколько констант с одним значением. Такая функциональность также была введена.

Уравнения непрерывного поведения начальной задачи. Для описания непрерывного поведения в начальных задачах используются обыкновенные дифференциальные уравнения. Левая часть уравнения является производной по времени, заданной именем фазовой переменной и знаком штриха «'».

Состояния. Для описания дискретного поведения гибридных динамических систем в языке *LISMA_PDE* используются имено-

ванные блоки, называемые состояниями. Они имеют идентификатор, предикат перехода в состояние и список состояний, из которых возможен переход. В теле состояния описываются мгновенные действия, смена правых частей уравнений и расширение модели новыми уравнениями. Грамматика состояний имеет следующий вид:

```
state → 'state' state_name '(' expression ')' state_body
[state_from] ';'
state_body → '{' {equation | setter} '}'
state_from → 'from' Identifier {' ',' Identifier}
```

Пример 6.5

Прыгающий мяч [51].

Прыгающий мяч описывается гибридной системой уравнений:

$$y' = v_y, y(t_0) = H, \quad (6.6)$$

$$v_y' = -g, v_y(t_0) = 0,$$

$$\text{if } (y \leq 0) \text{ and } (v_y < 0) \text{ then } v_y = -v_y.$$

Решение системы уравнений (6.6) при $H = 10$ представлено на рис. 6.11.

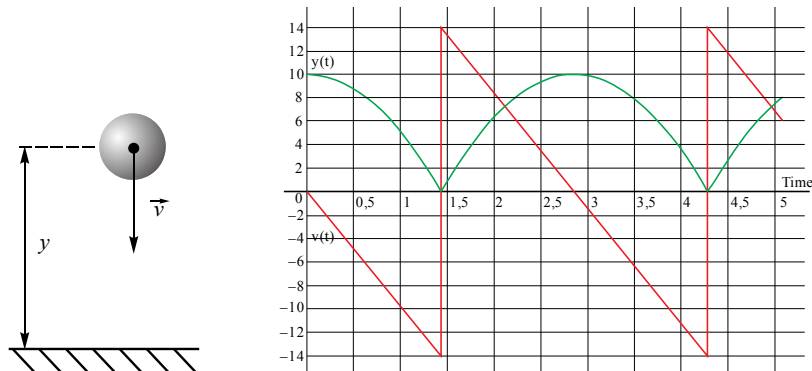


Рис. 6.11. Расстояние до поверхности отскока $y(t)$ и скорость $v(t) = v_y$

Приведем программную модель описания состояний в задаче прыгающего мячика (рис. 6.12) на языке *LISMA* и *LISMA_PDE*.



Рассматриваемая модель имеет два попеременно сменяющихся состояния, описывающих состояние системы в моменты взлета (**up**) и падения (**down**) мячика. Момент отскока мячика сопровождается мгновенной сменой направления движения мяча. Описание состояния начинается с ключевого слова **state** и имеет C-подобный синтаксис (фигурные скобки блока, условие внутри круглых скобок). Оператор присваивания значения фазовой переменной из языка *LISMA* также заменен на явные операции объявления начального условия задачи Коши (**ic**, сокращенно от *initial condition*) и мгновенной смены значения фазовой переменной в состоянии **set**. Эти изменения синтаксиса введены для обеспечения семантической согласованности языка и делают *LISMA_PDE* более выразительным.

<pre> 1 v' = -g; 2 y' = v; 3 y = 10; 4 5 up [y<0] is 6 v' = -g; 7 y' = v; 8 v = -v; 9 from init, down; 10 11 down [v<0] is 12 v' = -g; 13 y' = v; 14 from init, up; </pre>	<pre> 1 v' = -g; 2 y' = v; 3 ic y = 10; 4 5 state up (y<0) { 6 v' = -g; 7 y' = v; 8 set v = -v; 9 } from init, down; 10 11 state down (v<0) { 12 v' = -g; 13 y' = v; 14 } from init, up; </pre>
<i>a</i>	<i>б</i>

Рис. 6.12. Модель прыгающего мячика:

a – на языке *LISMA*; *б* – на языке *LISMA_PDE*

В методах интерпретации с языка *LISMA_PDE* реализованы механизмы наследования непрерывного поведения режимами ГС. Использование наследования возможно, когда все режимы имеют общие уравнения непрерывного поведения. С точки зрения спецификации это означает, что во всех состояниях все программные фрагменты описания уравнений (или их часть) одинаковы. Для иллюстрации механизма наследования рассмотрим фрагмент программной модели двух резервуаров (рис. 6.13) [48, 51]. В непрерывных

режимах *st1* и *st2* изменение уровней жидкости *h1* и *h2* описывается одинаковыми дифференциальными уравнениями. Режимы отличаются лишь значением переменной *v3*. Применение наследования в этом случае позволяет сделать модель более компактной и тем самым уменьшить вероятность ошибки при построении модели, уменьшить число операторов и сократить время работы анализатора.

<pre> 1 h1'=(1/S)*(Qp - Q1 - Q2 - V3*Q3); 2 h2'=(1/S)*(Q2 + V3*Q3 - V4*Q4); 3 4 state st1 (h1 <= hv3) { 5 V3 = 0; 6 h1'=(1/S)*(Qp - Q1 - Q2 - V3*Q3); 7 h2'=(1/S)*(Q2 + V3*Q3 - V4*Q4); 8 } from init, st2; 9 10 state st2 (h1 >= hv3) { 11 V3 = 1; 12 h1'=(1/S)*(Qp - Q1 - Q2 - V3*Q3); 13 h2'=(1/S)*(Q2 + V3*Q3 - V4*Q4); 14 } from init, st1;</pre>	<pre> 1 h1'=(1/S)*(Qp - Q1 - Q2 - V3*Q3); 2 h2'=(1/S)*(Q2 + V3*Q3 - V4*Q4); 3 4 state st1 (h1 <= hv3) { 5 V3 = 0; 6 } from init, st2; 7 8 state st2 (h1 >= hv3) { 9 V3 = 1; 10 } from init, st1;</pre>
--	--

a

б

Рис. 6.13. Фрагменты программных моделей ГС двух резервуаров:

a – без наследования; *б* – с применением наследования

Событийное управление. В языке *LISMA_PDE* также реализовано событийное управление [108] с использованием блоков *if...else*. С точки зрения теории гибридных систем это особый вид состояния, переход в который сопровождается мгновенным выходом из данного состояния в предыдущее. Переход в условные блоки возможен из любого состояния. Условные блоки служат для мгновенной смены правых частей уравнений и значений фазовых переменных без смены самого состояния. При помощи механизма событийного управления реализуются внешние воздействия на моделируемую систему, т. е. действия, имеющие другую природу происхождения и не зависящие от законов функционирования системы. Однако событие может косвенно привести к смене локального состояния, так как в результате его выполнения могут измениться значения параметров и (или) фазовых переменных. Использование событийного управления особенно удобно при первом типе гибридного поведения.



Пространственные переменные. Для описания системы дифференциальных уравнений в частных производных, краевых условий и начальных значений введены новые элементы языка. Для пространственных переменных введено явное объявление. В контексте рассматриваемых численных методов пространственные переменные подлежат дискретизации. С этой целью в описании следует указать необходимую информацию, а также граничные значения переменной. Грамматика записывается следующим образом:

```

approximated_var → 'var' Identifier '(' approximat-
ed_var_bound ',' approximated_var_bound ')' approximat-
ed_var_tail ';'
approximated_var_bound → ['-']DecimalLiteral
approximated_var_tail → 'apx' DecimalLiteral | 'step'
(FloatingPointLiteral|DecimalLiteral)

```

Рассмотрим объявление пространственной переменной на рис. 6.14.

```

1   var x[0, 20] apx 30;
2   var y[0, 30] step 0,5;

```

Рис. 6.14. Объявление пространственных переменных на языке *LISMA_PDE*

В этой конструкции в квадратных скобках задаются границы изменения переменной. Далее следует ключевое слово **apx** (approximation) или слово **step**. Ключевое слово **apx** используется в случае, если рассматриваемую область определения следует разбить на фиксированное количество отрезков. Ключевое слово **step** используется, если важен размер шага. После ключевого слова записывается значение количества отрезков или размера шага.

Краевые условия. Одним из элементов описания начально-краевой задачи являются краевые условия. В данной работе рассматриваются краевые условия первого и второго рода (краевые условия Дирихле и Неймана) [64]. Рассмотрим грамматику краевых условий:

```

edge → 'edge' edge_eq 'on' Identifier edge_side ';'
edge_eq → (Identifier | Partial_operand) '=' (FloatingPoint-
Literal | DecimalLiteral)
edge_side → 'left' | 'right' | 'both'

```

Конструкция содержит уравнение частной производной с заданным значением в правой части и типом края. Поскольку в настоящей работе рассматриваются области с простой прямоугольной геометрией, то могут быть указаны следующие границы: левая (*left*), правая (*right*) или одновременно обе (*both*). На рис. 6.15 приведен пример использования краевых условий первого и второго рода с различным типом границы.

<pre> 1 edge c1 = 0 on x left; 2 edge c1 = 20 on x right; 3 edge c1 = 0 on y both; </pre>	<pre> 1 edge D(c1, x) = 0 on left; 2 edge D(c1, x) = 20 on right; 3 edge D(c1, x) = 0 on both; </pre>
<i>a</i>	<i>б</i>

Рис. 6.15. Объявление краевых условий Дирихле (*a*) и краевых условий Неймана (*б*) на языке *LISMA_PDE*

Дифференциальные уравнения в частных производных.

Рассмотрим грамматику, описывающую ДАУ и ДУЧП, представленную в РБНФ:

```

Equation → ae | ode | pde | pde_param
ae → variable '=' expression ';'
ode → variable "' '=' expression ';'
pde → 'pde' pdeop ('+' pdeop)* '=' expression ';'
pde_param → 'pde' pde_op_param
           ('+' pde_op_param)* '=' expression ';'
pde_op_param → pde_param?pdeop
pde_param → pde_param_atom '*';
pde_param_atom → '(' expression ')' | Identifier | literal
pdeop → 'D'('Identifier ','Identifier ('Decimal)?)'
        | 'DD' '(' Identifier ',' Identifier ',' Identifier ('Decimal)? ')'
        | PDEF_spat_common('Identifier ('Decimal)? ')
        | PDEFspat '(' Identifier ')'
PDEF_spat_common → 'dx' | 'dy' | 'dz'
PDEFspat → 'dx2'|'dy2'|'dz2'|'dx3'|'dy3'|'dz3'|'dx4'|'dy4'|'dz4'

```




ОДУ (ode) являются разрешенными относительно производной по времени и описываются именем переменной, знаком штриха и выражением правой части. Аналогичной конструкцией без знака штриха (ae) описываются алгебраические уравнения. В случае если в правой части имеются частные производные, то описываемое уравнение соответствует гиперболическому ДУЧП, разрешенному относительно производной по времени. В общем случае в качестве аргументов используются имя дифференцируемой функции, независимая переменная и порядок производной. Например, $\partial^2 c / \partial x^2$ соответствует D(c,x,2). Если производная первого порядка, то последний аргумент опускается. Для упрощения записи поддерживается ряд предустановленных производных для пространственных переменных x , y и z с порядком не более четырех. Предыдущий пример может быть записан в сокращенной форме: dx2(c). Имеется поддержка смешанных производных. Например, $\partial^2 c / \partial x \partial y$ соответствует записи DD(c,x,y). Эллиптические ДУЧП (pde) представлены уравнениями Пуассона, для которых характерно наличие частных производных по пространственным переменным в левой части. Описание таких ДУЧП начинается с ключевого слова **pde**. При частных производных также возможно наличие параметра-множителя, представленного константой или числом. К примеру, эллиптическому уравнению

$$\lambda \frac{\partial^2 u}{\partial x^2} + \mu \frac{\partial^2 u}{\partial y^2} = (\lambda + \mu) \frac{\partial^2 v}{\partial x \partial y}$$

будет соответствовать запись

$$\text{pde } a * dx2(u) + u * dy2(u) = (a + u) * DD(v, x, y);$$

Ниже приведен пример использования данной языковой конструкции на модели двухмерного уравнения диффузии с краевыми условиями первого рода, математическое описание которого имеет вид

$$\frac{du}{dt} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}, \quad u(0, x, y) = e^{(x-0,5)^2 + (y-0,5)^2} / 0,02,$$

$$u(t, 0, y) = 0, u(t, 1, y) = 0, u(t, x, 0) = 0, u(t, x, 1) = 0,$$

$$0 < x < 1, 0 < y < 1.$$

Соответствующая программная модель представлена на рис. 6.16.

```

1 // пространственная переменная
2 var x[0, 1] apx 20;
3 // ДУЧП
4 u'=D(u, x, 2) + D(u, y, 2);
5 // начальные условия
6 ic u = exp(pow(x-0.5, 2) + pow(y-0.5, 2)/0.02);
7 // краевые условия Дирихле
8 edge u = 0 on x both;
9 edge u = 0 on y both;
```

Рис. 6.16. Объявление ДУЧП в модели уравнений диффузии

Циклы и индексная форма записи уравнений. Для формирования описания набора одностипных уравнений в *LISMA_PDE* впервые введен блок цикла. Он позволяет указывать переменную индекса в заданных интервалах.

Выражение цикла имеет тело, внутри которого с использованием индексной формы записи можно указать уравнения, начальные условия, объявление констант и событийное управление. Грамматика цикла в РБНФ записывается следующим образом:

```

for_cycle → 'for' Identifier '=' for_cycle_interval ('
              for_cycle_interval)* for_cycle_body
for_cycle_interval → Decimal (':' Decimal)?
for_cycle_body → '{' (equation | init_cond | constant | pseudo_state)* '}'
```

Внутри цикла допускается индексная запись имени переменной, которая соответствует грамматике:

```

variable → var_ident | derivative_ident
var_ident → Identifier (cycle_index) ?
cycle_index → '[' cycle_index_idx ']' cycle_index_posfix?
cycle_index_idx → Identifier (('+'|'-') Decimal)?
cycle_index_posfix → Identifier | Decimal
```



В качестве примера рассмотрим фрагмент модели электроэнергетической системы [104]:

$$\begin{aligned}\frac{d\Psi_{di}}{dt} &= -U_{di} - \omega_i \Psi_{qi}, \quad \frac{d\Psi_{qi}}{dt} = -U_{qi} + \omega_i \Psi_{di}, \\ \frac{d\Psi_{fi}}{dt} &= E_{qei} \frac{r_{fi}}{x_{adi}} - r_{fi} i_{fi}, \quad \frac{d\omega_i}{dt} = \frac{1}{T_{Ji}} (M_{Ti} - M_i), \\ i_{fi} &= \frac{1}{x_{adi}} (\omega_{\text{ном}} \Psi_{di} - x_{di} i_{di}), \quad M_i = i_{qi} \Psi_{di} - i_{di} \Psi_{qi}.\end{aligned}$$

В данном примере рассматриваются уравнения для синхронных машин, где $1 \leq i \leq 6$. Соответствующая программная модель приведена на рис. 6.17.

```

1 // 1. Уравнения для синхронных машин, i=1...6.
2 for i = 1:6 {
3     Fd[i]' = -Ud[i] - w[i]*Fq[i];
4     Fq[i]' = -Uq[i] + w[i]*Fd[i];
5     Ff[i]' = Eqe[i]*rf[i]/xad[i] - rf[i]*if_[i];
6     w[i]' = (MT[i] - M[i])/TJ[i];
7     if_[i] = (w_nom*Fd[i] - xd[i]*id[i]);
8     M[i] = iq[i]*Fd[i] - id[i]*Fq[i];
9 }

```

Рис. 6.17. Индексная форма записи уравнений для синхронных машин

Без использования индексной формы записи программная модель состояла бы из 36 уравнений.

Макроподстановка. В больших программных моделях ГС нередко возникают случаи, когда части отдельных уравнений повторяются. С целью сокращения текстовой модели и облегчения работы пользователя используются макроподстановки. Они имеют идентификатор и соответствующую ему правую часть. Макроподстановки не являются отдельными уравнениями. В ходе семантического анализа правая часть макроподстановок используется в местах вызова соответствующего идентификатора. Применение макроподстановок можно проиллюстрировать на приведенной в [48] модели проникновения помеченных радиоактивной меткой антител в пораженную опухолью ткань живого организма.

В результате дискретизации в исходной математической модели производных первого и второго порядка по пространственной переменной формируются уравнения

$$f_{2j-1} = \alpha_j \frac{y_{2j+1} - y_{2j-3}}{2\Delta\zeta} + \beta_j \frac{y_{2j-3} - 2y_{2j-1} + y_{2j+1}}{(\Delta\zeta)^2} - ky_{2j-1}y_{2j},$$

$$f_{2j} = -ky_{2j}y_{2j-1},$$

где

$$\alpha_j = 2(j\Delta\zeta - 1)^3 c^2, \quad \beta_j = (j\Delta\zeta - 1)^4 c^2, \quad 1 \leq j \leq N,$$

$$\Delta\zeta = \frac{1}{N}, \quad y_{-1}(t) = \varphi(t), \quad y_{2N+1} = y_{2N-1},$$

$$g \in R^{2N}, \quad g = (0, v_0, 0, v_0, \dots, 0, v_0)^T.$$

На рис. 6.18 представлен фрагмент программной модели на языке *LISMA_PDE* с макроподстановками **alpha** и **beta**. Уравнения программной модели приближены к математическим с минимальными отличиями. Макроподстановки позволяют значительно сократить вид правой части уравнений.

```
1 macro alpha = (2*pow(1/N-1, 3)/C2), beta = (pow(1/N-1, 4)/C2);
2 y1' = alpha*N*(y3-phi)/2 + beta*N*N*(phi-2*y1+y3) - k*y1*y2;
3 y3' = (y1 - 2*y3 + y5)*beta*N*N + alpha*N/2*(y5 - y1) - k*y3*y4;
```

Рис. 6.18. Фрагмент модели проникновения антител в ткань организма

Библиотечные функции. Для описания непрерывного поведения часто требуется наличие библиотечных функций. Поддерживаются следующие функции: абсолютное значение $\text{abs}(x)$, синус $\sin(x)$, косинус $\cos(x)$, тангенс $\text{tg}(x)$, экспонента $\exp(x)$, корень $\text{sqrt}(x)$, степень $\text{pow}(x, n)$, максимум $\text{max}(x, y)$, минимум $\text{min}(x, y)$, знак $\text{sign}(x)$.

В приложении Б приведена грамматика **G[LISMA_PDE]** полностью.

КЛАССИФИКАЦИЯ ГРАММАТИКИ

Проблема соответствий Поста означает, что в общем случае нельзя показать однозначность и безвозвратность КС-грамматик. Поэтому при проектировании языковых процессоров используются



подклассы КС-грамматик, для которых однозначность доказана. Доказана однозначность строгих КС-грамматик типа $LL(k)$, $k \geq 1$. Для построения распознавателей $LL(k)$ -грамматик используется множество $\mathbf{FIRST}_k(\alpha)$ – первые k символов терминальной цепочки $\alpha \in (V_T \cup V_N)^*$ при $|\alpha| > k$, либо сама цепочка α , если $|\alpha| \leq k$. Грамматика $G = (V_N, V_T, S, P)$ является $LL(k)$ -грамматикой тогда и только тогда, когда выполняется условие: $\forall A \rightarrow \beta \in P$ и $\forall A \rightarrow \gamma \in P$ ($\beta \neq \gamma$): $(\mathbf{FIRST}_k(\beta\omega) \cap \mathbf{FIRST}_k(\gamma\omega)) = \emptyset$ для всех цепочек ω , таких что $S \Rightarrow^* \alpha A \omega$.

Применим этот вывод для классификации $G_{\text{LISMA_PDE}}$. Рассмотрим грамматику спецификации уравнений различных типов G_{equation}

Equation $\rightarrow$ ae | ode | pde
 ae $\rightarrow$ variable '=' expression ';'
 ode $\rightarrow$ variable "' '=' expression ';"
 pde $\rightarrow$ 'pde' variable "' '=' expression ';"
 variable $\rightarrow$ Identifier

Для сокращения записи введем новые обозначения символов грамматики G_{equation} в соответствии с табл. 6.1.

Таблица 6.1

Сокращенные обозначения

equation	ae	ode	pde	variable	expression	'='	','	''	'pde'	Identifier
<i>E</i>	<i>A</i>	<i>O</i>	<i>P</i>	<i>V</i>	<i>X</i>	<i>e</i>	<i>s</i>	<i>a</i>	<i>p</i>	<i>i</i>

Обозначим грамматику G_{eq} и перепишем с учетом введенных обозначений:

$$G_{\text{eq}} = (\{E, A, O, P, V, X\}, \{e, s, a, p, i\}, E, \{E \rightarrow A \mid O \mid P, A \rightarrow VeXs, O \rightarrow VaeXs, P \rightarrow pVaeXs, V \rightarrow i\}). \quad (6.7)$$

Из записи (6.7) можно видеть, что G_{eq} соответствует определению КС-грамматик. Проверим подкласс грамматики. Первое правило (6.7) содержит три альтернативные части. Покажем выводимость цепочек:

- 1) $E \Rightarrow A \Rightarrow VeXs \Rightarrow ieXs$;
- 2) $E \Rightarrow O \Rightarrow VaeXs \Rightarrow iaeXs$;
- 3) $E \Rightarrow P \Rightarrow pVaeXs \Rightarrow piaeXs$.

Найдем $FIRST_1(\alpha)$ и $FIRST_2(\alpha)$ для полученных строк (табл. 6.2).

Т а б л и ц а 6.2

Начальные символы строк

Строка	$FIRST_1$	$FIRST_2$
<i>ieXs</i>	<i>i</i>	<i>ie</i>
<i>iaeXs</i>	<i>i</i>	<i>ia</i>
<i>piaeXs</i>	<i>p</i>	<i>pi</i>

В двух случаях получились совпадающие $FIRST_1(\alpha)$. Следовательно, G_{eq} не относится к классу LL(1). При этом строки $FIRST_2(\alpha)$ различны. Таким образом, G_{eq} является грамматикой типа LL(2), что и требовалось определить.

Рассматривая все правила G_{LISMA_PDE} и рассуждая аналогичным образом, можно доказать, что грамматика G_{LISMA_PDE} также является однозначной типа LL(2). Все сокращенные обозначения, упрощенная грамматика G_{LISMA_PDE} и полный список продукций грамматики с пересекающимися множествами $FIRST_1(\alpha)$ приведены в приложении Б (табл. ПБ.1–ПБ.4). В частности в табл. ПБ.4 видно, что G_{LISMA_PDE} является КС-грамматикой типа LL(2). Этот вывод позволяет строить безвозвратный анализатор с применением однозначного и безвозвратного метода рекурсивного спуска.

УПРАЖНЕНИЯ

Все упражнения практического приложения языковых процессоров содержат задачи с программной реализацией в отличие от теоретической части.



В качестве упражнения требуется разработать язык и порождающую грамматику для выбранного варианта задания. Выполнить программную реализацию алгоритма синтаксического анализа (парсер).

Варианты

1. Арифметический оператор условного перехода языка *BASIC*.
2. Условный оператор языка *C/C++* (*if-else*).
3. Условный оператор языка *C/C++* (*if* с блоком действий).
4. Оператор выбора языка *C/C++* (*switch*).
5. Арифметический оператор условного перехода языка *Fortran*.
6. Оператор перехода вычисляемый языка *Fortran*.
7. Оператор присваивания языка *Fortran*.
8. Оператор цикла языка *Fortran*.
9. Оператор цикла языка *BASIC*.
10. Оператор цикла *for* языка *C/C++*.
11. Оператор цикла *while* языка *C++*.
12. Оператор цикла *do...while* языка *C/C++*.
13. Функция ввода *scanf* языка *C*.
14. Оператор потокового ввода *>>* языка *C++*.
15. Функция вывода *Console::WriteLine()* языка *C++*.
16. Функция вывода *printf* языка *C*.
17. Оператор потокового вывода *<<* языка *C++*.
18. Функция ввода *Console::ReadLine()* языка *C++*.
19. Функция вывода *System.out.println()* языка *Java*.
20. Использование класса *Scanner* (*java.util*) для ввода строки.
21. Функция вывода *Console.WriteLine()* языка *C#*.
22. Функция ввода *Console.ReadLine()* языка *C#*.
23. Оператор вывода *write* языка *Pascal*.
24. Оператор ввода *INPUT* языка *BASIC*.
25. Оператор ввода *READ* языка *Fortran*.
26. Логические выражения языка *Fortran*.
27. Оператор вызова функций *CALL* языка *Fortran*.
28. Текстовые константы языка *Fortran*.
29. Декларирование комплексных констант языка *Fortran*.
30. Декларирование комплексных констант языка *Pascal*.
31. Объявление структуры на языке *C*.



- 32. Объявление прототипа функции на языке *C/C++*.
- 33. Объявление целочисленной переменной с инициализацией на языке *C/C++*.
- 34. Объявление целочисленной константы с инициализацией на языке *C/C++*.
- 35. Объявление целочисленной константы с инициализацией на языке *Java*.
- 36. Объявление массива символов с инициализацией строковой константой на языке *C/C++*.
- 37. Оператор описания **COMMON** языка *Fortran*.
- 38. Оператор описания данных **DATA** языка *Fortran*.
- 39. Оператор описания типов языка *BASIC*.
- 40. Оператор описания типов языка *Pascal*.
- 41. Оператор **new** для создания объекта с инициализацией (вызов конструктора) на языке *C++*.
- 42. Объявление перечисляемого типа на языке *C++*.
- 43. Тернарный оператор языка *C/C++*.
- 44. Оператор **foreach** языка *C#*.
- 45. Манипуляторы для объекта **cout** языка *C++*.
- 46. Операторы обработки исключений языка *C++*.

7. ГЕНЕРАТОРЫ ЯЗЫКОВЫХ ПРОЦЕССОРОВ

Исторически первым и наиболее универсальным механизмом разработки и реализации языков программирования являются порождающие грамматики. Однако при большом количестве продукций и нетерминальных символов разработанной грамматики вопросы реализации языков связаны с высокой размерностью рутинных процедур программирования анализаторов. Эта трудность мотивировала разработчиков системного ПО к созданию так называемых генераторов компиляторов [52, 24]. Суть генераторов состоит в автоматизации программирования шаблонов – процедур (функций) по заданной порождающей грамматике строго определенного класса в классификации Хомского.

Генератор языковых процессоров представляет собой программу-интерпретатор, которая генерирует лексер (лексический анализатор, сканер) и парсер языка (синтаксический анализатор) на основе входного файла порождающей грамматики. В зависимости от используемого генератора применяется ряд требований к грамматикам, с которыми он может работать. В большинстве генераторов такими требованиями является принадлежность разработанной грамматики к классу контекстно-свободной грамматики типа $LL(k)$.

Входной файл с порождающей грамматикой является текстовым файлом, в котором находится описание грамматики в нормальной форме Бэкуса – Наура (НБФ) или ее расширенной форме (РБНФ) в зависимости от генератора. Сам синтаксис, используемый для описания грамматики, имеет незначительные изменения, что позволяет уменьшить временные затраты при переходе с одного генератора на другой.

Примерами используемых генераторов являются ANTLR [25, 102], FLEX & BISON [101] (LEX & YACC в старом варианте), Roslyn [100]. Каждый из генераторов может скомпилировать анали-



заторы порождающих грамматик для определенного набора языков. Однако при создании сканера и парсера для одного языка программирования часть генераторов может справиться лучше, чем другие, поэтому при разработке не стоит ограничиваться одним генератором, лучше целенаправленно выбирать наиболее подходящий. В таблице ниже приведен список поддерживаемых языков для генераторов ANTLR v4, FLEX & BISON v3.7.1.

Т а б л и ц а

Поддерживаемые языки

Генератор	Язык программирования
ANTLR v4	<i>Java, JavaScript, C++, Python, Go, Swift, PHP, DART, C#</i>
FLEX & BISON v3.7.1	<i>C++, C, Java</i>
Roslyn	<i>C#</i>

В этом разделе будет продемонстрирована работа генератора ANTLR на C# и FLEX & BISON на C++ соответственно для грамматик арифметического оператора присваивания и оператора определения числовых констант.

7.1. ANTLR

ANTLR – мощный инструмент автоматизации программирования на этапе реализации новых языков программирования и обработки / перевода структурированных текстовых или двоичных файлов. ANTLR использует в качестве входных данных сконструированную пользователем КС-грамматику типа $LL(k)$ для генерации парсера и лексера. Парсер состоит из выходных файлов на указанном пользователем целевом языке (см. таблицу). Для работы со средами разработки (IDE) в пользовательском интерфейсе (GUI) существуют плагины для Visual Studio, IntelliJ, NetBeans и Eclipse.

РЕАЛИЗАЦИЯ

Пусть грамматика оператора присваивания арифметического выражения $G[<arithmetic>]$ разработана в соответствии с нотацией Хомского и имеет следующий вид:



$G[\langle \text{arithmetic} \rangle]$:

- 1) $\langle \text{arithmetic} \rangle \rightarrow \langle \text{statement} \rangle \mid \langle \text{statement} \rangle \langle \text{arithmetic} \rangle$
- 2) $\langle \text{statement} \rangle \rightarrow \langle \text{expr} \rangle \langle \text{newline} \rangle \mid \langle \text{id} \rangle = \langle \text{expr} \rangle \langle \text{newline} \rangle \mid \langle \text{newline} \rangle$
- 3) $\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ + \langle \text{term} \rangle \mid - \langle \text{term} \rangle \}$
- 4) $\langle \text{term} \rangle \rightarrow \langle \text{operand} \rangle \{ * \langle \text{operand} \rangle \mid / \langle \text{operand} \rangle \}$
- 5) $\langle \text{operand} \rangle \rightarrow \langle \text{id} \rangle \mid \langle \text{int} \rangle \mid (\langle \text{expr} \rangle)$
- 6) $\langle \text{id} \rangle \rightarrow \text{letter} \{ \text{letter} \mid \text{digit} \}$
- 7) $\langle \text{int} \rangle \rightarrow \text{digit} \{ \text{digit} \}$
- 8) $\langle \text{newline} \rangle \rightarrow [\text{r}] \backslash \text{n}$.

Представленную грамматику $G[\langle \text{arithmetic} \rangle]$ необходимо переписать в виде РБНФ-формата ANTLR. Для грамматики в формате ANTLR следует представить все продукции в виде

$\text{neterm} : \text{expr} \mid \text{alternative} ; \quad (*)$

где **neterm** – нетерминальный символ; **expr** – выражение из терминалов и нетерминалов; **alternative** – альтернативное выражение.

В отличие от традиционных обозначений грамматики в нотации Хомского (см. раздел 2), в РБНФ все продукции начинаются нетерминалом без угловых скобок и заканчиваются разделителем «;». Терминальные символы заключаются в апострофы «' '». Вместо символа выводимости « $\rightarrow$ » используется двоеточие «:». В формате ANTLR все нетерминалы для парсера записываются строчными буквами, для лексера – прописными. Знак «?» используется для умолчания конструкции слева. Границы синтаксической конструкции (цепочки) выделяются круглыми скобками, если цепочка длинная и границы не очевидны. Знаки «+» и «*» имеют прежний смысл усеченной итерации и итерации соответственно относительно конструкции слева. Две точки «..» между символами определяются как диапазон между ними и читаются «и так далее».

Приведем грамматику $G[\langle \text{arithmetic} \rangle]$ в формате ANTLR с учетом записи (*) и указанных новых обозначений РБНФ.

Листинг 7.1. Грамматика $G[\langle \text{arithmetic} \rangle]$ в ANTLR

```
grammar Calc;
arithmetic :
    statement+
    ;
```



```
statement :  
    expr NEWLINE  
    | ID '=' expr NEWLINE  
    | NEWLINE  
    ;  
expr :  
    term('+ ' term | '-' term)*  
    ;  
term  
    : operand('* ' operand | '/' operand)*  
    ;  
operand  
    : ID  
    | INT  
    | '(' expr ')'  
    ;  
ID  
    : ('a'..'z'|'A'..'Z'|'_') ('a'..'z'|'A'..'Z'|'0'..'9'|'_')*  
    ;  
INT  
    : '0'..'9'+  
    ;  
NEWLINE  
    : '\r'? '\n'  
    ;
```

ГЕНЕРАЦИЯ КОДА СРЕДСТВАМИ ANTLR

На следующем этапе необходимо скомпилировать классы лексера и парсера. Для генерации на C# во входную грамматику ANTLR сразу после имени **grammar Calc** добавляется следующий код:

```
options  
{language=CSharp;}
```

и организуется .bat-файл вида

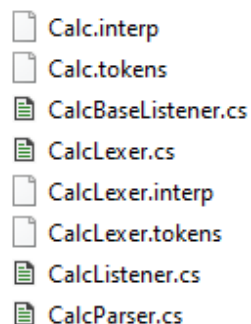
```
java -jar <ANTLR_jar> -o <Output_Folder> <File> ,
```

где <ANTLR_jar> – путь до jar-файла с ANTLR; <Output_Folder> – директория, в которую будут сгенерированы все результирующие файлы; <File> – файл с исходной грамматикой в формате ANTLR.



После успешной компиляции в указанной директории образуется набор файлов (рис. 7.1), который добавляется в программный проект.

Рис. 7.1. Сгенерированные файлы
(классы лексера и парсера)



Ниже приводится код, позволяющий использовать грамматику в проекте с разработанной консольной программой. Этот код добавляется в корневой сегмент консольной программы.

```
try
{
    // В качестве входного потока символов устанавливаем
    консольный ввод
    AntlrInputStream input = new AntlrInputStream(Console.In);
    // Настраиваем лексер на этот поток
    CalcLexer lexer = new CalcLexer(input);
    // Создаем поток токенов на основе лексера
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    // Создаем парсер
    CalcParser parser = new CalcParser(tokens);
    // Запускаем первое правило грамматики
    parser.arithmetic();
}

catch (Exception e)
{
    Console.WriteLine(e.Message);
}
```



Кроме того, для работы с полученным кодом необходимо подключить пространство имен с кодом:

```
//Создание собственного слушателя синтаксических ошибок
ErrorListener errors = new ErrorListener();
//Создание собственного слушателя лексических ошибок
ErrorLexerListener errorsLexer = new ErrorLexerListener();
// Определение символьного потока, parsedText – исходный текст
ICharInput input = CharInputs.fromString(parsedText);
//Создание лексера на потоке input
lexer.AddErrorListener(errorsLexer);
//Создание потока токенов на основе лексера
ITokenInput tokens = new CommonTokenInput(lexer);
//Удаление стандартных слушателей
parser.RemoveErrorListeners();
//Добавление своего слушателя синтаксических ошибок
parser.AddErrorListener(errors);
```

Этот код позволяет использовать грамматику в проекте и добавляется в файл Form1.cs, который называется *программная логика формы*. В результате работы в *слушателях* (listener) errors и errorsLexer будут находиться списки синтаксических и лексических ошибок соответственно.

7.2. ЛИСТИНГ ГРАММАТИКИ LISMA_PDE В ФОРМАТЕ ANTLR

Ниже приведен пример входного файла для ANTLR от порождающей грамматики языка LISMA_PDE, спроектированного в разделе 6 настоящего учебного пособия. Соответствующая полная порождающая грамматика представлена в приложении Б.

Листинг 7.2. Грамматика G[<arithmetic>] в ANTLR

```
grammar Lisma;
//
=====
// 1. Parser
//
=====
```



```

lisma : (statement)*;
statement :      constant | init_cond | equation | state | pseu-
do_state | approximated_var | edge | macros | start | end | step |
out;
// === 1.1 Constant
constant : 'const' (var_ident ASSIGN)+ expression SEMI;
// === 1.2 Partial
approximated_var
: 'var' var_ident LBRACK approximated_var_bound COMMA
approximated_var_bound RBRACK approximated_var_tail SEMI;
approximated_var_bound : (SUB)?DecimalLiteral;
approximated_var_tail : 'apx' DecimalLiteral | 'step' (Floating-
PointLiteral|DecimalLiteral);
partial_operand :      'D' LPAREN Identifier COMMA Identifi-
er (COMMA DecimalLiteral)? RPAREN;
edge :      'edge' edge_eq 'on' Identifier edge_side SEMI;
edge_eq :      Identifier ASSIGN expression ;
edge_side : 'left' | 'right' | 'both';
// === 1.3 Initial condition
init_cond :      variable LPAREN DecimalLiteral RPAREN
ASSIGN expression ';' ;
// === 1.4 Alg's and diff's equations
equation: variable ASSIGN expression SEMI;
// === 1.5 State
state:      'state' state_name LPAREN expression RPAREN
state_body (state_from)? SEMI;
state_body:      LBRACE (equation | setter)* RBRACE;
state_from :      'from' state_name (COMMA state_name)*;
state_name : Identifier ;
pseudo_state : 'if' LPAREN expression RPAREN pseu-
do_state_body ;
pseudo_state_body : LBRACE pseudo_state_elem*
RBRACE;
pseudo_state_elem: equation | setter;
// === 1.6 Function
func      :      Identifier LPAREN arg_list RPAREN ;
arg_list: expression ( COMMA expression )*;
// === 1.7 Derivative ident

```



```
derivative_ident :      var_ident derivative_quote_operant |
    'der' LPAREN var_ident COMMA DecimalLiteral RPAREN;
derivative_quote_operant : (QUOTE1)+;
// === 1.8 Variable
variable : var_ident | derivative_ident;
var_ident :      Identifier;
// === 1.9 Expression
parExpression : parExpressionLeftPar expression parExpressionRightPar ;
parExpressionLeftPar: LPAREN ;
parExpressionRightPar : RPAREN;
expression: conditionalExpression ;
conditionalExpression: conditionalOrExpression;
conditionalOrExpression: conditionalAndExpression (
or_operator conditionalAndExpression )*;
conditionalAndExpression : equalityExpression ( and_operator
equalityExpression )*;
equalityExpression : relationalExpression ( equalityExpressionOperator relationalExpression )*;
equalityExpressionOperator : (EQUAL | NOTEQUAL);
relationalExpression : additiveExpression ( relationalOp additiveExpression )*;
relationalOp : LT ASSIGN | GT ASSIGN | LT | GT ;
additiveExpression : multiplicativeExpression (additiveExpressionOperator multiplicativeExpression)*;
additiveExpressionOperator : ADD | SUB;
multiplicativeExpression : unaryExpression ( multiplicativeExpressionOperator unaryExpression )*;
multiplicativeExpressionOperator: MUL | DIV | MOD ;
unaryExpression: unaryExpressionOperator unaryExpression|
unaryExpressionNotPlusMinus;
unaryExpressionOperator: ADD | SUB ;
unaryExpressionNotPlusMinus: not_operator unaryExpression| primary;
primary: parExpression| primary_id | literal| partial_operand|
func;
primary_id: Identifier ;
literal : DecimalLiteral| FloatingPointLiteral;
```




```

    or_operator      :      OR | 'or' | 'OR';and_operator :      AND |
'and' | 'AND';not_operator :      BANG | 'not' | 'NOT';
    // === 1.10 macros
    macros : 'macro' MacroIdentifier ASSIGN expression SEMI;
    // === 1.11 setter
    setter: 'set' var_ident ASSIGN expression SEMI;
    // =====
    // 2. Lexer
    // =====
    DecimalLiteral : ('0' | '1'..'9' '0'..'9'*);
    FloatingPointLiteral: ('0'..'9')+ '.' ('0'..'9')* Exponent? | '.'
('0'..'9')+ Exponent? | ('0'..'9')+ Exponent | ('0'..'9')+ | '0' ('x'|'X')(
HexDigit+ ('.' HexDigit*)? HexExponent | '.' HexDigit+ HexExpo-
nent );
    fragment Exponent : ('e'|'E') ('+'|'-')? ('0'..'9')+ ;
    fragment HexExponent : ('p'|'P') ('+'|'-')? ('0'..'9')+ ;
    fragment HexDigit : ('0'..'9'|'a'..'f'|'A'..'F') ;
    Identifier : Letter (Letter|IDDigit)*;
    MacroIdentifier : '#'(Letter|IDDigit)*;
    fragment Letter : '\u0024' |\u0041'..\u005a' |\u005f
|\u0061'..\u007a' |\u00c0'..\u00d6' |\u00d8'..\u00f6'
|\u00f8'..\u00ff' |\u0100'..\u1fff' |\u3040'..\u318f'
|\u3300'..\u337f' |\u3400'..\u3d2d' |\u4e00'..\u9fff'
|\uf900'..\ufaff';
    fragment IDDigit: '\u0030'..\u0039' |\u0060'..\u0069'
|\u006f0'..\u006f9' |\u0096'..\u009f' |\u009e6'..\u009ef'
|\u00a66'..\u00a6f' |\u00ae6'..\u00aef' |\u00b66'..\u00b6f'
|\u00be7'..\u00bef' |\u00c66'..\u00c6f' |\u00ce6'..\u00cef'
|\u00d66'..\u00d6f' |\u00e50'..\u00e59' |\u00ed0'..\u00ed9'
|\u01040'..\u01049';
    WS : (' |\r|\t'|\u000C'|\n')+ -> channel(HIDDEN);
    COMMENT: '/*' .*? '*/' -> channel(HIDDEN);
    LINE_COMMENT: '//' ~('\n'|\r)* '\r'? '\n' -> channel(HIDDEN);
    // Keywords
    OR_KW1: 'or'; OR_KW2: 'OR';
    AND_KW1: 'and'; AND_KW2: 'AND';
    NOT_KW1: 'not'; NOT_KW2: 'NOT';
    // Separators

```



```
LPAREN : '('; RPAREN : ')';
LBRACE : '{'; RBRACE : '}';
LBRACK : '['; RBRACK : ']';
SEMI : ';'; COMMA : ','; DOT : '.';
// Operators
QUOTE1 : '\\'; ASSIGN : '='; GT : '>'; LT : '<'; BANG :
'!'; TILDE : '~'; QUESTION : '?'; COLON : ':'; EQUAL : '=='; LE
:<='; GE : '>='; NOTEQUAL : '!='; AND : '&&'; OR : '||'; INC
:'++; DEC : '--'; ADD : '+'; SUB : '-'; MUL : '*'; DIV : '/';
BITAND : '&'; BITOR : '|'; CARET : '^'; MOD : '%';
ADD_ASSIGN : '+='; SUB_ASSIGN : '-='; MUL_ASSIGN : '*=';
DIV_ASSIGN : '/='; AND_ASSIGN : '&='; OR_ASSIGN : '|=';
XOR_ASSIGN : '^='; MOD_ASSIGN : '%='; LSHIFT_ASSIGN :
'<<='; RSHIFT_ASSIGN : '>>='; URSHIFT_ASSIGN : '>>>=';
```

7.3. FLEX & BISON

Рассмотрим еще один, менее популярный генератор FLEX & BISON с иллюстрацией программирования входного файла для некоего оператора задания констант. Определим оператор задания констант следующим образом:

```
const <имя_константы> = <значение>;
```

Правильными языковыми цепочками являются следующие:

```
const exp=-5.01E-10;
const b=001.123;
const a=.001;
const max=128;
```

Тогда грамматика десятичных констант $G[\langle \text{Def} \rangle]$ в нотации Хомского имеет следующий вид.

1. $\langle \text{Def} \rangle \rightarrow \langle \text{Letter} \rangle \langle \text{IdRem} \rangle$
2. $\langle \text{IdRem} \rangle \rightarrow \langle \text{Letter} \rangle \langle \text{IdRem} \rangle$
3. $\langle \text{IdRem} \rangle \rightarrow _ \langle \text{Name} \rangle$
4. $\langle \text{Name} \rangle \rightarrow \langle \text{Letter} \rangle \langle \text{NameRem} \rangle$
5. $\langle \text{NameRem} \rangle \rightarrow \langle \text{Letter} \rangle \langle \text{NameRem} \rangle$
6. $\langle \text{NameRem} \rangle \rightarrow = \langle \text{Number} \rangle$
7. $\langle \text{Number} \rangle \rightarrow [+ | -] \langle \text{UnsignedNumber} \rangle$



8. $\langle \text{UnsignedNumber} \rangle \rightarrow \langle \text{Decimal} \rangle [\text{E} \langle \text{Integer} \rangle] \mid \text{E} \langle \text{Integer} \rangle$
9. $\langle \text{Decimal} \rangle \rightarrow [\langle \text{UnsignedInt} \rangle]. \langle \text{UnsignedInt} \rangle \mid \langle \text{UnsignedInt} \rangle$
10. $\langle \text{Integer} \rangle \rightarrow [+ \mid -] \langle \text{UnsignedInt} \rangle$
11. $\langle \text{UnsignedInt} \rangle \rightarrow \langle \text{Digit} \rangle \{ \langle \text{Digit} \rangle \}$
12. $\langle \text{Digit} \rangle \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
13. $\langle \text{Letter} \rangle \rightarrow a \mid b \mid c \mid \dots \mid z \mid A \mid B \mid C \mid \dots \mid Z.$

Для работы с генератором *лексических анализаторов* FLEX из грамматики $G[\langle \text{Def} \rangle]$ выделяются токены, для каждого из которых описаны следующие регулярные выражения:

```
CONST "const"
ID [a-zA-Z]+
EQUALS "="
SEMICOLON ";"
NUMBER [+-]?[0-9]+("."[0-9]+)?("E"[+-]?[0-9]+)?
```

В отличие от формата ANTLR здесь терминалы представлены в двойных апострофах, диапазон терминалов обозначается символом тире «—», число NUMBER представлено как регулярное выражение числа в экспоненциальной форме (см. раздел 4).

В ходе разбора программы компилятор может встретить символы табуляции и переноса строки

```
[ \t\r\n]+
```

и символы, которые не были перечислены выше и которые будут считаться компилятором за лексическую ошибку:

```
{
return LEXERROR;
}
```

Для работы с генератором *синтаксических анализаторов* BISON необходимо представить в соответствии с $G[\langle \text{Def} \rangle]$ следующие productions:

1. prog: def progRem ;
2. def: CONST ID EQUALS NUMBER SEMICOLON;
3. def: error SEMICOLON;
4. progRem: def progRem;
5. progRem: ;

В BISON всегда определен и зарезервирован терминальный символ (специальный предопределенный токен) `error` для обработ-



ки ошибок. Парсер BISON генерирует токен **error** всякий раз, когда происходит синтаксическая ошибка.

Допишем продукции в грамматику BISON следующим образом.

6. def: error ID EQUALS NUMBER SEMICOLON;
7. def: CONST error EQUALS NUMBER SEMICOLON;
8. def: CONST ID error NUMBER SEMICOLON;
9. def: CONST ID EQUALS error SEMICOLON;
10. def: CONST ID EQUALS NUMBER error;
11. def: error SEMICOLON;

Последняя продукция отображает важную стратегию в разработке синтаксического анализатора: если обнаружена ошибка, то необходимо пропустить оставшуюся часть текущей строки ввода до токена SEMICOLON (;).

ГЕНЕРАЦИЯ КОДА СРЕДСТВАМИ FLEX & BISON

Следующим этапом необходимо скомпилировать классы лексера и парсера на языке C++ средствами FLEX & BISON. Для генерации классов в командной строке должны быть выполнены следующие команды.

```
.\win_flex.exe -+ -o lex.cpp .\parser.l
```

```
win_bison -d -no-lines parser.y -o parser.tab.cpp
```

Результатом выполнения команд является получение заголовочных и исполняемых файлов, представленных на рис. 7.2.

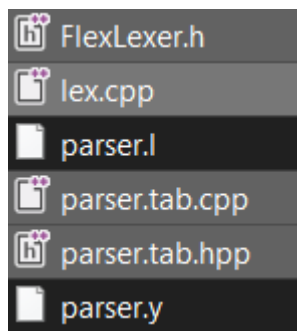


Рис. 7.2. Сгенерированные файлы
(классы лексера и парсера)

В заключение отметим, что использование FLEX & BISON или ANTLR связано с критериями пользовательского сервиса и особенностями грамматики входного языка программирования.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. *Грис Д.* Конструирование компиляторов для цифровых вычислительных машин / Д. Грис. – Москва : Мир, 1975. – 544 с.
2. *Льюис Ф.* Теоретические основы проектирования компиляторов / Ф. Льюис, Д. Розенкранц, Р. Стирнз. – Москва : Мир, 1979. – 654 с.
3. *Рейуорд-Смит В. Дж.* Теория формальных языков : вводный курс / В. Дж. Рейуорд-Смит. – Москва : Радио и связь, 1988. – 127 с.
4. *Бек Л.* Введение в системное программирование / Л. Бек. – Москва : Мир, 1988. – 448 с.
5. *Касьянов В. Н.* Методы построения трансляторов / В. Н. Касьянов, И. В. Поттосин. – Новосибирск : Наука, 1986. – 343 с.
6. *Вайнгартен Ф.* Трансляция языков программирования / Ф. Вайнгартен. – Москва : Мир, 1977. – 190 с.
7. *Хопгуд Ф.* Методы компиляции / Ф. Хопгуд. – Москва : Мир, 1972. – 160 с.
8. *Донован Дж.* Системное программирование / Дж. Донован. – Москва : Мир, 1975. – 540 с.
9. *Хантер Р.* Проектирование и конструирование компиляторов / Р. Хантер. – Москва : Мир, 1984. – 232 с.
10. *Кнут Д. Э.* Искусство программирования. Т. 3. Сортировка и поиск / Д. Э. Кнут. – 2-е изд. – Москва : Вильямс, 2000. – 822 с.
11. *Ахо А. В.* Теория синтаксического анализа, перевода и компиляции : в 2 т. / А. В. Ахо, Дж. Ульман. – Москва : Мир, 1978. – 2 т.
12. *Гордеев А. В.* Системное программное обеспечение / А. В. Гордеев, А. Ю. Молчанов. – Санкт-Петербург : Питер, 2001. – 734 с.
13. *Ахо А.* Компиляторы: принципы, технологии, инструменты : пер. с англ. / А. В. Ахо, С. Рави, Дж. Ульман. – Москва : Вильямс, 2001. – 768 с.
14. *Dijkstra E. W.* An Algol 60 translator for the XI // Annual Review in Automatic Programming. – 1963. – Vol. 3. – P. 329–345.
15. *Ершов А. П.* Введение в теоретическое программирование / А. П. Ершов. – Москва : Наука, 1977. – 288 с.
16. *Карпов Ю. Г.* Теория автоматов / Ю. Г. Карпов. – Санкт-Петербург : Питер, 2002. – 206 с. : ил.



17. Языки и автоматы : сб. переводов / под ред. А. Н. Маслова и Э. Д. Стоцкого. – Москва : Мир, 1975. – 358 с.
18. Компаниец Р. И. Системное программирование : основы построения трансляторов / Р. И. Компаниец, Е. В. Маньков, Н. Е. Филатов. – Санкт-Петербург : Корона принт, 2000. – 256 с. – (Учебное пособие для высших и средних учебных заведений).
19. Свердлов С. З. Языки программирования и методы трансляции / С. З. Свердлов. – Санкт-Петербург : Лань, 2019. – 564 с.
20. Jarvis J. F. Regular expressions as a feature selection language for pattern recognition // Proceedings of the Third International Joint Conference on Pattern Recognition. – Coronado, CA, 1976. – P. 189–192.
21. Майр Э. Популяции, виды и эволюция / Э. Майр. – Москва : Мир, 2012. – 460 с.
22. Visual REGEXP. – URL: <http://laurent.riesterer.free.fr/regexp/> (дата обращения: 15.04.2022).
23. Rhodes University. Computer Science. – URL: <http://cs.ru.ac.za/homes/cspt/cocor.htm/> (accessed: 15.04.2022).
24. Калачева С. Б. Сравнительный анализ генераторов-компиляторов LLgen и ACCENT // Труды СПИИРАН. – 2005. – Вып. 2, т. 2. – С. 184–190.
25. Пентус А. Е. Теория формальных языков : учеб. пособие / А. Е. Пентус, М. Р. Пентус. – Москва : Изд-во ЦПИ при мех.-мат. фак. МГУ, 2004. – 80 с.
26. Зубинский А. Транслятор – это не только просто... // Компьютерное обозрение. – 2001. – № 23.
27. Поттосин И. В. Текущее состояние российских исследований и разработок в области трансляции / И. В. Поттосин. – Новосибирск, 1995. – 32 с. – (Препринт / Рос. акад. наук, Сиб. отд-ние, Ин-т систем информатики им. А. П. Ершова ; 30).
28. Krukov V. A. RAMPA – CASE for portable programs development / V. A. Krukov, L. A. Pozdnjakov, I. B. Zadykhailo // Parallel Computing Technologies (РАСТ-93) : proceedings. – Obninsk, 1993. – Vol. 3. – P. 643–650.
29. Серебряков В. А. Синапс/3 – расширение языка Си для параллельных вычислений при решении научных задач / В. А. Серебряков, А. Н. Бездушный, К. Г. Белов // Программирование. – 1994. – Т. 20, № 2. – С. 3–15.
30. Серебряков В. А. Основные особенности входного языка и реализации СПТ Супер // Программирование. – 1982. – № 1. – С. 78–88.
31. Касьянов В. Н. Инструменты преобразования программ / В. Н. Касьянов, В. К. Сабельфельд // Автоматизированное рабочее место программиста. – Новосибирск, 1988. – С. 4–12.
32. Поттосин И. В. СОКРАТ – система окружения программирования для встроенных ЭВМ / И. В. Поттосин. – Новосибирск, 1992. – 18 с. –



(Препринт / Рос. акад. наук, Сиб. отд-ние, Ин-т систем информатики им. А. П. Ершова ; 11).

33. *Nedorya A.* Mithril – portable Oberon-2 environment / A. Nedorya, A. Nikitin // *Modular Magazine*. – 1993. – N 1. – P. 8–14.

34. *Терехов А. Н.* Технологические средства программирования – методы и инструменты // *Информатика и программирование*. – Новосибирск, 1989. – С. 5–16.

35. *Лаврова Ю. К.* Алгол-68 для IBM PC // *Информатика и программирование*. – Новосибирск, 1989. – С. 121–124.

36. *Хомский Н.* Три модели описания языка // *Кибернетический сборник*. – Москва, 1961. – Вып. 2. – С. 237–266.

37. *Вирт Н.* Алгоритмы + структуры данных = программы : пер. с англ. / Н. Вирт. – Москва : Мир, 1985. – 406 с.

38. *Вирт Н.* Программирование на языке Модула-2 : пер. с англ. / Н. Вирт. – Москва : Мир, 1987. – 224 с.: ил.

39. *Уолш Б.* Программирование на Бейсике : пер. с англ. / Б. Уолш. – Москва : Радио и связь, 1987. – 382 с.

40. *Бартенев О. В.* Фортран для студентов / О. В. Бартенев. – Москва : Диалог-МИФИ, 1999. – 397 с.

41. *Джехани Н.* Язык Ада : пер. с англ. / Н. Джехани. – Москва : Мир, 1988. – 549 с.

42. *Бар Р.* Язык Ада в проектировании систем : пер. с англ. / Р. Бар. – Москва : Мир, 1988. – 320 с.

43. *Жильцова Л. П.* Основы теории автоматов и формальных языков в примерах и задачах : учеб.-метод. пособие / Л. П. Жильцова, Т. Г. Смирнова. – Нижний Новгород : Нижегород. гос. ун-т, 2017. – 64 с.

44. *Марчук Г. И.* Математические модели в иммунологии / Г. И. Марчук. – Москва : Наука, 1980. – 264 с.

45. *Мелса Дж. Л.* Программы в помощь изучающим теорию линейных систем управления / Дж. Л. Мелса, Ст. К. Джонс ; пер. с англ. В. М. Герасимова. – Москва : Машиностроение, 1981. – 200 с.

46. *Бондарчук П. И.* Одношаговые итерационные численные методы для исследования жестких задач // *Численные решения ОДЕ*. – Москва : ИПМ АН СССР, 1978. – С. 111–123.

47. *Хайрер Э.* Решение обыкновенных дифференциальных уравнений : жесткие и дифференциально-алгебраические задачи / Э. Хайрер, Г. Ваннер. – Москва : Мир, 1999. – 685 с.

48. *Новиков Е. А.* Компьютерное моделирование жестких гибридных систем : [монография] / Е. А. Новиков, Ю. В. Шорников. – Новосибирск : Изд-во НГТУ, 2012. – 451 с.

49. *Самарский А. А.* Численные методы / А. А. Самарский, А. В. Гулин. – Москва : Наука, 1989. – 430 с.



50. Новиков Е. А. Явные методы для жестких систем / Е. А. Новиков. – Новосибирск : Наука, 1997. – 195 с.
51. Новиков Е. А. Моделирование жестких гибридных систем = Modeling and simulation of stiff hybrid systems : учеб. пособие / Е. А. Новиков, Ю. В. Шорников. – Санкт-Петербург : Лань, 2019. – 420 с.
52. Маккиман У. Генератор компиляторов : пер. с англ. / У. Маккиман, Дж. Хорнинг, Д. Уортман. – Москва : Статистика, 1980. – 527 с. : ил.
53. Непомнящий В. А. REAL 92: комбинированный язык спецификаций для систем и свойств взаимодействующих процессов реального времени / В. А. Непомнящий, Н. В. Шилов // Программирование. – 1993. – № 6. – С. 64–80.
54. Язык представления знаний «РЕПРО» / И. Л. Артемьева, А. С. Клешев, Ф. Я. Лифшиц, Г. Я. Плис // Объектно-ориентированное программирование : тез. докл. Всесоюз. конф. «Актуальные проблемы системного программирования». – Таллин, 1990. – С. 84–86.
55. Хоар Ч. Взаимодействующие последовательные процессы / Ч. Хоар. – Москва : Мир, 1989. – 264 с.
56. Mantsivoda A. Flang and its implementation // Lecture Notes in Computer Science. – 1993. – Vol. 714. – P. 151–166.
57. Сафонов В. О. Языки и методы программирования в системе «Эльбрус» / В. О. Сафонов. – Москва : Наука, 1989. – 392 с.
58. Как Паскаль и Оберон попадают на «Самсон», или Искусство создания трансляторов / С. К. Кожокар, М. В. Евстунин, А. Н. Терехов, В. А. Уфнаровский. – Кишинев : Штиинца, 1992. – 303 с.
59. Сайты РЕФАЛ-диаспоры. – URL: <http://www.refal.ru/diaspora.html> (дата обращения: 20.04.2022).
60. Turchin V. Metacomputation: metasystem transitions plus supercompilation // Partial Evaluation. – Springer, 1996. – P. 481–510. – (LNCS; vol. 1110). – DOI 10.1007/3-540-61580-6_24.
61. Турчин В. Ф. Программирование на языке Рефал. 1. Неформальное введение в программирование на языке Рефал / В. Ф. Турчин. – Москва, 1971. – 55 с. – (Препринт / Ин-т прикладной математики АН СССР).
62. Сибирское отделение Российской академии наук (СО РАН) : веб-сайт. – URL: <http://www.sbras.ru/> (дата обращения: 20.04.2022).
63. Анисимов В. А. Система параллельного программирования «Иня» // Математическое и архитектурное обеспечение параллельных вычислений. – Новосибирск, 1989. – С. 67–73.
64. Симонов С. А. Статический анализ параллельных программ на языке «Иня» // Математическое и архитектурное обеспечение параллельных вычислений. – Новосибирск, 1989. – С. 74–84.



65. *Симонов С. А.* Языково-независимый инструментальный комплекс для отладки параллельных программ //: Параллельные алгоритмы и структуры. – Новосибирск, 1991. – С. 144–154.
66. *Калверт Ч.* Delphi 5 : энциклопедия пользователя / Ч. Калверт. – Киев : ДиаСофт, 2000.
67. *Гуревич Н.* Освой самостоятельно Visual C++ 5 : пер. с англ. / Н. Гуревич, О. Гуревич. – Москва : Бином, 1998. – 624 с. : ил.
68. *Колецки П.* Oracle JDeveloper 10g. Руководство по разработке Интернет-приложений J2EE с помощью Oracle JDeveloper и Oracle ADF / П. Колецки, Д. Миллс. – Москва : Лори, 2012. – 640 с.
69. *Спольски Д.* Джозл: и снова о программировании / Д. Спольски. – Москва : Символ-плюс, 2009. – 320 с.
70. *Керниган Б.* Язык программирования Си. Задачи по языку Си : пер. с англ. / Б. Керниган, Д. Ритчи, А. Фьюэр. – Москва : Финансы и статистика, 1985. – 279 с.
71. Концептуальное моделирование информационных систем / под ред. В. В. Фильчакова. – Санкт-Петербург : СПБУРЭ ПВО, 1998. – 356 с.
72. *Потемкин В. Г.* Система MATLAB : справ. пособие / В. Г. Потемкин. – Москва : Диалог-МИФИ, 1998. – 350 с.
73. *Бенькович Е. А.* Практическое моделирование динамических систем / Е. А. Бенькович, Ю. Б. Колесов, Ю. Б. Сениченков. – Санкт-Петербург : БХВ-Петербург, 2002. – 464 с.
74. *Шорников Ю. В.* Инструментальные средства компьютерного моделирования динамических систем / Ю. В. Шорников, Т. А. Жданов, В. В. Ландовский // Компьютерное моделирование – 2003 : тр. 4-й Международ. науч.-техн. конф. – Санкт-Петербург : Нестор, 2003. – С. 250–257.
75. *Шорников Ю. В.* Теория и практика языковых процессоров / Ю. В. Шорников. – Новосибирск : Изд-во НГТУ, 2004. – 204 с. – (Учебники НГТУ).
76. *Медведев В. И.* Особенности объектно-ориентированного программирования на C++/CLI, C# и Java / В. И. Медведев. – 2-е изд. – Казань : Школа, 2010. – 437 с.
77. *Златопольский Д.* Основы программирования на Python / Д. Златопольский. – 2-е изд. – Москва : ДМК Пресс, 2018. – 396 с.
78. *Лебедев В. Н.* Введение в системы программирования / В. Н. Лебедев. – Москва : Статистика, 1975. – 311 с.
79. Компиляторы: принципы, технологии и инструментарий / А. В. Ахо, М. С. Лам, Р. Сети, Д. Д. Ульман. – 2-е изд. – Москва : Вильямс, 2008. – 1175 с.
80. *Шорников Ю. В.* Визуально-лингвистическое моделирование гибридных систем // Научный вестник НГТУ. – 2006. – № 2 (23). – С. 65–72.



81. *Шорников Ю. В.* Визуальное моделирование гибридных систем / Ю. В. Шорников, О. В. Никонова // 15-я Международная конференция по компьютерной графике и ее приложениям ГрафиКон'2005 : тр. Конф., 20–24 июня 2005 г. – Новосибирск : Ин-т вычисл. математики и мат. геофизики СО РАН, 2005. – С. 263–266.
82. *Borshchev A.* Java engine for UML based hybrid state machines / A. Borshchev, Yu. Kolesov, Yu. Senichenkov // Proceedings of the 2000 Winter Simulation Conference. – Orlando, FL, USA, 2000. – Vol. 2. – P. 1888–1894.
83. *Архангельский А. Я.* Программирование в C++ Builder 5 / А. Я. Архангельский. – Москва : Бином, 2000. – 1152 с.
84. *Шорников Ю. В.* Компьютерное моделирование динамических систем : учеб. пособие / Ю. В. Шорников, Д. Н. Достовалов. – Новосибирск : Изд-во НГТУ, 2017. – 68 с.
85. *Мэтьюз Д. Г.* Численные методы : использование MATLAB / Д. Г. Мэтьюз, К. Д. Финк. – 3-е изд. – Санкт-Петербург : Вильямс, 2001. – 713 с.
86. *Пратт Т.* Языки программирования: разработка и реализация / Т. Пратт, М. Зелковиц ; под общ. ред. А. Матросова. – 4-е изд. – Санкт-Петербург : Питер, 2002. – 688 с.
87. TOP500 : website. – URL: <https://www.top500.org/> (accessed: 21.04.2022).
88. The Java Virtual Machine Specification. Java SE 15 Edition / T. Lindholm, F. Yellin, G. Bracha, A. Buckley, D. Smith. – URL: <https://docs.oracle.com/javase/specs/jvms/se15/html/index.html> (accessed: 21.04.2022).
89. TIOBE Index : website. – URL: <https://www.tiobe.com/tiobe-index/> (accessed: 21.04.2022).
90. Mono project : website. – URL: <https://www.mono-project.com/> (accessed: 21.04.2022).
91. The Python language official site. – URL: <https://www.python.org/> (accessed: 21.04.2022).
92. V8 JavaScript engine : website. – URL: <https://v8.dev/> (accessed: 21.04.2022).
93. *Lattner C. A.* LLVM: an infrastructure for multi-stage optimization : Master's thesis / C. A. Lattner. – Computer Science Dept., University of Illinois at Urbana-Champaign, 2002. – 61 p.
94. LLVA: a low-level virtual instruction set architecture / V. Adve, C. Lattner, M. Brukman, A. Shukla, B. Gaeke // Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-36, San Diego, CA. – IEEE, 2003. – P. 205–216.



95. *Касьянов В. Н.* Язык программирования Cloud Sisal / В. Н. Касьянов, Е. В. Касьянова. – Новосибирск, 2018. – 42 с. – (Препринт / Рос. акад. наук, Сиб. отд-ние, Ин-т систем информатики им. А. П. Ершова ; 181).
96. *Городняя Л. В.* Язык параллельного программирования Синхро, предназначенный для обучения / Л. В. Городняя. – Новосибирск, 2016. – 30 с. – (Препринт / Рос. акад. наук, Сиб. отд-ние, Ин-т систем информатики им. А. П. Ершова ; 180).
97. *Ануреев И. С.* Концептуальный базис трехуровневого метода верификации C# программ / И. С. Ануреев. – Новосибирск, 2013. – 30 с. – (Препринт / Рос. акад. наук, Сиб. отд-ние, Ин-т систем информатики им. А. П. Ершова ; 170).
98. *Непомнящий В. А.* Применение языка Dynamic-Real для анализа и верификации распределенных систем, специфицированных на языке SDL / В. А. Непомнящий, Е. В. Бодин, С. О. Веретнов. – Новосибирск, 2011. – 52 с. – (Препринт / Рос. акад. наук, Сиб. отд-ние, Ин-т систем информатики им. А. П. Ершова ; 161).
99. *Parr T.* The definitive ANTLR 4 reference / T. Parr. – [S. l.] : Pragmatic Bookshelf, 2013. – 328 p.
100. Roslyn: the Roslyn.NET compiler // GitHub – dotnet : website. – URL: <https://github.com/dotnet/roslyn> (accessed: 21.04.2022).
101. *Levine J.* Flex & bison: text processing tools / J. Levine. – Sebastopol : O'Reilly Media, 2009. – 271 p.
102. ANTLR v4 // ANTLR (ANother Tool for Language Recognition) : website. – URL: <https://www.antlr.org/> (accessed: 21.04.2022).
103. *Карпов Ю. Г.* Имитационное моделирование систем: введение в моделирование с AnyLogic 5 / Ю. Г. Карпов. – Санкт-Петербург : БХВ-Петербург, 2006. – 390 с.
104. *Достовалов Д. Н.* Спецификация и интерпретация моделей переходных процессов в системах электроэнергетики : дис. ... канд. техн. наук / Достовалов Дмитрий Николаевич. – Новосибирск, 2014. – 136 с.
105. Программа графического редактора электроэнергетических систем EPS (ISMA Electric Power Systems) : свидетельство о гос. регистрации программы для ЭВМ № 2013617771 / Ю. В. Шорников, А. Н. Комаричев, Д. Н. Достовалов ; правообладатель : Федер. гос. бюджет. образоват. учреждение высш. проф. образования «Новосиб. гос. техн. ун-т». – Заявка № 2013615377 ; заявл. 27.06.2013 ; зарег. 22.08.2013.
106. Компонента спецификации моделей гибридных систем на языке LISMA_PDE : свидетельство о гос. регистрации программы для ЭВМ № 2015617191 / Ю. В. Шорников, А. В. Бессонов. – Москва : Федер. служба по интеллект. собственности, патентам и товарн. знакам, 2015.



107. Numerical solution of hybrid systems with PDE in the ISMA simulation environment / Y. V. Shornikov, A. V. Bessonov, M. S. Nasyrova, D. N. Dostovalov // Университетский научный журнал. – 2014. – № 10. – С. 189–202.

108. Бессонов А. В. Символьная спецификация и анализ программных моделей гибридных систем : дис. ... канд. техн. наук / Бессонов Алексей Владимирович. – Новосибирск, 2014. – 164 с.

109. *Esposito J.* Accurate event detection for simulating hybrid systems / J. Esposito, V. Kumar, G. J. Pappas // Hybrid systems: computation and control : 4th International Workshop, HSCC 2001, Rome, Italy, March 28–30, 2001 : Proceedings. – Springer, 2001. – P. 204–217. – (LNCS ; vol. 2034).

110. *Harel D.* Executable object modeling with statecharts / D. Harel, E. Gery // Computer. – 1997. – Vol. 30 (7). – P. 31–42.

111. *Abrial J.-R.* Modelling in Event-B: system and software engineering / J.-R. Abrial. – Cambridge ; New York : Cambridge University Press, 2010. – 586 p.

112. Composing modeling and simulation with machine learning in Julia / C. Rackauckas [et al.] // arXiv preprint arXiv:2105.05946. – 2021.

113. *Urqu'ia Moreleda A.* Modeling and simulation in engineering using Modelica / A. Urqu'ia Moreleda, C. Mart'ın Villalba. – Madrid, Spain : UNED Editorial, 2018.

114. Поликарпова Н. И. Автоматное программирование / Н. И. Поликарпова, А. А. Шалыто ; Санкт-Петербургский государственный университет информационных технологий, механики и оптики. – Санкт-Петербург, 2008. – 168 с.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

ВАРИАНТ 1

ЛИСТИНГ ИНТЕРПРЕТАТОРА ПРЕОБРАЗОВАНИЯ
ИНФИКСНОЙ ФОРМЫ В ПОЛИЗ.
ИНТЕРФЕЙС РЕДАКТОРА ВХОДНОГО ЯЗЫКА
АРИФМЕТИЧЕСКИХ ВЫРАЖЕНИЙ И ДИАГНОСТИКА
СИНТАКСИЧЕСКИХ ОШИБОК

Листинг интерпретатора ПОЛИЗ / C++

TO_POLIZ.CPP

```
#ifndef _TO_POLIZ_  
#define _TO_POLIZ_
```

```
#include <ctype.h>
```

```
// прототипы функций  
void Process(char *s, char *d);  
int GetPrior(char l);
```

```
// сам алгоритм  
// s - исходная строка (стек1), d - результирующая строка  
// (стек2)
```

```
void Process(char *s, char *d)  
{
```

```
    // дополнительный стек для операций (стек3) char  
    buf[100];
```

```
    // указатели в стеках s, d и buf соответственно
```

```
    int sp=0, dp=0, bp=0;
```

```
    // пока не конец строки s
```

```
    while(s[sp] != '\0')
```

```
    {
```

```
        // если текущий символ (ТС) - буква ..
```

```
        if(isalpha(s[sp]))
```



```
{
// .. перезапись в стек2
d[dp] = s[sp];
dp++;
sp++;
}
else
{
// .. иначе, если в стеке3 есть операции ..
if(bp>0)
{
// .. просмотр с вершины стека3
for(int i=bp-1; i>=0; i--)
{
// если TC=) вершине стека3 ="(" ..
if(buf[i]=='(' && s[sp]==')')
{
// .. взаимное исключение скобок
bp=i;
// выход из внутреннего цикла
break;
}
// если приоритет TC > приоритета символа в вершине
стека3 или приоритет TC =0 ..
if(GetPrior(s[sp])>GetPrior(buf[i]) || GetPrior(s[sp])==0)
{
// .. запись TC в вершину стека3
buf[i+1] = s[sp];
bp = i+2;
// и выход из внутреннего цикла
break;
}
}
else
{
// .. иначе выталкиваем символ из вершины стека3
в стек2
d[dp] = buf[i];
dp++;
// если цикл неожиданно закончился ..
```



```

        if(i==0)
        {
            // .. записать ТС в стек3
            buf[i] = s[sp];
            bp = 1;
        }
    }
}
sp++;
}
else
{
    // .. если не было операций, запись в стек3
    buf[bp]=s[sp];
    bp++;
    sp++;
}
}
}
// если в стеке3 что-нибудь осталось, то поочередное вы-
талкивание в стек2
for(int i=bp-1; i>=0; i--)
{
    d[dp] = buf[i];
    dp++;
}
// и закончить строку
d[dp]=0;
}

// возвращает приоритет символа l без проверки
int GetPrior(char l)
{
    // таблица приоритетов
    char prior[12][2] = {{ '(', 0}, { ')', 1}, { '|', 2},
                        { '&', 3}, { '<', 4}, { '=', 4},
                        { '>', 4}, { '+', 5}, { '-', 5},
                        { '*', 6}, { '/', 6}, { '^', 7}};

```



```
int pr;
// перебираем всю таблицу - верхнюю строку
for(int i=0; i<12; i++)
{
    // пока не найден символ I
    if(prior[i][0] == I)
    {
        // запомнить его приоритет и выход
        pr = prior[i][1];
        break;
    }
}
return pr;
}

#endif
```

SYNTAX.CPP

```
#ifndef _SYNTAX_
#define _SYNTAX_
// для функции isalpha()
#include <ctype.h>

// прототипы функций
int A(char* s, int& pos);
int B(char* s, int& pos);
int C(char* s, int& pos);
int D(char* s, int& pos);
int E(char* s, int& pos);
int F(char* s, int& pos);
int G(char* s, int& pos);

/*
    -- используются следующие правила --
    G[A]:
    1. A -> B{|B}
    2. B -> C{&C}
```




```
3. C -> D{=D}
4. D -> E{+E}
5. E -> F{*F}
6. F -> G{^G}
7. G -> Б
8. G -> (A)
*/

// -- функция соответствует правилу --
// нет алгоритма Айронса => при первой ошибке прекращает-
// ся разбор
// возвращаемое значение: 0 - ОК, -1 - ошибка
// параметр - текущая строка и позиция в ней

// содержит описание ошибки
char *err;

// TC - текущий символ
// соответствует двум последним правилам
int G(char* s, int& pos)
{
    // если TC - буква ..
    if(isalpha(s[pos]))
    {
        // .. сканирование
        pos++;
        // возврат без ошибок
        return 0;
    }
    // если TC = ( ..
    if(s[pos] == '(')
    {
        // .. сканирование
        pos++;
        // вызов A()
        if(A(s, pos) == -1)
            // если была ошибка - возврат с ошибкой
            return -1;
    }
}
```



```
// если TC = ) ..
if(s[pos] == ')')
{
    // .. сканирование
    pos++;
    // возврат без ошибок
    return 0;
}
else
{
    // .. иначе - сообщение
    err = "Не хватает закрывающей скобки '\')'";
    // возврат с ошибкой
    return -1;
}
}
// .. иначе - сообщение
err = "Неверный идентификатор";
// возврат с ошибкой
return -1;
}

// соответствует шестому правилу
// функции с 1 по 6 имеют одинаковую структуру
// => опишем только эту функцию
int F(char* s, int& pos)
{
    // если строка пуста (TC = \0) ..
    if(s[pos] == '\0')
    {
        // .. сообщение
        err = "Отсутствует выражение";
        // возврат с ошибкой
        return -1;
    }
    // вызов функции G()
    if(G(s, pos) == -1)
        // если были ошибки - возврат с ошибкой
```



```
    return -1;
    // проверка итерации {^G}
    // цикл пока встречаются знаки ^ (ТС = ^)
    while(s[pos]=='^')
    {
        // сканирование
        pos++;
        // вызов функции G()
        if(G(s, pos) == -1)
            // если были ошибки - возврат с ошибкой
            return -1;
    }
    // возврат без ошибки
    return 0;
}

int E(char* s, int& pos)
{
    if(s[pos] == '\0')
    {
        err = "Отсутствует выражение";
        return -1;
    }

    if(F(s, pos) == -1)
        return -1;

    while(s[pos]=='*' || s[pos]=='/')
    {
        pos++;
        if(F(s, pos) == -1)
            return -1;
    }
    return 0;
}

int D(char* s, int& pos)
{

```



```
    if(s[pos] == '\0')
    {
        err = "Отсутствует выражение";
        return -1;
    }

    if(E(s, pos) == -1)
        return -1;

    while(s[pos]=='+' || s[pos]=='-')
    {
        pos++;
        if(E(s, pos) == -1)
            return -1;
    }
    return 0;
}

int C(char* s, int& pos)
{
    if(s[pos] == '\0')
    {
        err = "Отсутствует выражение";
        return -1;
    }

    if(D(s, pos) == -1)
        return -1;

    while(s[pos]=='<' || s[pos]=='=' || s[pos]=='>')
    {
        pos++;
        if(D(s, pos) == -1)
            return -1;
    }
    return 0;
}

int B(char* s, int& pos)
{

```



```
    if(s[pos] == '\0')
    {
        err = "Отсутствует выражение";
        return -1;
    }

    if(C(s, pos) == -1)
        return -1;

    while(s[pos]!='&')
    {
        pos++;
        if(C(s, pos) == -1)
            return -1;
    }
    return 0;
}

int A(char* s, int& pos)
{
    if(s[pos] == '\0')
    {
        err = "Отсутствует выражение";
        return -1;
    }

    if(B(s, pos) == -1)
        return -1;

    while(s[pos]!='|')
    {
        pos++;
        if(B(s, pos) == -1)
            return -1;
    }
    return 0;
}

#endif
```

**Main.cpp**

```
//-----ИНТЕРФЕЙС-----
#include <vcl.h>
#pragma hdrstop

#include "Main.h"
#include "ChildWin.h"
#include "About.h"
#include "syntax.cpp"
#include "to_poliz.cpp"
//-----
#pragma resource "*.dfm"
TMainForm *MainForm;
//-----
// -- конструктор --
__fastcall TMainForm::TMainForm(TComponent *Owner)
    : TForm(Owner)
{
    Application->HelpFile = GetCurrentDir()+"\\main.hlp";
}
//-----
// -- функция создания подчиненной формы --
void __fastcall TMainForm::CreateMDIChild(String Name)
{
    // объект типа подчиненной формы (текстовое окно)
    TMDIChild *Child;
    // создание объекта
    Child = new TMDIChild(Application);
    // заполнили название
    Child->Caption = Name;
    // если существует файл с таким именем ..
    if(FileExists(Name))
    {
        // .. то загрузка текста из него
        Child->Мемо->Lines->LoadFromFile(Name);
        // запоминаем его имя
        Child->FileName = Name;
        // считаем, что он сохранен и не изменен
    }
}
```



```
        Child->Saved = true;
        Child->Memo->Modified = false;
    }
    else
    {
        // .. иначе текст остается пустым, и считается,
        // что текст не сохранен и изменен
        Child->Saved = false;
        Child->Memo->Modified = true;
    }
}
//-----
// -- создание нового чистого окна --
void __fastcall TMainForm::FileNew1Execute(TObject *Sender)
{
    // создание подчиненного окна с именем NONAME##
    CreateMDIChild("NONAME" + IntToStr((MDIChildCount +
1)/10) + IntToStr((MDIChildCount + 1)%10));
    // теперь можно разрешить пользоваться кнопками
    // Сохранить, Сохранить как и Запуск
    FileSave1->Enabled = true;
    FileSaveAs1->Enabled = true;
    ActionRun1->Enabled = true;
}
//-----
// -- функция открытия файла --
void __fastcall TMainForm::FileOpen1Execute(TObject *Sender)
{
    // если файл выбран в диалоге выбора ..
    if(OpenDialog->Execute())
    {
        // .. то создать новую подчиненную форму с
        // заголовком = имя открытого файла
        CreateMDIChild(OpenDialog->FileName);
        // теперь можно разрешить пользоваться кнопками
        // Сохранить, Сохранить как и Запуск
        FileSave1->Enabled = true;
        FileSaveAs1->Enabled = true;
    }
}
```



```
        ActionRun1->Enabled = true;
    }
}
//-----
// -- функция вывода окна "О программе" --
void __fastcall TMainForm::HelpAbout1Execute(TObject *Sender)
{
    // открыть окно "О программе" как модальное
    AboutBox->ShowModal();
}
//-----
// -- функция закрытия приложения --
void __fastcall TMainForm::FileExit1Execute(TObject *Sender)
{
    Close();
}
//-----
// -- функция сохранения файла --
void __fastcall TMainForm::FileSave1Execute(TObject *Sender)
{
    // проверим, есть ли у нас хоть одно открытое
    // подчиненное окно ..
    if(MDIChildCount > 0)
    {
        // .. ссылка
        // на активное окно, которое будем сохранять
        TMDIChild *Child;
        // приведение типов
        Child = (TMDIChild *)MainForm->ActiveMDIChild;
        if(Child->Memo->Modified)
        {
            // если содержимое окна изменялось ..
            if(Child->Saved)
            {
                // .. и если оно уже сохранялось ..
                // .. то сохранение под тем же именем
                Child->Memo->Lines-
                >SaveToFile(Child->FileName);
                Child->Memo->Modified = false;
            }
        }
    }
}
```



```
//-----  
// -- сохранить файл под другим именем --  
void __fastcall TMainForm::FileSaveAs1Execute(TObject  
*Sender)  
{  
    // проверить, есть хотя бы одно открытое подчиненное  
    окно?  
    if(MDICHildCount > 0)  
    {  
        // есть, даем возможность выбрать
```



```
        // новое имя файла
        if(SaveDialog->Execute())
        {
            // выбор сделан, создать ссылку на актив-
ную форму
            TMDIChild *Child;
            // в этом поможет СВОЙСТВО главной
формы
            Child = (TMDIChild *)MainForm-
>ActiveMDIChild;
            // сохранять текст в файл
            Child->запомнить имя файла
            Child->FileName = SaveDialog->FileName;
            // поменять заголовок
            Child->Caption = SaveDialog->FileName;
            // забыть про изменения
            Child->Memo->Modified = false;
            // напоминаем, что файл уже сохранялся
            Child->Saved = true;
        }
    }
}
//-----
// -- функция показать/убрать строку сообщения
void __fastcall TMainForm::N7Click(TObject *Sender)
{
    // проверить, отмечен ли пункт меню с именем N7 ..
    if(N7->Checked)
    {
        // .. отмечен => сделать неотмеченным
        N7->Checked = false;
        // скрыть Memo
        MessageMemo->Visible = false;
        // скрыть Splitter
        Splitter->Visible = false;
    }
    else
    {

```



```

        // .. не отмечен => сделать отмеченным
        N7->Checked = true;
        // показать Memo
        MessageMemo->Visible = true;
        // показать Splitter
        Splitter->Visible = true;
    }
}
//-----
// -- функция закрытия подчиненного окна --
// -- если сделана попытка закрыть приложение --
void __fastcall TMainForm::FormCloseQuery(TObject *Sender,
bool &CanClose)
{
    // CanClose = "Да" по умолчанию, т. е. можно закрыть

    TMDIChild *Child;
    // для каждого из всех открытых подчинённых окон ..
    for(int i=MDIChildCount-1; i>=0; i--)
    {
        // .. необходимо получить ссылку
        Child = (TMDIChild *)MainForm->MDIChildren[i];
        // и закрыть
        Child->Close();
    }
}
//-----
// -- запуск алгоритма Дейкстры --
void __fastcall TMainForm::ActionRun1Execute(TObject *Sender)
{
    TMDIChild* Child;
    // получить ссылку на активное подчиненное окно
    Child = (TMDIChild *)MainForm->ActiveMDIChild;
    AnsiString str;
    str = Child->Memo->Text;
    int i, len;
    len = str.Length();
    // удалить из текста все пробелы, ТАВы и ENTERы
    for(i=1; i<=len; i++)

```



```
{
    if(str[i]==' ' || str[i]=='\t' || str[i]=='\n' || str[i]=='\r')
    {
        str.Delete(i, 1);
        len = str.Length();
        i--;
    }
}
// очистить буфер сообщений и добавить туда .....
MainForm->MessageMemo->Lines->Clear();
MainForm->MessageMemo->Lines->Add("Исходное
выражение:");
MainForm->MessageMemo->Lines->Add(str);
MainForm->MessageMemo->Lines->Add("");
MainForm->MessageMemo->Lines->Add("Проверка
синтаксиса:");
// сначала ошибок нет
err = "Нет ошибок.";

int stat, pos=0;
// вызвать первое правило для строки str с начала
stat = A(str.c_str(), pos);
// особенность работы правил: проверка
// до конца ли разобрана строка ..
// комментарий к выражению в скобках:
// str - типа AnsiString
// str.c_str() - метод, возвращающий строку типа char*
// а раз (str.c_str()) типа char*, то (str.c_str())[pos] - типа
char - текущий символ
if((str.c_str())[pos] != '\0')
{
    // если не до конца - ошибка
    err = "Неверное выражение";
    stat = -1;
}
// вывод ошибки (она единственная)
MainForm->MessageMemo->Lines->Add(err);
MainForm->MessageMemo->Lines->Add("");
// если ошибок нет - перевод в ПОЛИЗ
```



```
        if(stat == 0)
        {
            char res[200];
            Process(str.c_str(), res);
            MainForm->MessageMemo->Lines-
>Add("ПОЛИЗ:");
            MainForm->MessageMemo->Lines->Add(res);
        }

        // показать буфер сообщений
        if(!N7->Checked)
        {
            N7Click(Sender);
        }
    }
//-----

void __fastcall TMainForm::N3Click(TObject *Sender)
{
    // вызов файла справки
    Application->HelpContext(10);
}
//-----

void __fastcall TMainForm::N4Click(TObject *Sender)
{
    Application->HelpContext(20);
}
//-----

void __fastcall TMainForm::N9Click(TObject *Sender)
{
    Application->HelpContext(30);
}
//-----

void __fastcall TMainForm::N10Click(TObject *Sender)
{
    Application->HelpContext(40);
}
```



```
}  
//-----  
  
void __fastcall TMainForm::N11Click(TObject *Sender)  
{  
    Application->HelpContext(50);  
}  
//-----  
  
void __fastcall TMainForm::N12Click(TObject *Sender)  
{  
    Application->HelpContext(100);  
}  
//-----
```

Childwin.cpp

```
//-----  
#include <vcl.h>  
#pragma hdrstop  
  
#include "Main.h"  
#include "ChildWin.h"  
//-----  
#pragma resource "*.dfm"  
//-----  
// -- конструктор --  
__fastcall TMDIChild::TMDIChild(TComponent *Owner)  
    : TForm(Owner)  
{  
    // файл заново создан и никогда не сохранялся  
    Saved = false;  
    // и не имеет имени  
    FileName = "";  
}  
//-----  
// -- если сделана попытка закрыть эту форму --  
void __fastcall TMDIChild::FormClose(TObject *Sender, TClose-  
Action &Action)  
{
```



```

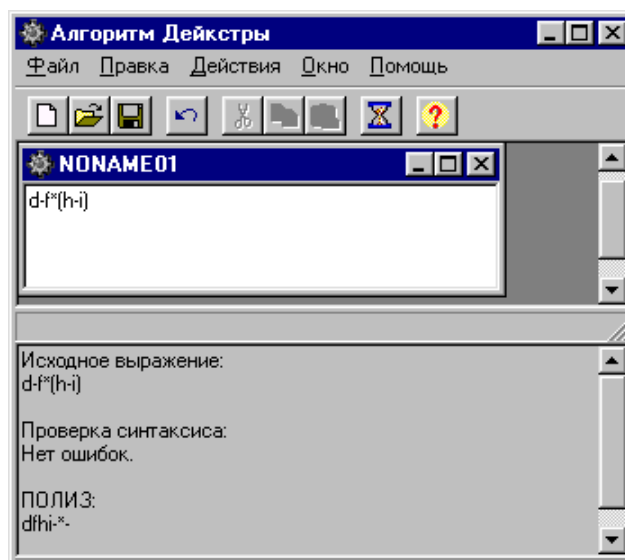
        // нужно ли его сохранять ?
        if(Memo->Modified)
        {
            int answ;
            // .. нужно, так как он был изменен, но надо
            // об этом спросить ..
            answ = Application->MessageBoxA("Данный файл был из-
менен. Сохранить эти изменения ?", FileName.c_str(),
MB_ICONQUESTION|MB_YESNO);
            if(answ == IDYES)
            {
                // если да
                // куда его сохранить ..
                if(Saved)
                {
                    // .. он уже сохранялся => туда же и
сохранить
                    Memo->Lines->SaveToFile(FileName);
                }
                else
                {
                    // .. его еще никто не сохранял
                    // изменить заголовок диалога со-
хранить
                    MainForm->SaveDialog->Title =
"Сохранение файла " + Caption;
                    // дать возможность выбрать имя
файла ..
                    if(MainForm->SaveDialog->Execute())
                    {
                        // .. имя выбрано, сохранить
                        Memo->Lines-
>SaveToFile(MainForm->SaveDialog->FileName);
                    }
                }
            }
        }
        // закроем его
        Action = caFree;

```



```
// проверка, было ли это последнее
// незакрытое окно ..
if(MainForm->MDIChildCount == 1)
{
    // .. было последнее, необходимо запретить до-
ступ к кнопкам
    // Сохранить, Сохранить как и Закрыть
    MainForm->FileSave1->Enabled = false;
    MainForm->FileSaveAs1->Enabled = false;
    MainForm->ActionRun1->Enabled = false;
}
}
//-----
```

СКРИНШОТ ИНТЕРФЕЙСА И ТЕСТА ПОЛИЗ



**ВАРИАНТ 2**

ЛИСТИНГ / C#
ИНТЕРФЕЙСА И ПАРСЕРА УСЛОВНОГО ОПЕРАТОРА if
С БЛОКОМ ДЕЙСТВИЙ ЯЗЫКА C/C++

```
using System;
using System.Windows.Forms;
using System.IO;
using System.Collections.Generic;
using System.Drawing;

namespace TFLC_CW
{
    // Для использования метода Interaction при создании файла
    using Microsoft.VisualBasic;

    public partial class MainForm : Form
    {
        // список позиций некорректных символов
        List<int> errlist = new List<int>();

        public MainForm()
        {
            InitializeComponent();
            // задать типы файлов для отображения
            // в диалоговом окне OpenFileDialog или SaveFileDialog
            openFileDialog1.Filter = "txt files (*.txt)|*.txt|All files (*.*)|*.*";
            saveFileDialog1.Filter = "txt files (*.txt)|*.txt|All files (*.*)|*.*";
            // задать атрибуты шрифта в текстовых полях
            richTextBox1.Font = new Font("Microsoft Sans Serif", 9);
            richTextBox2.Font = new Font("Microsoft Sans Serif", 9);
        }

        // Обработчик нажатия на кнопку Выход
        private void buttonExit_Click_1(object sender, EventArgs e)
        {
            Close(); // закрыть форму (приложение)
        }
    }
}
```



```
}

// Обработчик нажатия на кнопку Создать файл
private void buttonCreate_Click(object sender, EventArgs e)
{
    fileCreate(); // вызов функции Создать файл
}

// Обработчик нажатия на кнопку Открыть файл
private void buttonOpen_Click(object sender, EventArgs e)
{
    fileOpen(); // вызов функции Открыть файл
}

// Обработчик нажатия на кнопку Сохранить файл
private void buttonSave_Click(object sender, EventArgs e)
{
    fileSave(); // вызов функции Сохранить файл
}

// Обработчик нажатия на кнопку Отменить
private void buttonCancel_Click(object sender, EventArgs e)
{
    undoText(); // вызов функции Отменить
}

// Обработчик нажатия на кнопку Вырезать
private void buttonCut_Click(object sender, EventArgs e)
{
    cutText(); // вызов функции Вырезать
}

// Обработчик нажатия на кнопку Копировать
private void buttonCopy_Click(object sender, EventArgs e)
{
    copyText(); // вызов функции Копировать
}
```



```
// Обработчик нажатия на кнопку Вставить
private void buttonPaste_Click(object sender, EventArgs e)
{
    pasteText(); // вызов функции Вставить
}
```

```
// Обработчик нажатия на кнопку Создать файл (выпада-
ющее меню Файл)
private void createToolStripMenuItem_Click(object sender,
EventArgs e)
{
    fileCreate(); // вызов функции Создать файл
}
```

```
// Обработчик нажатия на кнопку Открыть файл (выпа-
дающее меню Файл)
private void openToolStripMenuItem_Click(object sender,
EventArgs e)
{
    fileOpen(); // вызов функции Открыть файл
}
```

```
// Обработчик нажатия на кнопку Сохранить файл (выпа-
дающее меню Файл)
private void saveToolStripMenuItem_Click(object sender,
EventArgs e)
{
    fileSave(); // Вызов функции Сохранить файл
}
```

```
// Обработчик нажатия на кнопку Отменить (выпадаю-
щее меню Правка)
private void cancelToolStripMenuItem_Click(object sender,
EventArgs e)
{
    undoText(); // вызов функции Отменить
}
```



// Обработчик нажатия на кнопку Копировать (выпадающее меню Правка)

```
private void copyToolStripMenuItem_Click(object sender, EventArgs e)
{
    copyText(); // вызов функции Копировать
}
```

// Обработчик нажатия на кнопку Вставить (выпадающее меню Правка)

```
private void pasteToolStripMenuItem_Click(object sender, EventArgs e)
{
    pasteText(); // вызов функции Вставить
}
```

// Обработчик нажатия на кнопку Вырезать (выпадающее меню Правка)

```
private void cutToolStripMenuItem_Click(object sender, EventArgs e)
{
    cutText(); // вызов функции Вырезать
}
```

// Обработчик нажатия на кнопку О программе (выпадающее меню Справка)

```
private void infoToolStripMenuItem_Click(object sender, EventArgs e)
{
    // создать экземпляр диалогового окна О программе
    ToP_Lab2.InfoForm info = new ToP_Lab2.InfoForm();
    // показать созданное диалоговое окно
    info.ShowDialog();
}
```

// Изменение размера шрифта выделенного фрагмента текста

```
private void buttonEditFont_Click(object sender, EventArgs e)
```



```
{ // показать панель редактирования цвета текста
fontDialog1.ShowColor = true;

// текущие параметры текста (размер шрифта и цвет)
fontDialog1.Font = richTextBox1.SelectionFont;
fontDialog1.Color = richTextBox1.SelectionColor;

// если изменение параметров текста не отменено
if (fontDialog1.ShowDialog() != DialogResult.Cancel)
{ // установить выбранные размер шрифта и цвет
  // для выбранного фрагмента текста
  richTextBox1.SelectionFont = fontDialog1.Font;
  richTextBox2.SelectionFont = fontDialog1.Font;
  richTextBox1.SelectionColor = fontDialog1.Color;
  richTextBox2.SelectionColor = fontDialog1.Color;
}
}

// Увеличение размера шрифта выделенного фрагмента
// текста
private void buttonUpperCase_Click(object sender, EventArgs e)
{ // переменная для работы с размером шрифта
  float currentSize;

  // убедиться, что текст в данный момент выделен в
  // текстовом поле 1
  if (richTextBox1.SelectedText != "")
  { // сохранить текущий размер шрифта
    currentSize = richTextBox1.SelectionFont.Size;
    // увеличить размер на 2.0F
    currentSize += 2.0F;
    // установить новый размер шрифта выделенному
    // фрагменту текста
    richTextBox1.SelectionFont = new Font("Microsoft Sans
    Serif", currentSize,
    richTextBox1.SelectionFont.Style, richTextBox1.SelectionFont.Unit);
  }
```



```
    }
    // убедиться, что текст в данный момент выделен в
текстовом поле 2
    if (richTextBox2.SelectedText != "")
    { // сохранить текущий размер шрифта
      currentSize = richTextBox2.SelectionFont.Size;
      // увеличить размер на 2.0F
      currentSize += 2.0F;
      // установить новый размер шрифта выделенному
фрагменту текста
      richTextBox2.SelectionFont = new Font("Microsoft Sans
Serif", currentSize,
      richTextBox2.SelectionFont.Style,           richText-
Box2.SelectionFont.Unit);
    }
    // убрать выделение текста
    richTextBox1.Select(0, 0);
    richTextBox2.Select(0, 0);
  }

  // Уменьшение размера шрифта выделенного фрагмента
текста
  private void buttonLowerCase_Click(object sender, Even-
tArgs e)
  { // // переменная для работы с размером шрифта
    float currentSize;

    // убедиться, что текст в данный момент выделен в
текстовом поле 1
    if (richTextBox1.SelectedText != "" && richText-
Box1.SelectionFont.Size > 10)
    { // сохранить текущий размер шрифта
      currentSize = richTextBox1.SelectionFont.Size;
      // уменьшить размер на 2.0F
      currentSize -= 2.0F;
      // установить новый размер шрифта выделенному
фрагменту текста
      richTextBox1.SelectionFont = new Font("Microsoft Sans
Serif", currentSize,
```



```

        richTextBox1.SelectionFont.Style,                richTextBox1.
Box1.SelectionFont.Unit);
    }
    // убедиться, что текст в данный момент выделен в
текстовом поле 2
    if (richTextBox2.SelectedText != "" && richTextBox2.
Box2.SelectionFont.Size > 10)
    { // сохранить текущий размер шрифта
        currentSize = richTextBox2.SelectionFont.Size;
        // уменьшить размер на 2.0F
        currentSize -= 2.0F;
        // установить новый размер шрифта выделенному
фрагменту текста
        richTextBox2.SelectionFont = new Font("Microsoft Sans
Serif", currentSize,
        richTextBox2.SelectionFont.Style,                richTextBox2.
Box2.SelectionFont.Unit);
    }
    // убрать выделение текста
    richTextBox1.Select(0, 0);
    richTextBox2.Select(0, 0);
}

// Создать файл
private void fileCreate()
{
    string str = "";
    // сохранить в str введенные пользователем данные
    str = Interaction.InputBox("Введите имя файла", "Созда-
ние файла");

    if (str != "") // если имя файла было введено
    {
        // строка path записывает путь к файлу
        string path =
Path.Combine(Environment.GetFolderPath(Environment.SpecialF
older.Desktop), str + ".txt");
        File.WriteAllText(path, "");
    }
}

```



```
// получить имя файла
openFileDialog1.FileName = str;

// если ввод имени файла был отменен - возврат
if (openFileDialog1.ShowDialog() == DialogResult.Cancel)
    return;

// считать название файла в строку filename
string filename = openFileDialog1.FileName;
// считать текст из файла в строку fileText
string fileText = System.IO.File.ReadAllText(filename);
// выгрузить текст из строки fileText в окно редактирования формы
richTextBox1.Text = fileText;
}
else // если имя файла не было введено
{
    MessageBox.Show("Имя файла не должно быть пустым");
}
}

// Открыть файл
private void fileOpen()
{
    // если открытие файла было отменено - возврат
    if (openFileDialog1.ShowDialog() == DialogResult.Cancel)
        return;

    // сохранить имя выбранного файла в строку filename
    string filename = openFileDialog1.FileName;
    // сохранить текст из файла в строку fileText
    string fileText = System.IO.File.ReadAllText(filename);
    // выгрузить строку fileText в окно редактирования формы
    richTextBox1.Text = fileText;

    MessageBox.Show("Файл открыт");
}
```




```
}

// Сохранить файл
private void fileSave()
{
    // если сохранение файла было отменено - возврат
    if (saveFileDialog1.ShowDialog() == DialogResult.Cancel)
        return;

    // сохранить имя выбранного файла в строку filename
    string filename = saveFileDialog1.FileName;
    // сохранить текст из файла в окно редактирования
    формы
    System.IO.File.WriteAllText(filename, richTextBox1.Text);

    MessageBox.Show("Файл сохранен");
}

// Отменить последнее действие с текстом
private void undoText()
{
    // определить, можно ли отменить последнюю опера-
    цию в текстовом поле
    if (richTextBox1.CanUndo == true)
    {
        // отменить последнюю операцию
        richTextBox1.Undo();
        // очистить буфер отмены
        richTextBox1.ClearUndo();
    }
}

// Вырезать выделенный текстовый фрагмент
private void cutText()
{
    // убедиться, что текст в данный момент выделен в
    текстовом поле
    if (richTextBox1.SelectedText != "")
```



```
// вырезать выделенный текст в текстовом поле и
вставить его в буфер обмена
richTextBox1.Cut();
}

// Копировать выделенный текстовый фрагмент
private void copyText()
{
    // убедиться, что текст в данный момент выделен в
    текстовом поле
    if (richTextBox1.SelectionLength > 0)
        // копировать текст в буфер обмена
        richTextBox1.Copy();
}

// Вставить текстовый фрагмент из буфера
private void pasteText()
{
    // определить, есть ли в буфере обмена текст для
    вставки в текстовое поле
    if (Clipboard.GetDataObject().GetDataPresent(DataFormats.Text) ==
true)
    {
        // определить, выделен ли какой-либо фрагмент
        текста в текстовом поле
        if (richTextBox1.SelectionLength > 0)
        {
            // спросить пользователя, хочет ли он вставить
            текст
            // поверх текущего выделенного текста
            if (MessageBox.Show("Вставить поверх текущего
выделения?", "", MessageBoxButtons.YesNo) == Di-
alogResult.No)
                // переместить текст после текущего выделения
                и вставить
                richTextBox1.SelectionStart = richTextBox1.
Box1.SelectionStart + richTextBox1.SelectionLength;
```



```
    }
    // вставить фрагмент текста из буфера обмена в
текстовое поле
    richTextBox1.Paste();
}

// Обработчик нажатия на кнопку Анализ
private void buttonAnalyze_Click(object sender, EventArgs e)
{
    // очистка окна вывода сообщений
    richTextBox2.Clear();
    errlist.Clear();
    // сохранение текста из окна редактирования
    string str = richTextBox1.Text;

    // процедура фильтрации строки (удалить из текста
неразрешенные символы)
    str = System.Text.RegularExpressions.Regex.Replace(str, "[^a-zA-Z0-9+-
=<>/*;(){}]", "");
    // вывод отфильтрованной строки
    richTextBox2.Text = "Исходное выражение: " + str + "\n";
    // переменная для посимвольной работы со строкой
    int pos = 0;
    // запуск анализирующей функции
    analyzer(str, ref pos);
}

// Запустить анализ строки на наличие
// корректного <IF с блоком действий> выражения
int analyzer(string s, ref int pos)
{
    if (instr(s, ref pos) == -1 || errlist.Count > 0)
    {
        // если ошибка в разборе строки
        // выделить некорректные символы
        foreach (int num in errlist)
```



```
{
    if (num <= s.Length)
    {
        richTextBox2.Select(num + 20, 1);
        richTextBox2.SelectionColor = Color.Red;
        richTextBox2.SelectionFont = new Font("Tahoma",
12, FontStyle.Bold);
    }
}
// возврат с ошибкой
return -1;
}
else
{ // если ошибок нет
    richTextBox2.Text = "Исходное выражение: " + s +
"\nПроверка синтаксиса: Ошибок нет";
    return 0;
}
}

// Основная функция разбора строки
int instr(string s, ref int pos)
{ // проверка наличия <if>-токен в начале строки
    if (pos + 2 <= s.Length && s.Substring(pos,
2).Contains("if"))
    {
        pos = pos + 2; // переход через if-токен
        // проверка наличия ( для условия
        if (pos >= s.Length)
        { // добавление строки ошибки в окно вывода
            richTextBox2.AppendText("\nERROR: Нет открывающей скобки для условного выражения");
            return -1;
        }
        else if (!s.Substring(pos, 1).Contains("("))
        {
            errlist.Add(pos); // добавление позиции символа в
список
            richTextBox2.AppendText("\nERROR: Нет открывающей скобки для условного выражения");
        }
    }
}
```



```
    }

    pos++; // следующий символ

    // проверка условного выражения
    if (condition(s, ref pos) == -1)
    { // добавление строки ошибки в окно вывода
        richTextBox2.AppendText("\nERROR: Некорректное
условное выражение");
    }

    // проверка наличия ) для условия
    if (pos >= s.Length)
    {
        richTextBox2.AppendText("\nERROR: Нет закрывающей
скобки для условного выражения");
        return -1;
    }
    else if (!s.Substring(pos, 1).Contains("("))
    {
        errlist.Add(pos); // добавление позиции символа в
список
        richTextBox2.AppendText("\nERROR: Нет закрывающей
скобки для условного выражения");
    }
    else
        pos++;

    // проверка наличия { для IF
    if (pos >= s.Length)
    {
        richTextBox2.AppendText("\nERROR: Нет открывающей
фигурной скобки для <if>");
        return -1;
    }
    else if (!s.Substring(pos, 1).Contains("{"))
    {
        errlist.Add(pos); // добавление позиции символа в
список
```



```
        richTextBox2.AppendText("\nERROR: Нет открывающей фигурной скобки для <if>");
        //return -1;
    }
    else
    {
        pos++; // следующий символ
    }

    // проверка выражений в блоке действий
    if (S(s, ref pos) == -1)
    {
        richTextBox2.AppendText("\nERROR: Некорректное выражение в блоке действий");
        // return -1;
    }

    // проверка наличия } для IF
    if (pos >= s.Length)
    {
        richTextBox2.AppendText("\nERROR: Нет закрывающей фигурной скобки для <if>");
        return -1;
    }
    else if (!s.Substring(pos, 1).Contains("}"))
    {
        errlist.Add(pos);
        richTextBox2.AppendText("\nERROR: Нет закрывающей фигурной скобки для <if>");
        return -1;
    }
}
else if (pos >= s.Length) // входная строка - пустая
{
    // вывод ошибки
    richTextBox2.AppendText("\nERROR: Отсутствует <IF с блоком действий> выражение");
    return -1;
}
```



```
    }
    else // входная строка не начинается с if-токена
    {
        // добавить позицию символа в список
        errlist.Add(pos);
        richTextBox2.AppendText("\nERROR:    Отсутствует
ключевое слово <if>");
        return -1;
    }
    return 0;
}

// Проверить условие (условного выражения)
int condition(string s, ref int pos)
{ // если строка пуста
    if (pos >= s.Length)
        return -1;
    // если первый символ - ')'
    if (s[pos] == ')')
    {
        errlist.Add(pos); // добавить ошибку в список
        richTextBox2.AppendText("\nERROR:    Условное    вы-
ражение не может быть пустым");
        return 0;
    }

    // проверка 1 символа условного выражения
    // если 1 символ - не буква
    if (!Char.IsLetter(s[pos]))
    { // добавление ошибки в список
        errlist.Add(pos);
        richTextBox2.AppendText("\nERROR:    Некорректная
левая часть условного выражения");
    }

    pos++; // следующий символ
    // если символ пустой
    if (pos >= s.Length)
```



```
{ // выход с ошибкой
    richTextBox2.AppendText("\nERROR:      Отсутствует
знак в условном выражении");
    return -1;
}

// проверка 2 символа условного выражения
// если 2 символ - не знак > | < | =
if (s[pos] != '>' && s[pos] != '<' && s[pos] != '=')
{ // добавление ошибки в список
    errlist.Add(pos);
    richTextBox2.AppendText("\nERROR:   Некорректный
знак в условном выражении");
    // если знак ')'
    if (s[pos] == ')')
    {
        richTextBox2.AppendText("\nERROR: Некорректное
условное выражение");
        return 0;
    }

    pos++; // следующий символ
    // текущий символ - буква
    if (pos < s.Length && Char.IsLetter(s[pos]))
    {
        pos++; // следующий символ
        // проверка следующих символов
        checkSymb(s, ref pos);
        return 0; // возврат без ошибки
    }
    // текущий символ - цифра
    else if (pos < s.Length && Char.IsDigit(s[pos]))
    { // считать все число в цикле
        while (pos < s.Length && Char.IsDigit(s[pos]))
            pos++;
        // проверка следующих символов
        checkSymb(s, ref pos);
        return 0; // возврат без ошибки
    }
}
```




```
    }
    // если текущий символ - не буква и не цифра
    else
    { // возврат с ошибкой
      if (pos >= s.Length)
      { // выход с ошибкой
        richTextBox2.AppendText("\nERROR: Некоррект-
ная правая часть условного выражения");
        return -1;
      }
      // проверка следующих символов
      checkSymb(s, ref pos);
      // добавление ошибки в список
      errlist.Add(pos);
      //richTextBox2.AppendText("\nERROR: Некоррект-
ная правая часть условного выражения");
      return 0;
    }
  }

  // если 2 символ - знак > | <
  if (pos < s.Length && (s[pos] == '>' || s[pos] == '<'))
  {
    pos++; // следующий символ
    // текущий символ - буква
    if (pos < s.Length && Char.IsLetter(s[pos]))
    {
      pos++; // следующий символ
      // проверка следующих символов
      checkSymb(s, ref pos);
      return 0; // возврат без ошибки
    }
    // текущий символ - цифра
    else if (pos < s.Length && Char.IsDigit(s[pos]))
    { // считать все число в цикле
      while (pos < s.Length && Char.IsDigit(s[pos]))
        pos++;
      // проверка следующих символов
```



```
        checkSymb(s, ref pos);
        return 0; // возврат без ошибки
    }
    // если текущий символ - не буква и не цифра
    else
    { // если конец строки
        if (pos >= s.Length)
        { // выход с ошибкой
            richTextBox2.AppendText("\nERROR: Некор-
ректная правая часть условного выражения");
            return -1;
        }
        // добавление в список ошибок
        errlist.Add(pos);
        //richTextBox2.AppendText("\nERROR: Некор-
ректная правая часть условного выражения");
        // проверка последующих символов
        checkSymb(s, ref pos);
        return 0;
    }
}
// если 2 символ - знак =
else if (pos < s.Length && s[pos] == '=')
{
    pos++; // следующий символ
    // если конец строки
    if (pos >= s.Length)
    { // возврат с ошибкой
        richTextBox2.AppendText("\nERROR: Знак в услов-
ном выражении некорректен");
        return -1;
    }
    // если знак == некорректен
    if (s[pos] != '=')
    { // запись ошибки в список
        errlist.Add(pos);
        richTextBox2.AppendText("\nERROR: Знак в услов-
ном выражении некорректен");
    }
}
```



```
// если символ ')'
if (s[pos] == ')')
{ // возврат в основную функцию
    richTextBox2.AppendText("\nERROR: Некоррект-
ная правая часть условного выражения");
    return 0;
}

// если текущий символ - буква
if (pos < s.Length && Char.IsLetter(s[pos]))
{
    pos++; // следующий символ
    // проверка следующих символов
    checkSymb(s, ref pos);
    return 0; // возврат без ошибки
}
// если текущий символ - цифра
else if (pos < s.Length && Char.IsDigit(s[pos]))
{ // считать все число в цикле
    while (pos < s.Length && Char.IsDigit(s[pos]))
        pos++;
    // проверка следующих символов
    checkSymb(s, ref pos);
    return 0; // возврат без ошибки
}
// если текущий символ - не буква и не цифра
else
{ // проверка следующих символов
    checkSymb(s, ref pos);
    richTextBox2.AppendText("\nERROR: Некоррект-
ная правая часть условного выражения");
    return 0;
}
}

// если текущий (3) символ - знак =
else
{
    pos++; // следующий символ
```



```
// если конец строки
if (pos >= s.Length)
{ // возврат с ошибкой
    richTextBox2.AppendText("\nERROR: Некоррект-
ная правая часть условного выражения");
    return -1;
}

// если текущий символ - буква
if (Char.IsLetter(s[pos]))
{
    pos++; // следующий символ
    // проверка следующих символов
    checkSymb(s, ref pos);
    return 0; // возврат без ошибки
}
// если текущий символ - цифра
else if (Char.IsDigit(s[pos]))
{ // считать все число в цикле
    while (pos < s.Length && Char.IsDigit(s[pos]))
        pos++;
    // проверка следующих символов
    checkSymb(s, ref pos);
    // возврат без ошибки
    return 0;
}
// если текущий символ - не буква и не цифра
else
{ // запись ошибки в список
    checkSymb(s, ref pos);
    return 0;
}
}
}
return 0; // возврат без ошибки
}
```

// Разобрать выражение в блоке действий



```
int S(string s, ref int pos)
{
    // если конец строки
    if (pos >= s.Length)
    {
        richTextBox2.AppendText("\nERROR: Некорректное
выражение в блоке действий");
        return -1;
    }

    if (s[pos] == '}')
    { // если первый символ - },
      // т.е. выражений в блоке действий нет
        return 0;
    }

    if (!Char.IsLetter(s[pos]))
    { // добавление ошибки в список
        errlist.Add(pos);
        richTextBox2.AppendText("\nERROR: Некорректное
выражение в блоке действий");
    }

    pos++; // следующий символ
    // проверка инкремента
    if (pos < s.Length && s[pos] == '+')
    {
        pos++; // следующий символ
        // если ошибка в записи инкремента
        if (pos >= s.Length)
        {
            richTextBox2.AppendText("\nERROR: Ошибка в
выражении инкремента");
            return -1;
        }
        if (s[pos] != '+')
        {
            errlist.Add(pos);
        }
    }
}
```



```
richTextBox2.AppendText("\nERROR: Ошибка в  
выражении инкремента");  
    // проверка следующих символов  
    if (s[pos] != ';')  
    {  
        while (pos < s.Length && s[pos] != ';')  
        {  
            if (s[pos] == '}')  
                return 0;  
  
            errlist.Add(pos);  
            pos++;  
        }  
    }  
    pos++;  
    if (pos < s.Length && Char.IsLetter(s[pos]))  
    { // если символ - буква, вызываем функцию S  
        // для разбора следующего выражения  
        S(s, ref pos);  
    }  
}  
else // если инкремент записан корректно  
{  
    pos++; // следующий символ  
    // проверка на наличие ; после выражения  
    if (pos >= s.Length)  
    {  
        richTextBox2.AppendText("\nERROR: Отсут-  
ствует ; после выражения в блоке действий");  
        return -1;  
    }  
    else if (s[pos] != ';')  
    {  
        errlist.Add(pos);  
        richTextBox2.AppendText("\nERROR: Отсут-  
ствует ; после выражения в блоке действий");  
        // проверка следующих символов  
        if (s[pos] != ';')
```



```

        {
            while (pos < s.Length && s[pos] != ';')
            {
                if (s[pos] == '}')
                    return 0;

                errlist.Add(pos);
                pos++;
            }
        }
        pos++;
        if (pos < s.Length && Char.IsLetter(s[pos]))
        { // если символ - буква, вызываем функцию
S
            // для разбора следующего выражения
            S(s, ref pos);
        }
    }
    else // ошибок нет, поэтому проверка
    { // следующего символа на наличие выраже-
ния
        pos++;

        if (pos < s.Length && Char.IsLetter(s[pos]))
        { // если символ - буква, вызываем функцию
S
            // для разбора следующего выражения
            S(s, ref pos);
        }
    }
}
}
else if (pos < s.Length && s[pos] == '=')
{ // если выражение - не инкремент
    pos++; // следующий символ
    // проверка правой части выражения
    if (pos < s.Length && (Char.IsLetter(s[pos]) ||
Char.IsDigit(s[pos])))

```



```
{ // если символ - буква/цифра, вызвать функцию S
  // для разбора следующего выражения
  if (A(s, ref pos) == -1)
    return -1;
}
else // иначе
{ // искать первый символ - букву/цифру
  errlist.Add(pos);
  richTextBox2.AppendText("\nERROR: Некоррект-
ное выражение в блоке действий");
  //pos++;
  while (pos < s.Length && !Char.IsLetter(s[pos]) &&
!Char.IsDigit(s[pos]) && s[pos] != ';')
  { // занести ошибку в список
    errlist.Add(pos);
    pos++;
  } // и вызвать S или A
  if (s[pos] == ';')
  {
    pos++;
    S(s, ref pos);
    return 0;
  }

  if (A(s, ref pos) == -1)
    return -1;
}
}
else if (pos >= s.Length)
{ // если второго символа нет
  richTextBox2.AppendText("\nERROR: Некорректное
выражение в блоке действий");
  return -1;
}
else // если второй символ выражения
{ // не является допустимым
  errlist.Add(pos);
  richTextBox2.AppendText("\nERROR: Некорректное
выражение в блоке действий");
```




```
        if (s[pos] != ';')
        {
            while (pos < s.Length && s[pos] != ';')
            {
                errlist.Add(pos);
                pos++;
            }
        }
        pos++;
        if (pos < s.Length && Char.IsLetter(s[pos]))
        { // если символ - буква, вызываем функцию S
            // для разбора следующего выражения
            S(s, ref pos);
        }
        else // иначе
        { // искать первый символ - букву
            while (pos < s.Length && !Char.IsLetter(s[pos]))
            { // занести ошибку в список
                errlist.Add(pos);
                pos++;
            } // и вызвать S
            S(s, ref pos);
        }
    }

    // если выражение не содержит ошибок
    return 0;
}

// Проверить правую часть выражения
// A -> B{+B}
int A(string s, ref int pos)
{
    // если конец строки
    if (pos >= s.Length)
    {
        richTextBox2.AppendText("\nERROR: Ошибка в пра-
вой части выражения в блоке действий");
    }
}
```



```
        return -1;
    }

    // вызов функции B
    if (B(s, ref pos) == -1)
    { // если были ошибки – возврат с ошибкой
        return -1;
    }

    // цикл пока встречаются знаки + и -
    while (pos < s.Length && (s[pos] == '+' || (s[pos] == '-')))
    {
        pos++; // следующий символ
        // вызов функции B
        // если были ошибки – возврат с ошибкой
        if (B(s, ref pos) == -1)
            return -1;
    }

    // если выражение закончилось - проверка следующего
символа
    if (pos < s.Length && s[pos] == ';')
    {
        pos++; // следующий символ
        if (pos < s.Length && s[pos] == '}')
            return 0;
        // если символ - буква, вызов функции S
        // для разбора следующего выражения
        if (pos < s.Length && Char.IsLetter(s[pos]))
        {
            if (S(s, ref pos) == -1)
            { // если в функции S были ошибки -
                return -1; // возврат с ошибкой
            }
            return 0; // если ошибок не было
        }
        else // иначе (символ - не буква)
        { // искать первый символ - букву
```



```
while (pos < s.Length && !Char.IsLetter(s[pos]))
{ // проверить символ на == '}'
  if (pos < s.Length && s[pos] == '}')
    return 0;
  // занести ошибку в список
  errlist.Add(pos);
  pos++;
} // и вызвать S для буквы или конца строки
S(s, ref pos);
}
}
else // если не найден знак ; после выражения
{ // добавление ошибки в список
  errlist.Add(pos);
  richTextBox2.AppendText("\nERROR: Отсутствует ;
после выражения в блоке действий");
  // проверка следующих символов
  if (pos < s.Length && s[pos] != ';')
  { // пока не найден символ ; или конец строки
    while (pos < s.Length && s[pos] != ';')
    { // если найден символ }
      if (pos < s.Length && s[pos] == '}')
        return 0; // возврат
      // добавление ошибки в список
      errlist.Add(pos);
      pos++; // следующий символ
    }
  }
  pos++; // следующий символ
  // если символ - буква, вызываем функцию S
  if (pos < s.Length && Char.IsLetter(s[pos]))
  { // если в функции S были ошибки
    if (S(s, ref pos) == -1)
      return -1; // возврат с ошибкой

    return 0; // если ошибок не было
  }
}
```



```
        return 0; // возврат без ошибки
    }

    // B -> C{*B}
    int B(string s, ref int pos)
    {
        // если строка пуста
        if (pos >= s.Length)
        { // вывод текста ошибки
            richTextBox2.AppendText("\nERROR: Ошибка в правой части выражения в блоке действий");
            return -1; // возврат с ошибкой
        }

        // вызов функции C
        if (C(s, ref pos) == -1)
        { // если были ошибки – возврат с ошибкой
            return -1;
        }

        // цикл пока встречаются знаки * и /
        while (pos < s.Length && (s[pos] == '*' || s[pos] == '/'))
        {
            pos++; // следующий символ
            // вызов функции C
            // если были ошибки – возврат с ошибкой
            if (C(s, ref pos) == -1)
                return -1;
        }
        // возврат без ошибки
        return 0;
    }

    // C -> Б | ЧК // C -> (A)
    int C(string s, ref int pos)
    {
        // если текущий символ (ТС) - буква или цифра
        if (Char.IsLetter(s[pos]) || Char.IsDigit(s[pos]))
```



```
{
    if (Char.IsDigit(s[pos]))
    { // если цифра - читаем все число в цикле
        while (pos < s.Length && Char.IsDigit(s[pos]))
        {
            pos++;
        }
        // возврат без ошибки
        return 0;
    }
    // следующий символ
    pos++;
    // возврат без ошибки
    return 0;
}
// следующий символ
pos++;
// вызов функции A
// если была ошибка - возврат с ошибкой
if (A(s, ref pos) == -1)
    return -1;

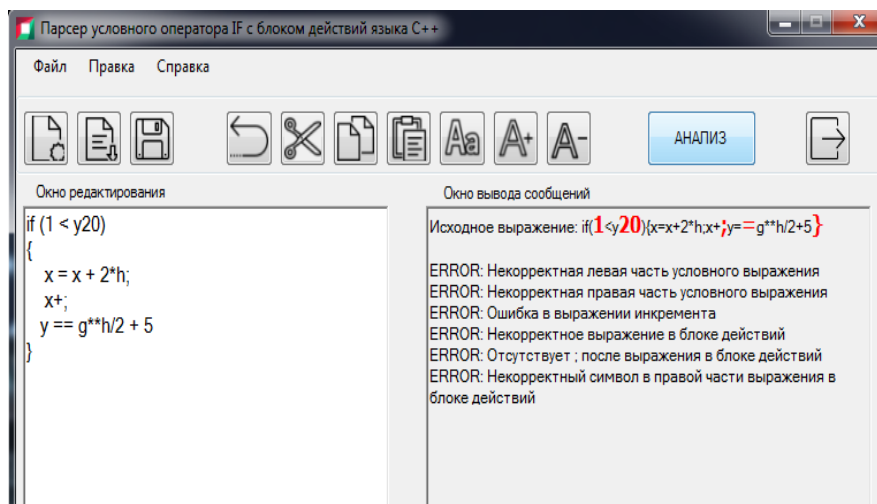
//MessageBox.Show("Исходное выражение: " + s +
"\nERROR: Некорректный символ в правой части выраже-
ния");
richTextBox2.AppendText("\nERROR:      Некорректный
символ в правой части выражения в блоке действий");
return -1; // возврат с ошибкой
}

// Проверить последовательность символов
// на содержание символа ')'
void checkSymb(string s, ref int pos)
{
    // проверка следующих символов
    if (pos < s.Length && s[pos] != ')')
    { // пока не конец строки или символ ')'
        if (s[pos] == '{')
```



```
        return;
        while (pos < s.Length && s[pos] != ')')
        { // добавить ошибку в список
            errlist.Add(pos);
            pos++; // следующий
        }
        richTextBox2.AppendText("\nERROR:   Некорректная
        правая часть условного выражения");
    }
}
}
```

СКРИНШОТ ИНТЕРФЕЙСА И ТЕСТА ПАРСЕРА if





ПРИЛОЖЕНИЕ Б

ГРАММАТИКА ЯЗЫКА *LISMA_PDE*

Таблица ПБ.1

Сокращенные обозначения нетерминальных символов

<i>multiplicativeExpressionOperator</i>	E10	<i>not operator</i>	E11
<i>partial operand spatial common</i>	P3	<i>ode equation</i>	E12
<i>unaryExpressionNotPlusMinus</i>	E18	<i>relationalOp</i>	E16
<i>additiveExpressionOperator</i>	E2	<i>and operator</i>	E3
<i>equalityExpressionOperator</i>	E8	<i>or operator</i>	E13
<i>partial operand spatial N</i>	P4	<i>pde equation</i>	D0
<i>conditionalAndExpression</i>	E4	<i>linear eq A</i>	L1
<i>derivative quote operant</i>	D1	<i>linear eq b</i>	L4
<i>multiplicativeExpression</i>	E9	<i>linear vars</i>	L5
<i>unaryExpressionOperator</i>	E19	<i>pseudo state</i>	L
<i>conditionalOrExpression</i>	E6	<i>spatial var</i>	V1
<i>partial operand common</i>	P1	<i>expression</i>	E0
<i>conditionalExpression</i>	E5	<i>state body</i>	S1
<i>linear eq A elem expr</i>	L3	<i>state from</i>	S2
<i>partial operand mixed</i>	P2	<i>edge side</i>	B2
<i>relationalExpression</i>	E15	<i>linear eq</i>	L0
<i>func and math mapping</i>	F	<i>pde param</i>	D3
<i>additiveExpression</i>	E1	<i>init cond</i>	I
<i>equalityExpression</i>	E7	<i>statement</i>	O
<i>pseudo state body</i>	L1	<i>arg list</i>	A
<i>pseudo state elem</i>	L2	<i>constant</i>	C
<i>spatial var bound</i>	V2	<i>edge eq</i>	B1
<i>linear eq A elem</i>	L2	<i>equation</i>	E
<i>spatial var tail</i>	V3	<i>primary</i>	P0
<i>unaryExpression</i>	E17	<i>variable</i>	V
<i>derivative ident</i>	D	<i>macros</i>	M
<i>parExpression</i>	E14	<i>setter</i>	G
<i>partial operand</i>	P	<i>lisma</i>	L
<i>pde param atom</i>	D4	<i>state</i>	S
		<i>edge</i>	B



Таблица ПБ.2

Сокращенные обозначения терминальных символов

'apx'	a	'on'	o	'*'	o2	'=='	l4	'dx4'	d7
'if'	i0	'pde'	p	'.'	c0	'>'	l5	'dy'	d8
'set'	g	'right'	r	'/'	o3	'>='	l6	'dy2'	d9
'from'	t	'step'	s	'.'	s0	'%'	o6	'dy3'	d10
'var'	v	Identifier	i	'/'	b3	' '	o7	'dy4'	d11
'ls'	l0	Decimal	f	'\"	d0	'&&'	o8	'dz'	d12
'both'	b	MacroIdentifier	n	'/'	b4	'!'	o9	'dz2'	d13
'const'	c	'='	q0	'/'	b5	'D'	d2	'dz3'	d14
'der'	d1	'_'	o1	'}'	b6	'DD'	d3	'dz4'	d15
'edge'	e	'!='	l1	'+'	o5	'dx'	d4		
'left'	l	'('	b1	'<'	l2	'dx2'	d5		
'macro'	m	')'	b2	'<='	l3	'dx3'	d6		

Таблица ПБ.3

Грамматика G_{LISMA_PDE} с сокращенными обозначениями

L ::= O*	D3 ::= D4 o2	E1 ::= E9 (E2 E9)*
O ::= C I E S L V1 B M L0 L5	D4 ::= f i b3 E0 b4	E2 ::= o1 o5
C ::= c (i q0)+ E0 s0	S ::= s I b1 E0 b2 S1	E9 ::= E17(E10 E17)*
V1 ::= v I b3 V2 c0 V2 b4 V3? s0	S2? s0	E10 ::= o2 o3 o6
V2 ::= o1? f i	S1 ::= b5 (E0 G)* b6	E17 ::= E19 E17 E18
V3 ::= a f s i	S2 ::= t i (c0 i)*	E19 ::= o1 o5
P ::= P1 P2 P3 P4	L ::= i0 b1 E0 b2 L1	E18 ::= E11 E17 P0
P1 ::= d2 b1 I c0 (c0 f)? b2	L1 ::= b5 L2 b6	P0 ::= E14 i f P F
P2 ::= d3 b1 i c0 i c0 i (c0 f)? b2	L2 ::= E G	E13 ::= o7
P3 ::= (d4 d8 d12) b1 i (c0 f)? b2	F ::= i b1 A b2	E3 ::= o8
P4 ::= (d5 d6 d7 d9 d10 d11 d13 d14 d15)b1 i b2	A ::= E0 (c0 E0)*	E11 ::= o9
B ::= e B1 o I B2 s0	D ::= i D1 d1 b1 i c0 f b2	M ::= n q0 E0 s0
B1 ::= (i P) q0 E0	D1 ::= d0+	G ::= g l q0 E0 s0
B2 ::= l r b	V ::= i D	L5 ::= v l0 i (c0 i)* s0
	E14 ::= b1 E0 b2	L0 ::= l0 L1 q0 L4 s0
	E0 ::= E5	L4 ::= E0
	E5 ::= E6	L1 ::= L2 (o5 L2)*
	E6 ::= E4 (E13 E4)*	
	E4 ::= E7 (E3 E7)*	



I ::= V b1 f b2 q0 E0 s0 E ::= E12 D0 E12 ::= V q0 E0 s0 D0 ::= p P(o5 P)*q0 E0 s0	E7 ::= E15 (E8 E15)* E8 ::= l4 l1 E15 ::= E1 (E16 E1)* E16 ::= l2 l3 l5 l6	L2 ::= i o2 L3 L3 ::= P0 E9
---	---	--

Таблица ПБ.4

Множества FIRST₁ и FIRST₂

№	Правило	FIRST ₁	FIRST ₂
O	C	c	ci
	I	i	ib1
	E	i, d1, p	iq0, id0, d1b1, p(d2-d15)
	S	s	si
	S0	i0	i0b1
	V1	v	vi
	B	e	ei, e(d2-d15)
	M	n	nq0
	L0	l0	l0i
	L5	v	vl0
E12	i q0 E0 s0	i	iq0
	D q0 E0 s0	i, d1	id0, d1b1
P0	E14	b1	b1b1, bli, b1f, b1(d2-d15), b1o1, b1o5, b1o9
	i	i	i
	f	f	f
	P	d2-d15	(d2-d15)b1
	F	i	ib1
S1	b5 E0+ b6	b5	b5i, b5d1, b5p
	b5 G+ b6	b5	b5g

УЧЕБНОЕ ИЗДАНИЕ

Шорников Юрий Владимирович

ТЕОРИЯ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ

Учебное пособие

Редактор *Е.Е. Татарникова*
Выпускающий редактор *И.П. Брованова*
Художественный редактор *А.В. Ладыжская*
Корректор *И.Е. Семенова*
Компьютерная верстка *А.В. Сухарева*

Подписано в печать 05.12.2022
Формат 70 × 100 1/16. Бумага офсетная
Уч.-изд. л. 23,54. Печ. л. 18,25.
Тираж 50 экз. (1-й з-д – 1–50 экз.)
Изд. № 58. Заказ № 327.

Налоговая льгота – Общероссийский классификатор продукции
Издание соответствует коду 95 3000 ОК 005-93 (ОКП)

Издательство Новосибирского государственного
технического университета
630073, г. Новосибирск, пр. К. Маркса, 20
Тел. (383) 346-31-87
E-mail: office@publish.nstu.ru

Отпечатано в типографии
Новосибирского государственного технического университета
630073, г. Новосибирск, пр. К. Маркса, 20